Kotlin Documentation

Kotlin Tour + Language Guide • Kotlin 2.4.21 (Oct 2026)

Source: kotlinlang.org/docs · github.com/JetBrains/kotlin-web-site (Apache License 2.0)

Typeset 2026-10-13

Contents

Kotlin Tour

- Welcome to our tour of Kotlin!
- Hello world
- Basic types
- Collections
- Control flow
- Functions
- Classes
- Null safety
- Extension functions
- Scope functions
- Lambda expressions with receiver
- Classes and interfaces
- Objects
- Open and special classes
- Properties
- Null safety
- Libraries and APIs

Language Guide

- Basic syntax overview
- Keywords and operators
- Packages and imports
- Annotations
- Visibility modifiers
- Coding conventions
- Idioms
- Types overview
- Numbers
- Unsigned integer types
- Booleans
- Characters
- Strings
- Arrays
- Type checks and casts
- Type aliases
- Conditions and loops
- Returns and jumps

Exception and error handling
Functions
Higher-order functions and lambdas
Type-safe builders
Using builders with builder type inference
Context parameters
Inline functions
Operator overloading
Unused return value checker
Classes
Data classes
Extensions
Interfaces
Delegation
Inheritance
Object declarations and expressions
This expressions
Sealed classes and interfaces
Enum classes
Inline value classes
Nested and inner classes
Functional \(SAM\) interfaces
Properties
Delegated properties
Null safety
Equality
Generics: in, out, where
Asynchronous control flow and suspending functions
Coroutines
Destructuring declarations

Android

Kotlin for Android

Kotlin Tour

Welcome to our tour of Kotlin!

> These tours can be completed entirely within your browser. There is no installation required.

Quickly learn the essentials of the Kotlin programming language through our tours.

Grasp the fundamentals.

```
    Start
</panel>
<panel>
    <title>Intermediate</title>
```

Take your understanding of Kotlin to the next level.

```
    Start
</panel>
```

Hello world

Here is a simple program that prints "Hello, world!":

```
fun main() {
    println("Hello, world!")
    // Hello, world!
}
```

In Kotlin:

- `fun` is used to declare a function
- The `main()` function is where your program starts from
- The body of a function is written within curly braces `{}`
- `println()` and `print()` functions print their arguments to standard output

A function is a set of instructions that performs a specific task. Once you create a function, you can use it whenever you need to perform that task, without having to write the instructions all over again. Functions are discussed in more detail in a couple of chapters. Until then, all examples use the `main()` function.

Variables

All programs need to be able to store data, and variables help you to do just that. In Kotlin, you can declare:

- Read-only variables with `val`
- Mutable variables with `var`

  Note: You can't change a read-only variable once you have given it a value.

To assign a value, use the assignment operator `=`.

For example:

```
fun main() {
    val popcorn = 5    // There are 5 boxes of popcorn
    val hotdog = 7     // There are 7 hotdogs
    var customers = 10 // There are 10 customers in the queue

    // Some customers leave the queue
    customers = 8
    println(customers)
    // 8
}
```

> **Tip:** Variables can be declared outside the `main()` function at the beginning of your program. Variables declared in this way are said to be declared at **top level**.

As `customers` is a mutable variable, its value can be reassigned after declaration.

> **Note:** We recommend declaring all variables as read-only (`val`) by default. Only use mutable variables (`var`) if you really need to. That way, you're less likely to accidentally change something that wasn't meant to change.

String templates

It's useful to know how to print the contents of variables to standard output. You can do this with **string templates**. You can use template expressions to access data stored in variables and other objects, and convert them into strings. A string value is a sequence of characters in double quotes `"`. Template expressions always start with a dollar sign `$`.

To evaluate a piece of code in a template expression, place the code within curly braces `{}` after the dollar sign `$`.

For example:

```
fun main() {
    val customers = 10
    println("There are $customers customers")
    // There are 10 customers

    println("There are ${customers + 1} customers")
    // There are 11 customers
}
```

For more information, see String templates.

You will notice that there aren't any types declared for variables. Kotlin has inferred the type itself: `Int`. This tour explains the different Kotlin basic types and how to declare them in the next chapter.

Practice

Print a statement using string templates

Complete the code to make the program print `"Mary is 20 years old"` to standard output:

```
fun main() {
    val name = "Mary"
    val age = 20


```

```
    // Write your code here
}
```

```
fun main() {
    val name = "Mary"
    val age = 20
    println("$name is $age years old")
}
```

```
<a as="button" href="kotlin-tour-basic-types.md" mode="classic" icon="arrow-right" icon-posi
tion="right">Next step</a>
```

Basic types

Every variable and data structure in Kotlin has a type. Types are important because they tell the compiler what you are allowed to do with that variable or data structure. In other words, what functions and properties it has.

In the last chapter, Kotlin was able to tell in the previous example that `customers` has type `Int`. Kotlin's ability to **infer** the type is called **type inference**. `customers` is assigned an integer value. From this, Kotlin infers that `customers` has a numerical type `Int`. As a result, the compiler knows that you can perform arithmetic operations with `customers`:

```
fun main() {
    var customers = 10

    // Some customers leave the queue
    customers = 8

    customers = customers + 3 // Example of addition: 11
    customers += 7            // Example of addition: 18
    customers -= 3            // Example of subtraction: 15
    customers *= 2            // Example of multiplication: 30
    customers /= 3            // Example of division: 10

    println(customers) // 10
}
```

> **Tip:** `+=`, `-=`, `*=`, `/=`, and `%=` are augmented assignment operators. For more information, see Augmented assignments.

In total, Kotlin has the following basic types:

Category	Basic types	Example code
Integers	`Byte`, `Short`, `Int`, `Long`	`val year: Int = 2020`
`val amount: Long = 350_000_000`		
Unsigned integers	`UByte`, `UShort`, `UInt`, `ULong`	`val score: UInt = 100u`
Floating-point numbers	`Float`, `Double`	`val currentTemp: Float = 24.5f`
`val price: Double = 19.99`		
Booleans	`Boolean`	`val isEnabled: Boolean = true`
Characters	`Char`	`val separator: Char = ','`

Category	Basic types	Example code
Strings	`String`	`val message: String = "Hello, world!"`

For more information on basic types and their properties, see Types overview.

With this knowledge, you can declare variables and initialize them later. Kotlin can manage this as long as variables are initialized before the first read.

To declare a variable without initializing it, specify its type with `:`. For example:

```
fun main() {
    // Variable declared without initialization
    val d: Int
    // Variable initialized
    d = 3

    // Variable explicitly typed and initialized
    val e: String = "hello"

    // Variables can be read because they have been initialized
    println(d) // 3
    println(e) // hello
}
```

If you don't initialize a variable before it is read, you see an error:

```
fun main() {
    // Variable declared without initialization
    val d: Int

    // Triggers an error
    println(d)
    // Variable 'd' must be initialized
}
```

Now that you know how to declare basic types, it's time to learn about collections.

Practice

Declare explicit types for variables

Explicitly declare the correct type for each variable:

```
fun main() {
    val a: Int = 1000
    val b = "log message"
    val c = 3.14
    val d = 100_000_000_000_000
```

```
    val e = false
    val f = '\n'
}
```

```
fun main() {
    val a: Int = 1000
    val b: String = "log message"
    val c: Double = 3.14
    val d: Long = 100_000_000_000_000
    val e: Boolean = false
    val f: Char = '\n'
}
```

```
<a as="button" href="kotlin-tour-hello-world.md" mode="outline" icon="arrow-left" icon-posit
ion="left">Previous step</a>



<a as="button" href="kotlin-tour-collections.md" mode="classic" icon="arrow-right" icon-posi
tion="right">Next step</a>
```

Collections

When programming, it is useful to be able to group data into structures for later processing. Kotlin provides collections for exactly this purpose.

Kotlin has the following collections for grouping items:

Collection type	Description
Lists	Ordered collections of items
Sets	Unique unordered collections of items
Maps	Sets of key-value pairs where keys are unique and map to only one value

Each collection type can be mutable or read only.

List

Lists store items in the order that they are added, and allow for duplicate items.

To create a read-only list (`List`), use the `listOf()` function.

To create a mutable list (`MutableList`), use the `mutableListOf()` function.

When creating lists, Kotlin can infer the type of items stored. To declare the type explicitly, add the type within angled brackets `<>` after the list declaration:

```
fun main() {
    // Read only list
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println(readOnlyShapes)
    // [triangle, square, circle]

    // Mutable list with explicit type declaration
    val shapes: MutableList<String> = mutableListOf("triangle", "square", "circle")
    println(shapes)
    // [triangle, square, circle]
}
```

> **Tip:** To prevent unwanted modifications, you can create a read-only view of a mutable list by assigning it to a `List`:
>
> ```
>     val shapes: MutableList<String> = mutableListOf("triangle", "square", "circl
> e")
>     val shapesLocked: List<String> = shapes
> ```
>
> This is also called **casting**.

Lists are ordered so to access an item in a list, use the indexed access operator `[]`:

```
fun main() {
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("The first item in the list is: ${readOnlyShapes[0]}")
    // The first item in the list is: triangle
}
```

To get the first or last item in a list, use `.first()` and `.last()` functions respectively:

```
fun main() {
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("The first item in the list is: ${readOnlyShapes.first()}")
    // The first item in the list is: triangle
}
```

> **Note:** `.first()` and `.last()` functions are examples of **extension** functions. To call an extension function on an object, write the function name after the object appended with a period `.`
>
> Extension functions are covered in detail in the intermediate tour. For now, you only need to know how to call them.

To get the number of items in a list, use the `.count()` function:

```
fun main() {
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("This list has ${readOnlyShapes.count()} items")
    // This list has 3 items
}
```

To check that an item is in a list, use the `in` operator:

```
fun main() {
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("circle" in readOnlyShapes)
    // true
}
```

To add or remove items from a mutable list, use `.add()` and `.remove()` functions respectively:

```
fun main() {
    val shapes: MutableList<String> = mutableListOf("triangle", "square", "circle")
    // Add "pentagon" to the list
    shapes.add("pentagon")
    println(shapes)
    // [triangle, square, circle, pentagon]

    // Remove the first "pentagon" from the list
    shapes.remove("pentagon")
```

```
    println(shapes)
    // [triangle, square, circle]
}
```

Set

Whereas lists are ordered and allow duplicate items, sets are **unordered** and only store **unique** items.

To create a read-only set (`Set`), use the `setOf()` function.

To create a mutable set (`MutableSet`), use the `mutableSetOf()` function.

When creating sets, Kotlin can infer the type of items stored. To declare the type explicitly, add the type within angled brackets `<>` after the set declaration:

```
fun main() {
    // Read-only set
    val readOnlyFruit = setOf("apple", "banana", "cherry", "cherry")
    // Mutable set with explicit type declaration
    val fruit: MutableSet<String> = mutableSetOf("apple", "banana", "cherry", "cherry")

    println(readOnlyFruit)
    // [apple, banana, cherry]
}
```

You can see in the previous example that because sets only contain unique elements, the duplicate `"cherry"` item is dropped.

> **Tip:** To prevent unwanted modifications, you can create a read-only view of a mutable set by assigning it to a `Set`:
>
> ```
>     val fruit: MutableSet<String> = mutableSetOf("apple", "banana", "cherry", "c
> herry")
>     val fruitLocked: Set<String> = fruit
> ```
>
> **Note:** As sets are **unordered**, you can't access an item at a particular index.

To get the number of items in a set, use the `.count()` function:

```
fun main() {
    val readOnlyFruit = setOf("apple", "banana", "cherry", "cherry")
    println("This set has ${readOnlyFruit.count()} items")
    // This set has 3 items
}
```

To check that an item is in a set, use the `in` operator:

```
fun main() {
    val readOnlyFruit = setOf("apple", "banana", "cherry", "cherry")
    println("banana" in readOnlyFruit)
    // true
}
```

To add or remove items from a mutable set, use `.add()` and `.remove()` functions respectively:

```
fun main() {
    val fruit: MutableSet<String> = mutableSetOf("apple", "banana", "cherry", "cherry")
    fruit.add("dragonfruit")    // Add "dragonfruit" to the set
    println(fruit)              // [apple, banana, cherry, dragonfruit]

    fruit.remove("dragonfruit") // Remove "dragonfruit" from the set
    println(fruit)              // [apple, banana, cherry]
}
```

Map

Maps store items as key-value pairs. You access the value by referencing the key. You can imagine a map like a food menu. You can find the price (value), by finding the food (key) you want to eat. Maps are useful if you want to look up a value without using a numbered index, like in a list.

> **Note:** * Every key in a map must be unique so that Kotlin can understand which value you want to get.
>
> - You can have duplicate values in a map.

To create a read-only map (`Map`), use the `mapOf()` function.

To create a mutable map (`MutableMap`), use the `mutableMapOf()` function.

When creating maps, Kotlin can infer the type of items stored. To declare the type explicitly, add the types of the keys and values within angled brackets `<>` after the map declaration. For example: `MutableMap<String, Int>`. The keys have type `String` and the values have type `Int`.

The easiest way to create maps is to use `to` between each key and its related value:

```
fun main() {
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println(readOnlyJuiceMenu)
    // {apple=100, kiwi=190, orange=100}

    // Mutable map with explicit type declaration
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi" to 190, "or
ange" to 100)
    println(juiceMenu)
```

```
    // {apple=100, kiwi=190, orange=100}
}
```

Tip: To prevent unwanted modifications, you can create a read-only view of a mutable map by assigning it to a `Map`:

```
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi"
to 190, "orange" to 100)
    val juiceMenuLocked: Map<String, Int> = juiceMenu
```

To access a value in a map, use the indexed access operator `[]` with its key:

```
fun main() {
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println("The value of apple juice is: ${readOnlyJuiceMenu["apple"]}")
    // The value of apple juice is: 100
}
```

Note: If you try to access a key-value pair with a key that doesn't exist in a map, you see a `null` value:

```
fun main() {
//sampleStart
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 10
0)
    println("The value of pineapple juice is: ${readOnlyJuiceMenu["pineappl
e"]}")
    // The value of pineapple juice is: null
//sampleEnd
}
```

This tour explains null values later in the Null safety chapter.

You can also use the indexed access operator `[]` to add items to a mutable map:

```
fun main() {
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi" to 190, "or
ange" to 100)
    juiceMenu["coconut"] = 150 // Add key "coconut" with value 150 to the map
    println(juiceMenu)
    // {apple=100, kiwi=190, orange=100, coconut=150}
}
```

To remove items from a mutable map, use the `.remove()` function:

```
fun main() {
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi" to 190, "or
ange" to 100)
```

```
    juiceMenu.remove("orange")    // Remove key "orange" from the map
    println(juiceMenu)
    // {apple=100, kiwi=190}
}
```

To get the number of items in a map, use the `.count()` function:

```
fun main() {
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println("This map has ${readOnlyJuiceMenu.count()} key-value pairs")
    // This map has 3 key-value pairs
}
```

To check if a specific key is already included in a map, use the `.containsKey()` function:

```
fun main() {
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println(readOnlyJuiceMenu.containsKey("kiwi"))
    // true
}
```

To obtain a collection of the keys or values of a map, use the `keys` and `values` properties respectively:

```
fun main() {
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println(readOnlyJuiceMenu.keys)
    // [apple, kiwi, orange]
    println(readOnlyJuiceMenu.values)
    // [100, 190, 100]
}
```

> **Note:** `keys` and `values` are examples of **properties** of an object. To access the property of an object, write the property name after the object appended with a period `.`
>
> Properties are discussed in more detail in the Classes chapter. At this point in the tour, you only need to know how to access them.

To check that a key or value is in a map, use the `in` operator:

```
fun main() {
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println("orange" in readOnlyJuiceMenu.keys)
    // true

    // Alternatively, you don't need to use the keys property
    println("orange" in readOnlyJuiceMenu)
    // true

    println(200 in readOnlyJuiceMenu.values)
```

```
    // false
}
```

For more information on what you can do with collections, see Collections.

Now that you know about basic types and how to manage collections, it's time to explore the control flow that you can use in your programs.

Practice

Count the total number of items in two lists

You have a list of “green” numbers and a list of “red” numbers. Complete the code to print how many numbers there are in total.

```
fun main() {
    val greenNumbers = listOf(1, 4, 23)
    val redNumbers = listOf(17, 2)
    // Write your code here
}
```

```
fun main() {
    val greenNumbers = listOf(1, 4, 23)
    val redNumbers = listOf(17, 2)
    val totalCount = greenNumbers.count() + redNumbers.count()
    println(totalCount)
}
```

Check whether a requested protocol is supported

You have a set of protocols supported by your server. A user requests to use a particular protocol. Complete the program to check whether the requested protocol is supported or not (`isSupported` must be a Boolean value).

```
fun main() {
    val SUPPORTED = setOf("HTTP", "HTTPS", "FTP")
    val requested = "smtp"
    val isSupported = // Write your code here
    println("Support for $requested: $isSupported")
}
```

Hint

```
    Make sure that you check the requested protocol in upper case. You can use the <code>.up
percase()</code>
```

function to help you with this.

```
fun main() {
    val SUPPORTED = setOf("HTTP", "HTTPS", "FTP")
    val requested = "smtp"
    val isSupported = requested.uppercase() in SUPPORTED
    println("Support for $requested: $isSupported")
}
```

Spell out a number using a map

Define a map that relates integer numbers from 1 to 3 to their corresponding spelling. Use this map to spell the given number.

```
fun main() {
    val number2word = // Write your code here
    val n = 2
    println("$n is spelled as '${<Write your code here >}'")
}
```

```
fun main() {
    val number2word = mapOf(1 to "one", 2 to "two", 3 to "three")
    val n = 2
    println("$n is spelled as '${number2word[n]}'")
}
```

```
<a as="button" href="kotlin-tour-basic-types.md" mode="outline" icon="arrow-left" icon-posit
ion="left">Previous step</a>



<a as="button" href="kotlin-tour-control-flow.md" mode="classic" icon="arrow-right" icon-pos
ition="right">Next step</a>
```

Control flow

Like other programming languages, Kotlin is capable of making decisions based on whether a piece of code is evaluated to be true. Such pieces of code are called **conditional expressions**. Kotlin is also able to create and iterate through loops.

Conditional expressions

Kotlin provides `if` and `when` for checking conditional expressions.

> **Note:** If you have to choose between `if` and `when`, we recommend using `when` because it:
>
> - Makes your code easier to read.
> - Makes it easier to add another branch.
> - Leads to fewer mistakes in your code.

If

To use `if`, add the conditional expression within parentheses `()` and the action to take if the result is true within curly braces `{}`:

```
fun main() {
    val d: Int
    val check = true

    if (check) {
        d = 1
    } else {
        d = 2
    }

    println(d)
    // 1
}
```

There is no ternary operator `condition ? then : else` in Kotlin. Instead, `if` can be used as an expression. If there is only one line of code per action, the curly braces `{}` are optional:

```
fun main() {
    val a = 1
    val b = 2

    println(if (a > b) a else b) // Returns a value: 2
}
```

When

Use `when` when you have a conditional expression with multiple branches.

To use `when`:

- Place the value you want to evaluate within parentheses `()`.
- Place the branches within curly braces `{}`.
- Use `->` in each branch to separate each check from the action to take if the check is successful.

`when` can be used either as a statement or as an expression. A **statement** doesn't return anything but performs actions instead.

Here is an example of using `when` as a statement:

```
fun main() {
    val obj = "Hello"

    when (obj) {
        // Checks whether obj equals to "1"
        "1" -> println("One")
        // Checks whether obj equals to "Hello"
        "Hello" -> println("Greeting")
        // Default statement
        else -> println("Unknown")
    }
    // Greeting
}
```

> **Note:** Note that all branch conditions are checked sequentially until one of them is satisfied. So only the first suitable branch is executed.

An **expression** returns a value that can be used later in your code.

Here is an example of using `when` as an expression. The `when` expression is assigned immediately to a variable which is later used with the `println()` function:

```
fun main() {
    val obj = "Hello"

    val result = when (obj) {
        // If obj equals "1", sets result to "one"
        "1" -> "One"
        // If obj equals "Hello", sets result to "Greeting"
        "Hello" -> "Greeting"
        // Sets result to "Unknown" if no previous condition is satisfied
        else -> "Unknown"
    }
    println(result)
```

```
    // Greeting
}
```

The examples of `when` that you've seen so far both had a subject: `obj`. But `when` can also be used without a subject.

This example uses a `when` expression **without** a subject to check a chain of Boolean expressions:

```
fun main() {
    val trafficLightState = "Red" // This can be "Green", "Yellow", or "Red"

    val trafficAction = when {
        trafficLightState == "Green" -> "Go"
        trafficLightState == "Yellow" -> "Slow down"
        trafficLightState == "Red" -> "Stop"
        else -> "Malfunction"
    }

    println(trafficAction)
    // Stop
}
```

However, you can have the same code but with `trafficLightState` as the subject:

```
fun main() {
    val trafficLightState = "Red" // This can be "Green", "Yellow", or "Red"

    val trafficAction = when (trafficLightState) {
        "Green" -> "Go"
        "Yellow" -> "Slow down"
        "Red" -> "Stop"
        else -> "Malfunction"
    }

    println(trafficAction)
    // Stop
}
```

Using `when` with a subject makes your code easier to read and maintain. When you use a subject with a `when` expression, it also helps Kotlin check that all possible cases are covered. Otherwise, if you don't use a subject with a `when` expression, you need to provide an else branch.

Practice: Conditional expressions

Check whether two dice rolls match

Create a simple game where you win if throwing two dice results in the same number. Use `if` to print `You win :)` if the dice match or `You lose :(` otherwise.

Tip: In this exercise, you import a package so that you can use the `Random.nextInt()` function to give you a random `Int`. For more information about importing packages, see Packages and imports.

Hint

```
    Use the equality operator (<code>==</code>) to compare the dice results.
```

```
import kotlin.random.Random

fun main() {
    val firstResult = Random.nextInt(6)
    val secondResult = Random.nextInt(6)
    // Write your code here
}
```

```
import kotlin.random.Random

fun main() {
    val firstResult = Random.nextInt(6)
    val secondResult = Random.nextInt(6)
    if (firstResult == secondResult)
        println("You win :)")
    else
        println("You lose :(")
}
```

Print the action for a game console button

Using a `when` expression, update the following program so that it prints the corresponding actions when you input the names of game console buttons.

Button	Action
A	Yes
B	No
X	Menu
Y	Nothing
Other	There is no such button

```
fun main() {
    val button = "A"

    println(
        // Write your code here
    )
}
```

```
fun main() {
    val button = "A"

    println(
        when (button) {
            "A" -> "Yes"
            "B" -> "No"
            "X" -> "Menu"
            "Y" -> "Nothing"
            else -> "There is no such button"
        }
    )
}
```

Ranges

Before talking about loops, it's useful to know how to construct ranges for loops to iterate over.

The most common way to create a range in Kotlin is to use the `..` operator. For example, `1..4` is equivalent to `1, 2, 3, 4`.

To declare a range that doesn't include the end value, use the `..<` operator. For example, `1..<4` is equivalent to `1, 2, 3`.

To declare a range in reverse order, use `downTo`. For example, `4 downTo 1` is equivalent to `4, 3, 2, 1`.

To declare a range that increments in a step that isn't 1, use `step` and your desired increment value. For example, `1..5 step 2` is equivalent to `1, 3, 5`.

You can also do the same with `Char` ranges:

- `'a'..'d'` is equivalent to `'a', 'b', 'c', 'd'`
- `'z' downTo 's' step 2` is equivalent to `'z', 'x', 'v', 't'`

Loops

The two most common loop structures in programming are `for` and `while`. Use `for` to iterate over a range of values and perform an action. Use `while` to continue an action until a particular condition is satisfied.

For

Using your new knowledge of ranges, you can create a `for` loop that iterates over numbers 1 to 5 and prints the number each time.

Place the iterator and range within parentheses `()` with keyword `in`. Add the action you want to complete within curly braces `{}`:

```
fun main() {
    for (number in 1..5) {
        // number is the iterator and 1..5 is the range
        print(number)
    }
    // 12345
}
```

Collections can also be iterated over by loops:

```
fun main() {
    val cakes = listOf("carrot", "cheese", "chocolate")

    for (cake in cakes) {
        println("Yummy, it's a $cake cake!")
    }
    // Yummy, it's a carrot cake!
    // Yummy, it's a cheese cake!
    // Yummy, it's a chocolate cake!
}
```

While

`while` can be used in two ways:

- To execute a code block while a conditional expression is true. (`while`)
- To execute the code block first and then check the conditional expression. (`do-while`)

In the first use case (`while`):

- Declare the conditional expression for your while loop to continue within parentheses `()`.
- Add the action you want to complete within curly braces `{}`.

  Tip: The following examples use the increment operator `++` to increment the value of the `cakesEaten` variable.

```
fun main() {
    var cakesEaten = 0
    while (cakesEaten < 3) {
        println("Eat a cake")
        cakesEaten++
    }
    // Eat a cake
    // Eat a cake
    // Eat a cake
}
```

In the second use case (`do-while`):

- Declare the conditional expression for your while loop to continue within parentheses `()`.
- Define the action you want to complete within curly braces `{}` with the keyword `do`.

```
fun main() {
    var cakesEaten = 0
    var cakesBaked = 0
    while (cakesEaten < 3) {
        println("Eat a cake")
        cakesEaten++
    }
    do {
        println("Bake a cake")
        cakesBaked++
    } while (cakesBaked < cakesEaten)
    // Eat a cake
    // Eat a cake
    // Eat a cake
    // Bake a cake
    // Bake a cake
    // Bake a cake
}
```

For more information and examples of conditional expressions and loops, see Conditions and loops.

Now that you know the fundamentals of Kotlin control flow, it's time to learn how to write your own functions.

Practice: Loops

Count pizza slices using while and do-while loops

You have a program that counts pizza slices until there's a whole pizza with 8 slices. Refactor this program in two ways:

- Use a `while` loop.
- Use a `do-while` loop.

```
fun main() {
    var pizzaSlices = 0
    // Start refactoring here
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
```

```
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    // End refactoring here
    println("There are $pizzaSlices slices of pizza. Hooray! We have a whole pizza! :D")
}
```

```
fun main() {
    var pizzaSlices = 0
    while ( pizzaSlices < 7 ) {
        pizzaSlices++
        println("There's only $pizzaSlices slice/s of pizza :(")
    }
    pizzaSlices++
    println("There are $pizzaSlices slices of pizza. Hooray! We have a whole pizza! :D")
}
```

```
fun main() {
    var pizzaSlices = 0
    pizzaSlices++
    do {
        println("There's only $pizzaSlices slice/s of pizza :(")
        pizzaSlices++
    } while ( pizzaSlices < 8 )
    println("There are $pizzaSlices slices of pizza. Hooray! We have a whole pizza! :D")
}
```

Play the Fizz buzz game

Write a program that simulates the Fizz buzz game. Your task is to print numbers from 1 to 100 incrementally, replacing any number divisible by three with the word "fizz", and any number divisible by five with the word "buzz". Any number divisible by both 3 and 5 must be replaced with the word "fizzbuzz".

Hint 1

```
    Use a <code>for</code> loop to count numbers and a <code>when</code> expression to decid
e what to print at each
    step.
```

Hint 2

```
    Use the modulo operator (<code>%</code>) to return the remainder of a number being divid
ed. Use the equality operator
    (<code>==</code>) to check if the remainder equals zero.
```

```
fun main() {
    // Write your code here
}
```

```
fun main() {
    for (number in 1..100) {
        println(
            when {
                number % 15 == 0 -> "fizzbuzz"
                number % 3 == 0 -> "fizz"
                number % 5 == 0 -> "buzz"
                else -> "$number"
            }
        )
    }
}
```

Print words that start with a given letter

You have a list of words. Use `for` and `if` to print only the words that start with the letter `l`.

Hint

```
    Use the  <code>.startsWith()</code>
     function for <code>String</code> type.
```

```
fun main() {
    val words = listOf("dinosaur", "limousine", "magazine", "language")
    // Write your code here
}
```

```
fun main() {
    val words = listOf("dinosaur", "limousine", "magazine", "language")
    for (w in words) {
        if (w.startsWith("l"))
            println(w)
    }
}
```

```
<a as="button" href="kotlin-tour-collections.md" mode="outline" icon="arrow-left" icon-posit
ion="left">Previous step</a>



<a as="button" href="kotlin-tour-functions.md" mode="classic" icon="arrow-right" icon-positi
on="right">Next step</a>
```

Functions

You can declare your own functions in Kotlin using the `fun` keyword.

```
fun hello() {
    return println("Hello, world!")
}

fun main() {
    hello()
    // Hello, world!
}
```

In Kotlin:

- Function parameters are written within parentheses `()`.
- Each parameter must have a type, and multiple parameters must be separated by commas `,`.
- The return type is written after the function's parentheses `()`, separated by a colon `:`.
- The body of a function is written within curly braces `{}`.
- The `return` keyword is used to exit or return something from a function.

  Note: If a function doesn't return anything useful, the return type and `return` keyword can be omitted. Learn more about this in Functions without return.

In the following example:

- `x` and `y` are function parameters.
- `x` and `y` have type `Int`.
- The function's return type is `Int`.
- The function returns a sum of `x` and `y` when called.

```
fun sum(x: Int, y: Int): Int {
    return x + y
}

fun main() {
    println(sum(1, 2))
    // 3
}
```

Note: We recommend in our coding conventions that you name functions starting with a lowercase letter and use camel case with no underscores.

Named arguments

For concise code, when calling your function, you don't have to include parameter names. However, including parameter names does make your code easier to read. This is called using **named arguments**. If you do include parameter names, then you can write the parameters in any order.

> **Tip:** In the following example, string templates (`$`) are used to access the parameter values, convert them to `String` type, and then concatenate them into a string for printing.

```
fun printMessageWithPrefix(message: String, prefix: String) {
    println("[$prefix] $message")
}

fun main() {
    // Uses named arguments with swapped parameter order
    printMessageWithPrefix(prefix = "Log", message = "Hello")
    // [Log] Hello
}
```

Default parameter values

You can define default values for your function parameters. Any parameter with a default value can be omitted when calling your function. To declare a default value, use the assignment operator `=` after the type:

```
fun printMessageWithPrefix(message: String, prefix: String = "Info") {
    println("[$prefix] $message")
}

fun main() {
    // Function called with both parameters
    printMessageWithPrefix("Hello", "Log")
    // [Log] Hello

    // Function called only with message parameter
    printMessageWithPrefix("Hello")
    // [Info] Hello

    printMessageWithPrefix(prefix = "Log", message = "Hello")
    // [Log] Hello
}
```

> **Note:** You can skip specific parameters with default values, rather than omitting them all. However, after the first skipped parameter, you must name all subsequent parameters.

Functions without return

If your function doesn't return a useful value, then its return type is `Unit`. `Unit` is a type with only one value – `Unit`. You don't have to declare that `Unit` is returned explicitly in your function body. This means that you don't have to use the `return` keyword or declare a return type:

```
fun printMessage(message: String) {
    println(message)
    // `return Unit` or `return` is optional
}

fun main() {
    printMessage("Hello")
    // Hello
}
```

Single-expression functions

To make your code more concise, you can use single-expression functions. For example, the `sum()` function can be shortened:

```
fun sum(x: Int, y: Int): Int {
    return x + y
}

fun main() {
    println(sum(1, 2))
    // 3
}
```

You can remove the curly braces `{}` and declare the function body using the assignment operator `=`. When you use the assignment operator `=`, Kotlin uses type inference, so you can also omit the return type. The `sum()` function then becomes one line:

```
fun sum(x: Int, y: Int) = x + y

fun main() {
    println(sum(1, 2))
    // 3
}
```

However, if you want your code to be quickly understood by other developers, it's a good idea to explicitly define the return type even when using the assignment operator `=`.

> **Note:** If you use `{}` curly braces to declare your function body, you must declare the return type unless it is the `Unit` type.

Early returns in functions

To stop the code in your function from being processed further than a certain point, use the `return` keyword. This example uses `if` to return from a function early if the conditional expression is found to be true:

```
// A list of registered usernames
val registeredUsernames = mutableListOf("john_doe", "jane_smith")

// A list of registered emails
val registeredEmails = mutableListOf("john@example.com", "jane@example.com")

fun registerUser(username: String, email: String): String {
    // Early return if the username is already taken
    if (username in registeredUsernames) {
        return "Username already taken. Please choose a different username."
    }

    // Early return if the email is already registered
    if (email in registeredEmails) {
        return "Email already registered. Please use a different email."
    }

    // Proceed with the registration if the username and email are not taken
    registeredUsernames.add(username)
    registeredEmails.add(email)

    return "User registered successfully: $username"
}

fun main() {
    println(registerUser("john_doe", "newjohn@example.com"))
    // Username already taken. Please choose a different username.
    println(registerUser("new_user", "newuser@example.com"))
    // User registered successfully: new_user
}
```

Practice: Functions

Calculate the area of a circle

Write a function called `circleArea` that takes the radius of a circle in integer format as a parameter and outputs the area of that circle.

> **Tip:** In this exercise, you import a package so that you can access the value of via `PI`.
> For more information about importing packages, see Packages and imports.

Hint

```
    The formula for calculating the area of a circle is <math>πr^2</math>, where <math>r</ma
th> is the radius.
```

```
import kotlin.math.PI

// Write your code here

fun main() {
    println(circleArea(2))
}
```

```
import kotlin.math.PI

fun circleArea(radius: Int): Double {
    return PI * radius * radius
}

fun main() {
    println(circleArea(2)) // 12.566370614359172
}
```

Rewrite a function as a single expression

Rewrite the `circleArea` function from the previous exercise as a single-expression function.

```
import kotlin.math.PI

// Write your code here

fun main() {
    println(circleArea(2))
}
```

```
import kotlin.math.PI

fun circleArea(radius: Int): Double = PI * radius * radius

fun main() {
    println(circleArea(2)) // 12.566370614359172
}
```

Refactor a function using default parameters and named arguments

You have a function that translates a time interval given in hours, minutes, and seconds into seconds. In most cases, you need to pass only one or two function parameters while the rest are equal to 0. Improve the function and the code that calls it by using default parameter values and named arguments so that the code is easier to read.

```
fun intervalInSeconds(hours: Int, minutes: Int, seconds: Int) =
    ((hours * 60) + minutes) * 60 + seconds
```

```
fun main() {
    println(intervalInSeconds(1, 20, 15))
    println(intervalInSeconds(0, 1, 25))
    println(intervalInSeconds(2, 0, 0))
    println(intervalInSeconds(0, 10, 0))
    println(intervalInSeconds(1, 0, 1))
}
```

```
fun intervalInSeconds(hours: Int = 0, minutes: Int = 0, seconds: Int = 0) =
    ((hours * 60) + minutes) * 60 + seconds

fun main() {
    println(intervalInSeconds(1, 20, 15))
    println(intervalInSeconds(minutes = 1, seconds = 25))
    println(intervalInSeconds(hours = 2))
    println(intervalInSeconds(minutes = 10))
    println(intervalInSeconds(hours = 1, seconds = 1))
}
```

Lambda expressions

Kotlin allows you to write even more concise code for functions by using lambda expressions.

For example, the following `uppercaseString()` function:

```
fun uppercaseString(text: String): String {
    return text.uppercase()
}
fun main() {
    println(uppercaseString("hello"))
    // HELLO
}
```

Can also be written as a lambda expression:

```
fun main() {
    val upperCaseString = { text: String -> text.uppercase() }
    println(upperCaseString("hello"))
    // HELLO
}
```

Lambda expressions can be hard to understand at first glance, so let's break it down. Lambda expressions are written within curly braces `{}`.

Within the lambda expression, you write:

- The parameters followed by an `->`.
- The function body after the `->`.

In the previous example:

- `text` is a function parameter.
- `text` has type `String`.
- The function returns the result of the `.uppercase()` function called on `text`.
- The entire lambda expression is assigned to the `upperCaseString` variable with the assignment operator `=`.
- The lambda expression is called by using the variable `upperCaseString` like a function and the string `"hello"` as a parameter.
- The `println()` function prints the result.

  Note: If you declare a lambda without parameters, then there is no need to use `->`. For example:

  ```
  { println("Log message") }
  ```

Lambda expressions can be used in a number of ways. You can:

- Pass a lambda expression as a parameter to another function
- Return a lambda expression from a function
- Invoke a lambda expression on its own

Pass to another function

A great example of when it is useful to pass a lambda expression to a function, is using the `.filter()` function on collections:

```
fun main() {
    val numbers = listOf(1, -2, 3, -4, 5, -6)

    val positives = numbers.filter ({ x -> x > 0 })

    val isNegative = { x: Int -> x < 0 }
    val negatives = numbers.filter(isNegative)

    println(positives)
    // [1, 3, 5]
    println(negatives)
    // [-2, -4, -6]
}
```

The `.filter()` function accepts a lambda expression as a predicate and applies it to each element of the list. The function keeps an element only if the predicate returns `true`:

- `{ x -> x > 0 }` returns `true` if the element is positive.
- `{ x -> x < 0 }` returns `true` if the element is negative.

This example demonstrates two ways of passing a lambda expression to a function:

- For positive numbers, the example adds the lambda expression directly in the `.filter()` function.
- For negative numbers, the example assigns the lambda expression to the `isNegative` variable. Then the `isNegative` variable is used as a function parameter in the `.filter()` function. In this case, you have to specify the type of function parameters (`x`) in the lambda expression.

  Note: If a lambda expression is the only function parameter, you can drop the function parentheses `()`:

  ```
  val positives = numbers.filter { x -> x > 0 }
  ```

  This is an example of a trailing lambda, which is discussed in more detail at the end of this chapter.

Another good example is using the `.map()` function to transform items in a collection:

```
fun main() {
    val numbers = listOf(1, -2, 3, -4, 5, -6)
    val doubled = numbers.map { x -> x * 2 }

    val isTripled = { x: Int -> x * 3 }
    val tripled = numbers.map(isTripled)

    println(doubled)
    // [2, -4, 6, -8, 10, -12]
    println(tripled)
    // [3, -6, 9, -12, 15, -18]
}
```

The `.map()` function accepts a lambda expression as a transform function:

- `{ x -> x * 2 }` takes each element of the list and returns that element multiplied by 2.
- `{ x -> x * 3 }` takes each element of the list and returns that element multiplied by 3.

Function types

Before you can return a lambda expression from a function, you first need to understand **function types**.

You have already learned about basic types but functions themselves also have a type. Kotlin's type inference can infer a function's type from the parameter type. But there may be times when you need to explicitly specify the function type. The compiler needs the function type so that it knows what is and isn't allowed for that function.

The syntax for a function type has:

- Each parameter's type written within parentheses `()` and separated by commas `,`.

- The return type written after `->`.

For example: `(String) -> String` or `(Int, Int) -> Int`.

This is what a lambda expression looks like if a function type for `upperCaseString()` is defined:

```
val upperCaseString: (String) -> String = { text -> text.uppercase() }

fun main() {
    println(upperCaseString("hello"))
    // HELLO
}
```

If your lambda expression has no parameters, then the parentheses `()` are left empty. For example: `() -> Unit`

> **Note:** You must declare parameter and return types either in the lambda expression or as a function type. Otherwise, the compiler won't be able to know what type your lambda expression is.
>
> For example, the following won't work:
>
> `val upperCaseString = { str -> str.uppercase() }`

Return from a function

Lambda expressions can be returned from a function. So that the compiler understands what type the lambda expression returned is, you must declare a function type.

In the following example, the `toSeconds()` function has function type `(Int) -> Int` because it always returns a lambda expression that takes a parameter of type `Int` and returns an `Int` value.

This example uses a `when` expression to determine which lambda expression is returned when `toSeconds()` is called:

```
fun toSeconds(time: String): (Int) -> Int = when (time) {
    "hour" -> { value -> value * 60 * 60 }
    "minute" -> { value -> value * 60 }
    "second" -> { value -> value }
    else -> { value -> value }
}

fun main() {
    val timesInMinutes = listOf(2, 10, 15, 1)
    val min2sec = toSeconds("minute")
    val totalTimeInSeconds = timesInMinutes.map(min2sec).sum()
    println("Total time is $totalTimeInSeconds secs")
    // Total time is 1680 secs
}
```

Invoke separately

Lambda expressions can be invoked on their own by adding parentheses `()` after the curly braces `{}` and including any parameters within the parentheses:

```
fun main() {
    println({ text: String -> text.uppercase() }("hello"))
    // HELLO
}
```

Trailing lambdas

As you have already seen, if a lambda expression is the only function parameter, you can drop the function parentheses `()`. If a lambda expression is passed as the last parameter of a function, then the expression can be written outside the function parentheses `()`. In both cases, this syntax is called a **trailing lambda**.

For example, the `.fold()` function accepts an initial value and an operation:

```
fun main() {
    // The initial value is zero.
    // The operation sums the initial value with every item in the list cumulatively.
    println(listOf(1, 2, 3).fold(0, { x, item -> x + item })) // 6

    // Alternatively, in the form of a trailing lambda
    println(listOf(1, 2, 3).fold(0) { x, item -> x + item })  // 6
}
```

For more information on lambda expressions, see Lambda expressions and anonymous functions.

The next step in our tour is to learn about classes in Kotlin.

Practice: Lambda expressions

Build a list of URLs using a lambda expression

You have a list of actions supported by a web service, a common prefix for all requests, and an ID of a particular resource. To request an action `title` over the resource with ID: 5, you need to create the following URL: `https://example.com/book-info/5/title`. Use a lambda expression to create a list of URLs from the list of actions.

```
fun main() {
    val actions = listOf("title", "year", "author")
    val prefix = "https://example.com/book-info"
    val id = 5
    val urls = // Write your code here
    println(urls)
}
```

```
fun main() {
    val actions = listOf("title", "year", "author")
    val prefix = "https://example.com/book-info"
    val id = 5
    val urls = actions.map { action -> "$prefix/$id/$action" }
    println(urls)
}
```

Repeat an action multiple times

Write a function that takes an `Int` value and an action (a function with type `() -> Unit`) which then repeats the action the given number of times. Then use this function to print “Hello” 5 times.

```
fun repeatN(n: Int, action: () -> Unit) {
    // Write your code here
}

fun main() {
    // Write your code here
}
```

```
fun repeatN(n: Int, action: () -> Unit) {
    for (i in 1..n) {
        action()
    }
}

fun main() {
    repeatN(5) {
        println("Hello")
    }
}
```

```
<a as="button" href="kotlin-tour-control-flow.md" mode="outline" icon="arrow-left" icon-posi
tion="left">Previous step</a>



<a as="button" href="kotlin-tour-classes.md" mode="classic" icon="arrow-right" icon-position
="right">Next step</a>
```

Classes

Kotlin supports object-oriented programming with classes and objects. Objects are useful for storing data in your program. Classes allow you to declare a set of characteristics for an object. When you create objects from a class, you can save time and effort because you don't have to declare these characteristics every time.

To declare a class, use the `class` keyword:

```
class Customer
```

Properties

Characteristics of a class's object can be declared in properties. You can declare properties for a class:

- Within parentheses `()` after the class name.

```
class Contact(val id: Int, var email: String)
```

- Within the class body defined by curly braces `{}`.

```
class Contact(val id: Int, var email: String) {
    val category: String = ""
}
```

We recommend that you declare properties as read-only (`val`) unless they need to be changed after an instance of the class is created.

You can declare properties without `val` or `var` within parentheses but these properties are not accessible after an instance has been created.

> **Note:** * The content contained within parentheses `()` is called the **class header**.
>
> - You can use a trailing comma when declaring class properties.

Just like with function parameters, class properties can have default values:

```
class Contact(val id: Int, var email: String = "example@gmail.com") {
    val category: String = "work"
}
```

Create instance

To create an object from a class, you declare a class **instance** using a **constructor**.

By default, Kotlin automatically creates a constructor with the parameters declared in the class header.

For example:

```
class Contact(val id: Int, var email: String)

fun main() {
    val contact = Contact(1, "mary@gmail.com")
}
```

In the example:

- `Contact` is a class.
- `contact` is an instance of the `Contact` class.
- `id` and `email` are properties.
- `id` and `email` are used with the default constructor to create `contact`.

Kotlin classes can have many constructors, including ones that you define yourself. To learn more about how to declare multiple constructors, see Constructors.

Access properties

To access a property of an instance, write the name of the property after the instance name appended with a period `.`:

```
class Contact(val id: Int, var email: String)

fun main() {
    val contact = Contact(1, "mary@gmail.com")

    // Prints the value of the property: email
    println(contact.email)
    // mary@gmail.com

    // Updates the value of the property: email
    contact.email = "jane@gmail.com"

    // Prints the new value of the property: email
    println(contact.email)
    // jane@gmail.com
}
```

Tip: To concatenate the value of a property as part of a string, you can use string templates (`$`). For example:

```
println("Their email address is: ${contact.email}")
```

Member functions

In addition to declaring properties as part of an object's characteristics, you can also define an object's behavior with member functions.

In Kotlin, member functions must be declared within the class body. To call a member function on an instance, write the function name after the instance name appended with a period `.`. For example:

```
class Contact(val id: Int, var email: String) {
    fun printId() {
        println(id)
    }
}

fun main() {
    val contact = Contact(1, "mary@gmail.com")
    // Calls member function printId()
    contact.printId()
    // 1
}
```

Data classes

Kotlin has **data classes** which are particularly useful for storing data. Data classes have the same functionality as classes, but they come automatically with additional member functions. These member functions allow you to easily print the instance to readable output, compare instances of a class, copy instances, and more. As these functions are automatically available, you don't have to spend time writing the same boilerplate code for each of your classes.

To declare a data class, use the keyword `data`:

```
data class User(val name: String, val id: Int)
```

The Kotlin compiler only uses the properties defined inside the primary constructor when generating member functions. If you declare properties in the data class body, they aren't included in the output of the generated functions.

The most useful predefined member functions of data classes are:

Function	Description
`toString()`	Prints a readable string of the class instance and its properties.
`equals()` or `==`	Compares instances of a class.
`copy()`	Creates a class instance by copying another, potentially with some different properties.

See the following sections for examples of how to use each function:

- Print as string
- Compare instances
- Copy instance

Print as string

To print a readable string of a class instance, you can explicitly call the `toString()` function, or use print functions (`println()` and `print()`) which automatically call `toString()` for you:

```
data class User(val name: String, val id: Int)

fun main() {
    val user = User("Alex", 1)

    // Automatically uses toString() function so that output is easy to read
    println(user)
    // User(name=Alex, id=1)
}
```

This is particularly useful when debugging or creating logs.

Compare instances

To compare data class instances, use the equality operator `==`:

```
data class User(val name: String, val id: Int)

fun main() {
    val user = User("Alex", 1)
    val secondUser = User("Alex", 1)
    val thirdUser = User("Max", 2)

    // Compares user to second user
    println("user == secondUser: ${user == secondUser}")
    // user == secondUser: true

    // Compares user to third user
    println("user == thirdUser: ${user == thirdUser}")
    // user == thirdUser: false
}
```

Copy instance

To create an exact copy of a data class instance, call the `copy()` function on the instance.

To create a copy of a data class instance **and** change some properties, call the `copy()` function on the instance **and** add replacement values for properties as function parameters.

For example:

```
data class User(val name: String, val id: Int)

fun main() {
    val user = User("Alex", 1)

    // Creates an exact copy of user
    println(user.copy())
    // User(name=Alex, id=1)

    // Creates a copy of user with name: "Max"
    println(user.copy("Max"))
    // User(name=Max, id=1)

    // Creates a copy of user with id: 3
    println(user.copy(id = 3))
    // User(name=Alex, id=3)
}
```

Creating a copy of an instance is safer than modifying the original instance because any code that relies on the original instance isn't affected by the copy and what you do with it.

For more information about data classes, see Data classes.

The last chapter of this tour is about Kotlin's null safety.

Practice

Declare a data class

Define a data class `Employee` with two properties: one for a name, and another for a salary. Make sure that the property for salary is mutable, otherwise you won't get a salary boost at the end of the year! The main function demonstrates how you can use this data class.

```
// Write your code here

fun main() {
    val emp = Employee("Mary", 20)
    println(emp)
    emp.salary += 10
    println(emp)
}
```

```
data class Employee(val name: String, var salary: Int)

fun main() {
    val emp = Employee("Mary", 20)
    println(emp)
    emp.salary += 10
    println(emp)
}
```

Declare nested data classes

Declare the additional data classes that are needed for this code to compile.

```
data class Person(val name: Name, val address: Address, val ownsAPet: Boolean = true)
// Write your code here
// data class Name(...)

fun main() {
    val person = Person(
        Name("John", "Smith"),
        Address("123 Fake Street", City("Springfield", "US")),
        ownsAPet = false
    )
}
```

```
data class Person(val name: Name, val address: Address, val ownsAPet: Boolean = true)
data class Name(val first: String, val last: String)
data class Address(val street: String, val city: City)
data class City(val name: String, val countryCode: String)

fun main() {
    val person = Person(
        Name("John", "Smith"),
        Address("123 Fake Street", City("Springfield", "US")),
        ownsAPet = false
    )
}
```

Generate random employees with a class

To test your code, you need a generator that can create random employees. Define a `RandomEmployeeGenerator` class with a fixed list of potential names (inside the class body). Configure the class with a minimum and maximum salary (inside the class header). In the class body, define the `generateEmployee()` function. Once again, the main function demonstrates how you can use this class.

> **Tip:** In this exercise, you import a package so that you can use the `Random.nextInt()` function. For more information about importing packages, see Packages and imports.

Hint 1

```
    Lists have an extension function called <code>.random()</code>
    that returns a random item within a list.
```

Hint 2

```
    <code>Random.nextInt(from = ..., until = ...)</code> gives you a random <code>Int</code>
number within specified limits.
```

```
import kotlin.random.Random

data class Employee(val name: String, var salary: Int)

// Write your code here

fun main() {
    val empGen = RandomEmployeeGenerator(10, 30)
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    empGen.minSalary = 50
    empGen.maxSalary = 100
    println(empGen.generateEmployee())
}
```

```
import kotlin.random.Random

data class Employee(val name: String, var salary: Int)

class RandomEmployeeGenerator(var minSalary: Int, var maxSalary: Int) {
    val names = listOf("John", "Mary", "Ann", "Paul", "Jack", "Elizabeth")
    fun generateEmployee() =
        Employee(names.random(),
            Random.nextInt(from = minSalary, until = maxSalary))
}

fun main() {
    val empGen = RandomEmployeeGenerator(10, 30)
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    println(empGen.generateEmployee())
    empGen.minSalary = 50
    empGen.maxSalary = 100
    println(empGen.generateEmployee())
}
```

```
<a as="button" href="kotlin-tour-functions.md" mode="outline" icon="arrow-left" icon-positio
n="left">Previous step</a>
```

```
<a as="button" href="kotlin-tour-null-safety.md" mode="classic" icon="arrow-right" icon-posi
tion="right">Next step</a>
```

Null safety

In Kotlin, it's possible to have a `null` value. Kotlin uses `null` values when something is missing or not yet set. You've already seen an example of Kotlin returning a `null` value in the Collections chapter when you tried to access a key-value pair with a key that doesn't exist in the map. Although it's useful to use `null` values in this way, you might run into problems if your code isn't prepared to handle them.

To help prevent issues with `null` values in your programs, Kotlin has null safety in place. Null safety detects potential problems with `null` values at compile time, rather than at run time.

Null safety is a combination of features that allow you to:

- Explicitly declare when `null` values are allowed in your program.
- Check for `null` values.
- Use safe calls to properties or functions that may contain `null` values.
- Declare actions to take if `null` values are detected.

Nullable types

Kotlin supports nullable types which allows the possibility for the declared type to have `null` values. By default, a type is **not** allowed to accept `null` values. Nullable types are declared by explicitly adding `?` after the type declaration.

For example:

```
fun main() {
    // neverNull has String type
    var neverNull: String = "This can't be null"

    // Throws a compiler error
    neverNull = null

    // nullable has nullable String type
    var nullable: String? = "You can keep a null here"

    // This is OK
    nullable = null

    // By default, null values aren't accepted
    var inferredNonNull = "The compiler assumes non-nullable"

    // Throws a compiler error
    inferredNonNull = null

    // notNull doesn't accept null values
    fun strLength(notNull: String): Int {
```

```
        return notNull.length
    }

    println(strLength(neverNull)) // 18
    println(strLength(nullable))  // Throws a compiler error
}
```

> **Tip:** `length` is a property of the String class that contains the number of characters within a string.

Check for null values

You can check for the presence of `null` values within conditional expressions. In the following example, the `describeString()` function has an `if` statement that checks whether `maybeString` is **not** `null` and if its `length` is greater than zero:

```
fun describeString(maybeString: String?): String {
    if (maybeString != null && maybeString.length > 0) {
        return "String of length ${maybeString.length}"
    } else {
        return "Empty or null string"
    }
}

fun main() {
    val nullString: String? = null
    println(describeString(nullString))
    // Empty or null string
}
```

Use safe calls

To safely access properties of an object that might contain a `null` value, use the safe call operator `?.`. The safe call operator returns `null` if either the object or one of its accessed properties is `null`. This is useful if you want to avoid the presence of `null` values triggering errors in your code.

In the following example, the `lengthString()` function uses a safe call to return either the length of the string or `null`:

```
fun lengthString(maybeString: String?): Int? = maybeString?.length

fun main() {
    val nullString: String? = null
    println(lengthString(nullString))
    // null
}
```

Tip: Safe calls can be chained so that if any property of an object contains a `null` value, then `null` is returned without an error being thrown. For example:

```
person.company?.address?.country
```

The safe call operator can also be used to safely call an extension or member function. In this case, a null check is performed before the function is called. If the check detects a `null` value, then the call is skipped and `null` is returned.

In the following example, `nullString` is `null` so the invocation of `.uppercase()` is skipped and `null` is returned:

```
fun main() {
    val nullString: String? = null
    println(nullString?.uppercase())
    // null
}
```

Use Elvis operator

You can provide a default value to return if a `null` value is detected by using the **Elvis operator** `?:`.

Write on the left-hand side of the Elvis operator what should be checked for a `null` value. Write on the right-hand side of the Elvis operator what should be returned if a `null` value is detected.

In the following example, `nullString` is `null` so the safe call to access the `length` property returns a `null` value. As a result, the Elvis operator returns `0`:

```
fun main() {
    val nullString: String? = null
    println(nullString?.length ?: 0)
    // 0
}
```

For more information about null safety in Kotlin, see Null safety.

Practice

Calculate an employee's salary

You have the `employeeById` function that gives you access to a database of employees of a company. Unfortunately, this function returns a value of the `Employee?` type, so the result can be `null`. Your goal is to write a function that returns the salary of an employee when their `id` is provided, or `0` if the employee is missing from the database.

```
data class Employee (val name: String, var salary: Int)

fun employeeById(id: Int) = when(id) {
    1 -> Employee("Mary", 20)
    2 -> null
    3 -> Employee("John", 21)
    4 -> Employee("Ann", 23)
    else -> null
}

fun salaryById(id: Int) = // Write your code here

fun main() {
    println((1..5).sumOf { id -> salaryById(id) })
}
```

```
data class Employee (val name: String, var salary: Int)

fun employeeById(id: Int) = when(id) {
    1 -> Employee("Mary", 20)
    2 -> null
    3 -> Employee("John", 21)
    4 -> Employee("Ann", 23)
    else -> null
}

fun salaryById(id: Int) = employeeById(id)?.salary ?: 0

fun main() {
    println((1..5).sumOf { id -> salaryById(id) })
}
```

What's next?

Congratulations! Now that you have completed the beginner tour, take your understanding of Kotlin to the next level with our intermediate tour:

```
<a as="button" href="kotlin-tour-classes.md" mode="outline" icon="arrow-left" icon-position
="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-extension-functions.md" mode="classic" icon="a
rrow-right" icon-position="right">Start intermediate Kotlin tour</a>
```

Extension functions

In this chapter, you'll explore special Kotlin functions that make your code more concise and readable. Learn how they can help you use efficient design patterns to take your projects to the next level.

Extension functions

In software development, you often need to modify a program's behavior without changing the original source code. For example, you might want to add extra functionality to a class from a third-party library.

You can do this by adding *extension functions* to extend a class. You call extension functions the same way you call member functions of a class, using a period `.`.

Before introducing the complete syntax for extension functions, you need to understand what a **receiver** is. The receiver is what the function is called on. In other words, the receiver is where or with whom the information is shared.

!An example of sender and receiver

In this example, the `main()` function calls the `.first()` function to return the first element in a list. The `.first()` function is called **on** the `readOnlyShapes` variable, so the `readOnlyShapes` variable is the receiver.

To create an extension function, write the name of the class that you want to extend followed by a `.` and the name of your function. Continue with the rest of the function declaration, including its parameters and return type.

For example:

```
fun String.bold(): String = "<b>$this</b>"

fun main() {
    // "hello" is the receiver
    println("hello".bold())
    // <b>hello</b>
}
```

In this example:

- `String` is the extended class.
- `bold` is the name of the extension function.
- The `.bold()` extension function's return type is `String`.

- `"hello"`, an instance of `String`, as the receiver.
- The receiver is accessed inside the body by the keyword: `this`.
- A string template (`$`) is used to access the value of `this`.
- The `.bold()` extension function takes a string and returns it in a `<b>` HTML element for bold text.

Extension-oriented design

You can define extension functions anywhere, which enables you to create extension-oriented designs. These designs separate core functionality from useful but non-essential features, making your code easier to read and maintain.

A good example is the `HttpClient` class from the Ktor library, which helps perform network requests. The core of its functionality is a single function `request()`, which takes all the information needed for an HTTP request:

```
class HttpClient {
    fun request(method: String, url: String, headers: Map<String, String>): HttpResponse {
        // Network code
    }
}
```

In practice, the most popular HTTP requests are GET or POST requests. It makes sense for the library to provide shorter names for these common use cases. However, these don't require writing new network code, only a specific request call. In other words, they are perfect candidates to be defined as separate `.get()` and `.post()` extension functions:

```
fun HttpClient.get(url: String): HttpResponse = request("GET", url, emptyMap())
fun HttpClient.post(url: String): HttpResponse = request("POST", url, emptyMap())
```

These `.get()` and `.post()` functions extend the `HttpClient` class. They can directly use the `request()` function from the `HttpClient` class because they're called on an instance of the `HttpClient` class as the receiver. You can use these extension functions to call the `request()` function with the appropriate HTTP method, which simplifies your code and makes it easier to understand:

```
class HttpClient {
    fun request(method: String, url: String, headers: Map<String, String>): HttpResponse {
        println("Requesting $method to $url with headers: $headers")
        return HttpResponse("Response from $url")
    }
}

fun HttpClient.get(url: String): HttpResponse = request("GET", url, emptyMap())

fun main() {
```

```
    val client = HttpClient()

    // Making a GET request using request() directly
    val getResponseWithMember = client.request("GET", "https://example.com", emptyMap())

    // Making a GET request using the get() extension function
    // The client instance is the receiver
    val getResponseWithExtension = client.get("https://example.com")
}
```

This extension-oriented approach is widely used in Kotlin's standard library and other libraries. For example, the `String` class has many extension functions to help you work with strings.

For more information about extension functions, see Extensions.

Practice

Check whether an integer is positive

Write an extension function called `isPositive` that takes an integer and checks whether it is positive.

```
fun Int.// Write your code here

fun main() {
    println(1.isPositive())
    // true
}
```

```
fun Int.isPositive(): Boolean = this > 0

fun main() {
    println(1.isPositive())
    // true
}
```

Convert a string to lowercase

Write an extension function called `toLowercaseString` that takes a string and returns a lowercase version.

Hint

```
    Use the  <code>.lowercase()</code>
     function for the <code>String</code> type.
```

```
fun // Write your code here

fun main() {
    println("Hello World!".toLowercaseString())
```

```
    // hello world!
}
```

```
fun String.toLowercaseString(): String = this.lowercase()

fun main() {
    println("Hello World!".toLowercaseString())
    // hello world!
}
```

```
<a as="button" href="kotlin-tour-null-safety.md" mode="outline" icon="arrow-left" icon-posit
ion="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-scope-functions.md" mode="classic" icon="arrow
-right" icon-position="right">Next step</a>
```

Scope functions

In this chapter, you'll build on your understanding of extension functions to learn how to use scope functions to write more idiomatic code.

Scope functions

In programming, a scope is the area in which your variable or object is recognized. The most commonly referred to scopes are the global scope and the local scope:

- **Global scope** – a variable or object that is accessible from anywhere in the program.
- **Local scope** – a variable or object that is only accessible within the block or function where it is defined.

In Kotlin, there are also scope functions that allow you to create a temporary scope around an object and execute some code.

Scope functions make your code more concise because you don't have to refer to the name of your object within the temporary scope. Depending on the scope function, you can access the object either by referencing it via the keyword `this` or using it as an argument via the keyword `it`.

Kotlin has five scope functions in total: `let`, `apply`, `run`, `also`, and `with`.

Each scope function takes a lambda expression and returns either the object or the result of the lambda expression. In this tour, we explain each scope function and how to use it.

> **Tip:** You can also watch the Back to the Stdlib: Making the Most of Kotlin's Standard Library talk on scope functions by Sebastian Aigner, Kotlin developer advocate.

Let

Use the `let` scope function when you want to perform null checks in your code and later perform further actions with the returned object.

Consider the example:

```
fun sendNotification(recipientAddress: String): String {
    println("Yo $recipientAddress!")
    return "Notification sent!"
}

fun getNextAddress(): String {
    return "sebastian@jetbrains.com"
}
```

```
fun main() {
    val address: String? = getNextAddress()
    sendNotification(address)
}
```

The example has two functions:

- `sendNotification()`, which has a function parameter `recipientAddress` and returns a string.
- `getNextAddress()`, which has no function parameters and returns a string.

The example creates a variable `address` that has a nullable `String` type. But this becomes a problem when you call the `sendNotification()` function because this function doesn't expect that `address` could be a `null` value. The compiler reports an error as a result:

```
Argument type mismatch: actual type is 'String?', but 'String' was expected.
```

From the beginner tour, you already know that you can perform a null check with an if condition or use the Elvis operator `?:`. But what if you want to use the returned object later in your code? You could achieve this with an if condition **and** an else branch:

```
fun sendNotification(recipientAddress: String): String {
    println("Yo $recipientAddress!")
    return "Notification sent!"
}

fun getNextAddress(): String {
    return "sebastian@jetbrains.com"
}

fun main() {
    val address: String? = getNextAddress()
    val confirm = if(address != null) {
        sendNotification(address)
    } else { null }
}
```

However, a more concise approach is to use the `let` scope function:

```
fun sendNotification(recipientAddress: String): String {
    println("Yo $recipientAddress!")
    return "Notification sent!"
}

fun getNextAddress(): String {
    return "sebastian@jetbrains.com"
}

fun main() {
    val address: String? = getNextAddress()
    val confirm = address?.let {
        sendNotification(it)
```

```
    }
}
```

The example:

- Creates variables called `address` and `confirm`.
- Uses a safe call for the `let` scope function on the `address` variable.
- Creates a temporary scope within the `let` scope function.
- Passes the `sendNotification()` function as a lambda expression into the `let` scope function.
- Refers to the `address` variable via `it`, using the temporary scope.
- Assigns the result to the `confirm` variable.

With this approach, your code can handle the `address` variable potentially being a `null` value, and you can use the `confirm` variable later in your code.

Apply

Use the `apply` scope function to initialize objects, like a class instance, at the time of creation rather than later on in your code. This approach makes your code easier to read and manage.

Consider the example:

```
class Client() {
    var token: String? = null
    fun connect() = println("connected!")
    fun authenticate() = println("authenticated!")
    fun getData() : String {
        println("getting data!")
        return "Mock data"
    }
}

val client = Client()

fun main() {
    client.token = "asdf"
    client.connect()
    // connected!
    client.authenticate()
    // authenticated!
    client.getData()
    // getting data!
}
```

The example has a `Client` class that contains one property called `token` and three member functions: `connect()`, `authenticate()`, and `getData()`.

The example creates `client` as an instance of the `Client` class before initializing its `token` property and calling its member functions in the `main()` function.

Although this example is compact, in the real world, it can be a while before you can configure and use the class instance (and its member functions) after you've created it. However, if you use the `apply` scope function you can create, configure and use member functions on your class instance all in the same place in your code:

```
class Client() {
    var token: String? = null
    fun connect() = println("connected!")
    fun authenticate() = println("authenticated!")
    fun getData() : String {
        println("getting data!")
        return "Mock data"
    }
}
val client = Client().apply {
    token = "asdf"
    connect()
    // connected!
    authenticate()
    // authenticated!
}

fun main() {
    client.getData()
    // getting data!
}
```

The example:

- Creates `client` as an instance of the `Client` class.
- Uses the `apply` scope function on the `client` instance.
- Creates a temporary scope within the `apply` scope function so that you don't have to explicitly refer to the `client` instance when accessing its properties or functions.
- Passes a lambda expression to the `apply` scope function that updates the `token` property and calls the `connect()` and `authenticate()` functions.
- Calls the `getData()` member function on the `client` instance in the `main()` function.

As you can see, this strategy is convenient when you are working with large pieces of code.

Run

Similar to `apply`, you can use the `run` scope function to initialize an object, but it's better to use `run` to initialize an object at a specific moment in your code **and** immediately compute a result.

Let's continue the previous example for the `apply` function, but this time, you want the `connect()` and `authenticate()` functions to be grouped so that they are called on every request.

For example:

```
class Client() {
    var token: String? = null
    fun connect() = println("connected!")
    fun authenticate() = println("authenticated!")
    fun getData() : String {
        println("getting data!")
        return "Mock data"
    }
}
val client: Client = Client().apply {
    token = "asdf"
}

fun main() {
    val result: String = client.run {
        connect()
        // connected!
        authenticate()
        // authenticated!
        getData()
        // getting data!
    }
}
```

The example:

- Creates `client` as an instance of the `Client` class.
- Uses the `apply` scope function on the `client` instance.
- Creates a temporary scope within the `apply` scope function so that you don't have to explicitly refer to the `client` instance when accessing its properties or functions.
- Passes a lambda expression to the `apply` scope function that updates the `token` property.

The `main()` function:

- Creates a `result` variable with type `String`.
- Uses the `run` scope function on the `client` instance.
- Creates a temporary scope within the `run` scope function so that you don't have to explicitly refer to the `client` instance when accessing its properties or functions.
- Passes a lambda expression to the `run` scope function that calls the `connect()`, `authenticate()`, and `getData()` functions.
- Assigns the result to the `result` variable.

Now you can use the returned result further in your code.

Also

Use the `also` scope function to complete an additional action with an object and then return the object to continue using it in your code, like writing a log.

Consider the example:

```
fun main() {
    val medals: List<String> = listOf("Gold", "Silver", "Bronze")
    val reversedLongUppercaseMedals: List<String> =
        medals
            .map { it.uppercase() }
            .filter { it.length > 4 }
            .reversed()
    println(reversedLongUppercaseMedals)
    // [BRONZE, SILVER]
}
```

The example:

- Creates the `medals` variable that contains a list of strings.
- Creates the `reversedLongUpperCaseMedals` variable that has the `List<String>` type.
- Uses the `.map()` extension function on the `medals` variable.
- Passes a lambda expression to the `.map()` function that refers to `medals` via the `it` keyword and calls the `.uppercase()` extension function on it.
- Uses the `.filter()` extension function on the `medals` variable.
- Passes a lambda expression as a predicate to the `.filter()` function that refers to `medals` via the `it` keyword and checks if the item in the list has more than 4 characters.
- Uses the `.reversed()` extension function on the `medals` variable.
- Assigns the result to the `reversedLongUpperCaseMedals` variable.
- Prints the list contained in the `reversedLongUpperCaseMedals` variable.

It would be useful to add some logging in between the function calls to see what is happening to the `medals` variable. The `also` function helps with that:

```
fun main() {
    val medals: List<String> = listOf("Gold", "Silver", "Bronze")
    val reversedLongUppercaseMedals: List<String> =
        medals
            .map { it.uppercase() }
            .also { println(it) }
            // [GOLD, SILVER, BRONZE]
            .filter { it.length > 4 }
            .also { println(it) }
            // [SILVER, BRONZE]
            .reversed()
    println(reversedLongUppercaseMedals)
    // [BRONZE, SILVER]
}
```

Now the example:

- Uses the `also` scope function on the `medals` variable.

- Creates a temporary scope within the `also` scope function so that you don't have to explicitly refer to the `medals` variable when using it as a function parameter.
- Passes a lambda expression to the `also` scope function that calls the `println()` function using the `medals` variable as a function parameter via the `it` keyword.

Since the `also` function returns the object, it is useful for not only logging but debugging, chaining multiple operations, and performing other side effect operations that don't affect the main flow of your code.

With

Unlike the other scope functions, `with` is not an extension function, so the syntax is different. You pass the receiver object to `with` as an argument.

Use the `with` scope function when you want to call multiple functions on an object.

Consider this example:

```
class Canvas {
    fun rect(x: Int, y: Int, w: Int, h: Int): Unit = println("$x, $y, $w, $h")
    fun circ(x: Int, y: Int, rad: Int): Unit = println("$x, $y, $rad")
    fun text(x: Int, y: Int, str: String): Unit = println("$x, $y, $str")
}

fun main() {
    val mainMonitorPrimaryBufferBackedCanvas = Canvas()

    mainMonitorPrimaryBufferBackedCanvas.text(10, 10, "Foo")
    mainMonitorPrimaryBufferBackedCanvas.rect(20, 30, 100, 50)
    mainMonitorPrimaryBufferBackedCanvas.circ(40, 60, 25)
    mainMonitorPrimaryBufferBackedCanvas.text(15, 45, "Hello")
    mainMonitorPrimaryBufferBackedCanvas.rect(70, 80, 150, 100)
    mainMonitorPrimaryBufferBackedCanvas.circ(90, 110, 40)
    mainMonitorPrimaryBufferBackedCanvas.text(35, 55, "World")
    mainMonitorPrimaryBufferBackedCanvas.rect(120, 140, 200, 75)
    mainMonitorPrimaryBufferBackedCanvas.circ(160, 180, 55)
    mainMonitorPrimaryBufferBackedCanvas.text(50, 70, "Kotlin")
}
```

The example creates a `Canvas` class that has three member functions: `rect()`, `circ()`, and `text()`. Each of these member functions prints a statement constructed from the function parameters that you provide.

The example creates `mainMonitorPrimaryBufferBackedCanvas` as an instance of the `Canvas` class before calling a sequence of member functions on the instance with different function parameters.

You can see that this code is hard to read. If you use the `with` function, the code is streamlined:

```
class Canvas {
    fun rect(x: Int, y: Int, w: Int, h: Int): Unit = println("$x, $y, $w, $h")
    fun circ(x: Int, y: Int, rad: Int): Unit = println("$x, $y, $rad")
    fun text(x: Int, y: Int, str: String): Unit = println("$x, $y, $str")
}

fun main() {
    val mainMonitorSecondaryBufferBackedCanvas = Canvas()
    with(mainMonitorSecondaryBufferBackedCanvas) {
        text(10, 10, "Foo")
        rect(20, 30, 100, 50)
        circ(40, 60, 25)
        text(15, 45, "Hello")
        rect(70, 80, 150, 100)
        circ(90, 110, 40)
        text(35, 55, "World")
        rect(120, 140, 200, 75)
        circ(160, 180, 55)
        text(50, 70, "Kotlin")
    }
}
```

This example:

- Uses the `with` scope function with the `mainMonitorSecondaryBufferBackedCanvas` instance as the receiver.
- Creates a temporary scope within the `with` scope function so that you don't have to explicitly refer to the `mainMonitorSecondaryBufferBackedCanvas` instance when calling its member functions.
- Passes a lambda expression to the `with` scope function that calls a sequence of member functions with different function parameters.

Now that this code is much easier to read, you are less likely to make mistakes.

Use case overview

This section has covered the different scope functions available in Kotlin and their main use cases for making your code more idiomatic. You can use this table as a quick reference. It's important to note that you don't need a complete understanding of how these functions work in order to use them in your code.

Function	Access to x via	Return value	Use case
`let`	`it`	Lambda result	Perform null checks in your code and later perform further actions with the returned object.
`apply`	`this`	`x`	Initialize objects at the time of creation.

Function	Access to x via	Return value	Use case
`run`	`this`	Lambda result	Initialize objects at the time of creation **AND** compute a result.
`also`	`it`	`x`	Complete additional actions before returning the object.
`with`	`this`	Lambda result	Call multiple functions on an object.

For more information about scope functions, see Scope functions.

Practice

Rewrite a function using safe calls and let

Rewrite the `.getPriceInEuros()` function as a single-expression function that uses safe call operators `?.` and the `let` scope function.

Hint


```
    Use safe call operators <code>?.</code> to safely access the <code>priceInDollars</code>
property from the <code>getProductInfo()</code>
    function. Then, use the <code>let</code> scope function to convert the value of <code>pr
iceInDollars</code> into euros.
```


```
data class ProductInfo(val priceInDollars: Double?)

class Product {
    fun getProductInfo(): ProductInfo? {
        return ProductInfo(100.0)
    }
}

// Rewrite this function
fun Product.getPriceInEuros(): Double? {
    val info = getProductInfo()
    if (info == null) return null
    val price = info.priceInDollars
    if (price == null) return null
    return convertToEuros(price)
}

fun convertToEuros(dollars: Double): Double {
    return dollars * 0.85
}

fun main() {
    val product = Product()
    val priceInEuros = product.getPriceInEuros()
```

```
    if (priceInEuros != null) {
        println("Price in Euros: €$priceInEuros")
        // Price in Euros: €85.0
    } else {
        println("Price information is not available.")
    }
}
```

```
data class ProductInfo(val priceInDollars: Double?)

class Product {
    fun getProductInfo(): ProductInfo? {
        return ProductInfo(100.0)
    }
}

fun Product.getPriceInEuros() = getProductInfo()?.priceInDollars?.let { convertToEuros(it) }

fun convertToEuros(dollars: Double): Double {
    return dollars * 0.85
}

fun main() {
    val product = Product()
    val priceInEuros = product.getPriceInEuros()

    if (priceInEuros != null) {
        println("Price in Euros: €$priceInEuros")
        // Price in Euros: €85.0
    } else {
        println("Price information is not available.")
    }
}
```

Chain apply and also

You have an `updateEmail()` function that updates the email address of a user. Use the `apply` scope function to update the email address and then the `also` scope function to print a log message: `Updating email for user with ID: ${it.id}`.

```
data class User(val id: Int, var email: String)

fun updateEmail(user: User, newEmail: String): User = // Write your code here

fun main() {
    val user = User(1, "old_email@example.com")
    val updatedUser = updateEmail(user, "new_email@example.com")
    // Updating email for user with ID: 1

    println("Updated User: $updatedUser")
    // Updated User: User(id=1, email=new_email@example.com)
}
```

```
data class User(val id: Int, var email: String)

fun updateEmail(user: User, newEmail: String): User = user.apply {
    this.email = newEmail
}.also { println("Updating email for user with ID: ${it.id}") }

fun main() {
    val user = User(1, "old_email@example.com")
    val updatedUser = updateEmail(user, "new_email@example.com")
    // Updating email for user with ID: 1

    println("Updated User: $updatedUser")
    // Updated User: User(id=1, email=new_email@example.com)
}
```

```
<a as="button" href="kotlin-tour-intermediate-extension-functions.md" mode="outline" icon="a
rrow-left" icon-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-lambdas-receiver.md" mode="classic" icon="arro
w-right" icon-position="right">Next step</a>
```

Lambda expressions with receiver

In this chapter, you'll learn how to use receivers with another type of function, lambda expressions, and how they can help you create a domain-specific language.

Lambda expressions with receiver

In the beginner tour, you learned how to use lambda expressions. Lambda expressions can also have a receiver. In this case, lambda expressions can access any member functions or properties of the receiver without having to explicitly specify the receiver each time. Without these additional references, your code is easier to read and maintain.

> **Tip:** Lambda expressions with receiver are also known as function literals with receiver.

The syntax for a lambda expression with receiver is different when you define the function type. First, write the receiver that you want to extend. Next, put a `.` and then complete the rest of your function type definition. For example:

```
MutableList<Int>.() -> Unit
```

This function type has:

- `MutableList<Int>` as the receiver.
- No function parameters within the parentheses `()`.
- No return value: `Unit`.

Consider this example that draws shapes on a canvas:

```
class Canvas {
    fun drawCircle() = println("🟠 Drawing a circle")
    fun drawSquare() = println("🟥 Drawing a square")
}

// Lambda expression with receiver definition
fun render(block: Canvas.() -> Unit): Canvas {
    val canvas = Canvas()
    // Use the lambda expression with receiver
    canvas.block()
    return canvas
}

fun main() {
    render {
        drawCircle()
        // 🟠 Drawing a circle
```

```
        drawSquare()
        // 🟥 Drawing a square
    }
}
```

In this example:

- The `Canvas` class has two functions that simulate drawing a circle or a square.
- The `render()` function takes a `block` parameter and returns an instance of the `Canvas` class.
- The `block` parameter is a lambda expression with receiver, where the `Canvas` class is the receiver.
- The `render()` function creates an instance of the `Canvas` class and calls the `block()` lambda expression on the `canvas` instance, using it as the receiver.
- The `main()` function calls the `render()` function with a lambda expression, which is passed to the `block` parameter.
- Inside the lambda passed to the `render()` function, the program calls the `drawCircle()` and `drawSquare()` functions on an instance of the `Canvas` class.

  Because the `drawCircle()` and `drawSquare()` functions are called in the lambda expression with receiver, they can be called directly as if they are inside the `Canvas` class.

Lambda expressions with receiver are helpful when you want to create a domain-specific language (DSL). Since you have access to the receiver's member functions and properties without explicitly referencing the receiver, your code becomes leaner.

To demonstrate this, consider an example that configures items in a menu. Let's begin with a `MenuItem` class and a `Menu` class that contains a function to add items to the menu called `item()`, as well as a list of all menu items `items`:

```
class MenuItem(val name: String)

class Menu(val name: String) {
    val items = mutableListOf<MenuItem>()

    fun item(name: String) {
        items.add(MenuItem(name))
    }
}
```

Let's use a lambda expression with receiver passed as a function parameter (`init`) to the `menu()` function that builds a menu as a starting point:

```
fun menu(name: String, init: Menu.() -> Unit): Menu {
    // Creates an instance of the Menu class
```

```
    val menu = Menu(name)
    // Calls the lambda expression with receiver init() on the class instance
    menu.init()
    return menu
}
```

Now you can use the DSL to configure a menu and create a `printMenu()` function to print the menu structure to the console:

```
class MenuItem(val name: String)

class Menu(val name: String) {
    val items = mutableListOf<MenuItem>()

    fun item(name: String) {
        items.add(MenuItem(name))
    }
}

fun menu(name: String, init: Menu.() -> Unit): Menu {
    val menu = Menu(name)
    menu.init()
    return menu
}
fun printMenu(menu: Menu) {
    println("Menu: ${menu.name}")
    menu.items.forEach { println("  Item: ${it.name}") }
}

// Use the DSL
fun main() {
    // Create the menu
    val mainMenu = menu("Main Menu") {
        // Add items to the menu
        item("Home")
        item("Settings")
        item("Exit")
    }

    // Print the menu
    printMenu(mainMenu)
    // Menu: Main Menu
    //   Item: Home
    //   Item: Settings
    //   Item: Exit
}
```

As you can see, using a lambda expression with receiver greatly simplifies the code needed to create your menu. Lambda expressions are not only useful for setup and creation but also for configuration. They are commonly used in building DSLs for APIs, UI frameworks, and configuration builders to produce streamlined code, allowing you to focus more easily on the underlying code structure and logic.

Kotlin's ecosystem has many examples of this design pattern, such as in the `buildList()` and `buildString()` functions from the standard library.

> **Tip:** Lambda expressions with receivers can be combined with **type-safe builders** in Kotlin to make DSLs that detect any problems with types at compile time rather than at runtime. To learn more, see Type-safe builders.

Practice

Process data with a lambda expression with receiver

You have a `fetchData()` function that accepts a lambda expression with receiver. Update the lambda expression to use the `append()` function so that the output of your code is: `Data received - Processed`.

```
fun fetchData(callback: StringBuilder.() -> Unit) {
    val builder = StringBuilder("Data received")
    builder.callback()
}

fun main() {
    fetchData {
        // Write your code here
        // Data received - Processed
    }
}
```

```
fun fetchData(callback: StringBuilder.() -> Unit) {
    val builder = StringBuilder("Data received")
    builder.callback()
}

fun main() {
    fetchData {
        append(" - Processed")
        println(this.toString())
        // Data received - Processed
    }
}
```

Handle a double-click event

You have a `Button` class and `ButtonEvent` and `Position` data classes. Write some code that triggers the `onEvent()` member function of the `Button` class to trigger a double-click event. Your code should print `"Double click!"`.

```
class Button {
    fun onEvent(action: ButtonEvent.() -> Unit) {
        // Simulate a double-click event (not a right-click)
```

```
        val event = ButtonEvent(isRightClick = false, amount = 2, position = Position(100, 2
00))
        event.action() // Trigger the event callback
    }
}

data class ButtonEvent(
    val isRightClick: Boolean,
    val amount: Int,
    val position: Position
)

data class Position(
    val x: Int,
    val y: Int
)

fun main() {
    val button = Button()

    button.onEvent {
        // Write your code here
        // Double click!
    }
}
```

```
class Button {
    fun onEvent(action: ButtonEvent.() -> Unit) {
        // Simulate a double-click event (not a right-click)
        val event = ButtonEvent(isRightClick = false, amount = 2, position = Position(100, 2
00))
        event.action() // Trigger the event callback
    }
}

data class ButtonEvent(
    val isRightClick: Boolean,
    val amount: Int,
    val position: Position
)

data class Position(
    val x: Int,
    val y: Int
)

fun main() {
    val button = Button()

    button.onEvent {
        if (!isRightClick && amount == 2) {
            println("Double click!")
            // Double click!
        }
```

```
    }
}
```

Create an incremented list

Write a function that creates a copy of a list of integers where every element is incremented by 1. Use the provided function skeleton that extends `List<Int>` with an `incremented` function.

```
fun List<Int>.incremented(): List<Int> {
    val originalList = this
    return buildList {
        // Write your code here
    }
}

fun main() {
    val originalList = listOf(1, 2, 3)
    val newList = originalList.incremented()
    println(newList)
    // [2, 3, 4]
}
```

```
fun List<Int>.incremented(): List<Int> {
    val originalList = this
    return buildList {
        for (n in originalList) add(n + 1)
    }
}

fun main() {
    val originalList = listOf(1, 2, 3)
    val newList = originalList.incremented()
    println(newList)
    // [2, 3, 4]
}
```

```
<a as="button" href="kotlin-tour-intermediate-scope-functions.md" mode="outline" icon="arrow
-left" icon-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-classes-interfaces.md" mode="classic" icon="ar
row-right" icon-position="right">Next step</a>
```

Classes and interfaces

In the beginner tour, you learned how to use classes and data classes to store data and maintain a collection of characteristics that can be shared in your code. Eventually, you will want to create a hierarchy to efficiently share code within your projects. This chapter explains the options Kotlin provides for sharing code and how they can make your code safer and easier to maintain.

Class inheritance

In a previous chapter, we covered how you can use extension functions to extend classes without modifying the original source code. But what if you are working on something complex where sharing code **between** classes would be useful? In such cases, you can use class inheritance.

By default, classes in Kotlin can't be inherited. Kotlin is designed this way to prevent unintended inheritance and make your classes easier to maintain.

Kotlin classes only support **single inheritance**, meaning it is only possible to inherit from **one class at a time**. This class is called the **parent**.

The parent of a class inherits from another class (the grandparent), forming a hierarchy. At the top of Kotlin's class hierarchy is the common parent class: `Any`. All classes ultimately inherit from the `Any` class:

!An example of the class hierarchy with Any type

The `Any` class provides the `toString()` function as a member function automatically. Therefore, you can use this inherited function in any of your classes. For example:

```
class Car(val make: String, val model: String, val numberOfDoors: Int)

fun main() {
    val car1 = Car("Toyota", "Corolla", 4)

    // Uses the .toString() function via string templates to print class properties
    println("Car1: make=${car1.make}, model=${car1.model}, numberOfDoors=${car1.numberOfDoor
s}")
    // Car1: make=Toyota, model=Corolla, numberOfDoors=4
}
```

If you want to use inheritance to share some code between classes, first consider using abstract classes.

Abstract classes

Abstract classes can be inherited by default. The purpose of abstract classes is to provide members that other classes inherit or implement. As a result, they have a constructor, but you can't create instances from them. Within the child class, you define the behavior of the parent's properties and functions with the `override` keyword. In this way, you can say that the child class "overrides" the members of the parent class.

> **Tip:** When you define the behavior of an inherited function or property, we call that an **implementation**.

Abstract classes can contain both functions and properties **with** implementation as well as functions and properties **without** implementation, known as abstract functions and properties.

To create an abstract class, use the `abstract` keyword:

```
abstract class Animal
```

To declare a function or a property **without** an implementation, you also use the `abstract` keyword:

```
abstract fun makeSound()
abstract val sound: String
```

For example, let's say that you want to create an abstract class called `Product` that you can create child classes from to define different product categories:

```
abstract class Product(val name: String, var price: Double) {
    // Abstract property for the product category
    abstract val category: String

    // A function that can be shared by all products
    fun productInfo(): String {
        return "Product: $name, Category: $category, Price: $price"
    }
}
```

In the abstract class:

- The constructor has two parameters for the product's `name` and `price`.
- There is an abstract property that contains the product category as a string.
- There is a function that prints information about the product.

Let's create a child class for electronics. Before you define an implementation for the `category` property in the child class, you must use the `override` keyword:

```
class Electronic(name: String, price: Double, val warranty: Int) : Product(name, price) {
    override val category = "Electronic"
```

```
}
```

The `Electronic` class:

- Inherits from the `Product` abstract class.
- Has an additional parameter in the constructor: `warranty`, which is specific to electronics.
- Overrides the `category` property to contain the string `"Electronic"`.

Now, you can use these classes like this:

```
abstract class Product(val name: String, var price: Double) {
    // Abstract property for the product category
    abstract val category: String

    // A function that can be shared by all products
    fun productInfo(): String {
        return "Product: $name, Category: $category, Price: $price"
    }
}

class Electronic(name: String, price: Double, val warranty: Int) : Product(name, price) {
    override val category = "Electronic"
}
fun main() {
    // Creates an instance of the Electronic class
    val laptop = Electronic(name = "Laptop", price = 1000.0, warranty = 2)

    println(laptop.productInfo())
    // Product: Laptop, Category: Electronic, Price: 1000.0
}
```

Although abstract classes are great for sharing code in this way, they are restricted because classes in Kotlin only support single inheritance. If you need to inherit from multiple sources, consider using interfaces.

Interfaces

Interfaces are similar to classes, but they have some differences:

- You can't create an instance of an interface. They don't have a constructor or header.
- Their functions and properties are implicitly inheritable by default. In Kotlin, we say that they are "open."
- You don't need to mark their functions as `abstract` if you don't give them an implementation.

Similar to abstract classes, you use interfaces to define a set of functions and properties that classes can inherit and implement later. This approach helps you focus on the abstraction described by the interface, rather than the specific implementation details. Using interfaces makes your code:

- More modular, as it isolates different parts, allowing them to evolve independently.
- Easier to understand by grouping related functions into a cohesive set.
- Easier to test, as you can quickly swap an implementation with a mock for testing.

To declare an interface, use the `interface` keyword:

```
interface PaymentMethod
```

Interface implementation

Interfaces support multiple inheritance so a class can implement multiple interfaces at once. First, let's consider the scenario where a class implements **one** interface.

To create a class that implements an interface, add a colon after your class header, followed by the interface name that you want to implement. You don't use parentheses `()` after the interface name because interfaces don't have a constructor:

```
class CreditCardPayment : PaymentMethod
```

For example:

```
interface PaymentMethod {
    // Functions are inheritable by default
    fun initiatePayment(amount: Double): String
}

class CreditCardPayment(val cardNumber: String, val cardHolderName: String, val expiryDate:
String) : PaymentMethod {
    override fun initiatePayment(amount: Double): String {
        // Simulate processing payment with credit card
        return "Payment of $$amount initiated using Credit Card ending in ${cardNumber.takeL
ast(4)}."
    }
}

fun main() {
    val paymentMethod = CreditCardPayment("1234 5678 9012 3456", "John Doe", "12/25")
    println(paymentMethod.initiatePayment(100.0))
    // Payment of $100.0 initiated using Credit Card ending in 3456.
}
```

In the example:

- `PaymentMethod` is an interface that has an `initiatePayment()` function without an implementation.
- `CreditCardPayment` is a class that implements the `PaymentMethod` interface.
- The `CreditCardPayment` class overrides the inherited `initiatePayment()` function.
- `paymentMethod` is an instance of the `CreditCardPayment` class.

- The overridden `initiatePayment()` function is called on the `paymentMethod` instance with a parameter of `100.0`.

To create a class that implements **multiple** interfaces, add a colon after your class header followed by the name of the interfaces that you want to implement separated by a comma:

```
class CreditCardPayment : PaymentMethod, PaymentType
```

For example:

```
interface PaymentMethod {
    fun initiatePayment(amount: Double): String
}

interface PaymentType {
    val paymentType: String
}

class CreditCardPayment(val cardNumber: String, val cardHolderName: String, val expiryDate:
String) : PaymentMethod,
    PaymentType {
    override fun initiatePayment(amount: Double): String {
        // Simulate processing payment with credit card
        return "Payment of $$amount initiated using Credit Card ending in ${cardNumber.takeL
ast(4)}."
    }

    override val paymentType: String = "Credit Card"
}

fun main() {
    val paymentMethod = CreditCardPayment("1234 5678 9012 3456", "John Doe", "12/25")
    println(paymentMethod.initiatePayment(100.0))
    // Payment of $100.0 initiated using Credit Card ending in 3456.

    println("Payment is by ${paymentMethod.paymentType}")
    // Payment is by Credit Card
}
```

In the example:

- `PaymentMethod` is an interface that has the `initiatePayment()` function without an implementation.
- `PaymentType` is an interface that has the `paymentType` property that isn't initialized.
- `CreditCardPayment` is a class that implements the `PaymentMethod` and `PaymentType` interfaces.
- The `CreditCardPayment` class overrides the inherited `initiatePayment()` function and the `paymentType` property.
- `paymentMethod` is an instance of the `CreditCardPayment` class.
- The overridden `initiatePayment()` function is called on the `paymentMethod` instance with a parameter of `100.0`.

- The overridden `paymentType` property is accessed on the `paymentMethod` instance.

For more information about interfaces and interface inheritance, see Interfaces.

Delegation

Interfaces are useful, but if your interface contains many functions, its child classes can end up with a lot of boilerplate code. If you only want to override a small part of a class's behavior, you need to repeat yourself a lot.

> **Tip:** Boilerplate code is a chunk of code that is reused with little or no alteration in multiple parts of a software project.

For example, let's say that you have an interface called `DrawingTool` that contains a number of functions and one property called `color`:

```
interface DrawingTool {
    val color: String
    fun draw(shape: String)
    fun erase(area: String)
    fun getToolInfo(): String
}
```

You create a class called `PenTool` which implements the `DrawingTool` interface and provides implementations for all of its members:

```
class PenTool : DrawingTool {
    override val color: String = "black"

    override fun draw(shape: String) {
        println("Drawing $shape using a pen in $color")
    }

    override fun erase(area: String) {
        println("Erasing $area with pen tool")
    }

    override fun getToolInfo(): String {
        return "PenTool(color=$color)"
    }
}
```

You want to create a class like `PenTool` with the same behavior but a different `color` value. One approach is to create a new class that expects an object implementing the `DrawingTool` interface as a parameter, like a `PenTool` class instance. Then, inside the class, you can override the `color` property.

But in this scenario, you need to add implementations for each member of the `DrawingTool` interface:

```
interface DrawingTool {
    val color: String
    fun draw(shape: String)
    fun erase(area: String)
    fun getToolInfo(): String
}

class PenTool : DrawingTool {
    override val color: String = "black"

    override fun draw(shape: String) {
        println("Drawing $shape using a pen in $color")
    }

    override fun erase(area: String) {
        println("Erasing $area with pen tool")
    }

    override fun getToolInfo(): String {
        return "PenTool(color=$color)"
    }
}
class CanvasSession(val tool: DrawingTool) : DrawingTool {
    override val color: String = "blue"

    override fun draw(shape: String) {
        tool.draw(shape)
    }

    override fun erase(area: String) {
        tool.erase(area)
    }

    override fun getToolInfo(): String {
        return tool.getToolInfo()
    }
}
fun main() {
    val pen = PenTool()
    val session = CanvasSession(pen)

    println("Pen color: ${pen.color}")
    // Pen color: black

    println("Session color: ${session.color}")
    // Session color: blue

    session.draw("circle")
    // Drawing circle with pen in black

    session.erase("top-left corner")
    // Erasing top-left corner with pen tool
```

```
    println(session.getToolInfo())
    // PenTool(color=black)
}
```

You can see that if you have a large number of member functions in the `DrawingTool` interface, the amount of boilerplate code in the `CanvasSession` class can be large. However, there is an alternative.

In Kotlin, you can delegate the interface implementation to a class instance using the `by` keyword. For example:

```
class CanvasSession(val tool: DrawingTool) : DrawingTool by tool
```

Here, `tool` is the name of the `PenTool` class instance where the implementations of member functions are delegated to.

Now you don't have to add implementations for the member functions in the `CanvasSession` class. The compiler does this for you automatically from the `PenTool` class. This saves you from having to write a lot of boilerplate code. Instead, you add code only for the behavior you want to change for your child class.

For example, if you want to change the value of the `color` property:

```
interface DrawingTool {
    val color: String
    fun draw(shape: String)
    fun erase(area: String)
    fun getToolInfo(): String
}

class PenTool : DrawingTool {
    override val color: String = "black"

    override fun draw(shape: String) {
        println("Drawing $shape using a pen in $color")
    }

    override fun erase(area: String) {
        println("Erasing $area with pen tool")
    }

    override fun getToolInfo(): String {
        return "PenTool(color=$color)"
    }
}
class CanvasSession(val tool: DrawingTool) : DrawingTool by tool {
    // No boilerplate code!
    override val color: String = "blue"
}
fun main() {
```

```
    val pen = PenTool()
    val session = CanvasSession(pen)

    println("Pen color: ${pen.color}")
    // Pen color: black

    println("Session color: ${session.color}")
    // Session color: blue

    session.draw("circle")
    // Drawing circle with pen in black

    session.erase("top-left corner")
    // Erasing top-left corner with pen tool

    println(session.getToolInfo())
    // PenTool(color=black)
}
```

If you want to, you can also override the behavior of an inherited member function in the `CanvasSession` class, but now you don't have to add new lines of code for every inherited member function.

For more information, see Delegation.

Practice

Implement smart devices with an abstract class

Imagine you're working on a smart home system. A smart home typically has different types of devices that all have some basic features but also unique behaviors. In the code sample below, complete the `abstract` class called `SmartDevice` so that the child class `SmartLight` can compile successfully.

Then, create another child class called `SmartThermostat` that inherits from the `SmartDevice` class and implements `turnOn()` and `turnOff()` functions that return print statements describing which thermostat is heating or turned off. Finally, add another function called `adjustTemperature()` that accepts a temperature measurement as an input and prints: `$name thermostat set to $temperature°C.`

Hint

```
    In the <code>SmartDevice</code> class, add the <code>turnOn()</code> and <code>turnOff()
</code> functions so that
    you can override their behavior later in the <code>SmartThermostat</code> class.
```

```
abstract class // Write your code here

class SmartLight(name: String) : SmartDevice(name) {
```

```
    override fun turnOn() {
        println("$name is now ON.")
    }

    override fun turnOff() {
        println("$name is now OFF.")
    }

   fun adjustBrightness(level: Int) {
        println("Adjusting $name brightness to $level%.")
    }
}

class SmartThermostat // Write your code here

fun main() {
    val livingRoomLight = SmartLight("Living Room Light")
    val bedroomThermostat = SmartThermostat("Bedroom Thermostat")

    livingRoomLight.turnOn()
    // Living Room Light is now ON.
    livingRoomLight.adjustBrightness(10)
    // Adjusting Living Room Light brightness to 10%.
    livingRoomLight.turnOff()
    // Living Room Light is now OFF.

    bedroomThermostat.turnOn()
    // Bedroom Thermostat thermostat is now heating.
    bedroomThermostat.adjustTemperature(5)
    // Bedroom Thermostat thermostat set to 5°C.
    bedroomThermostat.turnOff()
    // Bedroom Thermostat thermostat is now off.
}
```

```
abstract class SmartDevice(val name: String) {
    abstract fun turnOn()
    abstract fun turnOff()
}

class SmartLight(name: String) : SmartDevice(name) {
    override fun turnOn() {
        println("$name is now ON.")
    }

    override fun turnOff() {
        println("$name is now OFF.")
    }

   fun adjustBrightness(level: Int) {
        println("Adjusting $name brightness to $level%.")
    }
}

class SmartThermostat(name: String) : SmartDevice(name) {
    override fun turnOn() {
        println("$name thermostat is now heating.")
```

```
    }

    override fun turnOff() {
        println("$name thermostat is now off.")
    }

   fun adjustTemperature(temperature: Int) {
        println("$name thermostat set to $temperature°C.")
    }
}

fun main() {
    val livingRoomLight = SmartLight("Living Room Light")
    val bedroomThermostat = SmartThermostat("Bedroom Thermostat")

    livingRoomLight.turnOn()
    // Living Room Light is now ON.
    livingRoomLight.adjustBrightness(10)
    // Adjusting Living Room Light brightness to 10%.
    livingRoomLight.turnOff()
    // Living Room Light is now OFF.

    bedroomThermostat.turnOn()
    // Bedroom Thermostat thermostat is now heating.
    bedroomThermostat.adjustTemperature(5)
    // Bedroom Thermostat thermostat set to 5°C.
    bedroomThermostat.turnOff()
    // Bedroom Thermostat thermostat is now off.
}
```

Implement a media interface

Create an interface called `Media` that you can use to implement specific media classes like `Audio`, `Video`, or `Podcast`. Your interface must include:

- A property called `title` to represent the title of the media.
- A function called `play()` to play the media.

Then, create a class called `Audio` that implements the `Media` interface. The `Audio` class must use the `title` property in its constructor as well as have an additional property called `composer` that has `String` type. In the class, implement the `play()` function to print the following: `"Playing audio: $title, composed by $composer"`.

Hint

```
    You can use the <code>override</code> keyword in class headers to implement a property f
rom an interface in the constructor.
```

```
interface // Write your code here

class // Write your code here
```

```
fun main() {
    val audio = Audio("Symphony No. 5", "Beethoven")
    audio.play()
   // Playing audio: Symphony No. 5, composed by Beethoven
}
```

```
interface Media {
    val title: String
    fun play()
}

class Audio(override val title: String, val composer: String) : Media {
    override fun play() {
        println("Playing audio: $title, composed by $composer")
    }
}

fun main() {
    val audio = Audio("Symphony No. 5", "Beethoven")
    audio.play()
   // Playing audio: Symphony No. 5, composed by Beethoven
}
```

Combine an interface and an abstract class

You're building a payment processing system for an e-commerce application. Each payment method needs to be able to authorize a payment and process a transaction. Some payments also need to be able to process refunds.

1. In the `Refundable` interface, add a function called `refund()` to process refunds.
2. In the `PaymentMethod` abstract class:
    - Add a function called `authorize()` that takes an amount and prints a message containing the amount.
    - Add an abstract function called `processPayment()` that also takes an amount.
3. Create a class called `CreditCard` that implements the `Refundable` interface and `PaymentMethod` abstract class. In this class, add implementations for the `refund()` and `processPayment()` functions so that they print the following statements:
    - `"Refunding $amount to the credit card."`
    - `"Processing credit card payment of $amount."`

```
interface Refundable {
    // Write your code here
}

abstract class PaymentMethod(val name: String) {
    // Write your code here
}
```

```
class CreditCard // Write your code here

fun main() {
    val visa = CreditCard("Visa")

    visa.authorize(100.0)
    // Authorizing payment of $100.0.
    visa.processPayment(100.0)
    // Processing credit card payment of $100.0.
    visa.refund(50.0)
    // Refunding $50.0 to the credit card.
}
```

```
interface Refundable {
    fun refund(amount: Double)
}

abstract class PaymentMethod(val name: String) {
    fun authorize(amount: Double) {
        println("Authorizing payment of $$amount.")
    }

    abstract fun processPayment(amount: Double)
}

class CreditCard(name: String) : PaymentMethod(name), Refundable {
    override fun processPayment(amount: Double) {
        println("Processing credit card payment of $$amount.")
    }

    override fun refund(amount: Double) {
        println("Refunding $$amount to the credit card.")
    }
}

fun main() {
    val visa = CreditCard("Visa")

    visa.authorize(100.0)
    // Authorizing payment of $100.0.
    visa.processPayment(100.0)
    // Processing credit card payment of $100.0.
    visa.refund(50.0)
    // Refunding $50.0 to the credit card.
}
```

Customize behavior with interface delegation

You have a simple messaging app that has some basic functionality, but you want to add some functionality for *smart* messages without significantly duplicating your code.

In the code below, define a class called `SmartMessenger` that inherits from the `Messenger` interface but delegates the implementation to an instance of the `BasicMessenger` class.

In the `SmartMessenger` class, override the `sendMessage()` function to send smart messages. The function must accept a `message` as an input and return a printed statement: `"Sending a smart message: $message"`. In addition, call the `sendMessage()` function from the `BasicMessenger` class and prefix the message with `[smart]`.

> **Note:** You don't need to rewrite the `receiveMessage()` function in the `SmartMessenger` class.

```
interface Messenger {
    fun sendMessage(message: String)
    fun receiveMessage(): String
}

class BasicMessenger : Messenger {
    override fun sendMessage(message: String) {
        println("Sending message: $message")
    }

    override fun receiveMessage(): String {
        return "You've got a new message!"
    }
}

class SmartMessenger // Write your code here

fun main() {
    val basicMessenger = BasicMessenger()
    val smartMessenger = SmartMessenger(basicMessenger)

    basicMessenger.sendMessage("Hello!")
    // Sending message: Hello!
    println(smartMessenger.receiveMessage())
    // You've got a new message!
    smartMessenger.sendMessage("Hello from SmartMessenger!")
    // Sending a smart message: Hello from SmartMessenger!
    // Sending message: [smart] Hello from SmartMessenger!
}
```

```
interface Messenger {
    fun sendMessage(message: String)
    fun receiveMessage(): String
}

class BasicMessenger : Messenger {
    override fun sendMessage(message: String) {
        println("Sending message: $message")
    }

    override fun receiveMessage(): String {
        return "You've got a new message!"
    }
}
```

```
class SmartMessenger(val basicMessenger: BasicMessenger) : Messenger by basicMessenger {
    override fun sendMessage(message: String) {
        println("Sending a smart message: $message")
        basicMessenger.sendMessage("[smart] $message")
    }
}

fun main() {
    val basicMessenger = BasicMessenger()
    val smartMessenger = SmartMessenger(basicMessenger)

    basicMessenger.sendMessage("Hello!")
    // Sending message: Hello!
    println(smartMessenger.receiveMessage())
    // You've got a new message!
    smartMessenger.sendMessage("Hello from SmartMessenger!")
    // Sending a smart message: Hello from SmartMessenger!
    // Sending message: [smart] Hello from SmartMessenger!
}
```

```
<a as="button" href="kotlin-tour-intermediate-lambdas-receiver.md" mode="outline" icon="arro
w-left" icon-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-objects.md" mode="classic" icon="arrow-right"
icon-position="right">Next step</a>
```

Objects

In this chapter, you'll expand your understanding of classes by exploring object declarations. This knowledge will help you efficiently manage behavior across your projects.

Object declarations

In Kotlin, you can use **object declarations** to declare a class with a single instance. In a sense, you declare the class and create the single instance *at the same time*. Object declarations are useful when you want to create a class to use as a single reference point for your program or to coordinate behavior across a system.

> **Tip:** A class that has only one instance that is easily accessible is called a **singleton**.

Objects in Kotlin are **lazy**, meaning they are created only when accessed. Kotlin also ensures that all objects are created in a thread-safe manner so that you don't have to check this manually.

To create an object declaration, use the `object` keyword:

```
object DoAuth {}
```

Following the name of your `object`, add any properties or member functions within the object body defined by curly braces `{}`.

> **Note:** Objects can't have constructors, so they don't have headers like classes.

For example, let's say that you wanted to create an object called `DoAuth` that is responsible for authentication:

```
object DoAuth {
    fun takeParams(username: String, password: String) {
        println("input Auth parameters = $username:$password")
    }
}

fun main(){
    // The object is created when the takeParams() function is called
    DoAuth.takeParams("coding_ninja", "N1njaC0ding!")
    // input Auth parameters = coding_ninja:N1njaC0ding!
}
```

The object has a member function called `takeParams` that accepts `username` and `password` variables as parameters and prints a string to the console. The `DoAuth` object is only created when the function is called for the first time.

Note: Objects can inherit from classes and interfaces. For example:

```
interface Auth {
    fun takeParams(username: String, password: String)
}

object DoAuth : Auth {
    override fun takeParams(username: String, password: String) {
        println("input Auth parameters = $username:$password")
    }
}
```

Data objects

To make it easier to print the contents of an object declaration, Kotlin has **data** objects. Similar to data classes, which you learned about in the beginner tour, data objects automatically come with additional member functions: `toString()` and `equals()`.

> Unlike data classes, data objects do not come automatically with the `copy()` member function because they only have a single instance that can't be copied.

To create a data object, use the same syntax as for object declarations but prefix it with the `data` keyword:

```
data object AppConfig {}
```

For example:

```
data object AppConfig {
    var appName: String = "My Application"
    var version: String = "1.0.0"
}

fun main() {
    println(AppConfig)
    // AppConfig

    println(AppConfig.appName)
    // My Application
}
```

For more information about data objects, see .

Companion objects

In Kotlin, a class can have an object: a **companion** object. You can only have **one** companion object per class. A companion object is created only when its class is referenced for the first time.

Any properties or functions declared inside a companion object are shared across all class instances.

To create a companion object within a class, use the same syntax for an object declaration but prefix it with the `companion` keyword:

```
companion object Bonger {}
```

> **Note:** A companion object doesn't have to have a name. If you don't define one, the default is `Companion`.

To access any properties or functions of the companion object, reference the class name. For example:

```
class BigBen {
    companion object Bonger {
        fun getBongs(nTimes: Int) {
            repeat(nTimes) { print("BONG ") }
            }
        }
    }

fun main() {
    // Companion object is created when the class is referenced for the
    // first time.
    BigBen.getBongs(12)
    // BONG BONG BONG BONG BONG BONG BONG BONG BONG BONG BONG BONG
}
```

This example creates a class called `BigBen` that contains a companion object called `Bonger`. The companion object has a member function called `getBongs()` that accepts an integer and prints `"BONG"` to the console the same number of times as the integer.

In the `main()` function, the `getBongs()` function is called by referring to the class name. The companion object is created at this point. The `getBongs()` function is called with parameter `12`.

For more information, see .

Practice

Implement a data object for order tracking

You run a coffee shop and have a system for tracking customer orders. Consider the code below and complete the declaration of the second data object so that the following code in the `main()` function runs successfully:

```
interface Order {
    val orderId: String
```

```
    val customerName: String
    val orderTotal: Double
}

data object OrderOne: Order {
    override val orderId = "001"
    override val customerName = "Alice"
    override val orderTotal = 15.50
}

data object // Write your code here

fun main() {
    // Print the name of each data object
    println("Order name: $OrderOne")
    // Order name: OrderOne
    println("Order name: $OrderTwo")
    // Order name: OrderTwo

    // Check if the orders are identical
    println("Are the two orders identical? ${OrderOne == OrderTwo}")
    // Are the two orders identical? false

    if (OrderOne == OrderTwo) {
        println("The orders are identical.")
    } else {
        println("The orders are unique.")
        // The orders are unique.
    }

    println("Do the orders have the same customer name? ${OrderOne.customerName == OrderTwo.
customerName}")
    // Do the orders have the same customer name? false
}
```

```
interface Order {
    val orderId: String
    val customerName: String
    val orderTotal: Double
}

data object OrderOne: Order {
    override val orderId = "001"
    override val customerName = "Alice"
    override val orderTotal = 15.50
}

data object OrderTwo: Order {
    override val orderId = "002"
    override val customerName = "Bob"
    override val orderTotal = 12.75
}

fun main() {
    // Print the name of each data object
    println("Order name: $OrderOne")
```

```
    // Order name: OrderOne
    println("Order name: $OrderTwo")
    // Order name: OrderTwo

    // Check if the orders are identical
    println("Are the two orders identical? ${OrderOne == OrderTwo}")
    // Are the two orders identical? false

    if (OrderOne == OrderTwo) {
        println("The orders are identical.")
    } else {
        println("The orders are unique.")
        // The orders are unique.
    }

    println("Do the orders have the same customer name? ${OrderOne.customerName == OrderTwo.
customerName}")
    // Do the orders have the same customer name? false
}
```

Create an object declaration

Create an object declaration that inherits from the `Vehicle` interface to create a unique vehicle type: `FlyingSkateboard`. Implement the `name` property and the `move()` function in your object so that the following code in the `main()` function runs successfully:

```
interface Vehicle {
    val name: String
    fun move(): String
}

object // Write your code here

fun main() {
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.move()}")
    // Flying Skateboard: Glides through the air with a hover engine
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.fly()}")
    // Flying Skateboard: Wooooooooo
}
```

```
interface Vehicle {
    val name: String
    fun move(): String
}

object FlyingSkateboard : Vehicle {
    override val name = "Flying Skateboard"
    override fun move() = "Glides through the air with a hover engine"

   fun fly(): String = "Wooooooooo"
}

fun main() {
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.move()}")
```

```
    // Flying Skateboard: Glides through the air with a hover engine
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.fly()}")
    // Flying Skateboard: Wooooooooo
}
```

Validate an email address before creating a user

You are building a user registration module for an app. You want to keep email validation associated with the `User` class but don't want to create an unnecessary `User` instance if the email address is invalid.

For this exercise, consider an email address valid if it contains both `@` and `.`. Complete the data class so that the following code in the `main()` function runs successfully:

Hint

```
    Add an email validation function in a companion object for the `User` class so that you
can call the function directly on `User`.
```

```
data class User(val name: String, val email: String) {
    // Write your code here
}

fun main() {
    val candidates = listOf(
        Pair("Alice", "alice@example.com"),
        Pair("Bob", "bob2example-com")
    )

    for ((name, email) in candidates) {
        if (User.isValidEmail(email)) {
            val user = User(name, email)
            println("Registered: ${user.name}, ${user.email}")
            // Registered: Alice, alice@example.com
        } else {
            println("Error: '${email}' is not valid. The email should contain '@' and '.'")
            // Error: 'bob2example-com' is not valid. The email should contain '@' and '.'
        }
    }
}
```

```
data class User(val name: String, val email: String) {
    companion object {
        fun isValidEmail(email: String): Boolean =
            email.contains('@') && email.contains('.')
    }
}

fun main() {
    val candidates = listOf(
        Pair("Alice", "alice@example.com"),
        Pair("Bob", "bob2example-com")
    )
```

```
    for ((name, email) in candidates) {
        if (User.isValidEmail(email)) {
            val user = User(name, email)
            println("Registered: ${user.name}, ${user.email}")
            // Registered: Alice, alice@example.com
        } else {
            println("Error: '${email}' is not valid. The email should contain '@' and '.'")
            // Error: 'bob2example-com' is not valid. The email should contain '@' and '.'
        }
    }
}
```

Tip: As an extension of this exercise, try using functions in companion objects as factory methods to construct instances of a class. For an example and more information about this pattern, see .

```
<a as="button" href="kotlin-tour-intermediate-classes-interfaces.md" mode="outline" icon="ar
row-left" icon-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-open-special-classes.md" mode="classic" icon
="arrow-right" icon-position="right">Next step</a>
```

Open and special classes

In this chapter, you'll learn about open classes, how they work with interfaces, and other special types of classes available in Kotlin.

Open classes

If you can't use interfaces or abstract classes, you can explicitly make a class inheritable by declaring it as **open**. To do this, use the `open` keyword before your class declaration:

```
open class Vehicle(val make: String, val model: String)
```

To create a class that inherits from another, add a colon after your class header followed by a call to the constructor of the parent class that you want to inherit from. In this example, the `Car` class inherits from the `Vehicle` class:

```
open class Vehicle(val make: String, val model: String)

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make, model)

fun main() {
    // Creates an instance of the Car class
    val car = Car("Toyota", "Corolla", 4)

    // Prints the details of the car
    println("Car Info: Make - ${car.make}, Model - ${car.model}, Number of doors - ${car.num
berOfDoors}")
    // Car Info: Make - Toyota, Model - Corolla, Number of doors - 4
}
```

Just like when creating a normal class instance, if your class inherits from a parent class, then it must initialize all the parameters declared in the parent class header. So in the example, the `car` instance of the `Car` class initializes the parent class parameters: `make` and `model`.

Overriding inherited behavior

If you want to inherit from a class but change some of the behavior, you can override the inherited behavior.

By default, it's not possible to override a member function or property of a parent class. Just like with abstract classes, you need to add special keywords.

Member functions

To allow a function in the parent class to be overridden, use the `open` keyword before its declaration in the parent class:

```
open fun displayInfo() {}
```

To override an inherited member function, use the `override` keyword before the function declaration in the child class:

```
override fun displayInfo() {}
```

For example:

```
open class Vehicle(val make: String, val model: String) {
    open fun displayInfo() {
        println("Vehicle Info: Make - $make, Model - $model")
    }
}

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make, model) {
    override fun displayInfo() {
        println("Car Info: Make - $make, Model - $model, Number of Doors - $numberOfDoors")
    }
}

fun main() {
    val car1 = Car("Toyota", "Corolla", 4)
    val car2 = Car("Honda", "Civic", 2)

    // Uses the overridden displayInfo() function
    car1.displayInfo()
    // Car Info: Make - Toyota, Model - Corolla, Number of Doors - 4
    car2.displayInfo()
    // Car Info: Make - Honda, Model - Civic, Number of Doors - 2
}
```

This example:

- Creates two instances of the `Car` class that inherit from the `Vehicle` class: `car1` and `car2`.
- Overrides the `displayInfo()` function in the `Car` class to also print the number of doors.
- Calls the overridden `displayInfo()` function on `car1` and `car2` instances.

Properties

In Kotlin, it's not common practice to make a property inheritable by using the `open` keyword and overriding it later. Most of the time, you use an abstract class or an interface where properties are inheritable by default.

Properties inside open classes are accessible by their child class. In general, it's better to access them directly rather than override them with a new property.

For example, let's say that you have a property called `transmissionType` that you want to override later. The syntax for overriding properties is exactly the same as for overriding member functions. You can do this:

```
open class Vehicle(val make: String, val model: String) {
    open val transmissionType: String = "Manual"
}

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make, model) {
    override val transmissionType: String = "Automatic"
}
```

However, this is not good practice. Instead, you can add the property to the constructor of your inheritable class and declare its value when you create the `Car` child class:

```
open class Vehicle(val make: String, val model: String, val transmissionType: String = "Manu
al")

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make, model, "Autom
atic")
```

Accessing properties directly, instead of overriding them, leads to simpler and more readable code. By declaring properties once in the parent class and passing their values through the constructor, you eliminate the need for unnecessary overrides in child classes.

For more information about class inheritance and overriding class behavior, see Inheritance.

Open classes and interfaces

You can create a class that inherits a class **and** implements multiple interfaces. In this case, you must declare the parent class first, after the colon, before listing the interfaces:

```
// Define interfaces
interface EcoFriendly {
    val emissionLevel: String
}

interface ElectricVehicle {
    val batteryCapacity: Double
}

// Parent class
open class Vehicle(val make: String, val model: String)

// Child class
open class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make, model)
```

```
// New class that inherits from Car and implements two interfaces
class ElectricCar(
    make: String,
    model: String,
    numberOfDoors: Int,
    val capacity: Double,
    val emission: String
) : Car(make, model, numberOfDoors), EcoFriendly, ElectricVehicle {
    override val batteryCapacity: Double = capacity
    override val emissionLevel: String = emission
}
```

Special classes

In addition to abstract, open, and data classes, Kotlin has special types of classes designed for various purposes, such as restricting specific behavior or reducing the performance impact of creating small objects.

Sealed classes

There may be times when you want to restrict inheritance. You can do this with sealed classes. Sealed classes are a special type of abstract class. Once you declare that a class is sealed, you can only create child classes from it within the same package. It's not possible to inherit from the sealed class outside of this scope.

> **Tip:** A package is a collection of code with related classes and functions, typically within a directory. To learn more about packages in Kotlin, see Packages and imports.

To create a sealed class, use the `sealed` keyword:

```
sealed class Mammal
```

Sealed classes are particularly useful when combined with a `when` expression. By using a `when` expression, you can define the behavior for all possible child classes. For example:

```
sealed class Mammal(val name: String)

class Cat(val catName: String) : Mammal(catName)
class Human(val humanName: String, val job: String) : Mammal(humanName)

fun greetMammal(mammal: Mammal): String {
    when (mammal) {
        is Human -> return "Hello ${mammal.name}; You're working as a ${mammal.job}"
        is Cat -> return "Hello ${mammal.name}"
    }
}

fun main() {
    println(greetMammal(Cat("Snowy")))
```

```
    // Hello Snowy
}
```

In the example:

- There is a sealed class called `Mammal` that has the `name` parameter in the constructor.
- The `Cat` class inherits from the `Mammal` sealed class and uses the `catName` parameter in its own constructor as the `name` parameter from the `Mammal` class.
- The `Human` class inherits from the `Mammal` sealed class and uses the `humanName` parameter in its own constructor as the `name` parameter from the `Mammal` class. It also has the `job` parameter in its constructor.
- The `greetMammal()` function accepts an argument of `Mammal` type and returns a string.
- Within the `greetMammal()` function body, there's a `when` expression that uses the `is` operator to check the type of `mammal` and decide which action to perform.
- The `main()` function calls the `greetMammal()` function with an instance of the `Cat` class and `name` parameter called `Snowy`.

  This tour discusses the `is` operator in more detail in the Null safety chapter.

For more information about sealed classes and their recommended use cases, see Sealed classes and interfaces.

Enum classes

Enum classes are useful when you want to represent a finite set of distinct values in a class. An enum class contains enum constants, which are themselves instances of the enum class.

To create an enum class, use the `enum` keyword:

```
enum class State
```

Let's say that you want to create an enum class that contains the different states of a process. Each enum constant must be separated by a comma `,`:

```
enum class State {
    IDLE, RUNNING, FINISHED
}
```

The `State` enum class has enum constants: `IDLE`, `RUNNING`, and `FINISHED`. To access an enum constant, use the class name followed by a `.` and the name of the enum constant:

```
val state = State.RUNNING
```

You can use this enum class with a `when` expression to define the action to take depending on the value of the enum constant:

```
enum class State {
    IDLE, RUNNING, FINISHED
}

fun main() {
    val state = State.RUNNING
    val message = when (state) {
        State.IDLE -> "It's idle"
        State.RUNNING -> "It's running"
        State.FINISHED -> "It's finished"
    }
    println(message)
    // It's running
}
```

Enum classes can have properties and member functions just like normal classes.

For example, let's say you're working with HTML and you want to create an enum class containing some colors. You want each color to have a property, let's call it `rgb`, that contains their RGB value as a hexadecimal. When creating the enum constants, you must initialize it with this property:

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF),
    YELLOW(0xFFFF00)
}
```

> **Note:** Kotlin stores hexadecimals as integers, so the `rgb` property has the `Int` type, not the `String` type.

To add a member function to this class, separate it from the enum constants with a semicolon `;`:

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF),
    YELLOW(0xFFFF00);

    fun containsRed() = (this.rgb and 0xFF0000 != 0)
}

fun main() {
    val red = Color.RED

    // Calls containsRed() function on enum constant
    println(red.containsRed())
    // true

    // Calls containsRed() function on enum constants via class names
    println(Color.BLUE.containsRed())
    // false
```

```
    println(Color.YELLOW.containsRed())
    // true
}
```

In this example, the `containsRed()` member function accesses the value of the enum constant's `rgb` property using the `this` keyword and checks if the hexadecimal value contains `FF` as its first bits to return a boolean value.

For more information, see Enum classes.

Inline value classes

Sometimes in your code, you may want to create small objects from classes and use them only briefly. This approach can have a performance impact. Inline value classes are a special type of class that avoids this performance impact. However, they can only contain values.

To create an inline value class, use the `value` keyword and the `@JvmInline` annotation:

```
@JvmInline
value class Email
```

> **Tip:** The `@JvmInline` annotation instructs Kotlin to optimize the code when it is compiled. To learn more, see Annotations.

An inline value class **must** have a single property initialized in the class header.

Let's say that you want to create a class that collects an email address:

```
// The address property is initialized in the class header.
@JvmInline
value class Email(val address: String)

fun sendEmail(email: Email) {
    println("Sending email to ${email.address}")
}

fun main() {
    val myEmail = Email("example@example.com")
    sendEmail(myEmail)
    // Sending email to example@example.com
}
```

In the example:

- `Email` is an inline value class that has one property in the class header: `address`.
- The `sendEmail()` function accepts objects with type `Email` and prints a string to the standard output.
- The `main()` function:

- Creates an instance of the `Email` class called `myEmail`.
- Calls the `sendEmail()` function on the `myEmail` object.

By using an inline value class, you make the class inlined and can use it directly in your code without creating an object. This can significantly reduce memory footprint and improve your code's runtime performance.

For more information about inline value classes, see Inline value classes.

Practice

Model delivery statuses with a sealed class

You manage a delivery service and need a way to track the status of packages. Create a sealed class called `DeliveryStatus`, containing data classes to represent the following statuses: `Pending`, `InTransit`, `Delivered`, `Canceled`. Complete the `DeliveryStatus` class declaration so that the code in the `main()` function runs successfully:

```
sealed class // Write your code here

fun printDeliveryStatus(status: DeliveryStatus) {
    when (status) {
        is DeliveryStatus.Pending -> {
            println("The package is pending pickup from ${status.sender}.")
        }
        is DeliveryStatus.InTransit -> {
            println("The package is in transit and expected to arrive by ${status.estimatedD
eliveryDate}.")
        }
        is DeliveryStatus.Delivered -> {
            println("The package was delivered to ${status.recipient} on ${status.deliveryDa
te}.")
        }
        is DeliveryStatus.Canceled -> {
            println("The delivery was canceled due to: ${status.reason}.")
        }
    }
}

fun main() {
    val status1: DeliveryStatus = DeliveryStatus.Pending("Alice")
    val status2: DeliveryStatus = DeliveryStatus.InTransit("2024-11-20")
    val status3: DeliveryStatus = DeliveryStatus.Delivered("2024-11-18", "Bob")
    val status4: DeliveryStatus = DeliveryStatus.Canceled("Address not found")

    printDeliveryStatus(status1)
    // The package is pending pickup from Alice.
    printDeliveryStatus(status2)
    // The package is in transit and expected to arrive by 2024-11-20.
    printDeliveryStatus(status3)
    // The package was delivered to Bob on 2024-11-18.
```

```
    printDeliveryStatus(status4)
    // The delivery was canceled due to: Address not found.
}
```

```
sealed class DeliveryStatus {
    data class Pending(val sender: String) : DeliveryStatus()
    data class InTransit(val estimatedDeliveryDate: String) : DeliveryStatus()
    data class Delivered(val deliveryDate: String, val recipient: String) : DeliveryStatus()
    data class Canceled(val reason: String) : DeliveryStatus()
}

fun printDeliveryStatus(status: DeliveryStatus) {
    when (status) {
        is DeliveryStatus.Pending -> {
            println("The package is pending pickup from ${status.sender}.")
        }
        is DeliveryStatus.InTransit -> {
            println("The package is in transit and expected to arrive by ${status.estimatedD
eliveryDate}.")
        }
        is DeliveryStatus.Delivered -> {
            println("The package was delivered to ${status.recipient} on ${status.deliveryDa
te}.")
        }
        is DeliveryStatus.Canceled -> {
            println("The delivery was canceled due to: ${status.reason}.")
        }
    }
}

fun main() {
    val status1: DeliveryStatus = DeliveryStatus.Pending("Alice")
    val status2: DeliveryStatus = DeliveryStatus.InTransit("2024-11-20")
    val status3: DeliveryStatus = DeliveryStatus.Delivered("2024-11-18", "Bob")
    val status4: DeliveryStatus = DeliveryStatus.Canceled("Address not found")

    printDeliveryStatus(status1)
    // The package is pending pickup from Alice.
    printDeliveryStatus(status2)
    // The package is in transit and expected to arrive by 2024-11-20.
    printDeliveryStatus(status3)
    // The package was delivered to Bob on 2024-11-18.
    printDeliveryStatus(status4)
    // The delivery was canceled due to: Address not found.
}
```

Define problem types with an enum class

In your program, you want to be able to handle different statuses and types of errors. You have a sealed class to capture the different statuses which are declared in data classes or objects. Complete the code below by creating an enum class called `Problem` that represents the different problem types: `NETWORK`, `TIMEOUT`, and `UNKNOWN`.

```
sealed class Status {
    data object Loading : Status()
    data class Error(val problem: Problem) : Status() {
        // Write your code here
    }

    data class OK(val data: List<String>) : Status()
}

fun handleStatus(status: Status) {
    when (status) {
        is Status.Loading -> println("Loading...")
        is Status.OK -> println("Data received: ${status.data}")
        is Status.Error -> when (status.problem) {
            Status.Error.Problem.NETWORK -> println("Network issue")
            Status.Error.Problem.TIMEOUT -> println("Request timed out")
            Status.Error.Problem.UNKNOWN -> println("Unknown error occurred")
        }
    }
}

fun main() {
    val status1: Status = Status.Error(Status.Error.Problem.NETWORK)
    val status2: Status = Status.OK(listOf("Data1", "Data2"))

    handleStatus(status1)
    // Network issue
    handleStatus(status2)
    // Data received: [Data1, Data2]
}
```

```
sealed class Status {
    data object Loading : Status()
    data class Error(val problem: Problem) : Status() {
        enum class Problem {
            NETWORK,
            TIMEOUT,
            UNKNOWN
        }
    }

    data class OK(val data: List<String>) : Status()
}

fun handleStatus(status: Status) {
    when (status) {
        is Status.Loading -> println("Loading...")
        is Status.OK -> println("Data received: ${status.data}")
        is Status.Error -> when (status.problem) {
            Status.Error.Problem.NETWORK -> println("Network issue")
            Status.Error.Problem.TIMEOUT -> println("Request timed out")
            Status.Error.Problem.UNKNOWN -> println("Unknown error occurred")
        }
    }
}
```

```
fun main() {
    val status1: Status = Status.Error(Status.Error.Problem.NETWORK)
    val status2: Status = Status.OK(listOf("Data1", "Data2"))

    handleStatus(status1)
    // Network issue
    handleStatus(status2)
    // Data received: [Data1, Data2]
}
```

```
<a as="button" href="kotlin-tour-intermediate-objects.md" mode="outline" icon="arrow-left" i
con-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-properties.md" mode="classic" icon="arrow-righ
t" icon-position="right">Next step</a>
```

Properties

In the beginner tour, you learned how properties are used to declare characteristics of class instances and how to access them. This chapter digs deeper into how properties work in Kotlin and explores other ways that you can use them in your code.

Backing fields

In Kotlin, properties have default `get()` and `set()` functions, known as property accessors, which handle retrieving and modifying their values. While these default functions are not explicitly visible in the code, the compiler automatically generates them to manage property access behind the scenes. These accessors use a **backing field** to store the actual property value.

Backing fields exist if either of the following is true:

- You use the default `get()` or `set()` functions for the property.
- You try to access the property value in code by using the `field` keyword.

  Tip: `get()` and `set()` functions are also called getters and setters.

For example, this code has the `category` property that has no custom `get()` or `set()` functions and therefore uses the default implementations:

```
class Contact(val id: Int, var email: String) {
    var category: String = ""
}
```

Under the hood, this is equivalent to this pseudocode:

```
class Contact(val id: Int, var email: String) {
    var category: String = ""
        get() = field
        set(value) {
            field = value
        }
}
```

In this example:

- The `get()` function retrieves the property value from the field: `""`.
- The `set()` function accepts `value` as a parameter and assigns it to the field, where `value` is `""`.

Access to the backing field is useful when you want to add extra logic in your `get()` or `set()` functions without causing an infinite loop. For example, you have a `Person` class with a `name`

property:

```
class Person {
    var name: String = ""
}
```

You want to ensure that the first letter of the `name` property is capitalized, so you create a custom `set()` function that uses the `.replaceFirstChar()` and `.uppercase()` extension functions. However, if you refer to the property directly in your `set()` function, you create an infinite loop and see a `StackOverflowError` at runtime:

```
class Person {
    var name: String = ""
        set(value) {
            // This causes a runtime error
            name = value.replaceFirstChar { firstChar -> firstChar.uppercase() }
        }
}

fun main() {
    val person = Person()
    person.name = "kodee"
    println(person.name)
    // Exception in thread "main" java.lang.StackOverflowError
}
```

To fix this, you can use the backing field in your `set()` function instead by referencing it with the `field` keyword:

```
class Person {
    var name: String = ""
        set(value) {
            field = value.replaceFirstChar { firstChar -> firstChar.uppercase() }
        }
}

fun main() {
    val person = Person()
    person.name = "kodee"
    println(person.name)
    // Kodee
}
```

Backing fields are also useful when you want to add logging, send notifications when a property value changes, or use additional logic that compares the old and new property values.

For more information, see Backing fields.

Extension properties

Just like extension functions, there are also extension properties. Extension properties allow you to add new properties to existing classes without modifying their source code. However, extension properties in Kotlin do **not** have backing fields. This means that you need to write the `get()` and `set()` functions yourself. Additionally, the lack of a backing field means that they can't hold any state.

To declare an extension property, write the name of the class that you want to extend followed by a `.` and the name of your property. Just like with normal class properties, you need to declare a type for your property. For example:

```
val String.lastChar: Char
```

Extension properties are most useful when you want a property to contain a computed value without using inheritance. You can think of extension properties working like a function with only one parameter: the receiver.

For example, let's say that you have a data class called `Person` with two properties: `firstName` and `lastName`.

```
data class Person(val firstName: String, val lastName: String)
```

You want to be able to access the person's full name without modifying the `Person` data class or inheriting from it. You can do this by creating an extension property with a custom `get()` function:

```
data class Person(val firstName: String, val lastName: String)

// Extension property to get the full name
val Person.fullName: String
    get() = "$firstName $lastName"

fun main() {
    val person = Person(firstName = "John", lastName = "Doe")

    // Use the extension property
    println(person.fullName)
    // John Doe
}
```

> **Note:** Extension properties can't override existing properties of a class.

Just like with extension functions, the Kotlin standard library uses extension properties widely. For example, see the `lastIndex` property for a `CharSequence`.

Delegated properties

You already learned about delegation in the Classes and interfaces chapter. You can also use delegation with properties to delegate their property accessors to another object. This is useful when you have more complex requirements for storing properties that a simple backing field can't handle, such as storing values in a database table, browser session, or map. Using delegated properties also reduces boilerplate code because the logic for getting and setting your properties is contained only in the object that you delegate to.

The syntax is similar to using delegation with classes but operates on a different level. Declare your property, followed by the `by` keyword and the object you want to delegate to. For example:

```
val displayName: String by Delegate
```

Here, the delegated property `displayName` refers to the `Delegate` object for its property accessors.

Every object you delegate to **must** have a `getValue()` operator function, which Kotlin uses to retrieve the value of the delegated property. If the property is mutable, it must also have a `setValue()` operator function for Kotlin to set its value.

By default, the `getValue()` and `setValue()` functions have the following construction:

```
operator fun getValue(thisRef: Any?, property: KProperty<*>): String {}

operator fun setValue(thisRef: Any?, property: KProperty<*>, value: String) {}
```

In these functions:

- The `operator` keyword marks these functions as operator functions, enabling them to overload the `get()` and `set()` functions.
- The `thisRef` parameter refers to the object **containing** the delegated property. By default, the type is set to `Any?`, but you may need to declare a more specific type.
- The `property` parameter refers to the property whose value is accessed or changed. You can use this parameter to access information like the property's name or type. By default, the type is set to `KProperty<*>` but you can also use `Any?`. You don't need to worry about changing this in your code.

The `getValue()` function has a return type of `String` by default, but you can adjust this if you want.

The `setValue()` function has an additional parameter `value`, which is used to hold the new value that's assigned to the property.

So, how does this look in practice? Suppose you want to have a computed property, like a user's display name, that is calculated only once because the operation is expensive and your application

is performance-sensitive. You can use a delegated property to cache the display name so that it is only computed once but can be accessed anytime without performance impact.

First, you need to create the object to delegate to. In this case, the object will be an instance of the `CachedStringDelegate` class:

```
class CachedStringDelegate {
    var cachedValue: String? = null
}
```

The `cachedValue` property contains the cached value. Within the `CachedStringDelegate` class, add the behavior that you want from the `get()` function of the delegated property to the `getValue()` operator function body:

```
class CachedStringDelegate {
    var cachedValue: String? = null

    operator fun getValue(thisRef: Any?, property: Any?): String {
        if (cachedValue == null) {
            cachedValue = "Default Value"
            println("Computed and cached: $cachedValue")
        } else {
            println("Accessed from cache: $cachedValue")
        }
        return cachedValue ?: "Unknown"
    }
}
```

The `getValue()` function checks whether the `cachedValue` property is `null`. If it is, the function assigns the `"Default value"` and prints a string for logging purposes. If the `cachedValue` property has already been computed, the property isn't `null`. In this case, another string is printed for logging purposes. Finally, the function uses the Elvis operator to return the cached value or `"Unknown"` if the value is `null`.

Now you can delegate the property that you want to cache (`val displayName`) to an instance of the `CachedStringDelegate` class:

```
class CachedStringDelegate {
    var cachedValue: String? = null

    operator fun getValue(thisRef: User, property: Any?): String {
        if (cachedValue == null) {
            cachedValue = "${thisRef.firstName} ${thisRef.lastName}"
            println("Computed and cached: $cachedValue")
        } else {
            println("Accessed from cache: $cachedValue")
        }
        return cachedValue ?: "Unknown"
    }
}
```

```
class User(val firstName: String, val lastName: String) {
    val displayName: String by CachedStringDelegate()
}

fun main() {
    val user = User("John", "Doe")

    // First access computes and caches the value
    println(user.displayName)
    // Computed and cached: John Doe
    // John Doe

    // Subsequent accesses retrieve the value from cache
    println(user.displayName)
    // Accessed from cache: John Doe
    // John Doe
}
```

This example:

- Creates a `User` class that has two properties in the header, `firstName`, and `lastName`, and one property in the class body, `displayName`.
- Delegates the `displayName` property to an instance of the `CachedStringDelegate` class.
- Creates an instance of the `User` class called `user`.
- Prints the result of accessing the `displayName` property on the `user` instance.

Note that in the `getValue()` function, the type for the `thisRef` parameter is narrowed from `Any?` type to the object type: `User`. This is so that the compiler can access the `firstName` and `lastName` properties of the `User` class.

Standard delegates

The Kotlin standard library provides some useful delegates for you so you don't have to always create yours from scratch. If you use one of these delegates, you don't need to define `getValue()` and `setValue()` functions because the standard library automatically provides them.

Lazy properties

To initialize a property only when it's first accessed, use a lazy property. The standard library provides the `Lazy` interface for delegation.

To create an instance of the `Lazy` interface, use the `lazy()` function by providing it with a lambda expression to execute when the `get()` function is called for the first time. Any further calls of the `get()` function return the same result that was provided on the first call. Lazy properties use the trailing lambda syntax to pass the lambda expression.

For example:

```
class Database {
    fun connect() {
        println("Connecting to the database...")
    }

    fun query(sql: String): List<String> {
        return listOf("Data1", "Data2", "Data3")
    }
}

val databaseConnection: Database by lazy {
    val db = Database()
    db.connect()
    db
}

fun fetchData() {
    val data = databaseConnection.query("SELECT * FROM data")
    println("Data: $data")
}

fun main() {
    // First time accessing databaseConnection
    fetchData()
    // Connecting to the database...
    // Data: [Data1, Data2, Data3]

    // Subsequent access uses the existing connection
    fetchData()
    // Data: [Data1, Data2, Data3]
}
```

In this example:

- There is a `Database` class with `connect()` and `query()` member functions.
- The `connect()` function prints a string to the console, and the `query()` function accepts an SQL query and returns a list.
- There is a `databaseConnection` property that is a lazy property.
- The lambda expression provided to the `lazy()` function:
    - Creates an instance of the `Database` class.
    - Calls the `connect()` member function on this instance (`db`).
    - Returns the instance.
- There is a `fetchData()` function that:
    - Creates an SQL query by calling the `query()` function on the `databaseConnection` property.
    - Assigns the SQL query to the `data` variable.
    - Prints the `data` variable to the console.
- The `main()` function calls the `fetchData()` function. The first time it is called, the lazy property is initialized. The second time, the same result is returned as the first call.

Lazy properties are useful not only when initialization is resource-intensive but also when a property might not be used in your code. Additionally, lazy properties are thread-safe by default, which is particularly beneficial if you are working in a concurrent environment.

For more information, see Lazy properties.

Observable properties

To monitor whether the value of a property changes, use an observable property. An observable property is useful when you want to detect a change in the property value and use this knowledge to trigger a reaction. The standard library provides the `Delegates` object for delegation.

To create an observable property, you must first import `kotlin.properties.Delegates.observable`. Then, use the `observable()` function and provide it with a lambda expression to execute whenever the property changes. Just like with lazy properties, observable properties use the trailing lambda syntax to pass the lambda expression.

For example:

```
import kotlin.properties.Delegates.observable

class Thermostat {
    var temperature: Double by observable(20.0) { _, old, new ->
        if (new > 25) {
            println("Warning: Temperature is too high! ($old°C -> $new°C)")
        } else {
            println("Temperature updated: $old°C -> $new°C")
        }
    }
}

fun main() {
    val thermostat = Thermostat()
    thermostat.temperature = 22.5
    // Temperature updated: 20.0°C -> 22.5°C

    thermostat.temperature = 27.0
    // Warning: Temperature is too high! (22.5°C -> 27.0°C)
}
```

In this example:

- There is a `Thermostat` class that contains an observable property: `temperature`.
- The `observable()` function accepts `20.0` as a parameter and uses it to initialize the property.
- The lambda expression provided to the `observable()` function:
    - Has three parameters:
        - `_`, which refers to the property itself.
        - `old`, which is the old value of the property.
        - `new`, which is the new value of the property.

    - Checks if the `new` parameter is greater than `25` and, depending on the result, prints a string to console.
- The `main()` function:
    - Creates an instance of the `Thermostat` class called `thermostat`.
    - Updates the value of the `temperature` property of the instance to `22.5`, which triggers a print statement with a temperature update.
    - Updates the value of the `temperature` property of the instance to `27.0`, which triggers a print statement with a warning.

Observable properties are useful not only for logging and debugging purposes. You can also use them for use cases like updating a UI or to perform additional checks, like verifying the validity of data.

For more information, see Observable properties.

Practice

Find out-of-stock books

You manage an inventory system at a bookstore. The inventory is stored in a list where each item represents the quantity of a specific book. For example, `listOf(3, 0, 7, 12)` means the store has 3 copies of the first book, 0 of the second, 7 of the third, and 12 of the fourth.

Write a function called `findOutOfStockBooks()` that returns a list of indices for all the books that are out of stock.

Hint 1


```
    Use the <code>indices</code> extension property from the standard library.
```


Hint 2


```
    You can use the <code>buildList()</code> function to create and manage a list instead of
manually creating and returning a mutable list. The <code>buildList()</code> function uses a
lambda with a receiver, which you learned about in earlier chapters.
```


```
fun findOutOfStockBooks(inventory: List<Int>): List<Int> {
    // Write your code here
}

fun main() {
    val inventory = listOf(3, 0, 7, 0, 5)
    println(findOutOfStockBooks(inventory))
    // [1, 3]
}
```

```
fun findOutOfStockBooks(inventory: List<Int>): List<Int> {
    val outOfStockIndices = mutableListOf<Int>()
    for (index in inventory.indices) {
        if (inventory[index] == 0) {
            outOfStockIndices.add(index)
        }
    }
    return outOfStockIndices
}

fun main() {
    val inventory = listOf(3, 0, 7, 0, 5)
    println(findOutOfStockBooks(inventory))
    // [1, 3]
}
```

```
fun findOutOfStockBooks(inventory: List<Int>): List<Int> = buildList {
    for (index in inventory.indices) {
        if (inventory[index] == 0) {
            add(index)
        }
    }
}

fun main() {
    val inventory = listOf(3, 0, 7, 0, 5)
    println(findOutOfStockBooks(inventory))
    // [1, 3]
}
```

Convert kilometers to miles

You have a travel app that needs to display distances in both kilometers and miles. Create an extension property for the `Double` type called `asMiles` to convert a distance in kilometers to miles:

> **Note:** The formula to convert kilometers to miles is `miles = kilometers * 0.621371`.

Hint

```
    Remember that extension properties need a custom <code>get()</code> function.
```

```
val // Write your code here

fun main() {
    val distanceKm = 5.0
    println("$distanceKm km is ${distanceKm.asMiles} miles")
    // 5.0 km is 3.106855 miles

    val marathonDistance = 42.195
    println("$marathonDistance km is ${marathonDistance.asMiles} miles")
    // 42.195 km is 26.218757 miles
}
```

```
val Double.asMiles: Double
    get() = this * 0.621371

fun main() {
    val distanceKm = 5.0
    println("$distanceKm km is ${distanceKm.asMiles} miles")
    // 5.0 km is 3.106855 miles

    val marathonDistance = 42.195
    println("$marathonDistance km is ${marathonDistance.asMiles} miles")
    // 42.195 km is 26.218757 miles
}
```

Initialize health checks lazily

You have a system health checker that can determine the state of a cloud system. However, the two functions it can run to perform a health check are performance intensive. Use lazy properties to initialize the checks so that the expensive functions are only run when needed:

```
fun checkAppServer(): Boolean {
    println("Performing application server health check...")
    return true
}

fun checkDatabase(): Boolean {
    println("Performing database health check...")
    return false
}

fun main() {
    // Write your code here

    when {
        isAppServerHealthy -> println("Application server is online and healthy")
        isDatabaseHealthy -> println("Database is healthy")
        else -> println("System is offline")
    }
    // Performing application server health check...
    // Application server is online and healthy
}
```

```
fun checkAppServer(): Boolean {
    println("Performing application server health check...")
    return true
}

fun checkDatabase(): Boolean {
    println("Performing database health check...")
    return false
}

fun main() {
    val isAppServerHealthy by lazy { checkAppServer() }
```

```
    val isDatabaseHealthy by lazy { checkDatabase() }

    when {
        isAppServerHealthy -> println("Application server is online and healthy")
        isDatabaseHealthy -> println("Database is healthy")
        else -> println("System is offline")
    }
   // Performing application server health check...
   // Application server is online and healthy
}
```

Track budget changes

You're building a simple budget tracker app. The app needs to observe changes to the user's remaining budget and notify them whenever it goes below a certain threshold. You have a `Budget` class that is initialized with a `totalBudget` property that contains the initial budget amount. Within the class, create an observable property called `remainingBudget` that prints:

- A warning when the value is lower than 20% of the initial budget.
- An encouraging message when the budget is increased from the previous value.

```
import kotlin.properties.Delegates.observable

class Budget(val totalBudget: Int) {
    var remainingBudget: Int // Write your code here
}

fun main() {
    val myBudget = Budget(totalBudget = 1000)
    myBudget.remainingBudget = 800
    myBudget.remainingBudget = 150
    // Warning: Your remaining budget (150) is below 20% of your total budget.
    myBudget.remainingBudget = 50
    // Warning: Your remaining budget (50) is below 20% of your total budget.
    myBudget.remainingBudget = 300
    // Good news: Your remaining budget increased to 300.
}
```

```
import kotlin.properties.Delegates.observable

class Budget(val totalBudget: Int) {
    var remainingBudget: Int by observable(totalBudget) { _, oldValue, newValue ->
        if (newValue < totalBudget * 0.2) {
            println("Warning: Your remaining budget ($newValue) is below 20% of your total b
udget.")
        } else if (newValue > oldValue) {
            println("Good news: Your remaining budget increased to $newValue.")
        }
    }
}

fun main() {
    val myBudget = Budget(totalBudget = 1000)
```

```
    myBudget.remainingBudget = 800
    myBudget.remainingBudget = 150
    // Warning: Your remaining budget (150) is below 20% of your total budget.
    myBudget.remainingBudget = 50
    // Warning: Your remaining budget (50) is below 20% of your total budget.
    myBudget.remainingBudget = 300
    // Good news: Your remaining budget increased to 300.
}
```

```
<a as="button" href="kotlin-tour-intermediate-open-special-classes.md" mode="outline" icon
="arrow-left" icon-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-null-safety.md" mode="classic" icon="arrow-rig
ht" icon-position="right">Next step</a>
```

Null safety

In the beginner tour, you learned how to handle `null` values in your code. This chapter covers common use cases for null safety features and how to make the most of them.

Smart casts and safe casts

Kotlin can sometimes infer the type without explicit declaration. When you tell Kotlin to treat a variable or object as if it belongs to a specific type, this process is called **casting**. When a type is automatically cast, like when it's inferred, it's called **smart casting**.

is and !is operators

Before we explore how casting works, let's see how you can check if an object has a certain type. For this, you can use the `is` and `!is` operators with `when` or `if` conditional expressions:

- `is` checks if the object has the type and returns a boolean value.
- `!is` checks if the object **doesn't** have the type and returns a boolean value.

For example:

```
fun printObjectType(obj: Any) {
    when (obj) {
        is Int -> println("It's an Integer with value $obj")
        !is Double -> println("It's NOT a Double")
        else -> println("Unknown type")
    }
}

fun main() {
    val myInt = 42
    val myDouble = 3.14
    val myList = listOf(1, 2, 3)

    // The type is Int
    printObjectType(myInt)
    // It's an Integer with value 42

    // The type is List, so it's NOT a Double.
    printObjectType(myList)
    // It's NOT a Double

    // The type is Double, so the else branch is triggered.
    printObjectType(myDouble)
    // Unknown type
}
```

Tip: You've already seen an example of how to use a `when` conditional expression with the `is` and `!is` operators in the Open and other special classes chapter.

as and as? operators

To explicitly *cast* an object to any other type, use the `as` operator. This includes casting from a nullable type to its non-nullable counterpart. If the cast isn't possible, the program crashes **at runtime**. This is why it's called the **unsafe** cast operator.

```
fun main() {
    val a: String? = null
    val b = a as String

    // Triggers an error at runtime
    print(b)
}
```

To explicitly cast an object to a non-nullable type, but return `null` instead of throwing an error on failure, use the `as?` operator. Since the `as?` operator doesn't trigger an error on failure, it is called the **safe** operator.

```
fun main() {
    val a: String? = null
    val b = a as? String

    // Returns null value
    print(b)
    // null
}
```

You can combine the `as?` operator with the Elvis operator `?:` to reduce several lines of code down to one. For example, the following `calculateTotalStringLength()` function calculates the total length of all strings provided in a mixed list:

```
fun calculateTotalStringLength(items: List<Any>): Int {
    var totalLength = 0

    for (item in items) {
        totalLength += if (item is String) {
            item.length
        } else {
            0  // Add 0 for non-String items
        }
    }

    return totalLength
}
```

The example:

- Uses the `totalLength` variable as a counter.
- Uses a `for` loop to loop through every item in the list.
- Uses an `if` and the `is` operator to check if the current item is a string:
    - If it is, the string's length is added to the counter.
    - If it is not, the counter isn't incremented.
- Returns the final value of the `totalLength` variable.

This code can be reduced to:

```
fun calculateTotalStringLength(items: List<Any>): Int {
    return items.sumOf { (it as? String)?.length ?: 0 }
}
```

The example uses the `.sumOf()` extension function and provides a lambda expression that:

- For each item in the list, performs a safe cast to `String` using `as?`.
- Uses a safe call `?.` to access the `length` property if the call doesn't return a `null` value.
- Uses the Elvis operator `?:` to return `0` if the safe call returns a `null` value.

Null values and collections

In Kotlin, working with collections often involves handling `null` values and filtering out unnecessary elements. Kotlin has useful functions that you can use to write clean, efficient, and null-safe code when working with lists, sets, maps, and other types of collections.

To filter `null` values from a list, use the `filterNotNull()` function:

```
fun main() {
    val emails: List<String?> = listOf("alice@example.com", null, "bob@example.com", null,
"carol@example.com")

    val validEmails = emails.filterNotNull()

    println(validEmails)
    // [alice@example.com, bob@example.com, carol@example.com]
}
```

If you want to perform filtering of `null` values directly when creating a list, use the `listOfNotNull()` function:

```
fun main() {
    val serverConfig = mapOf(
        "appConfig.json" to "App Configuration",
        "dbConfig.json" to "Database Configuration"
    )

    val requestedFile = "appConfig.json"
    val configFiles = listOfNotNull(serverConfig[requestedFile])
```

```
    println(configFiles)
    // [App Configuration]
}
```

In both of these examples, if all items are `null` values, an empty list is returned.

Kotlin also provides functions that you can use to find values in collections. If a value isn't found, they return `null` values instead of triggering an error:

- `maxOrNull()` finds the highest value. If one doesn't exist, returns a `null` value.
- `minOrNull()` finds the lowest value. If one doesn't exist, returns a `null` value.

For example:

```
fun main() {
    // Temperatures recorded over a week
    val temperatures = listOf(15, 18, 21, 21, 19, 17, 16)

    // Find the highest temperature of the week
    val maxTemperature = temperatures.maxOrNull()
    println("Highest temperature recorded: ${maxTemperature ?: "No data"}")
    // Highest temperature recorded: 21

    // Find the lowest temperature of the week
    val minTemperature = temperatures.minOrNull()
    println("Lowest temperature recorded: ${minTemperature ?: "No data"}")
    // Lowest temperature recorded: 15
}
```

This example uses the Elvis operator `?:` to return a printed statement if the functions return a `null` value.

> **Note:** The `maxOrNull()`, and `minOrNull()` functions are designed to be used with collections that **don't** contain `null` values. Otherwise, you can't tell whether the function couldn't find the desired value or whether it found a `null` value.

You can use the `singleOrNull()` function with a lambda expression to find a single item that matches a condition. If one doesn't exist or there are multiple items that match, the function returns a `null` value:

```
fun main() {
    // Temperatures recorded over a week
    val temperatures = listOf(15, 18, 21, 21, 19, 17, 16)

    // Check if there was exactly one day with 30 degrees
    val singleHotDay = temperatures.singleOrNull{ it == 30 }
    println("Single hot day with 30 degrees: ${singleHotDay ?: "None"}")
    // Single hot day with 30 degrees: None
}
```

> **Note:** The `singleOrNull()` function is designed to be used with collections that **don't** contain `null` values.

Some functions use a lambda expression to transform a collection and return `null` values if they can't fulfill their purpose.

To transform a collection with a lambda expression and return the first value that isn't `null`, use the `firstNotNullOfOrNull()` function. If no such value exists, the function returns a `null` value:

```
fun main() {
    data class User(val name: String?, val age: Int?)

    val users = listOf(
        User(null, 25),
        User("Alice", null),
        User("Bob", 30)
    )

    val firstNonNullName = users.firstNotNullOfOrNull { it.name }
    println(firstNonNullName)
    // Alice
}
```

To use a lambda expression to process each collection item sequentially and create an accumulated value (or return a `null` value if the collection is empty) use the `reduceOrNull()` function:

```
fun main() {
    // Prices of items in a shopping cart
    val itemPrices = listOf(20, 35, 15, 40, 10)

    // Calculate the total price using the reduceOrNull() function
    val totalPrice = itemPrices.reduceOrNull { runningTotal, price -> runningTotal + price }
    println("Total price of items in the cart: ${totalPrice ?: "No items"}")
    // Total price of items in the cart: 120

    val emptyCart = listOf<Int>()
    val emptyTotalPrice = emptyCart.reduceOrNull { runningTotal, price -> runningTotal + pri
ce }
    println("Total price of items in the empty cart: ${emptyTotalPrice ?: "No items"}")
    // Total price of items in the empty cart: No items
}
```

This example also uses the Elvis operator `?:` to return a printed statement if the function returns a `null` value.

> **Note:** The `reduceOrNull()` function is designed to be used with collections that **don't** contain `null` values.

Explore Kotlin's standard library to find more functions that you can use to make your code safer.

Early returns and the Elvis operator

In the beginner tour, you learned how to use early returns to stop your function from being processed further than a certain point. You can use the Elvis operator `?:` with an early return to check preconditions in a function. This approach is a great way to keep your code concise because you don't need to use nested checks. The reduced complexity of your code also makes it easier to maintain. For example:

```
data class User(
    val id: Int,
    val name: String,
    // List of friend user IDs
    val friends: List<Int>
)

// Function to get the number of friends for a user
fun getNumberOfFriends(users: Map<Int, User>, userId: Int): Int {
    // Retrieves the user or return -1 if not found
    val user = users[userId] ?: return -1
    // Returns the number of friends
    return user.friends.size
}

fun main() {
    // Creates some sample users
    val user1 = User(1, "Alice", listOf(2, 3))
    val user2 = User(2, "Bob", listOf(1))
    val user3 = User(3, "Charlie", listOf(1))

    // Creates a map of users
    val users = mapOf(1 to user1, 2 to user2, 3 to user3)

    println(getNumberOfFriends(users, 1))
    // 2
    println(getNumberOfFriends(users, 2))
    // 1
    println(getNumberOfFriends(users, 4))
    // -1
}
```

In this example:

- There is a `User` data class that has properties for the user's `id`, `name` and list of friends.
- The `getNumberOfFriends()` function:
    - Accepts a map of `User` instances and a user ID as an integer.
    - Accesses the value of the map of `User` instances with the provided user ID.
    - Uses an Elvis operator to return the function early with the value of `-1` if the map value is a `null` value.
    - Assigns the value found from the map to the `user` variable.
    - Returns the number of friends in the user's friends list by using the `size` property.

- The `main()` function:
    - Creates three `User` instances.
    - Creates a map of these `User` instances and assigns them to the `users` variable.
    - Calls the `getNumberOfFriends()` function on the `users` variable with values `1` and `2` that returns two friends for `"Alice"` and one friend for `"Bob"`.
    - Calls the `getNumberOfFriends()` function on the `users` variable with value `4`, which triggers an early return with a value of `-1`.

You may notice that the code could be more concise without an early return. However, this approach needs multiple safe calls because the `users[userId]` might return a `null` value, making the code slightly harder to read:

```
fun getNumberOfFriends(users: Map<Int, User>, userId: Int): Int {
    // Retrieve the user or return -1 if not found
    return users[userId]?.friends?.size ?: -1
}
```

Although this example checks only one condition with the Elvis operator, you can add multiple checks to cover any critical error paths. Early returns with the Elvis operator prevent your program from doing unnecessary work and make your code safer by stopping as soon as a `null` value or invalid case is detected.

For more information about how you can use `return` in your code, see Returns and jumps.

Practice

Validate user data with safe casts and the Elvis operator

You are developing a notification system for an app where users can enable or disable different types of notifications. Complete the `getNotificationPreferences()` function so that:

1. The `validUser` variable uses the `as?` operator to check if `user` is an instance of the `User` class. If it isn't, return an empty list.
2. The `userName` variable uses the Elvis `?:` operator to ensure that the user's name defaults to `"Guest"` if it is `null`.
3. The final return statement uses the `.takeIf()` function to include email and SMS notification preferences only if they are enabled.
4. The `main()` function runs successfully and prints the expected output.

   The `takeIf()` function returns the original value if the given condition is true, otherwise it returns `null`. For example:

   ```
   fun main() {
       // The user is logged in
       val userIsLoggedIn = true
   ```

```
    // The user has an active session
    val hasSession = true

    // Gives access to the dashboard if the user is logged in
    // and has an active session
    val canAccessDashboard = userIsLoggedIn.takeIf { hasSession }

    println(canAccessDashboard ?: "Access denied")
    // true
}
```

```
data class User(val name: String?)

fun getNotificationPreferences(user: Any, emailEnabled: Boolean, smsEnabled: Boolean): List<
String> {
    val validUser = // Write your code here
    val userName = // Write your code here

    return listOfNotNull( /* Write your code here */)
}

fun main() {
    val user1 = User("Alice")
    val user2 = User(null)
    val invalidUser = "NotAUser"

    println(getNotificationPreferences(user1, emailEnabled = true, smsEnabled = false))
    // [Email Notifications enabled for Alice]
    println(getNotificationPreferences(user2, emailEnabled = false, smsEnabled = true))
    // [SMS Notifications enabled for Guest]
    println(getNotificationPreferences(invalidUser, emailEnabled = true, smsEnabled = true))
    // []
}
```

```
data class User(val name: String?)

fun getNotificationPreferences(user: Any, emailEnabled: Boolean, smsEnabled: Boolean): List<
String> {
    val validUser = user as? User ?: return emptyList()
    val userName = validUser.name ?: "Guest"

    return listOfNotNull(
        "Email Notifications enabled for $userName".takeIf { emailEnabled },
        "SMS Notifications enabled for $userName".takeIf { smsEnabled }
    )
}

fun main() {
    val user1 = User("Alice")
    val user2 = User(null)
    val invalidUser = "NotAUser"

    println(getNotificationPreferences(user1, emailEnabled = true, smsEnabled = false))
    // [Email Notifications enabled for Alice]
    println(getNotificationPreferences(user2, emailEnabled = false, smsEnabled = true))
```

```
    // [SMS Notifications enabled for Guest]
    println(getNotificationPreferences(invalidUser, emailEnabled = true, smsEnabled = true))
    // []
}
```

Find a single active subscription with singleOrNull()

You are working on a subscription-based streaming service where users can have multiple subscriptions, but **only one can be active at a time**. Complete the `getActiveSubscription()` function so that it uses the `singleOrNull()` function with a predicate to return a `null` value if there is more than one active subscription:

```
data class Subscription(val name: String, val isActive: Boolean)

fun getActiveSubscription(subscriptions: List<Subscription>): Subscription? // Write your co
de here

fun main() {
    val userWithPremiumPlan = listOf(
        Subscription("Basic Plan", false),
        Subscription("Premium Plan", true)
    )

    val userWithConflictingPlans = listOf(
        Subscription("Basic Plan", true),
        Subscription("Premium Plan", true)
    )

    println(getActiveSubscription(userWithPremiumPlan))
    // Subscription(name=Premium Plan, isActive=true)

    println(getActiveSubscription(userWithConflictingPlans))
    // null
}
```

```
data class Subscription(val name: String, val isActive: Boolean)

fun getActiveSubscription(subscriptions: List<Subscription>): Subscription? {
    return subscriptions.singleOrNull { subscription -> subscription.isActive }
}

fun main() {
    val userWithPremiumPlan = listOf(
        Subscription("Basic Plan", false),
        Subscription("Premium Plan", true)
    )

    val userWithConflictingPlans = listOf(
        Subscription("Basic Plan", true),
        Subscription("Premium Plan", true)
    )

    println(getActiveSubscription(userWithPremiumPlan))
```

```
    // Subscription(name=Premium Plan, isActive=true)

    println(getActiveSubscription(userWithConflictingPlans))
    // null
}
```

```
data class Subscription(val name: String, val isActive: Boolean)

fun getActiveSubscription(subscriptions: List<Subscription>): Subscription? =
    subscriptions.singleOrNull { it.isActive }

fun main() {
    val userWithPremiumPlan = listOf(
        Subscription("Basic Plan", false),
        Subscription("Premium Plan", true)
    )

    val userWithConflictingPlans = listOf(
        Subscription("Basic Plan", true),
        Subscription("Premium Plan", true)
    )

    println(getActiveSubscription(userWithPremiumPlan))
    // Subscription(name=Premium Plan, isActive=true)

    println(getActiveSubscription(userWithConflictingPlans))
    // null
}
```

Filter active usernames with mapNotNull()

You are working on a social media platform where users have usernames and account statuses. You want to see the list of currently active usernames. Complete the `getActiveUsernames()` function so that the `mapNotNull()` function has a predicate that returns the username if it is active or a `null` value if it isn't:

```
data class User(val username: String, val isActive: Boolean)

fun getActiveUsernames(users: List<User>): List<String> {
    return users.mapNotNull { /* Write your code here */ }
}

fun main() {
    val allUsers = listOf(
        User("alice123", true),
        User("bob_the_builder", false),
        User("charlie99", true)
    )

    println(getActiveUsernames(allUsers))
    // [alice123, charlie99]
}
```

Just like in Exercise 1, you can use the `takeIf()` function when you check if the user is active.

```
data class User(val username: String, val isActive: Boolean)

fun getActiveUsernames(users: List<User>): List<String> {
    return users.mapNotNull { user ->
        if (user.isActive) user.username else null
    }
}

fun main() {
    val allUsers = listOf(
        User("alice123", true),
        User("bob_the_builder", false),
        User("charlie99", true)
    )

    println(getActiveUsernames(allUsers))
    // [alice123, charlie99]
}
```

```
data class User(val username: String, val isActive: Boolean)

fun getActiveUsernames(users: List<User>): List<String> =
    users.mapNotNull { user -> user.username.takeIf { user.isActive } }

fun main() {
    val allUsers = listOf(
        User("alice123", true),
        User("bob_the_builder", false),
        User("charlie99", true)
    )

    println(getActiveUsernames(allUsers))
    // [alice123, charlie99]
}
```

Validate stock with early returns and the Elvis operator

You are working on an inventory management system for an e-commerce platform. Before processing a sale, you need to check if the requested quantity of a product is valid based on the available stock.

Complete the `validateStock()` function so that it uses early returns and the Elvis operator (where applicable) to check if:

- The `requested` variable is `null`.
- The `available` variable is `null`.
- The `requested` variable is a negative value.
- The amount in the `requested` variable is higher than in the `available` variable.

In all of the above cases, the function must return early with the value of `-1`.

```
fun validateStock(requested: Int?, available: Int?): Int {
    // Write your code here
}

fun main() {
    println(validateStock(5,10))
    // 5
    println(validateStock(null,10))
    // -1
    println(validateStock(-2,10))
    // -1
}
```

```
fun validateStock(requested: Int?, available: Int?): Int {
    val validRequested = requested ?: return -1
    val validAvailable = available ?: return -1

    if (validRequested < 0) return -1
    if (validRequested > validAvailable) return -1

    return validRequested
}

fun main() {
    println(validateStock(5,10))
    // 5
    println(validateStock(null,10))
    // -1
    println(validateStock(-2,10))
    // -1
}
```

```
<a as="button" href="kotlin-tour-intermediate-properties.md" mode="outline" icon="arrow-lef
t" icon-position="left">Previous step</a>



<a as="button" href="kotlin-tour-intermediate-libraries-and-apis.md" mode="classic" icon="ar
row-right" icon-position="right">Next step</a>
```

Libraries and APIs

To get the most out of Kotlin, use existing libraries and APIs so you can spend more time coding and less time reinventing the wheel.

Libraries distribute reusable code that simplifies common tasks. Within libraries, there are packages and objects that group related classes, functions, and utilities. Libraries expose APIs (Application Programming Interfaces) as a set of functions, classes, or properties that developers can use in their code.

!Kotlin libraries and APIs

Let's explore what's possible with Kotlin.

The standard library

Kotlin has a standard library that provides essential types, functions, collections, and utilities to make your code concise and expressive. A large portion of the standard library (everything in the `kotlin` package) is readily available in any Kotlin file without the need to import it explicitly:

```
fun main() {
    val text = "emosewa si niltoK"

   // Use the reversed() function from the standard library
    val reversedText = text.reversed()

    // Use the print() function from the standard library
    print(reversedText)
    // Kotlin is awesome
}
```

However, some parts of the standard library require an import before you can use them in your code. For example, if you want to use the standard library's time measurement features, you need to import the `kotlin.time` package.

At the top of your file, add the `import` keyword followed by the package that you need:

```
import kotlin.time.*
```

The asterisk `*` is a wildcard import that tells Kotlin to import everything within the package. You can't use the asterisk `*` with companion objects. Instead, you need to explicitly declare the members of a companion object that you want to use.

For example:

```
import kotlin.time.Duration
import kotlin.time.Duration.Companion.hours
import kotlin.time.Duration.Companion.minutes

fun main() {
    val thirtyMinutes: Duration = 30.minutes
    val halfHour: Duration = 0.5.hours
    println(thirtyMinutes == halfHour)
    // true
}
```

This example:

- Imports the `Duration` class and the `hours` and `minutes` extension properties from its companion object.
- Uses the `minutes` property to convert `30` into a `Duration` of 30 minutes.
- Uses the `hours` property to convert `0.5` into a `Duration` of 30 minutes.
- Checks if both durations are equal and prints the result.

Search before you build

Before you decide to write your own code, check the standard library to see if what you're looking for already exists. Here's a list of areas where the standard library already provides a number of classes, functions, and properties for you:

- Collections
- Sequences
- String manipulation
- Time management

To learn more about what else is in the standard library, explore its API reference.

Kotlin libraries

The standard library covers many common use cases, but there are some that it doesn't address. Fortunately, the Kotlin team and the rest of the community have developed a wide range of libraries to complement the standard library. For example, `kotlinx-datetime` helps you manage time across different platforms.

You can find useful libraries on our search platform. To use them, you need to take extra steps, like adding a dependency or plugin. Each library has a GitHub repository with instructions on how to include it in your Kotlin projects.

Once you add the library, you can import any package within it. Here's an example of how to import the `kotlinx-datetime` package to find the current time in New York:

```
import kotlinx.datetime.*

fun main() {
    val now = Clock.System.now() // Get current instant
    println("Current instant: $now")

    val zone = TimeZone.of("America/New_York")
    val localDateTime = now.toLocalDateTime(zone)
    println("Local date-time in NY: $localDateTime")
}
```

This example:

- Imports the `kotlinx.datetime` package.
- Uses the `Clock.System.now()` function to create an instance of the `Instant` class that contains the current time and assigns the result to the `now` variable.
- Prints the current time.
- Uses the `TimeZone.of()` function to find the time zone for New York and assigns the result to the `zone` variable.
- Calls the `.toLocalDateTime()` function on the instance containing the current time, with the New York time zone as an argument.
- Assigns the result to the `localDateTime` variable.
- Prints the time adjusted for the time zone in New York.

  Tip: To explore the functions and classes that this example uses in more detail, see the API reference.

Opt in to APIs

Library authors may mark certain APIs as requiring opt-in before you can use them in your code. They usually do this when an API is still in development and may change in the future. If you don't opt in, you see warnings or errors like this:

```
This declaration needs opt-in. Its usage should be marked with '@...' or '@OptIn(...)'
```

To opt in, write `@OptIn` followed by parentheses containing the class name that categorizes the API, appended by two colons `::` and `class`.

For example, the `uintArrayOf()` function from the standard library falls under `@ExperimentalUnsignedTypes`, as shown in the API reference:

```
@ExperimentalUnsignedTypes
inline fun uintArrayOf(vararg elements: UInt): UIntArray
```

In your code, the opt-in looks like:

```
@OptIn(ExperimentalUnsignedTypes::class)
```

Here's an example that opts in to use the `uintArrayOf()` function to create an array of unsigned integers and modifies one of its elements:

```
@OptIn(ExperimentalUnsignedTypes::class)
fun main() {
    // Create an unsigned integer array
    val unsignedArray: UIntArray = uintArrayOf(1u, 2u, 3u, 4u, 5u)

    // Modify an element
    unsignedArray[2] = 42u
    println("Updated array: ${unsignedArray.joinToString()}")
    // Updated array: 1, 2, 42, 4, 5
}
```

This is the easiest way to opt in, but there are other ways. To learn more, see Opt-in requirements.

Practice

Calculate compound interest

You are developing a financial application that helps users calculate the future value of their investments. The formula to calculate compound interest is:

Where:

- `A` is the amount of money accumulated after interest (principal + interest).
- `P` is the principal amount (the initial investment).
- `r` is the annual interest rate (decimal).
- `n` is the number of times interest is compounded per year.
- `t` is the time the money is invested for (in years).

Update the code to:

1. Import the necessary functions from the `kotlin.math` package.
2. Add a body to the `calculateCompoundInterest()` function that calculates the final amount after applying compound interest.

```
// Write your code here

fun calculateCompoundInterest(P: Double, r: Double, n: Int, t: Int): Double {
    // Write your code here
}

fun main() {
```

```
    val principal = 1000.0
    val rate = 0.05
    val timesCompounded = 4
    val years = 5
    val amount = calculateCompoundInterest(principal, rate, timesCompounded, years)
    println("The accumulated amount is: $amount")
    // The accumulated amount is: 1282.0372317085844
}
```

```
import kotlin.math.*

fun calculateCompoundInterest(P: Double, r: Double, n: Int, t: Int): Double {
    return P * (1 + r / n).pow(n * t)
}

fun main() {
    val principal = 1000.0
    val rate = 0.05
    val timesCompounded = 4
    val years = 5
    val amount = calculateCompoundInterest(principal, rate, timesCompounded, years)
    println("The accumulated amount is: $amount")
    // The accumulated amount is: 1282.0372317085844
}
```

Measure the time taken by data processing

You want to measure the time it takes to perform multiple data processing tasks in your program. Update the code to add the correct import statements and functions from the `kotlin.time` package:

```
// Write your code here

fun main() {
    val timeTaken = /* Write your code here */ {
        // Simulate some data processing
        val data = List(1000) { it * 2 }
        val filteredData = data.filter { it % 3 == 0 }

        // Simulate processing the filtered data
        val processedData = filteredData.map { it / 2 }
        println("Processed data")
    }

    println("Time taken: $timeTaken") // e.g. 16 ms
}
```

```
import kotlin.time.measureTime

fun main() {
    val timeTaken = measureTime {
        // Simulate some data processing
        val data = List(1000) { it * 2 }
        val filteredData = data.filter { it % 3 == 0 }
```

```
        // Simulate processing the filtered data
        val processedData = filteredData.map { it / 2 }
        println("Processed data")
    }

    println("Time taken: $timeTaken") // e.g. 16 ms
}
```

Opt in to an experimental API

There's a new feature in the standard library available in the latest Kotlin release. You want to try it out, but it requires opt-in. The feature falls under `@ExperimentalStdlibApi`. What should the opt-in look like in your code?

```
@OptIn(ExperimentalStdlibApi::class)
```

What's next?

Congratulations! You've completed the intermediate tour! Would you like to share your feedback about your experience?

As a next step, check out our tutorials for popular Kotlin applications:

Create a backend application with Spring Boot and Kotlin.

```
    Start
</panel>
<panel>
    <title>Kotlin Multiplatform</title>
```

Create a cross-platform application from scratch and share business logic and UI.

```
    Start
</panel>
```

```
<a as="button" href="kotlin-tour-intermediate-null-safety.md" mode="outline" icon="arrow-lef
t" icon-position="left">Previous step</a>
```

Language Guide

Basic syntax overview

This is a collection of basic syntax elements with examples. At the end of every section, you'll find a link to a detailed description of the related topic.

You can also learn all the Kotlin essentials with the free Kotlin Core track by JetBrains Academy.

Package definition and imports

Package specification should be at the top of the source file:

```
package my.demo

import kotlin.text.*

// ...
```

It is not required to match directories and packages: source files can be placed arbitrarily in the file system.

See Packages.

Program entry point

An entry point of a Kotlin application is the `main` function:

```
fun main() {
    println("Hello world!")
}
```

Another form of `main` accepts a variable number of `String` arguments:

```
fun main(args: Array<String>) {
    println(args.contentToString())
}
```

Print to the standard output

`print` prints its argument to the standard output:

```
fun main() {
    print("Hello ")
```

```
    print("world!")
}
```

`println` prints its arguments and adds a line break, so that the next thing you print appears on the next line:

```
fun main() {
    println("Hello world!")
    println(42)
}
```

Read from the standard input

The `readln()` function reads from the standard input. This function reads the entire line the user enters as a string.

You can use the `println()`, `readln()`, and `print()` functions together to print messages requesting and showing user input:

```
// Prints a message to request input
println("Enter any word: ")

// Reads and stores the user input. For example: Happiness
val yourWord = readln()

// Prints a message with the input
print("You entered the word: ")
print(yourWord)
// You entered the word: Happiness
```

For more information, see Read standard input.

Functions

A function with two `Int` parameters and `Int` return type:

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
fun main() {
    print("sum of 3 and 5 is ")
    println(sum(3, 5))
}
```

A function body can be an expression. Its return type is inferred:

```
fun sum(a: Int, b: Int) = a + b
fun main() {
```

```
    println("sum of 19 and 23 is ${sum(19, 23)}")
}
```

A function that returns no meaningful value:

```
fun printSum(a: Int, b: Int): Unit {
    println("sum of $a and $b is ${a + b}")
}
fun main() {
    printSum(-1, 8)
}
```

`Unit` return type can be omitted:

```
fun printSum(a: Int, b: Int) {
    println("sum of $a and $b is ${a + b}")
}
fun main() {
    printSum(-1, 8)
}
```

See Functions.

Variables

In Kotlin, you declare a variable starting with a keyword, `val` or `var`, followed by the name of the variable.

Use the `val` keyword to declare variables that are assigned a value only once. These are immutable, read-only local variables that can't be reassigned a different value after initialization:

```
fun main() {
    // Declares the variable x and initializes it with the value of 5
    val x: Int = 5
    // 5
    println(x)
}
```

Use the `var` keyword to declare variables that can be reassigned. These are mutable variables, and you can change their values after initialization:

```
fun main() {
    // Declares the variable x and initializes it with the value of 5
    var x: Int = 5
    // Reassigns a new value of 6 to the variable x
    x += 1
    // 6
    println(x)
}
```

Kotlin supports type inference and automatically identifies the data type of a declared variable. When declaring a variable, you can omit the type after the variable name:

```
fun main() {
    // Declares the variable x with the value of 5;`Int` type is inferred
    val x = 5
    // 5
    println(x)
}
```

You can use variables only after initializing them. You can either initialize a variable at the moment of declaration or declare a variable first and initialize it later. In the second case, you must specify the data type:

```
fun main() {
    // Initializes the variable x at the moment of declaration; type is not required
    val x = 5
    // Declares the variable c without initialization; type is required
    val c: Int
    // Initializes the variable c after declaration
    c = 3
    // 5
    // 3
    println(x)
    println(c)
}
```

You can declare variables at the top level:

```
val PI = 3.14
var x = 0

fun incrementX() {
    x += 1
}
// x = 0; PI = 3.14
// incrementX()
// x = 1; PI = 3.14
fun main() {
    println("x = $x; PI = $PI")
    incrementX()
    println("incrementX()")
    println("x = $x; PI = $PI")
}
```

For information about declaring properties, see Properties.

Creating classes and instances

To define a class, use the `class` keyword:

```
class Shape
```

Properties of a class can be listed in its declaration or body:

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
```

The default constructor with parameters listed in the class declaration is available automatically:

```
class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
fun main() {
    val rectangle = Rectangle(5.0, 2.0)
    println("The perimeter is ${rectangle.perimeter}")
}
```

Inheritance between classes is declared by a colon (`:`). Classes are `final` by default; to make a class inheritable, mark it as `open`:

```
open class Shape

class Rectangle(val height: Double, val length: Double): Shape() {
    val perimeter = (height + length) * 2
}
```

For more information about constructors and inheritance, see Classes and Objects and instances.

Comments

Just like most modern languages, Kotlin supports single-line (or *end-of-line*) and multi-line (*block*) comments:

```
// This is an end-of-line comment

/* This is a block comment
   on multiple lines. */
```

Block comments in Kotlin can be nested:

```
/* The comment starts here
/* contains a nested comment */
and ends here. */
```

See Documenting Kotlin Code for information on the documentation comment syntax.

String templates

```
fun main() {
    var a = 1
    // simple name in template:
    val s1 = "a is $a"

    a = 2
    // arbitrary expression in template:
    val s2 = "${s1.replace("is", "was")}, but now is $a"
    println(s2)
}
```

See String templates for details.

Conditional expressions

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

In Kotlin, `if` can also be used as an expression:

```
fun maxOf(a: Int, b: Int) = if (a > b) a else b
fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

See `if`-expressions.

for loop

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    for (item in items) {
        println(item)
    }
}
```

or:

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    for (index in items.indices) {
        println("item at $index is ${items[index]}")
    }
}
```

See for loop.

while loop

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    var index = 0
    while (index < items.size) {
        println("item at $index is ${items[index]}")
        index++
    }
}
```

See while loop.

when expression

```
fun describe(obj: Any): String =
    when (obj) {
        1          -> "One"
        "Hello"    -> "Greeting"
        is Long    -> "Long"
        !is String -> "Not a string"
        else       -> "Unknown"
    }
fun main() {
    println(describe(1))
    println(describe("Hello"))
    println(describe(1000L))
    println(describe(2))
    println(describe("other"))
}
```

See when expressions and statements.

Ranges

Check if a number is within a range using `in` operator:

```
fun main() {
    val x = 10
```

```
    val y = 9
    if (x in 1..y+1) {
        println("fits in range")
    }
}
```

Check if a number is out of range:

```
fun main() {
    val list = listOf("a", "b", "c")

    if (-1 !in 0..list.lastIndex) {
        println("-1 is out of range")
    }
    if (list.size !in list.indices) {
        println("list size is out of valid list indices range, too")
    }
}
```

Iterate over a range:

```
fun main() {
    for (x in 1..5) {
        print(x)
    }
}
```

Or over a progression:

```
fun main() {
    for (x in 1..10 step 2) {
        print(x)
    }
    println()
    for (x in 9 downTo 0 step 3) {
        print(x)
    }
}
```

See Ranges and progressions.

Collections

Iterate over a collection:

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
    for (item in items) {
        println(item)
```

```
    }
}
```

Check if a collection contains an object using `in` operator:

```
fun main() {
    val items = setOf("apple", "banana", "kiwifruit")
    when {
        "orange" in items -> println("juicy")
        "apple" in items -> println("apple is fine too")
    }
}
```

Use lambda expressions to filter and map collections:

```
fun main() {
    val fruits = listOf("banana", "avocado", "apple", "kiwifruit")
    fruits
      .filter { it.startsWith("a") }
      .sortedBy { it }
      .map { it.uppercase() }
      .forEach { println(it) }
}
```

See Collections overview.

Nullable values and null checks

A reference must be explicitly marked as nullable when a `null` value is possible. Nullable type names have `?` at the end. For example, `Int?`.

Return `null` if `str` does not hold an integer:

```
fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}
```

Use a function returning nullable value:

```
fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}
fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)

    // Using `x * y` yields error because they may hold nulls.
    if (x != null && y != null) {
        // x and y are automatically cast to non-nullable after null check
        println(x * y)
```

```
    }
    else {
        println("'$arg1' or '$arg2' is not a number")
    }
}
fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("a", "b")
}
```

or:

```
fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)
    // ...
    if (x == null) {
        println("Wrong number format in arg1: '$arg1'")
        return
    }
    if (y == null) {
        println("Wrong number format in arg2: '$arg2'")
        return
    }

    // x and y are automatically cast to non-nullable after null check
    println(x * y)
}

fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("99", "b")
}
```

See Null-safety.

Type checks and automatic casts

The `is` operator checks if an expression is an instance of a type. If an immutable local variable or property is checked for a specific type, there's no need to cast it explicitly:

```
fun getStringLength(obj: Any): Int? {
    if (obj is String) {
        // `obj` is automatically cast to `String` in this branch
        return obj.length
    }
```

```
    // `obj` is still of type `Any` outside of the type-checked branch
    return null
}
fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The
object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf(Any()))
}
```

or:

```
fun getStringLength(obj: Any): Int? {
    if (obj !is String) return null

    // `obj` is automatically cast to `String` in this branch
    return obj.length
}
fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The
object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf(Any()))
}
```

or even:

```
fun getStringLength(obj: Any): Int? {
    // `obj` is automatically cast to `String` on the right-hand side of `&&`
    if (obj is String && obj.length >= 0) {
        return obj.length
    }

    return null
}
fun main() {
    fun printLength(obj: Any) {
        println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The
object is not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength("")
    printLength(1000)
}
```

See Classes and Type casts.

Keywords and operators

Hard keywords

The following tokens are always interpreted as keywords and cannot be used as identifiers:

- `as`
  - is used for type casts.
  - specifies an alias for an import
- `as?` is used for safe type casts.
- `break` terminates the execution of a loop.
- `class` declares a class.
- `continue` proceeds to the next step of the nearest enclosing loop.
- `do` begins a do/while loop (a loop with a postcondition).
- `else` defines the branch of an if expression that is executed when the condition is false.
- `false` specifies the 'false' value of the Boolean type.
- `for` begins a for loop.
- `fun` declares a function.
- `if` begins an if expression.
- `in`
  - specifies the object being iterated in a for loop.
  - is used as an infix operator to check that a value belongs to a range, a collection, or another entity that defines a 'contains' method.
  - is used in when expressions for the same purpose.
  - marks a type parameter as contravariant.
- `!in`
  - is used as an operator to check that a value does NOT belong to a range, a collection, or another entity that defines a 'contains' method.
  - is used in when expressions for the same purpose.
- `interface` declares an interface.
- `is`
  - checks that a value has a certain type.
  - is used in when expressions for the same purpose.
- `!is`
  - checks that a value does NOT have a certain type.
  - is used in when expressions for the same purpose.
- `null` is a constant representing an object reference that doesn't point to any object.
- `object` declares a class and its instance at the same time.
- `package` specifies the package for the current file.
- `return` returns from the nearest enclosing function or anonymous function.

- `super`
  - refers to the superclass implementation of a method or property.
  - calls the superclass constructor from a secondary constructor.
- `this`
  - refers to the current receiver.
  - calls another constructor of the same class from a secondary constructor.
- `throw` throws an exception.
- `true` specifies the 'true' value of the Boolean type.
- `try` begins an exception-handling block.
- `typealias` declares a type alias.
- `typeof` is reserved for future use.
- `val` declares a read-only property or local variable.
- `var` declares a mutable property or local variable.
- `when` begins a when expression (executes one of the given branches).
- `while` begins a while loop (a loop with a precondition).

Soft keywords

The following tokens act as keywords in the context in which they are applicable, and they can be used as identifiers in other contexts:

- `by`
  - delegates the implementation of an interface to another object.
  - delegates the implementation of the accessors for a property to another object.
- `catch` begins a block that handles a specific exception type.
- `constructor` declares a primary or secondary constructor.
- `context` declares context parameters for functions and properties.
- `delegate` is used as an annotation use-site target.
- `dynamic` references a dynamic type in Kotlin/JS code.
- `field`
  - declares an explicit backing field.
  - is used as an annotation use-site target.
- `file` is used as an annotation use-site target.
- `finally` begins a block that is always executed when a try block exits.
- `get`
  - declares the getter of a property.
  - is used as an annotation use-site target.
- `import` imports a declaration from another package into the current file.
- `init` begins an initializer block.
- `param` is used as an annotation use-site target.
- `property` is used as an annotation use-site target.

- `receiver` is used as an annotation use-site target.
- `set`
    - declares the setter of a property.
    - is used as an annotation use-site target.
- `setparam` is used as an annotation use-site target.
- `value` with the `class` keyword declares an inline class.
- `where` specifies the constraints for a generic type parameter.

Modifier keywords

The following tokens act as keywords in modifier lists of declarations, and they can be used as identifiers in other contexts:

- `abstract` marks a class or member as abstract.
- `actual` denotes a platform-specific implementation in multiplatform projects.
- `annotation` declares an annotation class.
- `companion` declares a companion object.
- `const` marks a property as a compile-time constant.
- `crossinline` forbids non-local returns in a lambda passed to an inline function.
- `data` instructs the compiler to generate canonical members for a class.
- `enum` declares an enumeration.
- `expect` marks a declaration as platform-specific, expecting an implementation in platform modules.
- `external` marks a declaration as implemented outside of Kotlin (accessible through JNI or in JavaScript).
- `final` forbids overriding a member.
- `infix` allows calling a function using infix notation.
- `inline` tells the compiler to inline a function and the lambdas passed to it at the call site.
- `inner` allows referring to an outer class instance from a nested class.
- `internal` marks a declaration as visible in the current module.
- `lateinit` allows initializing a non-nullable property outside of a constructor.
- `noinline` turns off inlining of a lambda passed to an inline function.
- `open` allows subclassing a class or overriding a member.
- `operator` marks a function as overloading an operator or implementing a convention.
- `out` marks a type parameter as covariant.
- `override` marks a member as an override of a superclass member.
- `private` marks a declaration as visible in the current class or file.
- `protected` marks a declaration as visible in the current class and its subclasses.
- `public` marks a declaration as visible anywhere.
- `reified` marks a type parameter of an inline function as accessible at runtime.
- `sealed` declares a sealed class (a class with restricted subclassing).

- `suspend` marks a function or lambda as suspending (usable as a coroutine).
- `tailrec` marks a function as tail-recursive (allowing the compiler to replace recursion with iteration).
- `vararg` allows passing a variable number of arguments for a parameter.

Special identifiers

The following identifiers are defined by the compiler in specific contexts, and they can be used as regular identifiers in other contexts:

- `field` is used inside a property accessor to refer to the backing field of the property.
- `it` is used inside a lambda to refer to its parameter implicitly.

Operators and special symbols

Kotlin supports the following operators and special symbols:

- `+`, `-`, `*`, `/`, `%` - mathematical operators
  - `*` is also used to pass an array to a vararg parameter.
- `=`
  - assignment operator.
  - is used to specify default values for parameters.
- `+=`, `-=`, `*=`, `/=`, `%=` - augmented assignment operators.
- `++`, `--` - increment and decrement operators.
- `&&`, `||`, `!` - logical 'and', 'or', 'not' operators (for bitwise operations, use the corresponding infix functions instead).
- `==`, `!=` - equality operators (translated to calls of `equals()` for non-primitive types).
- `===`, `!==` - referential equality operators.
- `<`, `>`, `<=`, `>=` - comparison operators (translated to calls of `compareTo()` for non-primitive types).
- `[`, `]` - indexed access operator (translated to calls of `get` and `set`).
- `!!` asserts that an expression is non-nullable.
- `?.` performs a safe call (calls a method or accesses a property if the receiver is non-nullable).
- `?:` takes the right-hand value if the left-hand value is null (the elvis operator).
- `::` creates a member reference or a class reference.
- `.`
  - accesses members, including nested classes and enum entries.
  - defines and calls extensions.
  - qualifies names and packages.
  - separates the integer and fractional parts of a floating-point literal.
- `..`, `..<` create ranges.
- `:` separates a name from a type in a declaration.

- `?` marks a type as nullable.
- `->`
    - separates the parameters and body of a lambda expression.
    - separates the parameters and return type declaration in a function type.
    - separates the condition and body of a when expression branch.
- `@`
    - introduces an annotation.
    - introduces or references a loop label.
    - introduces or references a lambda label.
    - references a 'this' expression from an outer scope.
    - references an outer superclass.
- `;` separates multiple statements on the same line.
- `$` references a variable or expression in a string template.
- `_`
    - substitutes an unused parameter in a lambda expression.
    - substitutes an unused parameter in a destructuring declaration.

For operator precedence, see this reference in Kotlin grammar.

Packages and imports

In a Kotlin project, code is organized using packages and imports:

- A **package** is a container for one or more Kotlin files. Files are linked to a package using a `package` header.
- An **import** is a directive that makes entities from other packages available in the current file.

Package headers

A source file may start with a package header:

```
package org.example

fun printMessage() { /*...*/ }
class Message(val text: String) { /*...*/ }
```

All contents of the source file, such as classes and functions, belong to this package. Their fully qualified name combines the package name with the entity's name. In this example:

- The fully qualified name of `printMessage()` is `org.example.printMessage`.
- The fully qualified name of `Message` is `org.example.Message`.

If a file has no package header, its contents belong to the root package.

Imports

To use an entity from a file in a different package, use an `import` directive. In addition to the default imports, each file may declare its own imports.

Import a single entity

Import a specific entity so you can use it without qualification:

```
// Message is accessible without qualification
import org.example.Message

fun main() {
    val message = Message("Hello")
    println(message.text)
}
```

Import the contents of a scope

Star imports, ending in an asterisk `*`, import all named entities inside the corresponding scope:

```
// Everything in org.example is accessible
import org.example.*

fun main() {
    printMessage()
    val message = Message("Hi")
}
```

If you import an entity with both a star import and an explicit import, the explicit import takes priority during overload resolution.

Resolve name clashes with aliases

If two imported entities have the same name, use the `as` keyword to locally rename one of them:

```
// Message refers to org.example.Message
import org.example.Message

// TestMessage refers to org.test.Message
import org.test.Message as TestMessage

fun main() {
    val a = Message("from example")
    val b = TestMessage("from test")
}
```

What you can import

The `import` keyword is not limited to classes. You can import any of the following entities, whether they come from a package, a class, an object, or an enum:

- Top-level functions and properties declared directly inside a package:

  ```
  import org.example.printMessage // Top-level function
  import org.example.VERSION      // Top-level property
  ```

- Functions and properties from object declarations:

  ```
  import org.example.Config.DEFAULT_TIMEOUT // Property from an object
  import org.example.Config.loadSettings    // Function from an object
  ```

- Members of a companion object, referenced through the enclosing class name:

  ```
  import org.example.MyClass.create // Refers to MyClass.Companion.create
  ```

- Enum constants:

```
import org.example.Color.RED
import org.example.Color.GREEN
```

- Nested classes:

```
import org.example.Outer.Nested
```

Default imports

Kotlin includes the following imports by default:

- kotlin.*
- kotlin.annotation.*
- kotlin.collections.*
- kotlin.comparisons.*
- kotlin.io.*
- kotlin.ranges.*
- kotlin.sequences.*
- kotlin.text.*
- kotlin.math.*

Kotlin imports additional packages depending on the target platform:

- JVM:
    - java.lang.*
    - kotlin.jvm.*
- JS:
    - kotlin.js.*

Visibility and imports

The ability to import an entity depends on its visibility modifiers:

- `public` entities can be imported anywhere.
- `internal` entities can be imported only within the same module.
- `protected` entities cannot be imported.
- Top-level `private` entities are only accessible within their declaring file.
- Other `private` entities cannot be imported.

Annotations

Annotations are tags that you can use to attach metadata to elements in your code. Tools and frameworks process this metadata during compilation and runtime, and perform different actions based on it.

You can annotate your code to simplify and automate common tasks, such as generating boilerplate code, enforcing coding standards or writing documentation.

> **Tip:** If you want to develop your own annotation processors, you can use the Kotlin Symbol Processing (KSP) API.

Declaration

Annotations are a special type of class. To declare an annotation, use the `annotation` keyword before the class declaration:

```
annotation class Fancy
```

Additional attributes of the annotation can be specified by annotating the annotation class with meta-annotations:

- `@Target` specifies the possible kinds of elements which can be annotated with the annotation (such as classes, functions, properties, and expressions);
- `@Retention` specifies whether the annotation is stored in the compiled class files and whether it's visible through reflection at runtime (by default, both are true);
- `@Repeatable` allows using the same annotation on a single element multiple times;
- `@MustBeDocumented` specifies that the annotation is part of the public API and should be included in the class or method signature shown in the generated API documentation.

```
@Target(AnnotationTarget.CLASS, AnnotationTarget.FUNCTION,
        AnnotationTarget.TYPE_PARAMETER, AnnotationTarget.VALUE_PARAMETER,
        AnnotationTarget.EXPRESSION)
@Retention(AnnotationRetention.SOURCE)
@MustBeDocumented
annotation class Fancy
```

Usage

```
@Fancy class Foo {
    @Fancy fun baz(@Fancy foo: Int): Int {
        return (@Fancy 1)
```

```
    }
}
```

If you need to annotate the primary constructor of a class, you need to add the `constructor` keyword to the constructor declaration, and add the annotations before it:

```
class Foo @Inject constructor(dependency: MyDependency) { ... }
```

You can also annotate property accessors:

```
class Foo {
    var x: MyDependency? = null
        @Inject set
}
```

Constructors

Annotations can have constructors that take parameters.

```
annotation class Special(val why: String)

@Special("example") class Foo {}
```

Allowed parameter types are:

- Types that correspond to Java primitive types (Int, Long etc.)
- Strings
- Classes (`Foo::class`)
- Enums
- Other annotations
- Arrays of the types listed above

Annotation parameters cannot have nullable types, because the JVM does not support storing `null` as a value of an annotation attribute.

If an annotation is used as a parameter of another annotation, its name is not prefixed with the `@` character:

```
annotation class ReplaceWith(val expression: String)

annotation class Deprecated(
        val message: String,
        val replaceWith: ReplaceWith = ReplaceWith(""))

@Deprecated("This function is deprecated, use === instead", ReplaceWith("this === other"))
```

If you need to specify a class as an argument of an annotation, use a Kotlin class (KClass). The Kotlin compiler will automatically convert it to a Java class, so that the Java code can access the annotations and arguments normally.

```
import kotlin.reflect.KClass

annotation class Ann(val arg1: KClass<*>, val arg2: KClass<out Any>)

@Ann(String::class, Int::class) class MyClass
```

Instantiation

In Java, an annotation type is a form of an interface, so you can implement it and use an instance. As an alternative to this mechanism, Kotlin lets you call a constructor of an annotation class in arbitrary code and similarly use the resulting instance.

```
annotation class InfoMarker(val info: String)

fun processInfo(marker: InfoMarker): Unit = TODO()

fun main(args: Array<String>) {
    if (args.isNotEmpty())
        processInfo(getAnnotationReflective(args))
    else
        processInfo(InfoMarker("default"))
}
```

Learn more about instantiation of annotation classes in this KEEP.

Lambdas

Annotations can also be used on lambdas. They will be applied to the `invoke()` method into which the body of the lambda is generated. This is useful for frameworks like Quasar, which uses annotations for concurrency control.

```
annotation class Suspendable

val f = @Suspendable { Fiber.sleep(10) }
```

Annotation use-site targets

When you're annotating a property or a primary constructor parameter, there are multiple Java elements that are generated from the corresponding Kotlin element, and therefore multiple

possible locations for the annotation in the generated Java bytecode. To specify how exactly the annotation should be generated, use the following syntax:

```
class Example(@field:Ann val foo,    // annotate only the Java field
              @get:Ann val bar,      // annotate only the Java getter
              @param:Ann val quux)   // annotate only the Java constructor parameter
```

The same syntax can be used to annotate the entire file. To do this, put an annotation with the target `file` at the top level of a file, before the package directive or before all imports if the file is in the default package:

```
@file:JvmName("Foo")

package org.jetbrains.demo
```

If you have multiple annotations with the same target, you can avoid repeating the target by adding brackets after the target and putting all the annotations inside the brackets (except for the `all` meta-target):

```
class Example {
    @set:[Inject VisibleForTesting]
    var collaborator: Collaborator
}
```

The full list of supported use-site targets is:

- `file`
- `field`
- `property` (annotations with this target are not visible to Java)
- `get` (property getter)
- `set` (property setter)
- `all` (a meta-target for properties, see the `all` meta-target section for more information)
- `receiver` (receiver parameter of an extension function or property)

  To annotate the receiver parameter of an extension function, use the following syntax:

  ```
  fun @receiver:Fancy String.myExtension() { ... }
  ```

- `param` (constructor parameter)
- `setparam` (property setter parameter)

- `delegate` (the field storing the delegate instance for a delegated property)

Defaults when no use-site targets are specified

If you don't specify a use-site target, the compiler chooses the target according to the `@Target` annotation of the annotation you use. If there are multiple applicable targets, the compiler chooses one or more of them in the following order:

- The constructor parameter target (`param`).
- The property target (`property`).
- The field target (`field`), if it's applicable and the property target (`property`) isn't.

If none of `param`, `property`, or `field` are applicable, the annotation is invalid, and you need to specify a use-site target explicitly.

Let's use the `@Email` annotation from Jakarta Bean Validation:

```
@Target(value={METHOD,FIELD,ANNOTATION_TYPE,CONSTRUCTOR,PARAMETER,TYPE_USE})
public @interface Email { }
```

With this annotation, consider the following example:

```
data class User(val username: String,
                // @Email is now equivalent to @param:Email @field:Email
                @Email val email: String) {
    // @Email is still equivalent to @field:Email
    @Email val secondaryEmail: String? = null
}
```

In this example, the `@Email` annotation applies to both the constructor parameter and the field targets for the `email` property because the property:

- Is declared in the primary constructor.
- Has no custom getter or setter, so the compiler generates a backing field.

The `@Email` annotation only applies to the field target for the `secondaryEmail` property because the property:

- Isn't declared in the primary constructor.
- Has no custom getter or setter, so the compiler generates a backing field.

`all` meta-target

The `all` target makes it easier to apply the same annotation not only to the parameter and the property or field, but also to the corresponding getter and setter.

Specifically, the annotation marked with `all` is propagated, if applicable:

- To the constructor parameter (`param`) if the property is defined in the primary constructor.
- To the property itself (`property`).
- To the backing field (`field`) if the property has one.
- To the getter (`get`).
- To the setter parameter (`setparam`) if the property is defined as `var`.
- To the Java-only target `RECORD_COMPONENT` if the class has the `@JvmRecord` annotation.

Let's use the `@Email` annotation from Jakarta Bean Validation, which is defined as follows:

```
@Target(value={METHOD,FIELD,ANNOTATION_TYPE,CONSTRUCTOR,PARAMETER,TYPE_USE})
public @interface Email { }
```

In the example below, this `@Email` annotation is applied to all relevant targets:

```
data class User(
    val username: String,
    // Applies @Email to param, field, and get
    @all:Email val email: String,
    // Applies @Email to param, field, get, and setparam
    @all:Email var name: String,
) {
    // Applies @Email to field and getter (not param since it's not in the constructor)
    @all:Email val secondaryEmail: String? = null
}
```

You can use the `all` meta-target with any property, both inside and outside the primary constructor.

Limitations

The `all` target comes with some limitations:

- It does not propagate an annotation to types, potential extension receivers, or context receivers or parameters.
- It cannot be used with multiple annotations:

  ```
  @all:[A B] // forbidden, use @all:A @all:B
  val x: Int = 5
  ```

- It cannot be used with delegated properties.

Java annotations

Java annotations are 100% compatible with Kotlin:

```
import org.junit.Test
import org.junit.Assert.*
import org.junit.Rule
```

```
import org.junit.rules.*

class Tests {
    // apply @Rule annotation to property getter
    @get:Rule val tempFolder = TemporaryFolder()

    @Test fun simple() {
        val f = tempFolder.newFile()
        assertEquals(42, getTheAnswer())
    }
}
```

Since the order of parameters for an annotation written in Java is not defined, you can't use a regular function call syntax for passing the arguments. Instead, you need to use the named argument syntax:

```
// Java
public @interface Ann {
    int intValue();
    String stringValue();
}
```

```
// Kotlin
@Ann(intValue = 1, stringValue = "abc") class C
```

Just like in Java, a special case is the `value` parameter; its value can be specified without an explicit name:

```
// Java
public @interface AnnWithValue {
    String value();
}
```

```
// Kotlin
@AnnWithValue("abc") class C
```

Arrays as annotation parameters

If the `value` argument in Java has an array type, it becomes a `vararg` parameter in Kotlin:

```
// Java
public @interface AnnWithArrayValue {
    String[] value();
}
```

```
// Kotlin
@AnnWithArrayValue("abc", "foo", "bar") class C
```

For other arguments that have an array type, you need to use the array literal syntax or `arrayOf(...)`:

```
// Java
public @interface AnnWithArrayMethod {
    String[] names();
}
```

```
@AnnWithArrayMethod(names = ["abc", "foo", "bar"])
class C
```

Accessing properties of an annotation instance

Values of an annotation instance are exposed as properties to Kotlin code:

```
// Java
public @interface Ann {
    int value();
}
```

```
// Kotlin
fun foo(ann: Ann) {
    val i = ann.value
}
```

Ability to not generate JVM 1.8+ annotation targets

If a Kotlin annotation has `TYPE` among its Kotlin targets, the annotation maps to `java.lang.annotation.ElementType.TYPE_USE` in its list of Java annotation targets. This is just like how the `TYPE_PARAMETER` Kotlin target maps to the `java.lang.annotation.ElementType.TYPE_PARAMETER` Java target. This is an issue for Android clients with API levels less than 26, which don't have these targets in the API.

To avoid generating the `TYPE_USE` and `TYPE_PARAMETER` annotation targets, use the new compiler argument `-Xno-new-java-annotation-targets`.

Repeatable annotations

Just like in Java, Kotlin has repeatable annotations, which can be applied to a single code element multiple times. To make your annotation repeatable, mark its declaration with the `@kotlin.annotation.Repeatable` meta-annotation. This will make it repeatable both in Kotlin and Java. Java repeatable annotations are also supported from the Kotlin side.

The main difference with the scheme used in Java is the absence of a *containing annotation*, which the Kotlin compiler generates automatically with a predefined name. For an annotation in the example below, it will generate the containing annotation `@Tag.Container`:

```
@Repeatable
annotation class Tag(val name: String)

// The compiler generates the @Tag.Container containing annotation
```

You can set a custom name for a containing annotation by applying the `@kotlin.jvm.JvmRepeatable` meta-annotation and passing an explicitly declared containing annotation class as an argument:

```
@JvmRepeatable(Tags::class)
annotation class Tag(val name: String)

annotation class Tags(val value: Array<Tag>)
```

To extract Kotlin or Java repeatable annotations via reflection, use the `KAnnotatedElement.findAnnotations()` function.

Learn more about Kotlin repeatable annotations in this KEEP.

Visibility modifiers

Classes, objects, interfaces, constructors, and functions, as well as properties and their setters, can have *visibility modifiers*. Getters always have the same visibility as their properties.

There are four visibility modifiers in Kotlin: `private`, `protected`, `internal`, and `public`. The default visibility is `public`.

On this page, you'll learn how the modifiers apply to different types of declaring scopes.

Packages

Functions, properties, classes, objects, and interfaces can be declared at the "top-level" directly inside a package:

```
// file name: example.kt
package foo

fun baz() { ... }
class Bar { ... }
```

- If you don't use a visibility modifier, `public` is used by default, which means that your declarations will be visible everywhere.
- If you mark a declaration as `private`, it will only be visible inside the file that contains the declaration.
- If you mark it as `internal`, it will be visible everywhere in the same module.
- The `protected` modifier is not available for top-level declarations.

  Note: To use a visible top-level declaration from another package, you should import it.

Examples:

```
// file name: example.kt
package foo

private fun foo() { ... } // visible inside example.kt

public var bar: Int = 5 // property is visible everywhere
    private set         // setter is visible only in example.kt

internal val baz = 6    // visible inside the same module
```

Class members

For members declared inside a class:

- `private` means that the member is visible inside this class only (including all its members).
- `protected` means that the member has the same visibility as one marked as `private`, but that it is also visible in subclasses.
- `internal` means that any client *inside this module* who sees the declaring class sees its `internal` members.
- `public` means that any client who sees the declaring class sees its `public` members.

> **Note:** In Kotlin, an outer class does not see private members of its inner classes.

If you override a `protected` or an `internal` member and do not specify the visibility explicitly, the overriding member will also have the same visibility as the original.

Examples:

```
open class Outer {
    private val a = 1
    protected open val b = 2
    internal open val c = 3
    val d = 4  // public by default

    protected class Nested {
        public val e: Int = 5
    }
}

class Subclass : Outer() {
    // a is not visible
    // b, c and d are visible
    // Nested and e are visible

    override val b = 5   // 'b' is protected
    override val c = 7   // 'c' is internal
}

class Unrelated(o: Outer) {
    // o.a, o.b are not visible
    // o.c and o.d are visible (same module)
    // Outer.Nested is not visible, and Nested::e is not visible either
}
```

Constructors

Use the following syntax to specify the visibility of the primary constructor of a class:

> **Note:** You need to add an explicit `constructor` keyword.

```
class C private constructor(a: Int) { ... }
```

Here the constructor is `private`. By default, all constructors are `public`, which effectively amounts to them being visible everywhere the class is visible (this means that a constructor of an `internal` class is only visible within the same module).

For sealed classes, constructors are `protected` by default. For more information, see Sealed classes.

Local declarations

Local variables, functions, and classes can't have visibility modifiers.

Modules

The `internal` visibility modifier means that the member is visible within the same module. More specifically, a module is a set of Kotlin files compiled together, for example:

- An IntelliJ IDEA module.
- A Maven project.
- A Gradle source set (with the exception that the `test` source set can access the internal declarations of `main`).

Coding conventions

Commonly known and easy-to-follow coding conventions are vital for any programming language. Here we provide guidelines on the code style and code organization for projects that use Kotlin.

Configure style in IDE

Two most popular IDEs for Kotlin - IntelliJ IDEA and Android Studio provide powerful support for code styling. You can configure them to automatically format your code in consistence with the given code style.

Apply the style guide

1. Go to **Settings/Preferences | Editor | Code Style | Kotlin**.
2. Click **Set from...**.
3. Select **Kotlin style guide** .

Verify that your code follows the style guide

1. Go to **Settings/Preferences | Editor | Inspections | General**.
2. Switch on **Incorrect formatting** inspection. Additional inspections that verify other issues described in the style guide (such as naming conventions) are enabled by default.

For more information, see the Migrate to Kotlin code style with IntelliJ IDEA guide.

Source code organization

Directory structure

In pure Kotlin projects, the recommended directory structure follows the package structure with the common root package omitted. For example, if all the code in the project is in the `org.example.kotlin` package and its subpackages, files with the `org.example.kotlin` package should be placed directly under the source root, and files in `org.example.kotlin.network.socket` should be in the `network/socket` subdirectory of the source root.

> **Note:** On JVM: In projects where Kotlin is used together with Java, Kotlin source files should reside in the same source root as the Java source files, and follow the same directory structure: each file should be stored in the directory corresponding to each package statement.

Source file names

If a Kotlin file contains a single class or interface (potentially with related top-level declarations), its name should be the same as the name of the class, with the `.kt` extension appended. It applies to all types of classes and interfaces. If a file contains multiple classes, or only top-level declarations, choose a name describing what the file contains, and name the file accordingly. Use upper camel case, where the first letter of each word is capitalized. For example, `ProcessDeclarations.kt`.

The name of the file should describe what the code in the file does. Therefore, you should avoid using meaningless words such as `Util` in file names.

Multiplatform projects

In multiplatform projects, files with top-level declarations in platform-specific source sets should have a suffix associated with the name of the source set. For example:

- **jvm**Main/kotlin/Platform.**jvm**.kt
- **android**Main/kotlin/Platform.**android**.kt
- **ios**Main/kotlin/Platform.**ios**.kt

As for the common source set, files with top-level declarations should not have a suffix. For example, `commonMain/kotlin/Platform.kt`.

Technical details

We recommend following this file naming scheme in multiplatform projects due to JVM limitations: it doesn't allow top-level members (functions, properties).

To work around this, the Kotlin JVM compiler creates wrapper classes (so-called "file facades") that contain top-level member declarations. File facades have an internal name derived from the file name.

In turn, JVM doesn't allow several classes with the same fully qualified name (FQN). This might lead to situations when a Kotlin project cannot be compiled to JVM:

```
root
|- commonMain/kotlin/myPackage/Platform.kt // contains 'fun count() { }'
|- jvmMain/kotlin/myPackage/Platform.kt // contains 'fun multiply() { }'
```

Here both `Platform.kt` files are in the same package, so the Kotlin JVM compiler produces two file facades, both of which have FQN `myPackage.PlatformKt`. This produces the "Duplicate JVM classes" error.

The simplest way to avoid that is renaming one of the files according to the guideline above. This naming scheme helps avoid clashes while retaining code readability.

Tip: There are two scenarios where these recommendations may seem redundant, but we still advise to follow them:

- Non-JVM platforms don't have issues with duplicating file facades. However, this naming scheme can help you keep file naming consistent.
- On JVM, if source files don't have top-level declarations, the file facades aren't generated, and you won't face naming clashes.

  However, this naming scheme can help you avoid situations when a simple refactoring or an addition could include a top-level function and result in the same "Duplicate JVM classes" error.

Source file organization

Placing multiple declarations (classes, top-level functions or properties) in the same Kotlin source file is encouraged as long as these declarations are closely related to each other semantically, and the file size remains reasonable (not exceeding a few hundred lines).

In particular, when defining extension functions for a class which are relevant for all clients of this class, put them in the same file with the class itself. When defining extension functions that make sense only for a specific client, put them next to the code of that client. Avoid creating files just to hold all extensions of some class.

Class layout

The contents of a class should go in the following order:

1. Property declarations and initializer blocks
2. Secondary constructors
3. Method declarations
4. Companion object

Do not sort the method declarations alphabetically or by visibility, and do not separate regular methods from extension methods. Instead, put related stuff together, so that someone reading the class from top to bottom can follow the logic of what's happening. Choose an order (either higher-level stuff first, or vice versa) and stick to it.

Put nested classes next to the code that uses those classes. If the classes are intended to be used externally and aren't referenced inside the class, put them in the end, after the companion object.

Interface implementation layout

When implementing an interface, keep the implementing members in the same order as members of the interface (if necessary, interspersed with additional private methods used for the implementation).

Overload layout

Always put overloads next to each other in a class.

Naming rules

Package and class naming rules in Kotlin are quite simple:

- Names of packages are always lowercase and do not use underscores (`org.example.project`). Using multi-word names is generally discouraged, but if you do need to use multiple words, you can either just concatenate them together or use camel case (`org.example.myProject`).
- Names of classes and objects use upper camel case:

```
open class DeclarationProcessor { /*...*/ }

object EmptyDeclarationProcessor : DeclarationProcessor() { /*...*/ }
```

Function names

Names of functions, properties, and local variables start with a lowercase letter and use camel case without underscores:

```
fun processDeclarations() { /*...*/ }
var declarationCount = 1
```

Names for class-like functions

There are two exceptions where function names should follow class-naming convention instead. Functions of this kind are usually defined at the top level.

- Factory functions that create class instances can have the same name as the abstract return type:

  ```
  interface Foo { /*...*/ }

  class FooImpl : Foo { /*...*/ }

  fun Foo(): Foo { return FooImpl() }
  ```

- `@Composable` functions that return `Unit`:

  ```
  @Composable fun TabHeader { /*...*/ }
  ```

Names for test methods

In tests (and **only** in tests), you can use method names with spaces enclosed in backticks. Note that such method names are only supported by Android runtime from API level 30. Underscores in method names are also allowed in test code.

```
class MyTestCase {
    @Test fun `ensure everything works`() { /*...*/ }

    @Test fun ensureEverythingWorks_onAndroid() { /*...*/ }
}
```

Property names

Names of constants (properties marked with `const`, or top-level or object `val` properties with no custom `get` function that hold deeply immutable data) should use all uppercase, underscore-separated names following the screaming snake case convention:

```
const val MAX_COUNT = 8
val USER_NAME_FIELD = "UserName"
```

Names of top-level or object properties which hold objects with behavior or mutable data should use camel case names:

```
val mutableCollection: MutableSet<String> = HashSet()
```

Names of properties holding references to singleton objects can use the same naming style as `object` declarations:

```
val PersonComparator: Comparator<Person> = /*...*/
```

For enum constants, it's OK to use either all uppercase, underscore-separated (screaming snake case) names (`enum class Color { RED, GREEN }`) or upper camel case names, depending on the usage.

Names for backing properties

If a class has two properties which are conceptually the same but one is part of a public API and another is an implementation detail, use an underscore as the prefix for the name of the private property:

```
class C {
    private val _elementList = mutableListOf<Element>()

    val elementList: List<Element>
```

```
        get() = _elementList
}
```

Choose good names

The name of a class is usually a noun or a noun phrase explaining what the class *is*: `List`, `PersonReader`.

The name of a method is usually a verb or a verb phrase saying what the method *does*: `close`, `readPersons`. The name should also suggest if the method is mutating the object or returning a new one. For instance, `sort` is sorting a collection in place, while `sorted` is returning a sorted copy of the collection.

The names should make it clear what the purpose of the entity is, so it's best to avoid using meaningless words (`Manager`, `Wrapper`) in names.

When using an acronym as part of a declaration name, follow these rules:

- For two-letter acronyms, use uppercase for both letters. For example, `IOStream`.
- For acronyms longer than two letters, capitalize only the first letter. For example, `XmlFormatter` or `HttpInputStream`.

Formatting

Indentation

Use four spaces for indentation. Do not use tabs.

For curly braces, put the opening brace at the end of the line where the construct begins, and the closing brace on a separate line aligned horizontally with the opening construct.

```
if (elements != null) {
    for (element in elements) {
        // ...
    }
}
```

> **Note:** In Kotlin, semicolons are optional, and therefore line breaks are significant. The language design assumes Java-style braces, and you may encounter surprising behavior if you try to use a different formatting style.

Horizontal whitespace

- Put spaces around binary operators (`a + b`). Exception: don't put spaces around the "range to" operator (`0..i`).
- Do not put spaces around unary operators (`a++`).

- Put spaces between control flow keywords (`if`, `when`, `for`, and `while`) and the corresponding opening parenthesis.
- Do not put a space before an opening parenthesis in a primary constructor declaration, method declaration or method call.

```
class A(val x: Int)

fun foo(x: Int) { ... }

fun bar() {
    foo(1)
}
```

- Never put a space after `(`, `[`, or before `]`, `)`.
- Never put a space around `.` or `?.`: `foo.bar().filter { it > 2 }.joinToString()`, `foo?.bar()`.
- Put a space after `//`: `// This is a comment`.
- Do not put spaces around angle brackets used to specify type parameters: `class Map<K, V> { ... }`.
- Do not put spaces around `::`: `Foo::class`, `String::length`.
- Do not put a space before `?` used to mark a nullable type: `String?`.

As a general rule, avoid horizontal alignment of any kind. Renaming an identifier to a name with a different length should not affect the formatting of either the declaration or any of the usages.

Colon

Put a space before `:` in the following scenarios:

- When it's used to separate a type and a supertype.
- When delegating to a superclass constructor or a different constructor of the same class.
- After the `object` keyword.

Don't put a space before `:` when it separates a declaration and its type.

Always put a space after `:`.

```
abstract class Foo<out T : Any> : IFoo {
    abstract fun foo(a: Int): T
}

class FooImpl : Foo() {
    constructor(x: String) : this(x) { /*...*/ }

    val x = object : IFoo { /*...*/ }
}
```

Class headers

Classes with a few primary constructor parameters can be written in a single line:

```
class Person(id: Int, name: String)
```

Classes with longer headers should be formatted so that each primary constructor parameter is in a separate line with indentation. Also, the closing parenthesis should be on a new line. If you use inheritance, the superclass constructor call, or the list of implemented interfaces should be located on the same line as the parenthesis:

```
class Person(
    id: Int,
    name: String,
    surname: String
) : Human(id, name) { /*...*/ }
```

For multiple interfaces, the superclass constructor call should be located first and then each interface should be located in a different line:

```
class Person(
    id: Int,
    name: String,
    surname: String
) : Human(id, name),
    KotlinMaker { /*...*/ }
```

For classes with a long supertype list, put a line break after the colon and align all supertype names horizontally:

```
class MyFavouriteVeryLongClassHolder :
    MyLongHolder<MyFavouriteVeryLongClass>(),
    SomeOtherInterface,
    AndAnotherOne {

    fun foo() { /*...*/ }
}
```

To clearly separate the class header and body when the class header is long, either put a blank line following the class header (as in the example above), or put the opening curly brace on a separate line:

```
class MyFavouriteVeryLongClassHolder :
    MyLongHolder<MyFavouriteVeryLongClass>(),
    SomeOtherInterface,
    AndAnotherOne
{


```

```
    fun foo() { /*...*/ }
}
```

Use regular indent (four spaces) for constructor parameters. This ensures that properties declared in the primary constructor have the same indentation as properties declared in the body of a class.

Modifiers order

If a declaration has multiple modifiers, always put them in the following order:

```
public / protected / private / internal
expect / actual
final / open / abstract / sealed / const
external
override
lateinit
tailrec
vararg
suspend
inner
enum / annotation / fun // as a modifier in `fun interface`
companion
inline / value
infix
operator
data
```

Place all annotations before modifiers:

```
@Named("Foo")
private val foo: Foo
```

Unless you're working on a library, omit redundant modifiers (for example, `public`).

Annotations

Place annotations on separate lines before the declaration to which they are attached, and with the same indentation:

```
@Target(AnnotationTarget.PROPERTY)
annotation class JsonExclude
```

Annotations without arguments may be placed on the same line:

```
@JsonExclude @JvmField
var x: String
```

A single annotation without arguments may be placed on the same line as the corresponding declaration:

```
@Test fun foo() { /*...*/ }
```

File annotations

File annotations are placed after the file comment (if any), before the `package` statement, and are separated from `package` with a blank line (to emphasize the fact that they target the file and not the package).

```
/** License, copyright and whatever */
@file:JvmName("FooBar")

package foo.bar
```

Functions

If the function signature doesn't fit on a single line, use the following syntax:

```
fun longMethodName(
    argument: ArgumentType = defaultValue,
    argument2: AnotherArgumentType,
): ReturnType {
    // body
}
```

Use regular indent (four spaces) for function parameters. It helps ensure consistency with constructor parameters.

Prefer using an expression body for functions with the body consisting of a single expression.

```
fun foo(): Int {     // bad
    return 1
}

fun foo() = 1        // good
```

Expression bodies

If the function has an expression body whose first line doesn't fit on the same line as the declaration, put the `=` sign on the first line and indent the expression body by four spaces.

```
fun f(x: String, y: String, z: String) =
    veryLongFunctionCallWithManyWords(andLongParametersToo(), x, y, z)
```

Properties

For very simple read-only properties, consider one-line formatting:

```
val isEmpty: Boolean get() = size == 0
```

For more complex properties, always put `get` and `set` keywords on separate lines:

```
val foo: String
    get() { /*...*/ }
```

For properties with an initializer, if the initializer is long, add a line break after the `=` sign and indent the initializer by four spaces:

```
private val defaultCharset: Charset? =
    EncodingRegistry.getInstance().getDefaultCharsetForPropertiesFiles(file)
```

Control flow statements

If the condition of an `if` or `when` statement is multiline, always use curly braces around the body of the statement. Indent each subsequent line of the condition by four spaces relative to the statement start. Put the closing parentheses of the condition together with the opening curly brace on a separate line:

```
if (!component.isSyncing &&
    !hasAnyKotlinRuntimeInScope(module)
) {
    return createKotlinNotConfiguredPanel(module)
}
```

This helps align the condition and statement bodies.

Put the `else`, `catch`, `finally` keywords, as well as the `while` keyword of a `do-while` loop, on the same line as the preceding curly brace:

```
if (condition) {
    // body
} else {
    // else part
}

try {
    // body
} finally {
    // cleanup
}
```

In a `when` statement, if a branch is more than a single line, consider separating it from adjacent case blocks with a blank line:

```
private fun parsePropertyValue(propName: String, token: Token) {
    when (token) {
```

```
        is Token.ValueToken ->
            callback.visitValue(propName, token.value)

        Token.LBRACE -> { // ...
        }
    }
}
```

Put short branches on the same line as the condition, without braces.

```
when (foo) {
    true -> bar() // good
    false -> { baz() } // bad
}
```

Method calls

In long argument lists, put a line break after the opening parenthesis. Indent arguments by four spaces. Group multiple closely related arguments on the same line.

```
drawSquare(
    x = 10, y = 10,
    width = 100, height = 100,
    fill = true
)
```

Put spaces around the `=` sign separating the argument name and value.

Wrap chained calls

When wrapping chained calls, put the `.` character or the `?.` operator on the next line, with a single indent:

```
val anchor = owner
    ?.firstChild!!
    .siblings(forward = true)
    .dropWhile { it is PsiComment || it is PsiWhiteSpace }
```

The first call in the chain should usually have a line break before it, but it's OK to omit it if the code makes more sense that way.

Lambdas

In lambda expressions, spaces should be used around the curly braces, as well as around the arrow which separates the parameters from the body. If a call takes a single lambda, pass it outside parentheses whenever possible.

```
list.filter { it > 10 }
```

If assigning a label for a lambda, do not put a space between the label and the opening curly brace:

```
fun foo() {
    ints.forEach lit@{
        // ...
    }
}
```

When declaring parameter names in a multiline lambda, put the names on the first line, followed by the arrow and the newline:

```
appendCommaSeparated(properties) { prop ->
    val propertyValue = prop.get(obj)  // ...
}
```

If the parameter list is too long to fit on a line, put the arrow on a separate line:

```
foo {
    context: Context,
    environment: Env
    ->
    context.configureEnv(environment)
}
```

Trailing commas

A trailing comma is a comma symbol after the last item in a series of elements:

```
class Person(
    val firstName: String,
    val lastName: String,
    val age: Int, // trailing comma
)
```

Using trailing commas has several benefits:

- It makes version-control diffs cleaner – as all the focus is on the changed value.
- It makes it easy to add and reorder elements – there is no need to add or delete the comma if you manipulate elements.
- It simplifies code generation, for example, for object initializers. The last element can also have a comma.

Trailing commas are entirely optional – your code will still work without them. The Kotlin style guide encourages the use of trailing commas at the declaration site and leaves it at your discretion for the call site.

To enable trailing commas in the IntelliJ IDEA formatter, go to **Settings/Preferences | Editor | Code Style | Kotlin**, open the **Other** tab and select the **Use trailing comma** option.

Enumerations

```
enum class Direction {
    NORTH,
    SOUTH,
    WEST,
    EAST, // trailing comma
}
```

Value arguments

```
fun shift(x: Int, y: Int) { /*...*/ }
shift(
    25,
    20, // trailing comma
)
val colors = listOf(
    "red",
    "green",
    "blue", // trailing comma
)
```

Class properties and parameters

```
class Customer(
    val name: String,
    val lastName: String, // trailing comma
)
class Customer(
    val name: String,
    lastName: String, // trailing comma
)
```

Function value parameters

```
fun powerOf(
    number: Int,
    exponent: Int, // trailing comma
) { /*...*/ }
constructor(
    x: Comparable<Number>,
    y: Iterable<Number>, // trailing comma
) {}
fun print(
    vararg quantity: Int,
    description: String, // trailing comma
) {}
```

Parameters with optional type (including setters)

```
val sum: (Int, Int, Int) -> Int = fun(
    x,
    y,
    z, // trailing comma
): Int {
    return x + y + x
}
println(sum(8, 8, 8))
```

Indexing suffix

```
class Surface {
    operator fun get(x: Int, y: Int) = 2 * x + 4 * y - 10
}
fun getZValue(mySurface: Surface, xValue: Int, yValue: Int) =
    mySurface[
        xValue,
        yValue, // trailing comma
    ]
```

Parameters in lambdas

```
fun main() {
    val x = {
            x: Comparable<Number>,
            y: Iterable<Number>, // trailing comma
        ->
        println("1")
    }
    println(x)
}
```

when entry

```
fun isReferenceApplicable(myReference: KClass<*>) = when (myReference) {
    Comparable::class,
    Iterable::class,
    String::class, // trailing comma
        -> true
    else -> false
}
```

Collection literals (in annotations)

```
annotation class ApplicableFor(val services: Array<String>)
@ApplicableFor([
    "serializer",
    "balancer",
    "database",
```

```
    "inMemoryCache", // trailing comma
])
fun run() {}
```

Type arguments

```
fun <T1, T2> foo() {}
fun main() {
    foo<
            Comparable<Number>,
            Iterable<Number>, // trailing comma
            >()
}
```

Type parameters

```
class MyMap<
        MyKey,
        MyValue, // trailing comma
        > {}
```

Destructuring declarations

```
data class Car(val manufacturer: String, val model: String, val year: Int)
val myCar = Car("Tesla", "Y", 2019)
val (
    manufacturer,
    model,
    year, // trailing comma
) = myCar
val cars = listOf<Car>()
fun printMeanValue() {
    var meanValue: Int = 0
    for ((
        _,
        _,
        year, // trailing comma
    ) in cars) {
        meanValue += year
    }
    println(meanValue/cars.size)
}
printMeanValue()
```

Documentation comments

For longer documentation comments, place the opening `/**` on a separate line and begin each subsequent line with an asterisk:

```
/**
 * This is a documentation comment
 * on multiple lines.
 */
```

Short comments can be placed on a single line:

```
/** This is a short documentation comment. */
```

Generally, avoid using `@param` and `@return` tags. Instead, incorporate the description of parameters and return values directly into the documentation comment, and add links to parameters wherever they are mentioned. Use `@param` and `@return` only when a lengthy description is required which doesn't fit into the flow of the main text.

```
// Avoid doing this:

/**
 * Returns the absolute value of the given number.
 * @param number The number to return the absolute value for.
 * @return The absolute value.
 */
fun abs(number: Int): Int { /*...*/ }

// Do this instead:

/**
 * Returns the absolute value of the given [number].
 */
fun abs(number: Int): Int { /*...*/ }
```

Avoid redundant constructs

In general, if a certain syntactic construction in Kotlin is optional and highlighted by the IDE as redundant, you should omit it in your code. Do not leave unnecessary syntactic elements in code just "for clarity".

Unit return type

If a function returns Unit, the return type should be omitted:

```
fun foo() { // ": Unit" is omitted here

}
```

Semicolons

Omit semicolons whenever possible.

String templates

Don't use curly braces when inserting a simple variable into a string template. Use curly braces only for longer expressions:

```
println("$name has ${children.size} children")
```

Use multi-dollar string interpolation to treat the dollar sign chars `$` as string literals:

```
val KClass<*>.jsonSchema : String
    get() = $$"""
        {
            "$schema": "https://json-schema.org/draft/2020-12/schema",
            "$id": "https://example.com/product.schema.json",
            "$dynamicAnchor": "meta",
            "title": "$${simpleName ?: qualifiedName ?: "unknown"}",
            "type": "object"
        }
        """
```

Idiomatic use of language features

Immutability

Prefer using immutable data to mutable. Always declare local variables and properties as `val` rather than `var` if they are not modified after initialization.

Always use immutable collection interfaces (`Collection`, `List`, `Set`, `Map`) to declare collections which are not mutated. When using factory functions to create collection instances, always use functions that return immutable collection types when possible:

```
// Bad: use of a mutable collection type for value which will not be mutated
fun validateValue(actualValue: String, allowedValues: HashSet<String>) { ... }

// Good: immutable collection type used instead
fun validateValue(actualValue: String, allowedValues: Set<String>) { ... }

// Bad: arrayListOf() returns ArrayList<T>, which is a mutable collection type
val allowedValues = arrayListOf("a", "b", "c")

// Good: listOf() returns List<T>
val allowedValues = listOf("a", "b", "c")
```

Default parameter values

Prefer declaring functions with default parameter values to declaring overloaded functions.

```
// Bad
fun foo() = foo("a")
fun foo(a: String) { /*...*/ }

// Good
fun foo(a: String = "a") { /*...*/ }
```

Type aliases

If you have a functional type or a type with type parameters which is used multiple times in a codebase, prefer defining a type alias for it:

```
typealias MouseClickHandler = (Any, MouseEvent) -> Unit
typealias PersonIndex = Map<String, Person>
```

If you use a private or internal type alias for avoiding name collision, prefer the `import ... as ...` mentioned in Packages and Imports.

Lambda parameters

In lambdas which are short and not nested, it's recommended to use the `it` convention instead of declaring the parameter explicitly. In nested lambdas with parameters, always declare parameters explicitly.

Returns in a lambda

Avoid using multiple labeled returns in a lambda. Consider restructuring the lambda so that it will have a single exit point. If that's not possible or not clear enough, consider converting the lambda into an anonymous function.

Do not use a labeled return for the last statement in a lambda.

Named arguments

Use the named argument syntax when a method takes multiple parameters of the same primitive type, or for parameters of `Boolean` type, unless the meaning of all parameters is absolutely clear from context.

```
drawSquare(x = 10, y = 10, width = 100, height = 100, fill = true)
```

Conditional statements

Prefer using the expression form of `try`, `if`, and `when`.

```
return if (x) foo() else bar()
```

```
return when(x) {
    0 -> "zero"
    else -> "nonzero"
}
```

The above is preferable to:

```
if (x)
    return foo()
else
    return bar()
```

```
when(x) {
    0 -> return "zero"
    else -> return "nonzero"
}
```

if versus when

Prefer using `if` for binary conditions instead of `when`. For example, use this syntax with `if`:

```
if (x == null) ... else ...
```

Instead of this one with `when`:

```
when (x) {
    null -> // ...
    else -> // ...
}
```

Prefer using `when` if there are three or more options.

Guard conditions in when expression

Use parentheses when combining multiple boolean expressions in `when` expressions or statements with guard conditions:

```
when (status) {
    is Status.Ok if (status.info.isEmpty() || status.info.id == null) -> "no information"
}
```

Instead of:

```
when (status) {
    is Status.Ok if status.info.isEmpty() || status.info.id == null -> "no information"
}
```

Nullable Boolean values in conditions

If you need to use a nullable `Boolean` in a conditional statement, use `if (value == true)` or `if (value == false)` checks.

Loops

Prefer using higher-order functions (`filter`, `map` etc.) to loops. Exception: `forEach` (prefer using a regular `for` loop instead, unless the receiver of `forEach` is nullable or `forEach` is used as part of a longer call chain).

When making a choice between a complex expression using multiple higher-order functions and a loop, understand the cost of the operations being performed in each case and keep performance considerations in mind.

Loops on ranges

Use the `..<` operator to loop over an open-ended range:

```
for (i in 0..n - 1) { /*...*/ }  // bad
for (i in 0..<n) { /*...*/ }  // good
```

Strings

Prefer string templates to string concatenation.

Prefer multiline strings to embedding `\n` escape sequences into regular string literals.

To maintain indentation in multiline strings, use `trimIndent` when the resulting string does not require any internal indentation, or `trimMargin` when internal indentation is required:

```
fun main() {
    println("""
     Not
     trimmed
     text
     """
    )

    println("""
     Trimmed
     text
     """.trimIndent()
    )

    println()

    val a = """Trimmed to margin text:
            |if(a > 1) {
            |    return a
```

```
            |}""".trimMargin()

    println(a)
}
```

Learn the difference between Java and Kotlin multiline strings.

Functions vs properties

In some scenarios, functions with no arguments might be interchangeable with read-only properties. Although the semantics are similar, there are some stylistic conventions on when to prefer one to another.

Prefer a property over a function when the underlying algorithm:

- Does not throw.
- Is cheap to calculate (or cached on the first run).
- Returns the same result over invocations if the object state hasn't changed.

Extension functions

Use extension functions liberally. Every time you have a function that works primarily on an object, consider making it an extension function accepting that object as a receiver. To minimize API pollution, restrict the visibility of extension functions as much as it makes sense. As necessary, use local extension functions, member extension functions, or top-level extension functions with private visibility.

Infix functions

Declare a function as `infix` only when it works on two objects which play a similar role. Good examples: `and`, `to`, `zip`. Bad example: `add`.

Do not declare a method as `infix` if it mutates the receiver object.

Factory functions

If you declare a factory function for a class, avoid giving it the same name as the class itself. Prefer using a distinct name, making it clear why the behavior of the factory function is special. Only if there is really no special semantics, you can use the same name as the class.

```
class Point(val x: Double, val y: Double) {
    companion object {
        fun fromPolar(angle: Double, radius: Double) = Point(...)
    }
}
```

If you have an object with multiple overloaded constructors that don't call different superclass constructors and can't be reduced to a single constructor including parameters with default values, prefer to replace the overloaded constructors with factory functions.

Platform types

A public function/method returning an expression of a platform type must declare its Kotlin type explicitly:

```
fun apiCall(): String = MyJavaApi.getProperty("name")
```

Any property (package-level or class-level) initialized with an expression of a platform type must declare its Kotlin type explicitly:

```
class Person {
    val name: String = MyJavaApi.getProperty("name")
}
```

A local value initialized with an expression of a platform type may or may not have a type declaration:

```
fun main() {
    val name = MyJavaApi.getProperty("name")
    println(name)
}
```

Scope functions apply/with/run/also/let

Kotlin provides a set of functions to execute a block of code in the context of a given object: `let`, `run`, `with`, `apply`, and `also`. For the guidance on choosing the right scope function for your case, refer to Scope Functions.

Coding conventions for libraries

When writing libraries, it's recommended to follow an additional set of rules to ensure API stability:

- Always explicitly specify member visibility (to avoid accidentally exposing declarations as public API).
- Always explicitly specify function return types and property types (to avoid accidentally changing the return type when the implementation changes).
- Provide KDoc comments for all public members, except for overrides that do not require any new documentation (to support generating documentation for the library).

Learn more about best practices and ideas to consider when writing an API for your library in the Library authors' guidelines.

Idioms

A collection of random and frequently used idioms in Kotlin. If you have a favorite idiom, contribute it by sending a pull request.

Create DTOs (POJOs/POCOs)

```
data class Customer(val name: String, val email: String)
```

provides a `Customer` class with the following functionality:

- getters (and setters in case of `var`s) for all properties
- `equals()`
- `hashCode()`
- `toString()`
- `copy()`
- `component1()`, `component2()`, …, for all properties (see Data classes)

Default values for function parameters

```
fun foo(a: Int = 0, b: String = "") { ... }
```

Filter a list

```
val positives = list.filter { x -> x > 0 }
```

Or alternatively, even shorter:

```
val positives = list.filter { it > 0 }
```

Learn the difference between Java and Kotlin filtering.

Check the presence of an element in a collection

```
if ("john@example.com" in emailsList) { ... }

if ("jane@example.com" !in emailsList) { ... }
```

String interpolation

```
println("Name $name")
```

Learn the difference between Java and Kotlin string concatenation.

Read standard input safely

```
// Reads a string and returns null if the input can't be converted into an integer. For exam
ple: Hi there!
val wrongInt = readln().toIntOrNull()
println(wrongInt)
// null

// Reads a string that can be converted into an integer and returns an integer. For example:
13
val correctInt = readln().toIntOrNull()
println(correctInt)
// 13
```

For more information, see Read standard input.

Instance checks

```
when (x) {
    is Foo -> ...
    is Bar -> ...
    else   -> ...
}
```

Read-only list

```
val list = listOf("a", "b", "c")
```

Read-only map

```
val map = mapOf("a" to 1, "b" to 2, "c" to 3)
```

Access a map entry

```
println(map["key"])
map["key"] = value
```

Traverse a map or a list of pairs

```
for ((k, v) in map) {
    println("$k -> $v")
}
```

`k` and `v` can be any convenient names, such as `name` and `age`.

Iterate over a range

```
for (i in 1..100) { ... }  // closed-ended range: includes 100
for (i in 1..<100) { ... } // open-ended range: does not include 100
for (x in 2..10 step 2) { ... }
for (x in 10 downTo 1) { ... }
(1..10).forEach { ... }
```

Lazy property

```
val p: String by lazy { // the value is computed only on first access
    // compute the string
}
```

Extension functions

```
fun String.spaceToCamelCase() { ... }

"Convert this to camelcase".spaceToCamelCase()
```

Create a singleton

```
object Resource {
    val name = "Name"
}
```

Use inline value classes for type-safe values

```
@JvmInline
value class EmployeeId(private val id: String)

@JvmInline
value class CustomerId(private val id: String)
```

If you accidentally mix up `EmployeeId` and `CustomerId`, a compilation error is triggered.

> **Note:** The `@JvmInline` annotation is only needed for JVM backends.

Instantiate an abstract class

```
abstract class MyAbstractClass {
    abstract fun doSomething()
    abstract fun sleep()
}

fun main() {
    val myObject = object : MyAbstractClass() {
        override fun doSomething() {
            // ...
        }

        override fun sleep() { // ...
        }
    }
    myObject.doSomething()
}
```

If-not-null shorthand

```
val files = File("Test").listFiles()

println(files?.size) // size is printed if files is not null
```

If-not-null-else shorthand

```
val files = File("Test").listFiles()

// For simple fallback values:
println(files?.size ?: "empty") // if files is null, this prints "empty"

// To calculate a more complicated fallback value in a code block, use `run`
val filesSize = files?.size ?: run {
    val someSize = getSomeSize()
    someSize * 2
}
println(filesSize)
```

Execute an expression if null

```
val values = ...
```

```
val email = values["email"] ?: throw IllegalStateException("Email is missing!")
```

Get first item of a possibly empty collection

```
val emails = ... // might be empty
val mainEmail = emails.firstOrNull() ?: ""
```

Learn the difference between Java and Kotlin first item getting.

Execute if not null

```
val value = ...

value?.let {
    ... // execute this block if not null
}
```

Map nullable value if not null

```
val value = ...

val mapped = value?.let { transformValue(it) } ?: defaultValue
// defaultValue is returned if the value or the transform result is null.
```

Return on when statement

```
fun transform(color: String): Int {
    return when (color) {
        "Red" -> 0
        "Green" -> 1
        "Blue" -> 2
        else -> throw IllegalArgumentException("Invalid color param value")
    }
}
```

try-catch expression

```
fun test() {
    val result = try {
        count()
    } catch (e: ArithmeticException) {
        throw IllegalStateException(e)
    }
```

```
    // Working with result
}
```

if expression

```
val y = if (x == 1) {
    "one"
} else if (x == 2) {
    "two"
} else {
    "other"
}
```

Builder-style usage of methods that return Unit

```
fun arrayOfMinusOnes(size: Int): IntArray {
    return IntArray(size).apply { fill(-1) }
}
```

Single-expression functions

```
fun theAnswer() = 42
```

This is equivalent to

```
fun theAnswer(): Int {
    return 42
}
```

This can be effectively combined with other idioms, leading to shorter code. For example, with the `when` expression:

```
fun transform(color: String): Int = when (color) {
    "Red" -> 0
    "Green" -> 1
    "Blue" -> 2
    else -> throw IllegalArgumentException("Invalid color param value")
}
```

Call multiple methods on an object instance (with)

```
class Turtle {
    fun penDown()
    fun penUp()
```

```
    fun turn(degrees: Double)
    fun forward(pixels: Double)
}

val myTurtle = Turtle()
with(myTurtle) { //draw a 100 pix square
    penDown()
    for (i in 1..4) {
        forward(100.0)
        turn(90.0)
    }
    penUp()
}
```

Configure properties of an object (apply)

```
val myRectangle = Rectangle().apply {
    length = 4
    breadth = 5
    color = 0xFAFAFA
}
```

This is useful for configuring properties that aren't present in the object constructor.

Java 7's try-with-resources

```
val stream = Files.newInputStream(Paths.get("/some/file.txt"))
stream.buffered().reader().use { reader ->
    println(reader.readText())
}
```

Generic function that requires the generic type information

```
//  public final class Gson {
//     ...
//     public <T> T fromJson(JsonElement json, Class<T> classOfT) throws JsonSyntaxException
{
//     ...

inline fun <reified T: Any> Gson.fromJson(json: JsonElement): T = this.fromJson(json, T::cla
ss.java)
```

Swap two variables

```
var a = 1
var b = 2
```

```
a = b.also { b = a }
```

Mark code as incomplete (TODO)

Kotlin's standard library has a `TODO()` function that will always throw a `NotImplementedError`. Its return type is `Nothing` so it can be used regardless of expected type. There's also an overload that accepts a reason parameter:

```
fun calcTaxes(): BigDecimal = TODO("Waiting for feedback from accounting")
```

IntelliJ IDEA's kotlin plugin understands the semantics of `TODO()` and automatically adds a code pointer in the TODO tool window.

What's next?

- Solve Advent of Code puzzles using the idiomatic Kotlin style.
- Learn how to perform typical tasks with strings in Java and Kotlin.
- Learn how to perform typical tasks with collections in Java and Kotlin.
- Learn how to handle nullability in Java and Kotlin.

Types overview

In Kotlin, everything is an object in the sense that you can call member functions and properties on any variable. Some types, such as numbers, characters, and booleans, have an optimized internal representation as primitive values at runtime, but they appear and behave like regular classes in Kotlin code.

Basic types

This section describes the basic types used in Kotlin:

Category	Basic types	Definition
Integers	`Byte`, `Short`, `Int`, `Long`	Whole numbers
Unsigned integers	`UByte`, `UShort`, `UInt`, `ULong`	Non-negative whole numbers
Floating-point numbers	`Float`, `Double`	Numbers with a fractional part
Booleans	`Boolean`	Logical values: `true` and `false`
Characters	`Char`	A single character
Strings	`String`	A sequence of characters
Arrays	`Array<T>`, primitive-type arrays	A fixed-size sequence of values

> **Note:** By default, every type is non-nullable. To allow `null` values, declare a variable with a `?` sign right after the variable type. For example, `String?`. Learn more in Null safety.

To learn about other Kotlin types, such as `Nothing`, `Any`, and `Unit`, look through the Kotlin API reference:

- `Any` – The root of the Kotlin class hierarchy.
- `Nothing` – A type that has no values.
- `Unit` – A type with only one value (`Unit`). The compiler infers it as the return type of functions with a block body and no explicit return type. See Unit-returning functions.

Non-denotable types

Kotlin also has non-denotable types. They are the types that you can't write directly in the Kotlin code. Instead, the compiler uses them internally, for example, for interoperability with other languages. Kotlin creates non-denotable types to represent type information that is more precise than what Kotlin source syntax allows.

Even though you can't declare non-denotable types yourself, you may encounter them in compiler diagnostics, IDE tooltips, or inferred type displays. Learn more about non-denotable types in:

- Platform types
-
-
-
- Kotlin language specification: Type system

  Tip: Learn how to perform type checks and casts in Kotlin.

Numbers

The Kotlin number types represent:

- Integer values (Byte, Short, Int, and Long)
- Floating-point values (Float and Double)

Use number types to store and process numeric data, for example, in arithmetic, counters, measurements, and other calculations.

Choose a number type

In most cases, you can refer to the following rules to determine the correct number type for your task:

- Use `Int` for whole numbers.
- Use `Long` for whole numbers outside the `Int` range.
- Use `Double` for decimal numbers.
- Use `Float` when lower precision is acceptable or required.
- Use `Byte` and `Short` when an API or data format requires them.

  Tip: Kotlin also provides as a Beta feature.

Integer types

Kotlin provides four integer types with different sizes and value ranges:

Type	Size (bits)	Min value	Max value
`Byte`	8	-128	127
`Short`	16	-32768	32767
`Int`	32	-2,147,483,648 (-2^{31})	2,147,483,647 ($2^{31} - 1$)
`Long`	64	-9,223,372,036,854,775,808 (-2^{63})	9,223,372,036,854,775,807 ($2^{63} - 1$)

Declare integer values

Kotlin supports the following literal forms for integer values:

- Decimals: `123`
- Hexadecimals: `0x0F`
- Binaries: `0b00001011`

Note: Kotlin does not support octal literals.

To declare a numeric value, specify the type explicitly:

```
val one: Int = 1

// Use underscores to improve readability
val oneBillion: Long = 1_000_000_000
val hexBytes: Int = 0x7F_EC_DE_5E
val bytes: Int = 0b01010010_01101001_10010100_10010010

val oneByte: Byte = 1
val oneShort: Short = 1
```

You can also append the `L` suffix, to declare a `Long` value:

```
val oneLong = 1L
```

When you declare a numeric type explicitly, the compiler checks that the value fits in the range of that type:

```
// Value fits in Byte
val oneByte: Byte = 1

// Error: the value does not fit in Byte
val tooBig: Byte = 128
```

When you do not specify a numeric type, Kotlin infers `Int` if the value fits in the `Int` range. Otherwise, Kotlin infers `Long`:

```
val million = 1_000_000 // Int
val threeBillion = 3_000_000_000 // Long
```

If a value can be absent, use nullable types:

```
val maybeAbsent: Int? = null
```

Floating-point types

For numbers with a fractional part, Kotlin provides `Float` and `Double`.

Floating-point types follow the IEEE 754 standard. `Float` reflects the *single precision*. `Double` reflects the *double precision*.

Floating-point types differ in size and precision:

Type	Size (bits)	Significant bits	Exponent bits	Decimal digits
`Float`	32	24	8	6-7
`Double`	64	53	11	15-16

Declare floating-point values

To declare a floating-point literal, include a decimal point (`.`) or use exponent notation:

```
val pi = 3.14
val avogadro = 6.02214076e23
```

By default, Kotlin infers floating-point literals as `Double`. To declare a `Float`, add the `f` or `F` suffix:

```
val pi = 3.14 // Double
val eFloat = 2.7182817f // Float
```

> **Note:** Kotlin rounds a `Float` literal that contains more precision than `Float` can store.

If a value can be absent, use nullable types:

```
val maybeAbsent: Double? = null
```

Arithmetic operations

Kotlin supports the standard arithmetic operations on numbers: `+`, `-`, `*`, `/`, and `%`.

Use these operators to perform common calculations:

```
fun main() {
    println(1 + 2) // 3
    println(2_500_000_000L - 1L) // 2499999999
    println(3.14 * 2.71) // 8.5094
    println(10.0 / 3) // 3.3333333333333335
}
```

The result type depends on the types of the operands. Learn more in .

> **Tip:** You can override these operators in custom number classes. For more information, see Operator overloading.

Integer division

Division between integer values always returns an integer result. The compiler discards the fractional part:

```
fun main() {
    val intValue = 5 / 2
    println(intValue) // 2

    val longValue = 5L / 2
    println(longValue) // 2
}
```

To return a floating-point result, make at least one operand a `Float` or `Double`:

```
fun main() {
    val a = 5 / 2.0
    println(a) // 2.5

    val b = 5 / 2.toDouble()
    println(b) // 2.5
}
```

Type conversion

Numeric types are not subtypes of one another. Kotlin requires explicit conversions to avoid silent data loss and unexpected behavior.

For example, a function that expects `Double` cannot accept an `Int` or a `Float` value without conversion:

```
fun main() {
    fun printDouble(x: Double) {
        print(x)
    }

    val x = 1.0
    val xInt = 1
    val xFloat = 1.0f
    val one: Double = 1 // Error: initializer type mismatch

    printDouble(x) // OK
    printDouble(xInt) // Error: argument type mismatch
    printDouble(xFloat) // Error: argument type mismatch
}
```

All number types support conversions to other number types. To convert a number to another type, use an explicit conversion function:

- `toByte()`
- `toShort()`
- `toInt()`
- `toLong()`
- `toFloat()`

- `toDouble()`

For example, the following code converts an `Int` value to `Double`:

```
fun main() {
    val intValue: Int = 1
    val doubleValue = intValue.toDouble()

    println(doubleValue) // 1.0
}
```

When you convert a floating-point value to an integer type, the compiler discards the fractional part:

```
fun main() {
    val d: Double = 1.5
    val l: Long = d.toLong()

    println(l) // 1
}
```

Mixed numeric expressions

Kotlin does not support implicit conversion for assignments or function arguments. However, you can combine different numeric types in arithmetic expressions. In such cases, Kotlin determines a result type based on the operand types, and arithmetic operators handle the conversion automatically:

```
val intNumber: Int = 1
val longNumber: Long = 1000
val result = intNumber + longNumber // 1001, Long
```

If you try to assign the result to a smaller type, the compiler reports an error:

```
val intNumber: Int = 1
val longNumber: Long = 1000
val result: Int = intNumber + longNumber
// Error: Initializer type mismatch
```

Integer literal types

During type inference, Kotlin treats unsuffixed integer literals as a special Integer Literal Type (ILT) until the surrounding context determines a specific type:

```
fun List<Any>.log() {
    println(joinToString(" | ") { it::class.simpleName ?: "Unknown" })
}
```

```
fun main() {
    listOf(1, 2).log()
    // Int | Int

    listOf(1L, 2L).log()
    // Long | Long

    // Compiler interprets 1 as an ILT and resolves it to Long
    listOf(1, 2L).log()
    // Long | Long

    // .toInt() converts the literal to Int
    listOf(1.toInt(), 2L).log()
    // Int | Long
}
```

It's especially easy to miss with the `Int` and `Long` values because they have the same string representation at runtime. To avoid this, specify the expected type or convert values explicitly:

```
fun List<Any>.log() {
    println(joinToString(" | ") { it::class.simpleName ?: "Unknown" })
}

fun main() {
    val longValues: List<Long> = listOf(1, 2L)
    longValues.log()
    // Long | Long

    val numberValues: List<Number> = listOf(1.toInt(), 2L)
    numberValues.log()
    // Int | Long
}
```

You can also use an explicit type to catch unintended type inference:

```
fun main() {
    val intValues: List<Int> = listOf(1, 2L)
    // Error: initializer type mismatch
}
```

> **Tip:** Learn more about Integer literal types.

Data overflow

Numeric types can represent only values within their defined ranges.

If the result of an operation falls outside that range, overflow occurs. If you convert a value to a smaller numeric type, the converted value may not preserve the original numeric value.

This behavior can affect the result of your code even when the compiler accepts it.

Overflow in operations

Each integer type can store only values within its defined range. When the result of an arithmetic operation exceeds that range, *data overflow* occurs:

```
fun main(){
    val intNumber: Int = 2147483647
    // Max Int value is 2147483647
    println(intNumber + 1) // -2147483648
}
```

Here, the result wraps around because the value no longer fits in `Int`.

> **Note:** The compiler does not automatically produce an error when integer overflow occurs.

Overflow in negation

Overflow can also occur during negation. For example, you cannot represent the positive counterpart of `Int.MIN_VALUE` as an `Int`.

```
fun main(){
    val min = Int.MIN_VALUE
    println(-min) // -2147483648
}
```

Narrowing conversions

When you convert a value to a smaller integer type, the result may not preserve the original numeric value:

```
fun main() {
    val large: Int = 130
    val narrowed: Byte = large.toByte()

    println(narrowed) // -126
}
```

However, since floating-point types follow the IEEE 754 Standard, very large results can become `Infinity`:

```
fun main() {
    println(Double.MAX_VALUE * 2) // Infinity
}
```

Bitwise operations

Kotlin provides *bitwise operations* for `Int` and `Long`. These operations are represented by a set of infix functions and `inv()`.

```
fun main() {
    val x = 1

    println(x shl 2) // 4
    println(x and 0x000FF000) // 0
}
```

Bitwise operations include:

- `shl()` – signed shift left
- `shr()` – signed shift right
- `ushr()` – unsigned shift right
- `and()` – bitwise AND
- `or()` – bitwise OR
- `xor()` – bitwise XOR
- `inv()` – bitwise inversion

Floating-point number comparison

In Kotlin, floating-point comparison depends on the static type of the operands.

When the operands are statically known to be `Float` or `Double`, operations on the numbers and the range that they form follow the IEEE 754 Standard for Floating-Point Arithmetic.

However, in generic use cases (such as `Any`, `Comparable<...>`, or `Collection<T>`), behavior differs for operands that are not statically typed as floating-point numbers. In these cases, Kotlin uses the `equals()` and `compareTo()` implementations for `Float` and `Double`.

As a result:

- `NaN` is considered equal to itself
- `NaN` is considered greater than any other element including `POSITIVE_INFINITY`
- `-0.0` is considered less than `0.0`

The following example shows the difference between operands statically typed as floating-point numbers and operands used through generic types:

```
fun generalizedEquals(a: Any, b: Any): Boolean {
    return a == b
}

fun main() {
```

```
    // Operands statically typed as floating-point numbers
    println(Double.NaN == Double.NaN) // false
    println(0.0 == -0.0) // true

    // Operands used through a non-floating-point static type
    println(generalizedEquals(Double.NaN, Double.NaN)) // true
    println(generalizedEquals(0.0, -0.0)) // false
}
```

Boxing and caching numbers on the JVM

On the JVM, non-nullable numeric values are usually stored using primitive types, such as `int`, `long`, or `double`. However, when you use generic types or nullable numeric types like `Int?`, the value is boxed and represented as an object.

The JVM applies a memory optimization technique to small numbers by caching their boxed representations. As a result, boxed numbers with the same value can be referentially equal.

For example, the JVM caches boxed `Integer` values in the range `-128` to `127`. Therefore, the following code returns `true`:

```
fun main() {
    val score: Int = 100
    val savedScore: Int? = score
    val displayedScore: Int? = score

    println(savedScore === displayedScore) // true
}
```

For values outside the cached range, boxed values are separate objects. In that case, they are not referentially equal, even if their values are structurally equal. For this reason, use `==` to compare numeric values:

```
fun main() {
    val score: Int = 10000
    val savedScore: Int? = score
    val displayedScore: Int? = score

    println(savedScore === displayedScore) // false
    println(savedScore == displayedScore) // true
}
```

Unsigned integer types

In addition to integer types, Kotlin provides the following types for unsigned integer numbers:

Type	Size (bits)	Min value	Max value
`UByte`	8	0	255
`UShort`	16	0	65,535
`UInt`	32	0	4,294,967,295 (2^{32} - 1)
`ULong`	64	0	18,446,744,073,709,551,615 (2^{64} - 1)

Unsigned types support most of the operations of their signed counterparts.

> **Note:** Unsigned numbers are implemented as inline classes with a single storage property that contains the corresponding signed counterpart type of the same width. If you want to convert between unsigned and signed integer types, make sure you update your code so that any function calls and operations support the new type.

Unsigned arrays and ranges

> **Warning:** Unsigned arrays and operations on them are in Beta. They can be changed incompatibly at any time. Opt-in is required (see the details below).

Same as for primitives, each unsigned type has a corresponding type that represents arrays of that type:

- `UByteArray`: an array of unsigned bytes.
- `UShortArray`: an array of unsigned shorts.
- `UIntArray`: an array of unsigned ints.
- `ULongArray`: an array of unsigned longs.

Same as for signed integer arrays, they provide a similar API to the `Array` class without boxing overhead.

When you use unsigned arrays, you receive a warning that indicates that this feature is not stable yet. To remove the warning, opt-in with the `@ExperimentalUnsignedTypes` annotation. It's up to you to decide if your clients have to explicitly opt-in into usage of your API, but keep in mind that unsigned arrays are not a stable feature, so an API that uses them can be broken by changes in the language. Learn more about opt-in requirements.

Ranges and progressions are supported for `UInt` and `ULong` by classes `UIntRange`, `UIntProgression`, `ULongRange`, and `ULongProgression`. Together with the unsigned integer types, these classes are

stable.

Unsigned integers literals

To make unsigned integers easier to use, you can append a suffix to an integer literal indicating a specific unsigned type (similarly to `F` for `Float` or `L` for `Long`):

- `u` and `U` letters signify unsigned literals without specifying the exact type. If no expected type is provided, the compiler uses `UInt` or `ULong` depending on the size of the literal:

```
val b: UByte = 1u  // UByte, expected type provided
val s: UShort = 1u // UShort, expected type provided
val l: ULong = 1u  // ULong, expected type provided

val a1 = 42u // UInt: no expected type provided, constant fits in UInt
val a2 = 0xFFFF_FFFF_FFFFu // ULong: no expected type provided, constant doesn't fit i
n UInt
```

- `uL` and `UL` explicitly specify that literal should be an unsigned long:

```
val a = 1UL // ULong, even though no expected type provided and the constant fits into
UInt
```

Use cases

The main use case of unsigned numbers is utilizing the full bit range of an integer to represent positive values.
For example, to represent hexadecimal constants that do not fit in signed types such as color in 32-bit `AARRGGBB` format:

```
data class Color(val representation: UInt)

val yellow = Color(0xFFCC00CCu)
```

You can use unsigned numbers to initialize byte arrays without explicit `toByte()` literal casts:

```
val byteOrderMarkUtf8 = ubyteArrayOf(0xEFu, 0xBBu, 0xBFu)
```

Another use case is interoperability with native APIs. Kotlin allows representing native declarations that contain unsigned types in the signature. The mapping won't substitute unsigned integers with signed ones keeping the semantics unaltered.

Non-goals

While unsigned integers can only represent positive numbers and zero, it's not a goal to use them where application domain requires non-negative integers. For example, as a type of collection size or collection index value.

There are a couple of reasons:

- Using signed integers can help to detect accidental overflows and signal error conditions, such as `List.lastIndex` being -1 for an empty list.
- Unsigned integers cannot be treated as a range-limited version of signed ones because their range of values is not a subset of the signed integers range. Neither signed nor unsigned integers are subtypes of each other.

Booleans

The `Boolean` type represents logical values: `true` and `false`.

Use `Boolean` values in functions that answer yes-or-no questions, and in the `while`, `if`, and `when` conditions.

Declare a `Boolean` variable

To declare a `Boolean` variable, assign it `true` or `false`.

You can specify the `Boolean` type explicitly or let Kotlin infer it from the value:

```
val isTrue: Boolean = true
val isFalse = false // Kotlin infers Boolean
```

If a value can be `null`, use `Boolean?`:

```
val isEnabled: Boolean? = null
```

> **Note:** You cannot assign an integer value to a `Boolean` variable. In Kotlin, `0` and `1` are not `Boolean` values.

Produce `Boolean` values

You can use comparison expressions and functions to produce `Boolean` values:

```
fun main() {
    val number = 10
    val isPositive = number > 0
    println(isPositive) // true

    val language = "Kotlin"
    val isEmpty = language.isEmpty()
    println(isEmpty) // false
}
```

You can use the results in conditions and other expressions as well:

```
fun main() {
    val number = 10
    val isPositive = number > 0 // true

    if (isPositive) {
        println("The number is positive.")
```

```
    }
}
```

`Boolean` operations

Kotlin provides operators and infix functions for working with `Boolean` values. You can use them to invert a `Boolean` value or combine multiple `Boolean` values into a single result.

Negation (NOT)

The NOT operator inverts a `Boolean` value.

To use NOT, place the `!` operator before a `Boolean` value:

```
val isOn = true
val isOff = !isOn // isOff is false
```

Logical AND

The AND operator returns `true` only if both operands are `true`.

To use logical AND, place the `&&` operator between operands:

```
val a = false && false // false
val b = false && true // false
val c = true && false // false
val d = true && true  // true
```

> **Note:** If the first operand is `false`, the `&&` operator skips the second operand. To evaluate both operands, use the `and` infix function instead.

Logical OR

The OR operator returns `true` if at least one operand is `true`.

To use logical OR, place the `||` operator between operands:

```
val a = false || false // false
val b = false || true  // true
val c = true || false  // true
val d = true || true   // true
```

> **Note:** If the first operand is `true`, the `||` operator skips the second operand. To evaluate both operands, use the `or` infix function instead.

Exclusive OR (XOR)

The exclusive OR (XOR) operation returns `true` if the operands have different values.

To use XOR, write `xor` between operands:

```
val a = false xor false // false
val b = false xor true  // true
val c = true xor false  // true
val d = true xor true   // false
```

> **Note:** `xor` is an infix function, not an operator.
>
> Learn more about `Boolean` functions in the API Reference.

Operator precedence

If an expression contains multiple logical operations and no parentheses to specify the evaluation order, Kotlin applies precedence rules. Operations with higher precedence are evaluated before operations with lower precedence.

For the `Boolean` operations described in this section, the precedence order is as follows:

1. `!`
2. `xor` (and other infix functions)
3. `&&`
4. `||`

In the following example, the compiler evaluates `&&` before `||`:

```
fun main() {
    val result = true || false && false
    println(result) // true
}
```

To make evaluation order explicit, use parentheses:

```
fun main() {
    val result = (true || false) && false
    println(result) // false
}
```

`Boolean` in conditions

`if`, `when`, and `while` evaluate `Boolean` expressions to direct program flow.

`if` expressions

```
fun main() {
    val number = 4
    val isEven = number % 2 == 0

    // Condition already has the `Boolean` type
    // You do not need to compare it to `true` or `false`
    if (isEven) {
        println("The number is even.")
    } else {
        println("The number is odd.")
    }
}
```

`when` expressions

```
fun main() {
    val number = 3

    when {
        number > 0 -> println("The number is positive.")
        number < 0 -> println("The number is negative.")
        else -> println("The number is zero.")
    }
}
```

`while` loops

```
fun main() {
    var isCalculating = true

    while (isCalculating) {
        println("Calculating...")
        isCalculating = false
    }
}
```

Characters

The `Char` type represents a single character as a UTF-16 code unit.

Use `Char` for individual character values, such as letters, digits, punctuation marks, or whitespace. For sequences of characters, use `String`.

> **Tip:** `Char` is not a numeric type, but each character has a numeric Unicode value that you can access. See .

Syntax

To declare a character, enclose the value in single quotes (`' '`). You can specify the `Char` type explicitly or let Kotlin infer it from the value:

```
val letter: Char = 'a'

// Kotlin infers Char because the values are written in single quotes
val digit = '1'
val symbol = '!'
val space = ' '
val separator = ':'
```

A character literal must contain exactly one character. Otherwise, the Kotlin compiler reports an error:

```
val invalid = 'AB' // Error
val invalidEmpty = '' // Error
```

Nullable values

To store a nullable value, use `Char?`:

```
val maybeAbsent: Char? = null
```

> **Note:** On the JVM, nullable `Char` values are boxed when needed. The same applies to numeric types.

Unicode support

Kotlin represents `Char` values as UTF-16 code units. This means that a single `Char` stores one UTF-16 code unit, not necessarily one complete Unicode character.

Basic Multilingual Plane

A single `Char` can store values in the range from `\u0000` to `\uFFFF`. This range covers the Basic Multilingual Plane (BMP) that includes characters for almost all modern languages and a large number of symbols.

To specify a character by the Unicode value, use `\u` followed by four-digit hexadecimal value from the Unicode table:

```
val unicodeNumber = '\u0031' // Equals '1'
```

Supplementary characters

Unicode characters outside the BMP, such as emojis and some historic scripts, cannot be represented by a single `Char`. In UTF-16, they are encoded as a *surrogate pair*, where two `Char` values together represent one Unicode character in a `String`:

```
fun main() {
    val emoji = "🥦"

    println(emoji.length) // 2
    println(emoji[0])     // First surrogate
    println(emoji[1])     // Second surrogate
}
```

> **Tip:** To handle 32-bit symbols individually, use Unicode code points stored as `Int` values.

Escape sequences

Use escape sequences for special characters that are difficult to write directly in source code or have a special meaning.

Every escape sequence begins with a backslash (`\`).

Supported sequence	Description
`\t`	Tab
`\b`	Backspace
`\n`	New Line (LF)
`\r`	Carriage Return (CR)
`\'`	Single quotation mark
`\"`	Double quotation mark

Supported sequence	Description
`\\`	Backslash
`\$`	Dollar sign

For example:

```
val newLine = '\n'
val dollar = '\$'
val backslash = '\\'
```

Operations

`Char` supports comparison, inspection, case conversion, and explicit numeric conversion.

Character comparison

To compare `Char` values, use standard comparison operators such as `==`, `!=`, `<`, `>`, `<=`, and `>=`.

Kotlin compares `Char` values by their numeric Unicode values and returns a `Boolean` value:

```
val before = 'a' < 'b' // true
val after = 'c' > 'd' // false
val different = 'A' == 'a' // false
val equal = 'A' == 'A' // true
```

Character processing

Kotlin provides functions for inspection and case conversion of character values. For example:

```
fun main() {
    val myChar = 'A'
    // Checks if the character represents a digit
    println(myChar.isDigit()) // false
    // Checks if the character represents an uppercase letter
    println(myChar.isUpperCase()) // true
    // Returns a lowercase version
    println(myChar.lowercaseChar()) // 'a'
}
```

> **Note:** Learn more about available functions in the API Reference.

Character arithmetic

You can create another character value by adding or subtracting an integer:

```
fun main() {
    val a = 'a'
```

```
    println(a + 1)  // b
    println(a + 2)  // c
    println(a - 32) // A
}
```

Note: These operations follow Unicode values, not language-specific alphabet rules.

You can also use the increment (`++`) and decrement (`--`) operators in the prefix and postfix forms with mutable variables:

```
fun main() {
    var a = 'A'

    a += 10
    println(a)   // 'K'

    println(++a) // 'L'  prefix increment
    println(a++) // 'L'  postfix increment
    println(a)   // 'M'

    println(--a) // 'L'  prefix decrement
    println(a--) // 'L'  postfix decrement
    println(a)   // 'K'
}
```

Character conversion

To convert `Char` to a numeric type, use explicit conversion:

- Use `.code` to get the numeric Unicode value of a character:

  ```
  fun main() {
      val letter = 'A'
      println(letter.code) // 65
  }
  ```

- If a character represents a decimal digit, use `digitToInt()`:

  ```
  fun main() {
      val digit = '7'
      println(digit.digitToInt()) // 7
  }
  ```

  If the character may not be a valid digit, use `digitToIntOrNull()`.

Strings

The `String` type represents a sequence of characters. You can use it for text values, such as words, sentences, messages, or structured text.

The `String` type is immutable. After you create a `String` object, its contents stay the same for the rest of its lifetime. Any operation that appears to modify the string actually creates a new string.

Declare strings

To declare a `String` literal, enclose the value in double quotes (`""`). You can specify the `String` type explicitly or let Kotlin infer it from the value:

```
val name: String = "Kotlin"
val message = "Hello, world!" // Kotlin infers String
```

Double-quoted string literals support escape sequences such as `\n` or `\t`:

```
val message = "Hello,\nworld!"
val quote = "Kotlin says, \"Hi\"."
```

Multiline strings

To store text that consists of multiple lines or contains quotes that you don't want to escape, use a multiline string enclosed in triple quotes (`""" """`):

```
val text = """
Hello,
Kotlin
"""

val quote = """Kotlin says, "Hi"."""
```

> **Note:** Multiline strings don't support escape sequences. Kotlin treats these characters as regular text.

Multiline strings preserve line breaks and indentations as written in the source code. This behavior is useful when you want the runtime value to match the text layout in your file.

In the following example, the spaces before each line are part of the resulting string:

```
val text = """
    Hello,
```

```
    Kotlin
"""
```

To remove common leading indentation, use the `trimIndent()` function. It detects the common minimal indent of non-empty lines and removes it:

```
fun main() {
    val text = """
        Hello,
        Kotlin
    """.trimIndent()

    println(text)
}
```

To control indentation removal more explicitly, use the `trimMargin()` function. It removes everything before and including the margin prefix on each line:

```
fun main() {
    val text = """
        |Hello,
        |Kotlin
    """.trimMargin()

    println(text)
}
```

By default, the `trimMargin()` function uses a pipe symbol (`|`) as the margin prefix, but you can pass another character as a parameter. For example: `trimMargin(">")`.

> **Note:** When you process a string with functions like `trimIndent()` or `trimMargin()`, the resulting string uses only newline (`\n`) separators, regardless of the platform.

String templates

String templates let you embed variables and expressions directly inside a `String` literal. This process is called *interpolation*. You can use string templates in both regular and multiline strings.

To insert a variable into a string, use the `$` symbol:

```
fun main() {
    val name = "Kotlin"
    println("Hello, $name")
    // Hello, Kotlin
}
```

To insert an expression into a string or to place a variable directly next to other text, use `${}`:

```
fun main() {
    val text = "abc"
    println("The length of $text is ${text.length}")
    // The length of abc is 3

    val language = "Kotlin"
    println("${language}Lang")
    // KotlinLang
}
```

> **Tip:** You can also combine strings with the `+` operator. However, string templates are usually easier to read and more idiomatic.

Template expressions can also contain double-quoted strings without escaping:

```
// Double-quoted string
val test = "${"test".uppercase()}"

// Multiline string
val result = """
Result: ${"OK".lowercase()}
"""
```

Nullable values in string templates

If an interpolated expression or variable evaluates to `null`, the Kotlin compiler inserts the text `null` into the resulting string. To replace `null` with another value, use the Elvis operator (`?:`):

```
fun main(){
    val text: String? = null

    println("Hello, $text")
    // Hello, null

    println("Hello, ${text ?: "Kotlin"}")
    // Hello, Kotlin
}
```

Multi-dollar string interpolation

In regular string templates, a single dollar sign (`$`) starts interpolation. If you need to include literal dollar signs in a string, use **multi-dollar string interpolation**.

Multi-dollar string interpolation allows you to specify how many consecutive dollar signs are required to trigger interpolation. Dollar signs below that number are treated as literal characters.

For example, when you use `$$` before a string literal, interpolation begins only with two consecutive dollar signs:

```
val KClass<*>.jsonSchema : String
    get() = $$"""
    {
      "$schema": "https://json-schema.org/draft/2020-12/schema",
      "$id": "https://example.com/product.schema.json",
      "$dynamicAnchor": "meta",
      "title": "$${simpleName ?: qualifiedName ?: "unknown"}",
      "type": "object"
    }
    """
```

Tip: If you use single-dollar string interpolation, multi-dollar string interpolation doesn't affect your code. You can continue using a single `$` and apply multi-dollar signs when needed.

Basic string operations

Kotlin provides a range of operations for working with strings. This section introduces some of the most commonly used operations.

Tip: Learn more about all available functions in the API Reference.

Get string length

To get the number of characters in a string, use the `length` property:

```
fun main (){
    val language = "Kotlin"
    println(language.length)
    // 6
}
```

Access characters

You can access an individual character in a string with the indexing operator (`[]`):

```
fun main (){
    val language = "Kotlin"

    println(language[0])
    // K
    println(language[5])
    // n
}
```

Tip: A string index starts at zero. If you try to access an index outside the valid range, Kotlin throws an exception.

You can also iterate over the characters in a string:

```
fun main(){
    for (char in "Kotlin") {
      println(char)
    }
}
```

Extract parts of a string

To extract parts of a string, use one of the following functions:

- `substring()` to return a new string with the selected part of the original text.
- `subSequence()` to return a `CharSequence` with the selected part of the original text.

For example:

```
fun main() {
    val text = "Kotlin"
    println(text.substring(1))
    // otlin
    println(text.substring(1, 5))
    // otli
    println(text.subSequence(1, 5))
    // otli
}
```

Since the `String` type is immutable, these functions don't modify the original string.

Compare strings

You can check whether two strings have the same content with the `==` operator:

```
fun main(){
    println("kotlin" == "kotlin")
    // true

    println("kotlin" == "Kotlin")
    // false
}
```

You can also compare strings lexicographically (character by character) with the `compareTo()` function. It scans both strings until it finds the first differing pair of characters and returns:

- `0` when the strings are equal.
- A value less than `0` when the receiver is smaller than the argument.
- A value greater than `0` when the receiver is greater than the argument.

```
fun main() {
    println("abc".compareTo("abd") < 0)
    // true

    println("abc".compareTo("ABC") > 0)
    // true

    // Pass true to ignore case differences
    println("abc".compareTo("ABC", ignoreCase = true) == 0)
    // true
}
```

Work with string content

If you want to change the content of a string, create a modified copy of it with functions like `.trim()`, `.replace()`, `.uppercase()`, and `.lowercase()`:

```
fun main() {
    val text = "  Hello, Kotlin  "

    println(text.trim())
    // Hello, Kotlin

    println(text.replace("Kotlin", "world"))
    //   Hello, world

    println(text.uppercase())
    //   HELLO, KOTLIN

    println(text.lowercase())
    //   hello, kotlin
}
```

You can also inspect the string content with the `contains()`, `startsWith()`, and `endsWith()` functions:

```
fun main() {
    val domain = "kotlinlang.org"

    // Checks if the string contains "."
    println(domain.contains("."))
    // true

    // Checks if the string starts with "kotlin"
    println(domain.startsWith("kotlin"))
    // true

    // Checks if the string ends with ".org"
    println(domain.endsWith(".org"))
    // true
}
```

Split strings

You can divide a string into parts around a delimiter with the `split()` function:

```
fun main() {
    val numbers = "one, two, three"
    println(numbers.split(", "))
    // [one, two, three]
}
```

If you want to split a string into individual lines, use the `lines()` function:

```
fun main() {
    val numbers = "one\ntwo\nthree"
    println(numbers.lines())
    // [one, two, three]
}
```

Build and format strings

> **Tip:** For most formatting tasks in Kotlin, use string templates.

When you concatenate strings with the `+` operator, Kotlin creates a new `String` object for each operation. However, this approach may not be beneficial in loops or when you assemble many pieces. To avoid such issues, you can use the `buildString()` function or `StringBuilder`. They collect all pieces in a single mutable buffer and produce only one string at the end.

Use the `buildString()` function when the logic that determines what to append is complex. For example, when you have multiple conditions that contribute a different fragment. With `buildString()`, you don't handle the buffer directly. The function creates a `StringBuilder` internally, runs your block, and returns the resulting string.

```
fun main() {
    val hasErrors = true
    val hasWarnings = true
    val isComplete = false

    // buildString creates an empty buffer
    val status = buildString {
        // Appends "Errors found" to the buffer
        if (hasErrors) append("Errors found")
        if (hasWarnings) {
            // The buffer is not empty, appends "; "
            if (isNotEmpty()) append("; ")
            // Appends "Warnings found"
            append("Warnings found")
        }
        // isComplete = false, nothing to append
        if (isComplete) {
            if (isNotEmpty()) append("; ")
```

```
            append("Completed")
        }
        // The buffer is not empty, skips the fallback
        if (isEmpty()) append("OK")
    }

    println(status)
    // Errors found; Warnings found
}
```

Use `StringBuilder` when you need the buffer as an explicit value. For example, to change the existing text:

```
fun main() {
    val text = "Hello, Kotlin"
    val builder = StringBuilder(text)

    builder.replace(7, 13, "world")
    println(builder.toString())
    // Hello, world
}
```

On the JVM, you can also format a string with the `String.format()` function:

```
val text = String.format("Hello, %s", "Kotlin")
```

> **Note:** Use the `String.format()` function only when you specifically need formatter-style specifiers on the JVM. Learn more about format specifiers in the Java Class Formatter documentation.

String conversion

Often you may use strings to represent values of other types, such as numbers, `Boolean` values, or identifiers from the input. Kotlin provides functions for converting values to strings and for parsing strings into other types.

To return a string representation of a value, use the `toString()` function:

```
val number = 10
val text = number.toString()
```

In string templates and string concatenation, Kotlin converts values to strings automatically.

To convert a string to another type, use the corresponding parsing functions:

- For integer values: `toByte()`, `toShort()`, `toInt()`, `toLong()`
- For floating-point values: `toDouble()`, `toFloat()`
- For booleans: `toBoolean()`, `toBooleanStrict()`

These functions return a value of the requested type if the string has a valid format. If the input may be invalid, use the `OrNull` variants. These functions return `null` instead of throwing an exception making them safe for user input or data that you don't fully control:

```
val toInt = "10".toInt() // 10

// 1000000000000 exceeds maximum value of Int
val toIntInvalid = "1000000000000".toIntOrNull()

val toBoolean = "true".toBooleanStrict() // true
val toBooleanInvalid = "yes".toBooleanStrictOrNull() // null
```

Arrays

An array is a data structure that holds a fixed number of values of the same type or its subtypes. Array elements are ordered and accessed by index.

Kotlin provides the `Array<T>` class and primitive-type arrays.

When to use arrays

Use arrays for interoperability with Java APIs or low-level requirements. For example, if you have performance requirements beyond what is needed for regular applications, or you need to build custom data structures.

For most use cases, prefer collections instead.

Functionality	Arrays	Collections
Size	Fixed	Depends on the type
Read-only variant	No, always mutable	Yes (`List` and `Set`)
Adding and removing elements	No native support.	
Allocate and copy to a new array	Yes (mutable collections)	
Structural equality with `==`	No, compares references.	
Use `.contentEquals()` instead	Yes	
Primitive values	Primitive-type arrays store values	
without boxing	Usually boxed	
Java interoperability	Maps to `T[]`	Maps to `java.util.List` and
`java.util.Set`		
Functional-style filtering and		
transformations	Limited	Extensive

Learn how to convert arrays to collections.

Create arrays

To create arrays, you can use:

- The `arrayOf()`, `arrayOfNulls()`, or `emptyArray()` functions.
- The `Array` constructor.

> **Note:** Declaring an array with `val` only prevents reassigning the variable but doesn't make the contents read-only. Elements are mutable in both `val` and `var` arrays. The distinction only affects the reference, not the contents. For a read-only view, use collections instead.

Array with values

To create a typed array from a known set of values, use the `arrayOf()` function. Kotlin infers the type automatically:

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3) // Array<Int>
    println(simpleArray.joinToString())
    // 1, 2, 3
}
```

Empty array

To create an array with no elements, use the `emptyArray()` function. You can specify the type of elements on the left-hand or right-hand side of the assignment:

```
val emptyArrayRight = emptyArray<String>()
val emptyArrayLeft: Array<String> = emptyArray()
```

Learn how to add elements to an array.

Array with nulls

To create an array of a given size filled with `null` elements, use the `arrayOfNulls()` function:

```
fun main() {
    val nullArray: Array<Int?> = arrayOfNulls(3)
    println(nullArray.joinToString())
    // null, null, null
}
```

Array constructor

The `Array` constructor takes the array size and a function that returns values of array elements:

```
fun main() {
    val zeroes = Array<Int>(3) { 0 }
    println(zeroes.joinToString())
    // 0, 0, 0

    val squares = Array(5) { i -> i * i }
    println(squares.joinToString())
```

```
    // 0, 1, 4, 9, 16
}
```

Nested arrays

To create a nested or multidimensional array, use an array of arrays. Nested arrays don't have to be the same type or the same size.

```
fun main() {
    // Creates a two-dimensional array
    val twoDArray = Array(2) { Array<Int>(2) { 0 } }
    println(twoDArray.contentDeepToString())
    // [[0, 0], [0, 0]]

    // Creates a three-dimensional array
    val threeDArray = Array(3) { Array(3) { Array<Int>(3) { 0 } } }
    println(threeDArray.contentDeepToString())
    // [[[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0],
[0, 0, 0], [0, 0, 0]]]
}
```

Primitive-type arrays

If you use the `Array` class with primitive values, the compiler boxes these values into objects. To avoid boxing overhead, you can use dedicated primitive-type arrays. They are not subclasses of the `Array<T>` class, but they provide a similar set of functions and properties.

Kotlin type	Java equivalent
`BooleanArray`	`boolean[]`
`ByteArray`	`byte[]`
`CharArray`	`char[]`
`DoubleArray`	`double[]`
`FloatArray`	`float[]`
`IntArray`	`int[]`
`LongArray`	`long[]`
`ShortArray`	`short[]`

> **Note:** Kotlin doesn't have a dedicated `StringArray` type. `String` is not a primitive, so use the `arrayOf()` or `arrayOf<String>()` functions with inferred type instead.

To create a primitive-type array, use one of the following options:

- Constructor functions:

```
fun main() {
    // Creates an Int array of size 5 with the values initialized to zero
    val primitiveTypeArray = IntArray(5)
    println(primitiveTypeArray.joinToString())
    // 0, 0, 0, 0, 0

    // Creates an Int array and takes an initializer function
    val squares = IntArray(5) { i -> i * i }
    println(squares.joinToString())
    // 0, 1, 4, 9, 16
}
```

- Factory functions:

```
fun main() {
    // Creates an Int array with 5 elements
    val numbers = intArrayOf(1, 2, 3, 4, 5)
    println(numbers.joinToString())
    // 1, 2, 3, 4, 5

    // Creates a Char array with 3 elements
    val characters = charArrayOf('K', 't', 'l')
    println(characters.joinToString())
    // K, t, l

    // Creates a Double array with 3 elements
    val doubles = doubleArrayOf(0.22, 4.16, 0.5)
    println(doubles.joinToString())
    // 0.22, 4.16, 0.5
}
```

Note: To convert primitive-type arrays to object-type arrays, use the `.toTypedArray()` function.

To convert object-type arrays to primitive-type arrays, use `.toBooleanArray()`, `.toByteArray()`, `.toCharArray()`, and so on.

Work with arrays

Arrays support many of the same operations as collections, including iteration, searching, sorting, and transformations. In Kotlin, you can work with arrays by using them to pass a variable number of arguments to a function or perform operations on the arrays themselves. Find the most common properties and functions in the following table:

Member	Returns
`size`	The number of elements
`indices`	The range of valid indices

Member	Returns
`lastIndex`	The last valid index
`first()` and `last()`	First and last element
`isEmpty()` and `isNotEmpty()`	`true` if array is empty or not empty
`contains()`	`true` if array has the element

Tip: Learn more about array properties and functions in the API reference.

This section introduces some of the most commonly used operations.

Access and modify elements

To access and modify elements in an array, use the indexed access operator (`[]`):

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3)
    val twoDArray = Array(2) { Array<Int>(2) { 0 } }

    // Accesses the element and modifies it
    simpleArray[0] = 10
    twoDArray[0][0] = 2

    // Prints the modified element
    println(simpleArray[0])
    // 10
    println(twoDArray[0][0])
    // 2
}
```

Note: If you try to access an index outside the bounds of an array, Kotlin throws `ArrayIndexOutOfBoundsException` at runtime.

You can also use the `fill(element, fromIndex, toIndex)` function to replace elements in a range in place. `fromIndex` is inclusive, and `toIndex` is exclusive:

```
fun main() {
    val arr = IntArray(3)
    println(arr.joinToString())
    // 0, 0, 0

   arr.fill(1)
   println(arr.joinToString())
   // 1, 1, 1

   arr.fill(0, 0, 2)
   println(arr.joinToString())
   // 0, 0, 1
}
```

In Kotlin, arrays are *invariant*. This means that `Array<String>` is not a subtype of `Array<Any>`. This prevents possible runtime type failures. To express covariance, use the `Array<out Any>` type projection:

```
fun main() {
    fun printArr(arr: Array<out Any>) {
        for (item in arr) print("$item, ")
    }

    printArr(arrayOf("k", "t", "n"))
    // k, t, n,
}
```

Add and remove elements

Since arrays have a fixed size, they don't support the `.add()` and `.remove()` functions. To perform these operations, you need to create a new array. For that, you can use one of the following options:

- Use the `.copyOf` function:

  ```
  fun main() {
      var arr = intArrayOf(0, 1, 2)

      arr = arr.copyOf(arr.size + 1)
      println(arr.joinToString())
      // 0, 1, 2, 0

      arr[arr.lastIndex] = 3
      println(arr.joinToString())
      // 0, 1, 2, 3
  }
  ```

- Use the `+` or `+=` operators:

  ```
  fun main() {
      var arr = intArrayOf(0, 1, 2)

      arr += 3
      println(arr.joinToString())
      // 0, 1, 2, 3

      arr = arr + intArrayOf(4, 5)
      println(arr.joinToString())
      // 0, 1, 2, 3, 4, 5
  }
  ```

  Tip: If you need to frequently add or remove elements, use mutable collections instead.

Compare arrays

To compare whether two arrays have the same elements in the same order, use the `.contentEquals()` and `.contentDeepEquals()` functions:

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3)
    val anotherArray = arrayOf(1, 2, 3)

    // Compares contents of arrays
    println(simpleArray.contentEquals(anotherArray))
    // true

    // Using infix notation, compares contents of arrays after an element
    // is changed
    simpleArray[0] = 10
    println(simpleArray contentEquals anotherArray)
    // false
}
```

> **Warning:** Don't use equality (`==`) and inequality (`!=`) operators to compare the contents of arrays. These operators check whether the assigned variables point to the same object.
>
> To learn more about why arrays in Kotlin behave this way, see our blog post.

Transform arrays

Kotlin has many useful functions to transform arrays. This section highlights some of them. See the complete list in our API reference.

Sum

To return the sum of all elements in an array, use the `.sum()` function:

```
fun main() {
    val sumArray = arrayOf(1, 2, 3)
    println(sumArray.sum())
    // 6
}
```

> **Note:** The `.sum()` function can only be used with arrays of numeric data types, such as `Int`.

Sort and shuffle

You can sort the elements in the array according to their natural order with the `.sort()` function or randomly shuffle them with the `.shuffle()` function:

```
fun main() {
    val simpleArray = arrayOf(1, 2, 3)

    // Randomly shuffles elements
    simpleArray.shuffle()
    println(simpleArray.joinToString())

    // Sorts elements
    simpleArray.sort()
    println(simpleArray.joinToString())
    // 1, 2, 3
}
```

To get a new sorted array without modifying the original, use the `.sortedArray()` function instead.

Pass variable number of arguments to a function

In Kotlin, you can pass a variable number of arguments to a function via the `vararg` parameter. This is useful when you don't know the number of arguments in advance, like when formatting a message or creating an SQL query.

To pass an array containing a variable number of arguments to a function, use the *spread operator* (`*`). The spread operator passes each element of the array as individual arguments to your chosen function:

```
fun main() {
    val lettersArray = arrayOf("c", "d")
    printAllStrings("a", "b", *lettersArray)
    // abcd
}

fun printAllStrings(vararg strings: String) {
    for (string in strings) {
        print(string)
    }
}
```

For more information, see Variable number of arguments (varargs).

Convert to collections

If you work with different APIs where some use arrays and some use collections, you can convert your arrays to collections and vice versa. For that, use the `.toList()`, `.toSet()`, and `.toMap()` functions. These functions copy the content from your array to the independent copy. They don't reflect subsequent changes to the array.

Convert to List or Set

To convert an array to a `List` or `Set`, use the `.toList()` and `.toSet()` functions:

```
fun main() {
    val simpleArray = arrayOf("a", "b", "c", "c")

    // Converts to a Set
    println(simpleArray.toSet())
    // [a, b, c]

    // Converts to a List
    println(simpleArray.toList())
    // [a, b, c, c]
}
```

Unless you are completely sure that the original array isn't changed or shared elsewhere, don't use `.asList()` and related `as*` functions. These functions wrap the original array instead of copying it. Therefore, changes to the array are reflected in the list and vice versa.

```
fun main() {
    val simpleArray = arrayOf("a", "b", "c")

    val list = simpleArray.asList()
    simpleArray[0] = "d"
    println(list)
    // [d, b, c]
}
```

Convert to Map

To convert an array to a `Map`, use the `.toMap()` function.

You can convert only an array of `Pair<K,V>` to a `Map`. The first value of a `Pair` instance becomes a key, and the second becomes a value. If the same key appears more than once, the last value is used.

This example uses the infix notation to call the `.to` function to create tuples of `Pair`:

```
fun main() {
    val pairArray = arrayOf("apple" to 120, "banana" to 150, "cherry" to 90, "apple" to 140)

    // Converts to a Map
    // Fruits are keys, calorie numbers are values
    // The latest "apple" value overwrites the first one
    println(pairArray.toMap())
    // {apple=140, banana=150, cherry=90}
}
```

What's next?

- Learn more about why we recommend using collections for most use cases in the Collections overview.
- Learn about other basic types.
- If you are a Java developer, read our Java to Kotlin migration guide for Collections.

Type checks and casts

In Kotlin, you can do two things with types at runtime: check whether an object is a specific type, or convert it to another type. Type **checks** help you confirm the kind of object you're dealing with, while type **casts** attempt to convert the object to another type.

> **Tip:** To learn specifically about **generics** type checks and casts, for example `List<T>`, `Map<K,V>`, see Generics type checks and casts.

Checks with `is` and `!is` operators

Use the `is` operator (or `!is` to negate it) to check if an object matches a type at runtime:

```
fun main() {
    val input: Any = "Hello, Kotlin"

    if (input is String) {
        println("Message length: ${input.length}")
        // Message length: 13
    }

    if (input !is String) { // Same as !(input is String)
        println("Input is not a valid message")
    } else {
        println("Processing message: ${input.length} characters")
        // Processing message: 13 characters
    }
}
```

You can also use `is` and `!is` operators to check if an object matches a subtype:

```
interface Animal {
    val name: String
    fun speak()
}

class Dog(override val name: String) : Animal {
    override fun speak() = println("$name says: Woof!")
}

class Cat(override val name: String) : Animal {
    override fun speak() = println("$name says: Meow!")
}
fun handleAnimal(animal: Animal) {
    println("Handling animal: ${animal.name}")
    animal.speak()

    // Use is operator to check for subtypes
    if (animal is Dog) {
```

```
        println("Special care instructions: This is a dog.")
    } else if (animal is Cat) {
        println("Special care instructions: This is a cat.")
    }
}
fun main() {
    val pets: List<Animal> = listOf(
        Dog("Buddy"),
        Cat("Whiskers"),
        Dog("Rex")
    )

    for (pet in pets) {
        handleAnimal(pet)
        println("---")
    }
    // Handling animal: Buddy
    // Buddy says: Woof!
    // Special care instructions: This is a dog.
    // ---
    // Handling animal: Whiskers
    // Whiskers says: Meow!
    // Special care instructions: This is a cat.
    // ---
    // Handling animal: Rex
    // Rex says: Woof!
    // Special care instructions: This is a dog.
    // ---
}
```

This example uses the `is` operator to check if the `Animal` class instance has subtype `Dog` or `Cat` to print the relevant care instructions.

You can check if an object is a supertype of its declared type, but it's not worthwhile because the answer is always true. Every class instance is already an instance of its supertypes.

> To identify the type of an object at runtime, see Reflection.

Type casts

To convert the type of an object in Kotlin to another type is called **casting**.

In some cases, the compiler automatically casts objects for you. This is called smart-casting.

If you need to explicitly cast a type, use `as?` or `as` cast operators.

Smart casts

The compiler tracks the type checks and explicit casts for immutable values and inserts implicit (safe) casts automatically:

```
fun logMessage(data: Any) {
    // data is automatically cast to String
    if (data is String) {
        println("Received text: ${data.length} characters")
    }
}

fun main() {
    logMessage("Server started")
    // Received text: 14 characters
    logMessage(404)
}
```

The compiler is even smart enough to know that a cast is safe if a negative check leads to a return:

```
fun logMessage(data: Any) {
    // data is automatically cast to String
    if (data !is String) return

    println("Received text: ${data.length} characters")
}

fun main() {
    logMessage("User signed in")
    // Received text: 14 characters
    logMessage(true)
}
```

Control flow

Smart casts work not only for `if` conditional expressions, but also for `when` expressions:

```
fun processInput(data: Any) {
    when (data) {
        // data is automatically cast to Int
        is Int -> println("Log: Assigned new ID ${data + 1}")
        // data is automatically cast to String
        is String -> println("Log: Received message \"$data\"")
        // data is automatically cast to IntArray
        is IntArray -> println("Log: Processed scores, total = ${data.sum()}")
    }
}

fun main() {
    processInput(1001)
    // Log: Assigned new ID 1002
    processInput("System rebooted")
    // Log: Received message "System rebooted"
    processInput(intArrayOf(10, 20, 30))
    // Log: Processed scores, total = 60
}
```

And for `while` loops:

```
sealed interface Status
data class Ok(val currentRoom: String) : Status
data object Error : Status

class RobotVacuum(val rooms: List<String>) {
    var index = 0

    fun status(): Status =
        if (index < rooms.size) Ok(rooms[index])
        else Error

    fun clean(): Status {
        println("Finished cleaning ${rooms[index]}")
        index++
        return status()
    }
}

fun main() {
    val robo = RobotVacuum(listOf("Living Room", "Kitchen", "Hallway"))

    var status: Status = robo.status()
    while (status is Ok) {
        // The compiler smart casts status to OK type, so the currentRoom
        // property is accessible.
        println("Cleaning ${status.currentRoom}...")
        status = robo.clean()
    }
    // Cleaning Living Room...
    // Finished cleaning Living Room
    // Cleaning Kitchen...
    // Finished cleaning Kitchen
    // Cleaning Hallway...
    // Finished cleaning Hallway
}
```

In this example, the sealed interface `Status` has two implementations: the data class `Ok` and the data object `Error`. Only the `Ok` data class has the `currentRoom` property. When the `while` loop condition evaluates to true, the compiler smart casts the `status` variable to `Ok` type, making the `currentRoom` property accessible within the loop body.

If you declare a variable of `Boolean` type before using it in your `if`, `when`, or `while` condition, any information collected by the compiler about the variable is accessible in the corresponding block for smart-casting.

This can be useful when you want to do things like extract boolean conditions into variables. Then, you can give the variable a meaningful name, which improves your code readability and makes it possible to reuse the variable later in your code. For example:

```
class Cat {
    fun purr() {
        println("Purr purr")
    }
}
fun petAnimal(animal: Any) {
    val isCat = animal is Cat
    if (isCat) {
        // The compiler can access information about
        // isCat, so it knows that animal was smart-cast
        // to the type Cat.
        // Therefore, the purr() function can be called.
        animal.purr()
    }
}

fun main(){
    val kitty = Cat()
    petAnimal(kitty)
    // Purr purr
}
```

Logical operators

The compiler can perform smart casts on the right-hand side of `&&` or `||` operators if there is a type check (regular or negative) on the left-hand side:

```
// x is automatically cast to String on the right-hand side of `||`
if (x !is String || x.length == 0) return

// x is automatically cast to String on the right-hand side of `&&`
if (x is String && x.length > 0) {
    print(x.length) // x is automatically cast to String
}
```

If you combine type checks for objects with an `and` operator (`&&`), the compiler smart-casts the object to all checked types simultaneously. Learn more in Intersection types.

If you combine type checks for objects with an `or` operator (`||`), a smart cast is made to their closest common supertype:

```
interface Status {
    fun signal() {}
}

interface Ok : Status
interface Postponed : Status
interface Declined : Status

fun signalCheck(signalStatus: Any) {
    if (signalStatus is Postponed || signalStatus is Declined) {
        // signalStatus is smart-cast to a common supertype Status
```

```
        signalStatus.signal()
    }
}
```

> **Note:** The common supertype is an **approximation** of a union type. Union types are not currently supported in Kotlin.

Intersection types

When the compiler smart-casts an object through multiple `&&` checks, it infers an *intersection type*. This is an internal type that simultaneously satisfies all the checked constraints:

```
interface Bird {
    fun fly()
}

interface Fish {
    fun swim()
}

fun describe(animal: Any) {
    // Infers the Bird and Fish types
    if (animal is Bird && animal is Fish) {
        // Accesses fly() and swim() without additional checks or casts
        animal.fly()
        animal.swim()
    }
}
```

Intersection types are non-denotable. They only exist in the compiler's internal type system to preserve type information during type checking. You can't write them directly in Kotlin code. You may encounter intersection types in compiler error messages and IDE tooltips, typically displayed as `A & B`. The one exception is `T & Any` that declares a definitely non-nullable type. This syntax is reserved specifically to combine a type parameter with `Any`.

```
fun  <T> T.assertNotNull(): T & Any = this ?: throw IllegalStateException("null value")
```

Inline functions

The compiler can smart-cast variables captured within lambda functions that are passed to inline functions.

Inline functions are treated as having an implicit `callsInPlace` contract. This means that any lambda functions passed to an inline function are called in place. Since lambda functions are called in place, the compiler knows that a lambda function can't leak references to any variables contained within its function body.

The compiler uses this knowledge, along with other analyses, to decide whether it's safe to smart-cast any of the captured variables. For example:

```
interface Processor {
    fun process()
}

inline fun inlineAction(f: () -> Unit) = f()

fun nextProcessor(): Processor? = null

fun runProcessor(): Processor? {
    var processor: Processor? = null
    inlineAction {
        // The compiler knows that processor is a local variable and inlineAction()
        // is an inline function, so references to processor can't be leaked.
        // Therefore, it's safe to smart-cast processor.

        // If processor isn't null, processor is smart-cast
        if (processor != null) {
            // The compiler knows that processor isn't null, so no safe call
            // is needed
            processor.process()
        }

        processor = nextProcessor()
    }

    return processor
}
```

Exception handling

Smart cast information is passed on to `catch` and `finally` blocks. This makes your code safer as the compiler tracks whether your object has a nullable type. For example:

```
fun testString() {
    var stringInput: String? = null
    // stringInput is smart-cast to String type
    stringInput = ""
    try {
        // The compiler knows that stringInput isn't null
        println(stringInput.length)
        // 0

        // The compiler rejects previous smart cast information for
        // stringInput. Now stringInput has the String? type.
        stringInput = null

        // Trigger an exception
        if (2 > 1) throw Exception()
        stringInput = ""
    } catch (exception: Exception) {
```

```
        // The compiler knows stringInput can be null
        // so stringInput stays nullable.
        println(stringInput?.length)
        // null
    }
}
fun main() {
    testString()
}
```

Smart cast prerequisites

Smart casts work only when the compiler can guarantee that the variable won't change between the check and its usage. They can be used in the following conditions:

`val` local variables	Always, except local delegated properties.
`val` properties	If the property is `private`, `internal`, or if the check is performed in the same module where the property is declared. Smart casts can't be used on `open` properties or properties that have custom getters.
`var` local variables	If the variable is not modified between the check and its usage, is not captured in a lambda that modifies it, and is not a local delegated property.
`var` properties	Never, because the variable can be modified at any time by other code.

`as` and `as?` cast operators

Kotlin has two cast operators: `as` and `as?`. You can use both to cast, but they have different behaviors.

If a cast fails with the `as` operator, a `ClassCastException` is thrown at runtime. That's why it's also called the **unsafe** operator. You can use `as` when casting to a non-null type:

```
fun main() {
    val rawInput: Any = "user-1234"

    // Casts to String successfully
    val userId = rawInput as String
    println("Logging in user with ID: $userId")
    // Logging in user with ID: user-1234

    // Triggers ClassCastException
    val wrongCast = rawInput as Int
    println("wrongCast contains: $wrongCast")
    // Exception in thread "main" java.lang.ClassCastException
}
```

If you use the `as?` operator instead, and the cast fails, the operator returns `null`. That's why it's also called the **safe** operator:

```
fun main() {
    val rawInput: Any = "user-1234"

    // Casts to String successfully
    val userId = rawInput as? String
    println("Logging in user with ID: $userId")
    // Logging in user with ID: user-1234

    // Assigns a null value to wrongCast
    val wrongCast = rawInput as? Int
    println("wrongCast contains: $wrongCast")
    // wrongCast contains: null
}
```

To cast a nullable type safely, use the `as?` operator to prevent triggering a `ClassCastException` if the cast fails.

You *can* use `as` with a nullable type. This allows the result to be `null`, but it still throws a `ClassCastException` if the cast is unsuccessful. For this reason, `as?` is the safer option:

```
fun main() {
    val config: Map<String, Any?> = mapOf(
        "username" to "kodee",
        "alias" to null,
        "loginAttempts" to 3
    )

    // Unsafely casts to a nullable String
    val username: String? = config["username"] as String?
    println("Username: $username")
    // Username: kodee

    // Unsafely casts a null value to a nullable String
    val alias: String? = config["alias"] as String?
    println("Alias: $alias")
    // Alias: null

    // Fails to cast to nullable String and throws ClassCastException
    // val unsafeAttempts: String? = config["loginAttempts"] as String?
    // println("Login attempts (unsafe): $unsafeAttempts")
    // Exception in thread "main" java.lang.ClassCastException

    // Fails to cast to nullable String and returns null
    val safeAttempts: String? = config["loginAttempts"] as? String
    println("Login attempts (safe): $safeAttempts")
    // Login attempts (safe): null
}
```

Up and downcasting

In Kotlin, you can cast objects to supertypes and subtypes.

Casting an object to an instance of its superclass is called **upcasting**. Upcasting doesn't need any special syntax or cast operators. For example:

```
interface Animal {
    fun makeSound()
}

class Dog : Animal {
    // Implements behavior for makeSound()
    override fun makeSound() {
        println("Dog says woof!")
    }
}

fun printAnimalInfo(animal: Animal) {
    animal.makeSound()
}

fun main() {
    val dog = Dog()
    // Upcasts Dog instance to Animal
    printAnimalInfo(dog)
    // Dog says woof!
}
```

In this example, when the `printAnimalInfo()` function is called with a `Dog` instance, the compiler upcasts it to `Animal` because that's the expected parameter type. Since the actual object is still a `Dog` instance, the compiler dynamically resolves the `makeSound()` function from the `Dog` class, printing `"Dog says woof!"`.

You'll often see explicit upcasting in Kotlin APIs where behavior depends on an abstract type. It's also common in Jetpack Compose and UI toolkits, which typically treat all UI elements as supertypes and later operate on specific subclasses:

```
    val textView = TextView(this)
    textView.text = "Hello, View!"

    // Upcasts from TextView to View
    val view: View = textView

    // Use View functions
    view.setPadding(20, 20, 20, 20)
    // Activity expects a View type
    setContentView(view)
```

Casting an object to an instance of a subclass is called **downcasting**. Because downcasting can be unsafe, you need to use explicit cast operators. To avoid throwing exceptions on failed casts, we

recommend using the safe cast operator `as?`, to return `null` if the cast fails:

```
interface Animal {
    fun makeSound()
}

class Dog : Animal {
    override fun makeSound() {
        println("Dog says woof!")
    }

    fun bark() {
        println("BARK!")
    }
}

fun main() {
    // Creates animal as a Dog instance with Animal
    // type
    val animal: Animal = Dog()

    // Safely downcasts animal to Dog type
    val dog: Dog? = animal as? Dog

    // Uses a safe call to call bark() if dog isn't null
    dog?.bark()
    // "BARK!"
}
```

In this example, `animal` is declared as type `Animal`, but it holds a `Dog` instance. The code safely casts `animal` to `Dog` type and uses a safe call (`?.`) to access the `bark()` function.

You'll use downcasting in serialization when deserializing a base class to a specific subtype. It's also common when working with Java libraries that return supertype objects, which you may need to downcast in Kotlin.

Type aliases

Type aliases provide alternative names for existing types. They can make long or frequently used type expressions shorter and easier to understand.

For example, you can create aliases for generic types, function types, and nested or inner classes:

```
// Generic types
typealias UserIndex = Map<Long, User>
typealias FileTable<K> = MutableMap<K, MutableList<File>>

// Function types
typealias RequestHandler = (Request) -> Response
typealias Predicate<T> = (T) -> Boolean

// Inner and nested classes
class Database {
    inner class Transaction
}
typealias DatabaseTransaction = Database.Transaction
```

A type alias doesn't create a new type. It introduces an alternative name for an existing type. The alias and its underlying type are interchangeable. For example, when you add `typealias Predicate<T>` and use `Predicate<Int>`, the compiler expands it to `(Int) -> Boolean`. You can use a value declared with the alias wherever the underlying type is expected, and the other way around:

```
typealias Predicate<T> = (T) -> Boolean

fun evaluate(predicate: Predicate<Int>) = predicate(42)

fun main() {
    val isPositive: (Int) -> Boolean = { it > 0 }
    println(evaluate(isPositive))
    // true

    val isValid: Predicate<Int> = { it > 0 }
    println(listOf(1, -2).filter(isValid))
    // [1]
}
```

Declare type aliases

You can declare a type alias:

- At the top level of a Kotlin file, as a top-level type alias.
- Inside a class, interface, or object, as a nested type alias.

You can't declare a type alias in a local scope, such as inside a function or lambda expression.

The declaration location determines the scope of a type alias, while its visibility determines which code can access it. By default, a type alias is `public`. A nested type alias is accessible only where its containing class, interface, or object is accessible. For example, a `public` alias inside an `internal` class isn't accessible from outside the module.

A type alias can't expose an underlying type with more restrictive visibility than its own. For example, a `public` type alias can't refer to a `private` class.

Top-level type aliases

A top-level type alias is a package-level declaration. Within the same package, you can refer to an alias by its unqualified name. To use the alias from another package, import the alias or refer to it by its qualified name:

```
// UserId.kt
package org.example.users

typealias UserId = Long

// Refers to the alias within the same package by its unqualified name
fun createUser(id: UserId) {
    // ...
}

// UserService.kt
package org.example.services

import org.example.users.UserId

// Uses the imported alias by its unqualified name
fun findUser(id: UserId) {
    // ...
}

// Uses the fully qualified name
fun deleteUser(id: org.example.users.UserId) {
    // ...
}
```

Nested type aliases

Nested type aliases allow for cleaner, more maintainable code by improving encapsulation, reducing package-level clutter, and simplifying internal implementations. Nested type aliases follow the same scope and name-resolution rules as nested classes.

Declare a type alias inside a class, interface, or object when the alternative name is relevant only in the context of that declaration. This keeps the alias close to the code that uses it and avoids adding

another name to the package scope.

Within the containing declaration, you can refer to the alias by its unqualified name. Outside the declaration, qualify the alias with the name of its containing declaration:

```
class UserRepository {
    typealias UserIndex = Map<UserId, User>

    // Refers to the alias by its unqualified name inside UserRepository
    fun saveAll(users: UserIndex) {
        // ...
    }
}

// Refers to the alias by its qualified name outside UserRepository
fun synchronizeUsers(users: UserRepository.UserIndex) {
    // ...
}
```

> **Note:** Nested type aliases aren't supported in Kotlin Multiplatform `expect/actual` declarations.

Type parameters

To use type parameters in a nested type alias, add them to the alias declaration:

```
class Graph<Node> {
    typealias Path<T> = List<T>
}

val cityPath: Graph.Path<String> = listOf("London", "Berlin")
```

In this example, `Path` declares its type parameter `T`. In `Graph.Path<String>`, `String` is the type argument for `T` and is independent of the `Node` type parameter declared by `Graph`.

If you refer to a type parameter declared by its containing class or interface, the compiler reports an error:

```
class Graph<Node> {
    typealias Path = List<Node>
    // Unresolved reference 'Node'.
}
```

Here, `Path` refers to `Node` from `Graph` instead of declaring its own type parameter.

Conditions and loops

Kotlin gives you flexible tools to control your program's flow. Use `if`, `when`, and loops to define clear, expressive logic for your conditions.

If expression

To use `if` in Kotlin, add the condition to check within parentheses `()` and the action to take if the result is true within curly braces `{}`. You can use `else` and `else if` for additional branches and checks.

You can also write `if` as an expression, which lets you assign its returned value directly to a variable. In this form, an `else` branch is required. The `if` expression serves the same purpose as the ternary operator (`condition ? then : else`) found in other languages.

For example:

```
fun main() {
    val heightAlice = 160
    val heightBob = 175
    var taller = heightAlice
    if (heightAlice < heightBob) taller = heightBob

    // Uses an else branch
    if (heightAlice > heightBob) {
        taller = heightAlice
    } else {
        taller = heightBob
    }

    // Uses if as an expression
    taller = if (heightAlice > heightBob) heightAlice else heightBob

    // Uses else if as an expression:
    val heightLimit = 150
    val heightOrLimit = if (heightLimit > heightAlice) heightLimit else if (heightAlice > he
ightBob) heightAlice else heightBob

    println("Taller height is $taller")
    // Taller height is 175
    println("Height or limit is $heightOrLimit")
    // Height or limit is 175
}
```

Each branch in an `if` expression can be a block, where the value of the last expression becomes the result:

```
fun main() {
    val heightAlice = 160
    val heightBob = 175

    val taller = if (heightAlice > heightBob) {
        print("Choose Alice\n")
        heightAlice
    } else {
        print("Choose Bob\n")
        heightBob
    }

    println("Taller height is $taller")
}
```

When expressions and statements

`when` is a conditional expression that runs code based on multiple possible values or conditions. It's similar to the `switch` statement in Java, C, and other languages. `when` evaluates its argument and compares the result against each branch in order until one branch condition is satisfied. For example:

```
fun main() {
    val userRole = "Editor"
    when (userRole) {
        "Viewer" -> print("User has read-only access")
        "Editor" -> print("User can edit content")
        else -> print("User role is not recognized")
    }
    // User can edit content
}
```

You can use `when` either as an **expression** or a **statement**. As an expression, `when` returns a value you can use later in your code. As a statement, `when` completes an action without returning a result:

Expression	Statement

`// Returns a string assigned to the`
`// text variable`
`val text = when (x) {`
`  1 -> "x == 1"`
`  2 -> "x == 2"`
`  else -> "x is neither 1 nor 2"`
`}`
	`// Returns no result but triggers a`
	`// print statement`
	`when (x) {`
	`  1 -> print("x == 1")`
	`  2 -> print("x == 2")`
	`  else -> print("x is neither 1 nor 2")`
	`}`

Secondly, you can use `when` with or without a subject. The behavior stays the same either way. Using a subject usually makes your code more readable and maintainable because it clearly shows

what you're checking.

With subject `x`	Without subject
`when(x) { ... }`	`when { ... }`

How you use `when` determines whether you need to cover all possible cases in your branches. Covering all possible cases is called being *exhaustive*.

Statements

If you use `when` as a statement, you don't need to cover all possible cases. In this example, some cases aren't covered, so no branch is triggered. However, no error occurs:

```
fun main() {
    val deliveryStatus = "OutForDelivery"
    when (deliveryStatus) {
        // Not all cases are covered
        "Pending" -> print("Your order is being prepared")
        "Shipped" -> print("Your order is on the way")
    }
}
```

Just like with `if`, each branch can be a block, and its value is the value of the last expression in the block.

Expressions

If you use `when` as an expression, you **must** cover all possible cases. The value of the first matching branch becomes the value of the overall expression. If you don't cover all cases, the compiler throws an error.

If your `when` expression has a subject, you can use an `else` branch to make sure that all possible cases are covered, but it isn't mandatory. For example, if your subject is a `Boolean`, `enum` class, `sealed` class, or one of their nullable counterparts, you can cover all cases without an `else` branch:

```
import kotlin.random.Random
enum class Bit {
    ZERO, ONE
}

fun getRandomBit(): Bit {
    return if (Random.nextBoolean()) Bit.ONE else Bit.ZERO
}

fun main() {
    val numericValue = when (getRandomBit()) {
        // No else branch is needed because all cases are covered
```

```
        Bit.ZERO -> 0
        Bit.ONE -> 1
    }

    println("Random bit as number: $numericValue")
    // Random bit as number: 0
}
```

> **Tip:** To simplify `when` expressions and reduce repetition, try out context-sensitive resolution (currently in preview). This feature allows you to omit the type name when using enum entries or sealed class members in `when` expressions if the expected type is known.
>
> For more information, see Preview of context-sensitive resolution or the related KEEP proposal.

If your `when` expression **doesn't** have a subject, you **must** have an `else` branch or the compiler throws an error. The `else` branch is evaluated when none of the other branch conditions are satisfied:

```
fun main() {
    val localFileSize = 1200
    val remoteFileSize = 1200

    val message = when {
        localFileSize > remoteFileSize -> "Local file is larger than remote file"
        localFileSize < remoteFileSize -> "Local file is smaller than remote file"
        else -> "Local and remote files are the same size"
    }

    println(message)
    // Local and remote files are the same size
}
```

Other ways to use when

`when` expressions and statements offer different ways to simplify your code, handle multiple conditions, and perform type checks.

Group multiple conditions into a single branch using commas:

```
fun main() {
    val ticketPriority = "High"
    when (ticketPriority) {
        "Low", "Medium" -> print("Standard response time")
        else -> print("High-priority handling")
    }
}
```

Use expressions that evaluate to `true` or `false` as branch conditions:

```
fun main() {
    val storedPin = "1234"
    val enteredPin = 1234
    when (enteredPin) {
        // Expression
        storedPin.toInt() -> print("PIN is correct")
        else -> print("Incorrect PIN")
    }
}
```

Check whether a value is or isn't contained in a range or collection using the `in` or `!in` keywords:

```
fun main() {
    val x = 7
    val validNumbers = setOf(15, 16, 17)
    when (x) {
        in 1..10 -> print("x is in the range")
        in validNumbers -> print("x is valid")
        !in 10..20 -> print("x is outside the range")
        else -> print("none of the above")
    }
}
```

Check a value's type using the `is` or `!is` keywords. Due to smart casts, you can access the member functions and properties of the type directly:

```
fun hasPrefix(input: Any): Boolean = when (input) {
    is String -> input.startsWith("ID-")
    else -> false
}

fun main() {
    val testInput = "ID-98345"
    println(hasPrefix(testInput))
    // true
}
```

Use `when` instead of a traditional `if-else if` chain. Without a subject, the branch conditions are simply boolean expressions. The first branch with a `true` condition runs:

```
fun Int.isOdd() = this % 2 != 0
fun Int.isEven() = this % 2 == 0

fun main() {
    val x = 5
    val y = 8

    when {
        x.isOdd() -> print("x is odd")
        y.isEven() -> print("y is even")
        else -> print("x+y is odd")
    }
```

```
    // x is odd
}
```

Finally, capture the subject in a variable by using the following syntax:

```
fun main() {
    val message = when (val input = "yes") {
        "yes" -> "You said yes"
        "no" -> "You said no"
        else -> "Unrecognized input: $input"
    }

    println(message)
    // You said yes
}
```

The scope of a variable introduced as the subject is restricted to the body of the `when` expression or statement.

Guard conditions

Guard conditions allow you to include more than one condition to the branches of a `when` expression or statement, making complex control flow more explicit and concise. You can use guard conditions with `when` as long as it has a subject.

Place a guard condition after the primary condition in the same branch, separated by `if`:

```
sealed interface Animal {
    data class Cat(val mouseHunter: Boolean) : Animal
    data class Dog(val breed: String) : Animal
}

fun feedDog() = println("Feeding a dog")
fun feedCat() = println("Feeding a cat")
fun feedAnimal(animal: Animal) {
    when (animal) {
        // Branch with only primary condition
        // Calls feedDog() when animal is Dog
        is Animal.Dog -> feedDog()
        // Branch with both primary and guard conditions
        // Calls feedCat() when animal is Cat and not mouseHunter
        is Animal.Cat if !animal.mouseHunter -> feedCat()
        // Prints "Unknown animal" if none of the above conditions match
        else -> println("Unknown animal")
    }
}

fun main() {
    val animals = listOf(
        Animal.Dog("Beagle"),
        Animal.Cat(mouseHunter = false),
        Animal.Cat(mouseHunter = true)
```

```
    )

    animals.forEach { feedAnimal(it) }
    // Feeding a dog
    // Feeding a cat
    // Unknown animal
}
```

You can't use guard conditions when you have multiple conditions separated by a comma. For example:

```
0, 1 -> print("x == 0 or x == 1")
```

In a single `when` expression or statement, you can combine branches with and without guard conditions. The code in a branch with a guard condition runs only if both the primary condition and the guard condition evaluate to `true`. If the primary condition doesn't match, the guard condition isn't evaluated.

Since `when` statements don't need to cover all cases, using guard conditions in `when` statements without an `else` branch means that if no conditions match, no code is run.

Unlike statements, `when` expressions must cover all cases. If you use guard conditions in `when` expressions without an `else` branch, the compiler requires you to handle every possible case to avoid runtime errors.

Combine multiple guard conditions within a single branch using the boolean operators `&&` (AND) or `||` (OR). Use parentheses around the boolean expressions to avoid confusion:

```
when (animal) {
    is Animal.Cat if (!animal.mouseHunter && animal.hungry) -> feedCat()
}
```

Guard conditions also support `else if`:

```
when (animal) {
    // Checks if `animal` is `Dog`
    is Animal.Dog -> feedDog()
    // Guard condition that checks if `animal` is `Cat` and not `mouseHunter`
    is Animal.Cat if !animal.mouseHunter -> feedCat()
    // Calls giveLettuce() if none of the above conditions match and animal.eatsPlants is tr
ue
    else if animal.eatsPlants -> giveLettuce()
    // Prints "Unknown animal" if none of the above conditions match
    else -> println("Unknown animal")
}
```

Bytecode generation on the JVM

When you compile Kotlin code for JVM 21 or later, the compiler generates an `invokedynamic` instruction for eligible `when` expressions. This produces smaller bytecode, similar to the bytecode produced by Java `switch` statements.

The compiler uses `invokedynamic` with the `SwitchBootstraps.typeSwitch()` method when all the following conditions are met:

- All conditions except for `else` are `is` or `null` checks.
- The `when` expression doesn't contain guard conditions (`if`).
- The conditions don't include types that can't be type-checked directly, such as mutable Kotlin collections (`MutableList`) or function types (`kotlin.Function1`, `kotlin.Function2`, and so on).
- The `when` expression has at least two conditions besides `else`.
- All branches check the same subject of the `when` expression.

For example:

```
open class Shape

class Circle : Shape()
class Rectangle : Shape()
class Triangle : Shape()

fun countCorners(shape: Shape) = when (shape) {
    is Circle -> 0
    is Rectangle -> 4
    is Triangle -> 3
    else -> -1
}
```

The `when (shape)` expression here compiles to a single `invokedynamic` type switch instead of multiple `instanceof` checks in the bytecode.

For loops

Use the `for` loop to iterate through a collection, array, or range:

```
for (item in collection) print(item)
```

The body of a `for` loop can be a block with curly braces `{}`.

```
fun main() {
    val shoppingList = listOf("Milk", "Bananas", "Bread")
    println("Things to buy:")
    for (item in shoppingList) {
        println("- $item")
    }
```

```
    // Things to buy:
    // - Milk
    // - Bananas
    // - Bread
}
```

Ranges

To iterate over a range of numbers, use a range expression with `..` and `..<` operators:

```
fun main() {
    println("Closed-ended range:")
    for (i in 1..6) {
        print(i)
    }
    // Closed-ended range:
    // 123456

    println("\nOpen-ended range:")
    for (i in 1..<6) {
        print(i)
    }
    // Open-ended range:
    // 12345

    println("\nReverse order in steps of 2:")
    for (i in 6 downTo 0 step 2) {
        print(i)
    }
    // Reverse order in steps of 2:
    // 6420
}
```

Arrays

If you want to iterate through an array or a list with an index, you can use the `indices` property:

```
fun main() {
    val routineSteps = arrayOf("Wake up", "Brush teeth", "Make coffee")
    for (i in routineSteps.indices) {
        println(routineSteps[i])
    }
    // Wake up
    // Brush teeth
    // Make coffee
}
```

Alternatively, you can use the `.withIndex()` function from the standard library:

```
fun main() {
    val routineSteps = arrayOf("Wake up", "Brush teeth", "Make coffee")
    for ((index, value) in routineSteps.withIndex()) {
```

```
        println("The step at $index is \"$value\"")
    }
    // The step at 0 is "Wake up"
    // The step at 1 is "Brush teeth"
    // The step at 2 is "Make coffee"
}
```

Iterators

The `for` loop iterates through anything that provides an iterator. Collections provide iterators by default, whereas ranges and arrays are compiled into index-based loops.

You can create your own iterators by providing a member or extension function called `iterator()` that returns an `Iterator<>`. The `iterator()` function must have a `next()` function and a `hasNext()` function that returns a `Boolean`.

The easiest way to create your own iterator for a class is to inherit from the `Iterable<T>` interface and override the `iterator()`, `next()`, and `hasNext()` functions that are already there. For example:

```
class Booklet(val totalPages: Int) : Iterable<Int> {
    override fun iterator(): Iterator<Int> {
        return object : Iterator<Int> {
            var current = 1
            override fun hasNext() = current <= totalPages
            override fun next() = current++
        }
    }
}

fun main() {
    val booklet = Booklet(3)
    for (page in booklet) {
        println("Reading page $page")
    }
    // Reading page 1
    // Reading page 2
    // Reading page 3
}
```

Tip: Learn more about interfaces and inheritance.

Alternatively, you can create the functions from scratch. In this case, add the `operator` keyword to the functions:

```
class Booklet(val totalPages: Int) {
    operator fun iterator(): Iterator<Int> {
        return object {
            var current = 1

            operator fun hasNext() = current <= totalPages
            operator fun next() = current++
```

```
        }.let {
            object : Iterator<Int> {
                override fun hasNext() = it.hasNext()
                override fun next() = it.next()
            }
        }
    }
}
fun main() {
    val booklet = Booklet(3)
    for (page in booklet) {
        println("Reading page $page")
    }
    // Reading page 1
    // Reading page 2
    // Reading page 3
}
```

While loops

`while` and `do-while` loops run the code in their body continuously while the condition is satisfied. The difference between them is the condition checking time:

- `while` checks the condition and, if it's satisfied, runs the code in its body and then returns to the condition check.
- `do-while` runs the code in its body and then checks the condition. If it's satisfied, the loop repeats. So, the body of `do-while` runs at least once, regardless of the condition.

For a `while` loop, place the condition to check in parentheses `()` and the body within curly braces `{}`:

```
fun main() {
    var carsInGarage = 0
    val maxCapacity = 3
    while (carsInGarage < maxCapacity) {
        println("Car entered. Cars now in garage: ${++carsInGarage}")
    }
    // Car entered. Cars now in garage: 1
    // Car entered. Cars now in garage: 2
    // Car entered. Cars now in garage: 3

    println("Garage is full!")
    // Garage is full!
}
```

For a `do-while` loop, place the body within curly braces `{}` first before the condition to check in parentheses `()`:

```
import kotlin.random.Random

```

```
fun main() {
    var roll: Int
    do {
        roll = Random.nextInt(1, 7)
        println("Rolled a $roll")
    } while (roll != 6)
    // Rolled a 2
    // Rolled a 6

    println("Got a 6! Game over.")
    // Got a 6! Game over.
}
```

Break and continue in loops

Kotlin supports traditional `break` and `continue` operators in loops. See Returns and jumps.

Returns and jumps

Kotlin has three structural jump expressions:

- `return` by default returns from the nearest enclosing function or anonymous function.
- `break` terminates the nearest enclosing loop.
- `continue` proceeds to the next step of the nearest enclosing loop.

All of these expressions can be used as part of larger expressions:

```
val s = person.name ?: return
```

The type of these expressions is the Nothing type.

Break and continue labels

Any expression in Kotlin may be marked with a *label*. Labels have the form of an identifier followed by the `@` sign, such as `abc@` or `fooBar@`. To label an expression, just add a label in front of it.

```
loop@ for (i in 1..100) {
    // ...
}
```

Now, you can qualify a `break` or a `continue` with a label:

```
loop@ for (i in 1..100) {
    for (j in 1..100) {
        if (...) break@loop
    }
}
```

A `break` qualified with a label jumps to the execution point right after the loop marked with that label. A `continue` proceeds to the next iteration of that loop.

> **Note:** In some cases, you can apply `break` and `continue` *non-locally* without explicitly defining labels. Such non-local usages are valid in lambda expressions used in enclosing inline functions.

Return to labels

In Kotlin, functions can be nested using function literals, local functions, and object expressions. A qualified `return` allows you to return from an outer function.

The most important use case is returning from a lambda expression. To return from a lambda expression, label it and qualify the `return`:

```
fun foo() {
    listOf(1, 2, 3, 4, 5).forEach lit@{
        if (it == 3) return@lit // local return to the caller of the lambda - the forEach lo
op
        print(it)
    }
    print(" done with explicit label")
}
fun main() {
    foo()
}
```

Now, it returns only from the lambda expression. Often it is more convenient to use *implicit labels*, because such a label has the same name as the function to which the lambda is passed.

```
fun foo() {
    listOf(1, 2, 3, 4, 5).forEach {
        if (it == 3) return@forEach // local return to the caller of the lambda - the forEac
h loop
        print(it)
    }
    print(" done with implicit label")
}
fun main() {
    foo()
}
```

Alternatively, you can replace the lambda expression with an anonymous function. A `return` statement in an anonymous function will return from the anonymous function itself.

```
fun foo() {
    listOf(1, 2, 3, 4, 5).forEach(fun(value: Int) {
        if (value == 3) return  // local return to the caller of the anonymous function - th
e forEach loop
        print(value)
    })
    print(" done with anonymous function")
}
fun main() {
    foo()
}
```

Note that the use of local returns in the previous three examples is similar to the use of `continue` in regular loops.

There is no direct equivalent for `break`, but it can be simulated by adding an outer `run` lambda and non-locally returning from it:

```
fun foo() {
    run loop@{
        listOf(1, 2, 3, 4, 5).forEach {
            if (it == 3) return@loop // non-local return from the lambda passed to run
            print(it)
        }
    }
    print(" done with nested loop")
}
fun main() {
    foo()
}
```

The non-local return here is possible since the nested `forEach()` lambda acts as an inline function.

When returning a value, the parser gives preference to the qualified return:

```
return@a 1
```

This means "return `1` at label `@a`" rather than "return a labeled expression `(@a 1)`".

> **Note:** In some cases, you can return from a lambda expression without using labels. Such *non-local* returns are located in a lambda but exit the enclosing inline function.

Exception and error handling

Exceptions help your code run more predictably, even when runtime errors occur that could disrupt program execution. Kotlin treats all exceptions as *unchecked* by default. Unchecked exceptions simplify the exception handling process: you can catch exceptions, but you don't need to explicitly handle or declare them.

> **Tip:** Learn more about how Kotlin handles exceptions when interacting with Java, Swift, and Objective-C in the Exception interoperability with Java, Swift, and Objective-C section.

Working with exceptions consists of two primary actions:

- **Throwing exceptions:** indicate when a problem occurs.
- **Catching exceptions:** handle the unexpected exception manually by resolving the issue or notifying the developer or application user.

Exceptions are represented by subclasses of the `Exception` class, which is a subclass of the `Throwable` class. For more information about the hierarchy, see the Exception hierarchy section. Since `Exception` is an `open class`, you can create custom exceptions to suit your application's specific needs.

Throw exceptions

You can manually throw exceptions with the `throw` keyword. Throwing an exception indicates that an unexpected runtime error has occurred in the code. Exceptions are objects, and throwing one creates an instance of an exception class.

You can throw an exception without any parameters:

```
throw IllegalArgumentException()
```

To better understand the source of the problem, include additional information, such as a custom message and the original cause:

```
val cause = IllegalStateException("Original cause: illegal state")

// Throws an IllegalArgumentException if userInput is negative
// Additionally, it shows the original cause, represented by the cause IllegalStateException
if (userInput < 0) {
    throw IllegalArgumentException("Input must be non-negative", cause)
}
```

In this example, an `IllegalArgumentException` is thrown when the user inputs a negative value. You can create custom error messages and keep the original cause (`cause`) of the exception, which will be included in the stack trace.

Throw exceptions with precondition functions

Kotlin offers additional ways to automatically throw exceptions using precondition functions. Precondition functions include:

Precondition function	Use case	Exception thrown
`require()`	Checks user input validity	`IllegalArgumentException`
`check()`	Checks object or variable state validity	`IllegalStateException`
`error()`	Indicates an illegal state or condition	`IllegalStateException`

These functions are suitable for situations where the program's flow cannot continue if specific conditions aren't met. This streamlines your code and makes handling these checks efficient.

require() function

Use the `require()` function to validate input arguments when they are crucial for the function's operation, and the function can't proceed if these arguments are invalid.

If the condition in `require()` is not met, it throws an `IllegalArgumentException`:

```
fun getIndices(count: Int): List<Int> {
    require(count >= 0) { "Count must be non-negative. You set count to $count." }
    return List(count) { it + 1 }
}

fun main() {
    // This fails with an IllegalArgumentException
    println(getIndices(-1))

    // Uncomment the line below to see a working example
    // println(getIndices(3))
    // [1, 2, 3]
}
```

> **Note:** The `require()` function allows the compiler to perform smart casting. After a successful check, the variable is automatically cast to a non-nullable type. These functions are often used for nullability checks to ensure that the variable is not null before proceeding. For example:
>
> ```
> fun printNonNullString(str: String?) {
>     // Nullability check
>     require(str != null)
>     // After this successful check, 'str' is guaranteed to be
> ```

```
    // non-null and is automatically smart cast to non-nullable String
    println(str.length)
}
```

check() function

Use the `check()` function to validate the state of an object or variable. If the check fails, it indicates a logic error that needs to be addressed.

If the condition specified in the `check()` function is `false`, it throws an `IllegalStateException`:

```
fun main() {
    var someState: String? = null

    fun getStateValue(): String {

        val state = checkNotNull(someState) { "State must be set beforehand!" }
        check(state.isNotEmpty()) { "State must be non-empty!" }
        return state
    }
    // If you uncomment the line below then the program fails with IllegalStateException
    // getStateValue()

    someState = ""

    // If you uncomment the line below then the program fails with IllegalStateException
    // getStateValue()
    someState = "non-empty-state"

    // This prints "non-empty-state"
    println(getStateValue())
}
```

Note: The `check()` function allows the compiler to perform smart casting. After a successful check, the variable is automatically cast to a non-nullable type. These functions are often used for nullability checks to ensure that the variable is not null before proceeding. For example:

```
fun printNonNullString(str: String?) {
    // Nullability check
    check(str != null)
    // After this successful check, 'str' is guaranteed to be
    // non-null and is automatically smart cast to non-nullable String
    println(str.length)
}
```

error() function

The `error()` function is used to signal an illegal state or a condition in the code that logically should not occur. It's suitable for scenarios when you want to throw an exception intentionally in

your code, such as when the code encounters an unexpected state. This function is particularly useful in `when` expressions, providing a clear way to handle cases that shouldn't logically happen.

In the following example, the `error()` function is used to handle an undefined user role. If the role is not one of the predefined ones, an `IllegalStateException` is thrown:

```
class User(val name: String, val role: String)

fun processUserRole(user: User) {
    when (user.role) {
        "admin" -> println("${user.name} is an admin.")
        "editor" -> println("${user.name} is an editor.")
        "viewer" -> println("${user.name} is a viewer.")
        else -> error("Undefined role: ${user.role}")
    }
}

fun main() {
    // This works as expected
    val user1 = User("Alice", "admin")
    processUserRole(user1)
    // Alice is an admin.

    // This throws an IllegalStateException
    val user2 = User("Bob", "guest")
    processUserRole(user2)
}
```

Handle exceptions using try-catch blocks

When an exception is thrown, it interrupts the normal execution of the program. You can handle exceptions gracefully with the `try` and `catch` keywords to keep your program stable. The `try` block contains the code that might throw an exception, while the `catch` block catches and handles the exception if it occurs. The exception is caught by the first `catch` block that matches its specific type or a superclass of the exception.

Here's how you can use the `try` and `catch` keywords together:

```
try {
    // Code that may throw an exception
} catch (e: SomeException) {
    // Code for handling the exception
}
```

It's a common approach to use `try-catch` as an expression, so it can return a value from either the `try` block or the `catch` block:

```
fun main() {
    val num: Int = try {
```

```
        // If count() completes successfully, its return value is assigned to num
        count()

    } catch (e: ArithmeticException) {

        // If count() throws an exception, the catch block returns -1,
        // which is assigned to num
        -1
    }
    println("Result: $num")
}

// Simulates a function that might throw ArithmeticException
fun count(): Int {

    // Change this value to return a different value to num
    val a = 0

    return 10 / a
}
```

You can handle an exception without using the exception instance. For example, you can provide a fallback value or a generic error message in the `catch` block.

Use an underscore (`_`) instead of the exception parameter name to indicate that the exception instance is intentionally ignored:

```
import java.io.File
import java.io.IOException
fun main() {
    val userSettings = try {
        File("user-settings.json").readText()

    // Catches IOException without using the exception instance
    } catch (_: IOException) {
        // Uses a fallback value if loading the file fails
        "{}"
    }

    println(userSettings)
}
```

You can use multiple `catch` handlers for the same `try` block. You can add as many `catch` blocks as needed to handle different exceptions distinctively. When you have multiple `catch` blocks, it's important to order them from the most specific to the least specific exception, following a top-to-bottom order in your code. This ordering aligns with the program's execution flow.

Consider this example with custom exceptions:

```
open class WithdrawalException(message: String) : Exception(message)
class InsufficientFundsException(message: String) : WithdrawalException(message)
```

```
fun processWithdrawal(amount: Double, availableFunds: Double) {
    if (amount > availableFunds) {
        throw InsufficientFundsException("Insufficient funds for the withdrawal.")
    }
    if (amount < 1 || amount % 1 != 0.0) {
        throw WithdrawalException("Invalid withdrawal amount.")
    }
    println("Withdrawal processed")
}

fun main() {
    val availableFunds = 500.0

    // Change this value to test different scenarios
    val withdrawalAmount = 500.5

    try {
        processWithdrawal(withdrawalAmount.toDouble(), availableFunds)

    // The order of catch blocks is important!
    } catch (e: InsufficientFundsException) {
        println("Caught an InsufficientFundsException: ${e.message}")
    } catch (e: WithdrawalException) {
        println("Caught a WithdrawalException: ${e.message}")
    }
}
```

A general catch block handling `WithdrawalException`, catches all exceptions of its type, including specific ones like `InsufficientFundsException`, unless they are caught earlier by a more specific catch block.

The finally block

The `finally` block contains code that always executes, regardless of whether the `try` block completes successfully or throws an exception. With the `finally` block, you can clean up code after the execution of `try` and `catch` blocks. This is especially important when working with resources like files or network connections, as `finally` guarantees they are properly closed or released.

Here is how you would typically use the `try-catch-finally` blocks together:

```
try {
    // Code that may throw an exception
}
catch (e: YourException) {
    // Exception handler
}
finally {
    // Code that is always executed
}
```

The returned value of a `try` expression is determined by the last executed expression in either the `try` or `catch` block. If no exceptions occur, the result comes from the `try` block; if an exception is handled, it comes from the `catch` block. The `finally` block is always executed, but it doesn't change the result of the `try-catch` block.

Let's look at an example to demonstrate:

```
fun divideOrNull(a: Int): Int {

    // The try block is always executed
    // An exception here (division by zero) causes an immediate jump to the catch block
    try {
        val b = 44 / a
        println("try block: Executing division: $b")
        return b
    }

    // The catch block is executed due to the ArithmeticException (division by zero if a ==
0)
    catch (e: ArithmeticException) {
        println("catch block: Encountered ArithmeticException $e")
        return -1
    }
    finally {
        println("finally block: The finally block is always executed")
    }
}

fun main() {

    // Change this value to get a different result. An ArithmeticException will return: -1
    divideOrNull(0)
}
```

> **Note:** In Kotlin, the idiomatic way to manage resources that implement the `AutoClosable` interface, such as file streams like `FileInputStream` or `FileOutputStream`, is to use the `.use()` function. This function automatically closes the resource when the block of code completes, regardless of whether an exception is thrown, thereby eliminating the need for a `finally` block. Consequently, Kotlin does not require a special syntax like Java's try-with-resources for resource management.
>
> ```
> FileWriter("test.txt").use { writer ->
>     writer.write("some text")
>     // After this block, the .use function automatically calls writer.close(), s
> imilar to a finally block
> }
> ```

If your code requires resource cleanup without handling exceptions, you can also use `try` with the `finally` block without `catch` blocks:

```
class MockResource {
    fun use() {
        println("Resource being used")
        // Simulate a resource being used
        // This throws an ArithmeticException if division by zero occurs
        val result = 100 / 0

        // This line is not executed if an exception is thrown
        println("Result: $result")
    }

    fun close() {
        println("Resource closed")
    }
}

fun main() {
    val resource = MockResource()
    try {

        // Attempts to use the resource
        resource.use()

    } finally {

        // Ensures that the resource is always closed, even if an exception occurs
        resource.close()
    }

    // This line is not printed if an exception is thrown
    println("End of the program")
}
```

As you can see, the `finally` block guarantees that the resource is closed, regardless of whether an exception occurs.

In Kotlin, you have the flexibility to use only a `catch` block, only a `finally` block, or both, depending on your specific needs, but a `try` block must always be accompanied by at least one `catch` block or a `finally` block.

Create custom exceptions

In Kotlin, you can define custom exceptions by creating classes that extend the built-in `Exception` class. This allows you to create more specific error types tailored to your application's needs.

To create one, you can define a class that extends `Exception`:

```
class MyException: Exception("My message")
```

In this example, there is a default error message, "My message", but you can leave it blank if you want.

> **Tip:** Exceptions in Kotlin are stateful objects, carrying information specific to the context of their creation, referred to as the stack trace. Avoid creating exceptions using object declarations. Instead, create a new instance of the exception every time you need one. This way, you can ensure the exception's state accurately reflects the specific context.

Custom exceptions can also be a subclass of any pre-existent exception subclass, like the `ArithmeticException` subclass:

```
class NumberTooLargeException: ArithmeticException("My message")
```

> **Note:** If you want to create subclasses of custom exceptions, you must declare the parent class as `open` because classes are final by default and cannot be subclassed otherwise.
>
> For example:
>
> ```
> // Declares a custom exception as an open class, making it subclassable
> open class MyCustomException(message: String): Exception(message)
>
> // Creates a subclass of the custom exception
> class SpecificCustomException: MyCustomException("Specific error message")
> ```

Custom exceptions behave just like built-in exceptions. You can throw them using the `throw` keyword, and handle them with `try-catch-finally` blocks. Let's look at an example to demonstrate:

```
class NegativeNumberException: Exception("Parameter is less than zero.")
class NonNegativeNumberException: Exception("Parameter is a non-negative number.")

fun myFunction(number: Int) {
    if (number < 0) throw NegativeNumberException()
    else if (number >= 0) throw NonNegativeNumberException()
}

fun main() {

    // Change the value in this function to a get a different exception
    myFunction(1)
}
```

In applications with diverse error scenarios, creating a hierarchy of exceptions can help make the code clearer and more specific. You can achieve this by using an abstract class or a sealed class as a base for common exception features and creating specific subclasses for detailed exception

types. Additionally, custom exceptions including parameters with default values offer flexibility, allowing initialization with varied messages, which enables more granular error handling.

Let's look at an example using the sealed class `AccountException` as the base for an exception hierarchy, and class `APIKeyExpiredException`, a subclass, which showcases the use of parameters with default values for improved exception detail:

```
// Creates a sealed class as the base for an exception hierarchy for account-related errors
sealed class AccountException(message: String, cause: Throwable? = null):
Exception(message, cause)

// Creates a subclass of AccountException
class InvalidAccountCredentialsException : AccountException("Invalid account credentials det
ected")

// Creates a subclass of AccountException, which allows the addition of custom messages and
causes
class APIKeyExpiredException(message: String = "API key expired", cause: Throwable? = null)
: AccountException(message, cause)

// Change values of placeholder functions to get different results
fun areCredentialsValid(): Boolean = true
fun isAPIKeyExpired(): Boolean = true
// Validates account credentials and API key
fun validateAccount() {
    if (!areCredentialsValid()) throw InvalidAccountCredentialsException()
    if (isAPIKeyExpired()) {
        // Example of throwing APIKeyExpiredException with a specific cause
        val cause = RuntimeException("API key validation failed due to network error")
        throw APIKeyExpiredException(cause = cause)
    }
}

fun main() {
    try {
        validateAccount()
        println("Operation successful: Account credentials and API key are valid.")
    } catch (e: AccountException) {
        println("Error: ${e.message}")
        e.cause?.let { println("Caused by: ${it.message}") }
    }
}
```

The Nothing type

In Kotlin, every expression has a type. The type of the expression `throw IllegalArgumentException()` is `Nothing`, a built-in type that is a subtype of all other types, also known as the bottom type. This means `Nothing` can be used as a return type or generic type where any other type is expected, without causing type errors.

`Nothing` is a special type in Kotlin used to represent functions or expressions that never complete successfully, either because they always throw an exception or enter an endless execution path like an infinite loop. You can use `Nothing` to mark functions that are not yet implemented or are designed to always throw an exception, clearly indicating your intentions to both the compiler and code readers. If the compiler infers a `Nothing` type in a function signature, it will warn you. Explicitly defining `Nothing` as the return type can eliminate this warning.

This Kotlin code demonstrates the use of the `Nothing` type, where the compiler marks the code following the function call as unreachable:

```
class Person(val name: String?)

fun fail(message: String): Nothing {
    throw IllegalArgumentException(message)
    // This function will never return successfully.
    // It will always throw an exception.
}

fun main() {
    // Creates an instance of Person with 'name' as null
    val person = Person(name = null)

    val s: String = person.name ?: fail("Name required")

    // 's' is guaranteed to be initialized at this point
    println(s)
}
```

Kotlin's `TODO()` function, which also uses the `Nothing` type, serves as a placeholder to highlight areas of the code that need future implementation:

```
fun notImplementedFunction(): Int {
    TODO("This function is not yet implemented")
}

fun main() {
    val result = notImplementedFunction()
    // This throws a NotImplementedError
    println(result)
}
```

As you can see, the `TODO()` function always throws a `NotImplementedError` exception.

Exception classes

Let's explore some common exception types found in Kotlin, which are all subclasses of the `RuntimeException` class:

- `ArithmeticException`: This exception occurs when an arithmetic operation is impossible to perform, like division by zero.

```
val example = 2 / 0 // throws ArithmeticException
```

- `IndexOutOfBoundsException`: This exception is thrown to indicate that an index of some sort, such as an array or string, is out of range.

```
val myList = mutableListOf(1, 2, 3)
myList.removeAt(3)  // throws IndexOutOfBoundsException
```

  To avoid this exception, use a safer alternative, such as the `getOrNull()` function:

```
val myList = listOf(1, 2, 3)
// Returns null, instead of IndexOutOfBoundsException
val element = myList.getOrNull(3)
println("Element at index 3: $element")
```

- `NoSuchElementException`: This exception is thrown when an element that does not exist in a particular collection is accessed. It occurs when using methods that expect a specific element, such as `first()` or `last()`.

```
val emptyList = listOf<Int>()
val firstElement = emptyList.first()  // throws NoSuchElementException
```

  To avoid this exception, use a safer alternative, such as the `firstOrNull()` function:

```
val emptyList = listOf<Int>()
// Returns null, instead of NoSuchElementException
val firstElement = emptyList.firstOrNull()
println("First element in empty list: $firstElement")
```

- `NumberFormatException`: This exception occurs when attempting to convert a string to a numeric type, but the string doesn't have an appropriate format.

```
val string = "This is not a number"
val number = string.toInt() // throws NumberFormatException
```

  To avoid this exception, use a safer alternative, such as the `toIntOrNull()` function:

```
val nonNumericString = "not a number"
// Returns null, instead of NumberFormatException
val number = nonNumericString.toIntOrNull()
println("Converted number: $number")
```

- `NullPointerException`: This exception is thrown when an application attempts to use an object reference that has the `null` value. Even though Kotlin's null safety features significantly reduce the risk of NullPointerExceptions, they can still occur either through deliberate use of the `!!` operator or when interacting with Java, which lacks Kotlin's null safety.

  ```
  val text: String? = null
  println(text!!.length)  // throws a NullPointerException
  ```

While all exceptions are unchecked in Kotlin, and you don't have to catch them explicitly, you still have the flexibility to catch them if desired.

Exception hierarchy

The root of the Kotlin exception hierarchy is the `Throwable` class. It has two direct subclasses, `Error` and `Exception`:

- The `Error` subclass represents serious fundamental problems that an application might not be able to recover from by itself. These are problems that you generally would not attempt to handle, such as `OutOfMemoryError` or `StackOverflowError`.
- The `Exception` subclass is used for conditions that you might want to handle. Subtypes of the `Exception` type, such as the `RuntimeException` and `IOException` (Input/Output Exception), deal with exceptional events in applications.

!Exception hierarchy - the Throwable class

`RuntimeException` is usually caused by insufficient checks in the program code and can be prevented programmatically. Kotlin helps prevent common `RuntimeExceptions` like `NullPointerException` and provides compile-time warnings for potential runtime errors, such as division by zero. The following picture demonstrates a hierarchy of subtypes descended from `RuntimeException`:

!Hierarchy of RuntimeExceptions

Stack trace

The *stack trace* is a report generated by the runtime environment, used for debugging. It shows the sequence of function calls leading to a specific point in the program, especially where an error or exception occurred.

Let's see an example where the stack trace is automatically printed because of an exception in a JVM environment:

```
fun main() {
    throw ArithmeticException("This is an arithmetic exception!")
}
```

Running this code in a JVM environment produces the following output:

```
Exception in thread "main" java.lang.ArithmeticException: This is an arithmetic exception!
    at MainKt.main(Main.kt:3)
    at MainKt.main(Main.kt)
```

The first line is the exception description, which includes:

- Exception type: `java.lang.ArithmeticException`
- Thread: `main`
- Exception message: `"This is an arithmetic exception!"`

Each other line that starts with an `at` after the exception description is the stack trace. A single line is called a *stack trace element* or a *stack frame*:

- `at MainKt.main (Main.kt:3)`: This shows the method name (`MainKt.main`) and the source file and line number where the method was called (`Main.kt:3`).
- `at MainKt.main (Main.kt)`: This shows that the exception occurs in the `main()` function of the `Main.kt` file.

Exception interoperability with Java, Swift, and Objective-C

Since Kotlin treats all exceptions as unchecked, it can lead to complications when such exceptions are called from languages that distinguish between checked and unchecked exceptions. To address this disparity in exception handling between Kotlin and languages like Java, Swift, and Objective-C, you can use the `@Throws` annotation. This annotation alerts callers about possible exceptions. For more information, see Calling Kotlin from Java and Interoperability with Swift/Objective-C.

Functions

To declare a function in Kotlin:

- Use the `fun` keyword.
- Specify the parameters in parentheses `()`.
- Include the return type if needed.

For example:

```
// 'double' is the name of the function
// 'x' is a parameter of Int type
// The expected return value is of Int type too
fun double(x: Int): Int {
    return 2 * x
}
fun main() {
    println(double(5))
    // 10
}
```

Function usage

Functions are called using the standard approach:

```
val result = double(2)
```

To call a member or extension function, use a period `.`:

```
// Creates an instance of the Stream class and calls read()
Stream().read()
```

Parameters

Declare function parameters using Pascal notation: `name: Type`. You must separate parameters using commas and give each parameter a type explicitly:

```
fun powerOf(number: Int, exponent: Int): Int { /*...*/ }
```

When you pass an object to a function, the compiler passes a copy of the reference to that object. The copied reference points to the same object, so the function can modify the object's mutable state.

Function parameters are read-only inside the function body (implicitly declared as `val`), so you can't reassign them:

```
class Counter(var value: Int)

fun reset(counter: Counter) {
    counter.value = 0    // Allowed: modifies the object
    counter = Counter(0) // Error: 'val' cannot be reassigned
}
```

You can use a trailing comma when declaring function parameters:

```
fun powerOf(
    number: Int,
    exponent: Int, // trailing comma
) { /*...*/ }
```

Trailing commas help with refactorings and code maintenance: you can move parameters within the declaration without worrying about which is going to be the last one.

> **Note:** Kotlin functions can receive other functions as parameters — and be passed as arguments. For more information, see .

Parameters with default values

You can make a function parameter optional by specifying a default value for it. Kotlin uses the default value when you call the function without providing an argument that corresponds to that parameter. Parameters with default values are also known as *optional parameters*.

Optional parameters reduce the need for multiple overloads, since you don't have to declare different versions of a function just to allow skipping a parameter with a reasonable default.

Set a default value by appending `=` to the parameter declaration:

```
fun read(
    b: ByteArray,
    // The default value of 'off' is 0
    off: Int = 0,
    // The default value of 'len' is calculated
    // as the size of the 'b' array
    len: Int = b.size,
) { /*...*/ }
```

When you declare a parameter **with** a default value before a parameter **without** a default value, you can only use the default value by naming the argument:

```
fun greeting(
    userId: Int = 0,
    message: String,
```

```
) { /*...*/ }

fun main() {
    // Uses 0 as the default value for 'userId'
    greeting(message = "Hello!")

    // Error: No value passed for parameter 'message'
    greeting("Hello!")
}
```

Trailing lambdas are an exception to this rule, since the last parameter must correspond to the passed function:

```
fun main () {
fun greeting(
    userId: Int = 0,
    message: () -> Unit,
)
{ println(userId)
  message() }

// Uses the default value for 'userId'
greeting() { println ("Hello!") }
// 0
// Hello!
}
```

Overriding methods always use the base method's default parameter values. When you override a method that has default parameter values, you must omit the default parameter values from the signature:

```
open class Shape {
    open fun draw(width: Int = 10, height: Int = 5) { /*...*/ }
}

class Rectangle : Shape() {
    // It's not allowed to specify default values here
    // but this function also uses 10 for 'width' and 5 for 'height'
    // by default.
    override fun draw(width: Int, height: Int) { /*...*/ }
}
```

Non-constant expressions as default values

You can assign a parameter a default value that isn't constant. For example, the default can be the result of a function call or a calculation that uses the values of other arguments, like the `len` parameter in this example:

```
fun read(
    b: ByteArray,
    off: Int = 0,
```

```
    len: Int = b.size,
) { /*...*/ }
```

Parameters that refer to the values of other parameters must be declared later in the order. In this example, `len` must be declared after `b`.

In general, you can assign any expression as the default value of a parameter. However, default values are only evaluated when the function is called **without** the corresponding parameter and a default value needs to be assigned. For example, this function prints out a line only when it is called without the `print` parameter:

```
fun main() {
    fun read(
        b: Int,
        print: Unit? = println("No argument passed for 'print'")
    ) { println(b) }

    // Prints "No argument passed for 'print'", then "1"
    read(1)
    // Prints only "1"
    read(1, null)
}
```

If the last parameter in a function declaration has a functional type, you can pass the corresponding lambda argument either as a named argument or outside the parentheses:

```
fun main() {
    fun log(
        level: Int = 0,
        code:  Int = 1,
        action: () -> Unit,
    ) { println (level)
        println (code)
        action() }

    // Passes 1 for 'level' and uses the default value 1 for 'code'
    log(1) { println("Connection established") }

    // Uses both default values, 0 for 'level' and 1 for 'code'
    log(action = { println("Connection established") })

    // Equivalent to the previous call, uses both default values
    log { println("Connection established") }
}
```

Named arguments

You can name one or more of a function's arguments when calling it. This can be helpful when a function call has many arguments. In such cases, it's difficult to associate a value with an argument, especially if it's `null` or a boolean value.

When you use named arguments in a function call, you can list them in any order.

Consider the `reformat()` function, which has 4 arguments with default values:

```
fun reformat(
    str: String,
    normalizeCase: Boolean = true,
    upperCaseFirstLetter: Boolean = true,
    divideByCamelHumps: Boolean = false,
    wordSeparator: Char = ' ',
) { /*...*/ }
```

When calling this function, you can name some of the arguments:

```
reformat(
    "String!",
    normalizeCase = false,
    upperCaseFirstLetter = false,
    divideByCamelHumps = true,
    '_'
)
```

You can skip all the arguments with default values:

```
reformat("This is a long String!")
```

You can also skip *some* arguments with default values, rather than omitting them all. However, after the first skipped argument, you must name all subsequent arguments:

```
reformat(
    "This is a short String!",
    upperCaseFirstLetter = false,
    wordSeparator = '_'
)
```

You can pass a variable number of arguments (`vararg`) by naming the corresponding argument. In this example, it's an array:

```
fun mergeStrings(vararg strings: String) { /*...*/ }

mergeStrings(strings = arrayOf("a", "b", "c"))
```

> **Note:** When calling Java functions on the JVM, you can't use the named argument syntax because Java bytecode does not always preserve the names of function parameters.

Return types

When you declare a function with a block body (by putting instructions within curly braces `{}`), you must always specify a return type explicitly. The only exception is when the function returns `Unit`,in which case specifying the return type is optional.

Kotlin doesn't infer return types for functions with block bodies. Their control flow can be complex, which makes the return type unclear to the reader and sometimes even to the compiler. However, Kotlin can infer the return type for single-expression functions if you don't specify it.

Kotlin functions return a single value, but that value can contain multiple pieces of data. For ways to represent these values, see Return multiple values.

Return multiple values

When you need to return multiple related values with distinct meanings, declare a data class, even if you only use it with one function:

```
data class OrderSummary(
    val subtotal: Double,
    val tax: Double,
)

fun calculateOrderSummary(prices: List<Double>): OrderSummary {
    val subtotal = prices.sum()
    val tax = subtotal * 0.2
    return OrderSummary(subtotal, tax)
}

fun main() {
    val summary = calculateOrderSummary(listOf(12.50, 8.00, 4.50))

    println(summary.subtotal)
    // 25.0
    println(summary.tax)
    // 5.0
}
```

A data class works well when you need to return multiple values with distinct meanings. If the returned values are of the same kind and you want to handle them as a group, consider returning a collection instead:

```
data class Person(val name: String)

val friendGroups = listOf(
    listOf(Person("Alice"), Person("Bob")),
    listOf(Person("Charlie"), Person("Diana"), Person("Eve")),
    listOf(Person("Frank"))
)

fun findLargestGroupOfFriends(): List<Person> {
```

```
    return friendGroups.maxByOrNull { it.size } ?: emptyList()
}

fun main() {
    val largestGroup = findLargestGroupOfFriends()

    println(largestGroup.map { it.name })
    // [Charlie, Diana, Eve]
}
```

If you need to return a fixed number of values, you can also use `Pair` or `Triple` data classes from the standard library. However, their properties have generic names such as `first`, `second`, and `third`, which can make the result difficult to understand.

For example, although the `calculateOrderTotals()` function returns a `Pair`, it's not clear what each `Double` represents:

```
fun calculateOrderTotals(prices: List<Double>): Pair<Double, Double> {
    val subtotal = prices.sum()
    val tax = subtotal * 0.2
    return Pair(subtotal, tax)
}

fun main() {
    val totals = calculateOrderTotals(listOf(12.50, 8.00, 4.50))

    // What does 'first' mean?
    println(totals.first)
    // 25.0

    // What does 'second' mean?
    println(totals.second)
    // 5.0
}
```

For results with distinct meanings, prefer a data class with descriptive property names, as demonstrated in the `OrderSummary` data class example.

Single-expression functions

When the function body consists of a single expression, you can omit the curly braces and specify the body after an `=` symbol:

```
fun double(x: Int): Int = x * 2
```

Most of the time you don't have to explicitly declare the return type:

```
// Compiler infers that the function returns Int
fun double(x: Int) = x * 2
```

The compiler can sometimes run into problems when inferring return types from single expressions. In such cases, you should add the return type explicitly. For example, functions that are recursive or mutually recursive (calling each other) and functions with typeless expressions like `fun empty() = null` always require a return type.

When you do use an inferred return type, make sure to check the actual result because the compiler may infer a type that is less useful to you. In the example above, if you want the `double()` function to return `Number` instead of `Int`, you have to declare this explicitly.

If you use a `return` statement inside an expression body, you must specify the return type explicitly:

```
fun getDisplayNameOrDefault(userId: String?): String =
    getDisplayName(userId ?: return "default")
```

Unit-returning functions

If a function has a block body and no explicit return type, the compiler infers its return type as `Unit`.

`Unit` is a type that has only one value, also called `Unit`. This value is an object, so only a single `Unit` instance exists. Unlike `void` in other programming languages, `Unit` is an actual type, and you can use its value if your code requires one. This way functions that produce results and functions that only perform actions fit the same type model: both return a value and can be represented by function types or used with generic APIs.

You don't need to specify `Unit` in function declarations or return its value explicitly. However, you must specify `Unit` when it is part of a function type, such as `() -> Unit`. For example, let's declare a `printHello()` function without returning `Unit`:

```
// The declaration of the functional type parameter ('action') still
// needs an explicit return type
fun printHello(name: String?, action: () -> Unit) {
    if (name != null)
        println("Hello $name")
    else
        println("Hi there!")

    action()
}

fun main() {
    printHello("Kodee") {
        println("This runs after the greeting.")
        // Hello Kodee
        // This runs after the greeting.
    }

    printHello(null) {
```

```
        println("No name provided, but action still runs.")
        // Hi there!
        // No name provided, but action still runs.
    }
}
```

This is equivalent to the following more verbose declaration, which specifies the return type and value explicitly:

```
// `Unit` is required in the `() -> Unit` function type,
// but declaring `: Unit` as the return type of `printHello()` is optional
fun printHello(name: String?, action: () -> Unit): Unit {
    if (name != null)
        println("Hello $name")
    else
        println("Hi there!")

    action()
    // Returning the `Unit` value explicitly is optional
    return Unit
}
fun main() {
    printHello("Kodee") {
        println("This action runs when a name is given.")
        // Hello Kodee
        // This action runs when a name is given.
    }
    printHello(null) {
        println("This action runs when no name is provided.")
        // Hi there!
        // This action runs when no name is provided.
    }
}
```

Variable number of arguments (varargs)

To pass a variable number of arguments to a function, you can mark one of its parameters (usually the last one) with the `vararg` modifier. Inside a function, you can use a `vararg`-parameter of type `T` as an array of `T`:

```
fun <T> asList(vararg ts: T): List<T> {
    val result = ArrayList<T>()
    for (t in ts) // ts is an Array
        result.add(t)
    return result
}
```

Then you can pass a variable number of arguments to the function:

```
fun <T> asList(vararg ts: T): List<T> {
    val result = ArrayList<T>()
    for (t in ts) // ts is an Array
```

```
        result.add(t)
    return result
}

fun main() {
    val list = asList(1, 2, 3)
    println(list)
    // [1, 2, 3]
}
```

Only one parameter can be marked as `vararg`. If you declare a `vararg` parameter anywhere other than last in the parameter list, you must pass values for the following parameters using named arguments. If a parameter has a function type, you can also pass its value by placing a lambda outside the parentheses.

When you call a `vararg`-function, you can pass arguments individually, as in the example of `asList(1, 2, 3)`. If you already have an array and want to pass its contents to a function as a `vararg` parameter or as a part of it, use the spread operator by prefixing the array name with `*`:

```
fun <T> asList(vararg ts: T): List<T> {
    val result = ArrayList<T>()
    for (t in ts)
        result.add(t)
    return result
}

fun main() {
    val a = arrayOf(1, 2, 3)

    // The function receives the array [-1, 0, 1, 2, 3, 4]
    list = asList(-1, 0, *a, 4)

    println(list)
    // [-1, 0, 1, 2, 3, 4]
}
```

If you want to pass a primitive type array as `vararg`, you need to convert it to a regular (typed) array using the `.toTypedArray()` function:

```
// 'a' is an IntArray, which is a primitive type array
val a = intArrayOf(1, 2, 3)
val list = asList(-1, 0, *a.toTypedArray(), 4)
```

Infix notation

You can declare functions that can be called without parentheses or the period by using the `infix` keyword. This can help make simple function calls in your code easier to read.

```
infix fun Int.shl(x: Int): Int { /*...*/ }
```

```
// Calls the function using the general notation
1.shl(2)

// Calls the function using the infix notation
1 shl 2
```

Infix functions must meet the following requirements:

- They must be member functions of a class or extension functions.
- They must have a single parameter.
- The parameter must not accept a variable number of arguments (`vararg`) and must have no default value.

  Note: Infix function calls have lower precedence than arithmetic operators, type casts, and the `rangeTo` operator. The following expressions are equivalent:

  - `1 shl 2 + 3` is equivalent to `1 shl (2 + 3)`
  - `0 until n * 2` is equivalent to `0 until (n * 2)`
  - `xs union ys as Set<*>` is equivalent to `xs union (ys as Set<*>)`

  On the other hand, an infix function call's precedence is higher than that of the boolean operators `&&` and `||`, `is`- and `in`-checks, and some other operators. These expressions are equivalent as well:

  - `a && b xor c` is equivalent to `a && (b xor c)`
  - `a xor b in c` is equivalent to `(a xor b) in c`

Note that infix functions always require both the receiver and the parameter to be specified. When you call a method on the current receiver using the infix notation, use `this` explicitly. This ensures unambiguous parsing.

```
class MyStringCollection {
    val items = mutableListOf<String>()

    infix fun add(s: String) {
        println("Adding: $s")
        items += s
    }

    fun build() {
        add("first")      // Correct: ordinary function call
        this add "second" // Correct: infix call with an explicit receiver
        // add "third"    // Compiler error: needs an explicit receiver
    }

    fun printAll() = println("Items = $items")
}

fun main() {
    val myStrings = MyStringCollection()
```

```
    // Adds "first" and "second" to the list
    myStrings.build()

    myStrings.printAll()
    // Adding: first
    // Adding: second
    // Items = [first, second]
}
```

Function scope

You can declare Kotlin functions at the top level in a file, meaning you do not need to create a class to hold a function. Functions can also be declared locally as *member functions* or *extension functions*.

Local functions

Kotlin supports local functions, which are functions declared inside other functions. For example, the following code implements the Depth-first search algorithm for a given graph. The local `dfs()` function inside the outer `dfs()` function to hide the implementation and handle recursive calls:

```
class Person(val name: String) {
    val friends = mutableListOf<Person>()
}
class SocialGraph(val people: List<Person>)
fun dfs(graph: SocialGraph) {
    fun dfs(current: Person, visited: MutableSet<Person>) {
        if (!visited.add(current)) return
        println("Visited ${current.name}")
        for (friend in current.friends)
            dfs(friend, visited)
    }
    dfs(graph.people[0], HashSet())
}
fun main() {
    val alice = Person("Alice")
    val bob = Person("Bob")
    val charlie = Person("Charlie")
    alice.friends += bob
    bob.friends += charlie
    charlie.friends += alice
    val network = SocialGraph(listOf(alice, bob, charlie))
    dfs(network)
}
```

A local function can access local variables of outer functions (the closure). In the case above, the `visited` function parameter can be a local variable:

```
class Person(val name: String) {
    val friends = mutableListOf<Person>()
```

```
}
class SocialGraph(val people: List<Person>)
fun dfs(graph: SocialGraph) {
    val visited = HashSet<Person>()
    fun dfs(current: Person) {
        if (!visited.add(current)) return
        println("Visited ${current.name}")
        for (friend in current.friends)
            dfs(friend)
    }
    dfs(graph.people[0])
}
fun main() {
    val alice = Person("Alice")
    val bob = Person("Bob")
    val charlie = Person("Charlie")
    alice.friends += bob
    bob.friends += charlie
    charlie.friends += alice
    val network = SocialGraph(listOf(alice, bob, charlie))
    dfs(network)
}
```

Member functions

A member function is a function that is defined inside a class or object:

```
class Sample {
    fun foo() { print("Foo") }
}
```

To call member functions, write the instance or object name, then add a `.` and write the function name:

```
// Creates an instance of the Stream class and calls read()
Stream().read()
```

For more information on classes and overriding members see Classes and Inheritance.

Generic functions

You can specify generic parameters for a function by using angle brackets `<>` before the function name:

```
fun <T> singletonList(item: T): List<T> { /*...*/ }
```

For more information on generic functions, see Generics.

Tail recursive functions

Kotlin supports a style of functional programming known as tail recursion. For some algorithms that would normally use loops, you can use a recursive function instead without the risk of stack overflow. When a function is marked with the `tailrec` modifier and meets the required formal conditions, the compiler optimizes out the recursion, leaving behind a fast and efficient loop based version instead:

```
import kotlin.math.cos
import kotlin.math.abs

// An arbitrary "good enough" precision
val eps = 1E-10

tailrec fun findFixPoint(x: Double = 1.0): Double =
    if (abs(x - cos(x)) < eps) x else findFixPoint(cos(x))
```

This code calculates the fixed point of cosine (a mathematical constant). The function calls `cos()` repeatedly starting at `1.0` until the result no longer changes, yielding a result of `0.7390851332151611` for the specified `eps` precision. The code is equivalent to this more traditional style:

```
import kotlin.math.cos
import kotlin.math.abs

// An arbitrary "good enough" precision
val eps = 1E-10

private fun findFixPoint(): Double {
    var x = 1.0
    while (true) {
        val y = cos(x)
        if (abs(x - y) < eps) return x
        x = cos(x)
    }
}
```

You can apply the `tailrec` modifier to a function only when it calls itself as its final operation. You cannot use tail recursion when there is more code after the recursive call, within `try`/`catch`/`finally` blocks, or when the function is open.

See also:

- Inline functions
- Extension functions
- Higher-order functions and lambdas

Higher-order functions and lambdas

Kotlin functions are first-class, which means they can be stored in variables and data structures, and can be passed as arguments to and returned from other higher-order functions. You can perform any operations on functions that are possible for other non-function values.

To facilitate this, Kotlin, as a statically typed programming language, uses a family of function types to represent functions, and provides a set of specialized language constructs, such as lambda expressions.

Higher-order functions

A higher-order function is a function that takes functions as parameters, or returns a function.

A good example of a higher-order function is the functional programming idiom `fold` for collections. It takes an initial accumulator value and a combining function and builds its return value by consecutively combining the current accumulator value with each collection element, replacing the accumulator value each time:

```
fun <T, R> Collection<T>.fold(
    initial: R,
    combine: (acc: R, nextElement: T) -> R
): R {
    var accumulator: R = initial
    for (element: T in this) {
        accumulator = combine(accumulator, element)
    }
    return accumulator
}
```

In the code above, the `combine` parameter has the function type `(R, T) -> R`, so it accepts a function that takes two arguments of types `R` and `T` and returns a value of type `R`. It is invoked inside the `for` loop, and the return value is then assigned to `accumulator`.

To call `fold`, you need to pass an instance of the function type to it as an argument, and lambda expressions (described in more detail below) are widely used for this purpose at higher-order function call sites:

```
fun main() {
    val items = listOf(1, 2, 3, 4, 5)

    // Lambdas are code blocks enclosed in curly braces.
    items.fold(0, {
        // When a lambda has parameters, they go first, followed by '->'
        acc: Int, i: Int ->
```

```
        print("acc = $acc, i = $i, ")
        val result = acc + i
        println("result = $result")
        // The last expression in a lambda is considered the return value:
        result
    })

    // Parameter types in a lambda are optional if they can be inferred:
    val joinedToString = items.fold("Elements:", { acc, i -> acc + " " + i })

    // Function references can also be used for higher-order function calls:
    val product = items.fold(1, Int::times)
    println("joinedToString = $joinedToString")
    println("product = $product")
}
```

Function types

Kotlin uses function types, such as `(Int) -> String`, for declarations that deal with functions: `val onClick: () -> Unit = ...`.

These types have a special notation that corresponds to the signatures of the functions - their parameters and return values:

- All function types have a parenthesized list of parameter types and a return type: `(A, B) -> C` denotes a type that represents functions that take two arguments of types `A` and `B` and return a value of type `C`. The list of parameter types may be empty, as in `() -> A`. The `Unit` return type cannot be omitted.
- Function types can optionally have an additional *receiver* type, which is specified before the dot in the notation: the type `A.(B) -> C` represents functions that can be called on a receiver object `A` with a parameter `B` and return a value `C`. Function literals with receiver are often used along with these types.
- Suspending functions belong to a special kind of function type that have a *suspend* modifier in their notation, such as `suspend () -> Unit` or `suspend A.(B) -> C`.

The function type notation can optionally include names for the function parameters: `(x: Int, y: Int) -> Point`. These names can be used for documenting the meaning of the parameters.

To specify that a function type is nullable, use parentheses as follows: `((Int, Int) -> Int)?`.

Function types can also be combined using parentheses: `(Int) -> ((Int) -> Unit)`.

> **Note:** The arrow notation is right-associative, `(Int) -> (Int) -> Unit` is equivalent to the previous example, but not to `((Int) -> (Int)) -> Unit`.

You can also give a function type an alternative name by using a type alias:

```
typealias ClickHandler = (Button, ClickEvent) -> Unit
```

Instantiating a function type

There are several ways to obtain an instance of a function type:

- Use a code block within a function literal, in one of the following forms:
  - a lambda expression: `{ a, b -> a + b }`,
  - an anonymous function: `fun(s: String): Int { return s.toIntOrNull() ?: 0 }`

  Function literals with receiver can be used as values of function types with receiver.
- Use a callable reference to an existing declaration:
  - a top-level, local, member, or extension function: `::isOdd`, `String::toInt`,
  - a top-level, member, or extension property: `List<Int>::size`,
  - a constructor: `::Regex`

  These include bound callable references that point to a member of a particular instance: `foo::toString`.
- Use instances of a custom class that implements a function type as an interface:

```
class IntTransformer: (Int) -> Int {
    override operator fun invoke(x: Int): Int = TODO()
}

val intFunction: (Int) -> Int = IntTransformer()
```

The compiler can infer the function types for variables if there is enough information:

```
val a = { i: Int -> i + 1 } // The inferred type is (Int) -> Int
```

Non-literal values of function types with and without a receiver are interchangeable, so the receiver can stand in for the first parameter, and vice versa. For instance, a value of type `(A, B) -> C` can be passed or assigned where a value of type `A.(B) -> C` is expected, and the other way around:

```
fun main() {
    val repeatFun: String.(Int) -> String = { times -> this.repeat(times) }
    val twoParameters: (String, Int) -> String = repeatFun // OK

    fun runTransformation(f: (String, Int) -> String): String {
        return f("hello", 3)
    }
    val result = runTransformation(repeatFun) // OK
```

```
    println("result = $result")
}
```

> **Note:** A function type with no receiver is inferred by default, even if a variable is initialized with a reference to an extension function. To alter that, specify the variable type explicitly.

Invoking a function type instance

A value of a function type can be invoked by using its `invoke(...)` operator: `f.invoke(x)` or just `f(x)`.

If the value has a receiver type, the receiver object should be passed as the first argument. Another way to invoke a value of a function type with receiver is to prepend it with the receiver object, as if the value were an extension function: `1.foo(2)`.

Example:

```
fun main() {
    val stringPlus: (String, String) -> String = String::plus
    val intPlus: Int.(Int) -> Int = Int::plus

    println(stringPlus.invoke("<-", "->"))
    println(stringPlus("Hello, ", "world!"))

    println(intPlus.invoke(1, 1))
    println(intPlus(1, 2))
    println(2.intPlus(3)) // extension-like call
}
```

Inline functions

Sometimes it is beneficial to use inline functions, which provide flexible control flow, for higher-order functions.

Callable reference

Callable references let you refer to a function, property, or constructor without calling or accessing it immediately. You can store the reference in a variable, pass it to another function, or invoke it later.

To create a callable reference, use the `::` operator:

```
fun isOdd(number: Int) = number % 2 != 0

fun main() {
    val numbers = listOf(1, 2, 3, 4, 5)
```

```
    // Pass a reference to isOdd()
    println(numbers.filter(::isOdd))
    // [1, 3, 5]
}
```

Callable-reference objects implement reflection interfaces, such as `KFunction` and `KProperty`.

Function references

Use a function reference to pass or store an existing function as a value. To refer to a top-level or local named function, use `::functionName`:

```
fun calculateLength(text: String) = text.length

fun main() {
    val length: (String) -> Int = ::calculateLength

    println(length("Kotlin"))
    // 6
    println(::calculateLength.name)
    // calculateLength
}
```

Function references belong to one of the `KFunction<out R>` subtypes, depending on the parameter count. For instance, `KFunction3<T1, T2, T3, R>`.

If several functions have the same name, the compiler uses the expected function type to select the matching overload:

```
fun parse(value: String) = "String: $value"
fun parse(value: Int) = "Int: $value"

fun main() {
    val parseString: (String) -> String = ::parse
    val parseInt: (Int) -> String = ::parse

    println(parseString("Kotlin"))
    // String: Kotlin
    println(parseInt(22))
    // Int: 22
}
```

Property references

Use a property reference when you need to pass a statically checked property accessor. For example, another function can use the reference to select which value to read or update.

To create a property reference, use the `::` operator. You can read the property by calling the reference as a function or by using the `get()` function. A mutable property reference also provides `set()`:

```
var greeting = "Hello"

class User(var name: String)

fun main() {
    // A top-level property reference
    // No receiver is needed
    println(::greeting.get())
    // Hello

    ::greeting.set("Hi")
    println(greeting)
    // Hi

    val user = User("Jane")

    // An unbound member property reference
    // Pass a User receiver explicitly
    val unboundName = User::name
    println(unboundName.get(user))
    // Jane

    unboundName.set(user, "Jane Doe")
    println(user.name)
    // Jane Doe

    // A bound member property reference
    // Captures the user receiver
    val boundName = user::name

    println(boundName.get())
    // Jane Doe

    boundName.set("Jane Smith")
    println(user.name)
    // Jane Smith
}
```

Property-reference types indicate how many receivers are required:

- `KProperty0<V>` requires no receiver. This includes top-level properties and properties bound to an object.
- `KProperty1<T, V>` requires one receiver.
- `KProperty2<D, E, V>` requires two receivers and represents a member extension property: one receiver for the containing class and another for the extended type.

Mutable properties have corresponding `KMutableProperty0`, `KMutableProperty1`, and `KMutableProperty2` types. Usually, you don't need to write these types explicitly because the compiler infers them from the reference.

> **Tip:** To discover the property at runtime, use Kotlin reflection.

Constructor references

Use a constructor reference when an API needs a factory and an existing constructor already has the required parameters and return type. The constructor reference lets the API create new instances through a function value.

To create a reference to a constructor, use `::ClassName`:

```
data class User(val name: String, val age: Int)

fun createUser(factory: (String, Int) -> User): User =
    factory("Jane Doe", 22)

fun main() {
    val user = createUser(::User)
    println(user)
    // User(name=Jane Doe, age=22)
}
```

Callable references to constructors are typed as one of the `KFunction<out R>` subtypes depending on the parameter count.

Bound and unbound references

A reference to a member can be *bound* and *unbound*:

- Bound reference captures a particular instance and requires only the function's declared arguments. Use it when every invocation should use the same object.
- Unbound reference isn't attached to an instance, so its function type includes the receiver. Use it when the caller should provide the object.

```
class User(val name: String) {
    fun hasName(prefix: String) = name.startsWith(prefix)
}

fun main() {
    // The first parameter of an unbound reference is the User receiver
    val unbound: (User, String) -> Boolean = User::hasName
    println(unbound(User("Jane"), "Ja"))
    // true

    val jane = User("Jane")

    // Reference captures jane as its receiver
    val bound: (String) -> Boolean = jane::hasName
    println(bound("Doe"))
    // false
}
```

Extension functions follow the same rule. For example, `String::isBlank` is unbound and needs a `String` receiver. `"Kotlin"::isBlank` is bound and takes no parameters.

Lambda expressions and anonymous functions

Lambda expressions and anonymous functions are *function literals*. Function literals are functions that are not declared but are passed immediately as an expression. Consider the following example:

```
max(strings, { a, b -> a.length < b.length })
```

The function `max` is a higher-order function, as it takes a function value as its second argument. This second argument is an expression that is itself a function, called a function literal, which is equivalent to the following named function:

```
fun compare(a: String, b: String): Boolean = a.length < b.length
```

You can also create a *suspending lambda expression* using the `suspend` keyword. A suspending lambda has the function type `suspend () -> Unit` and can call other suspending functions:

```
val suspendingTask = suspend { doSuspendingWork() }
```

Lambda expression syntax

The full syntactic form of lambda expressions is as follows:

```
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
```

- A lambda expression is always surrounded by curly braces.
- Parameter declarations in the full syntactic form go inside curly braces and have optional type annotations.
- The body goes after the `->`.
- If the inferred return type of the lambda is not `Unit`, the last (or possibly single) expression inside the lambda body is treated as the return value.

If you leave all the optional annotations out, what's left looks like this:

```
val sum = { x: Int, y: Int -> x + y }
```

Passing trailing lambdas

According to Kotlin convention, if the last parameter of a function is a function, then a lambda expression passed as the corresponding argument can be placed outside the parentheses:

```
val product = items.fold(1) { acc, e -> acc * e }
```

Such syntax is also known as *trailing lambda*.

If the lambda is the only argument in that call, the parentheses can be omitted entirely:

```
run { println("...") }
```

it: implicit name of a single parameter

It's very common for a lambda expression to have only one parameter.

If the compiler can parse the signature without any parameters, the parameter does not need to be declared and `->` can be omitted. The parameter will be implicitly declared under the name `it`:

```
ints.filter { it > 0 } // this literal is of type '(it: Int) -> Boolean'
```

Returning a value from a lambda expression

You can explicitly return a value from the lambda using the qualified return syntax. Otherwise, the value of the last expression is implicitly returned.

Therefore, the two following snippets are equivalent:

```
ints.filter {
    val shouldFilter = it > 0
    shouldFilter
}

ints.filter {
    val shouldFilter = it > 0
    return@filter shouldFilter
}
```

This convention, along with passing a lambda expression outside of parentheses, allows for LINQ-style code:

```
strings.filter { it.length == 5 }.sortedBy { it }.map { it.uppercase() }
```

Underscore for unused variables

If the lambda parameter is unused, you can place an underscore instead of its name:

```
map.forEach { (_, value) -> println("$value!") }
```

Destructuring in lambdas

Destructuring in lambdas is described as a part of destructuring declarations.

Anonymous functions

The lambda expression syntax above is missing one thing – the ability to specify the function's return type. In most cases, this is unnecessary because the return type can be inferred automatically. However, if you do need to specify it explicitly, you can use an alternative syntax: an *anonymous function*.

```
fun(x: Int, y: Int): Int = x + y
```

An anonymous function looks very much like a regular function declaration, except its name is omitted. Its body can be either an expression (as shown above) or a block:

```
fun(x: Int, y: Int): Int {
    return x + y
}
```

The parameters and the return type are specified in the same way as for regular functions, except the parameter types can be omitted if they can be inferred from the context:

```
ints.filter(fun(item) = item > 0)
```

The return type inference for anonymous functions works just like for normal functions: the return type is inferred automatically for anonymous functions with an expression body, but it has to be specified explicitly (or is assumed to be `Unit`) for anonymous functions with a block body.

> **Note:** When passing anonymous functions as parameters, place them inside the parentheses. The shorthand syntax that allows you to leave the function outside the parentheses works only for lambda expressions.

Another difference between lambda expressions and anonymous functions is the behavior of non-local returns. A `return` statement without a label always returns from the function declared with the `fun` keyword. This means that a `return` inside a lambda expression will return from the enclosing function, whereas a `return` inside an anonymous function will return from the anonymous function itself.

Closures

A lambda expression or anonymous function (as well as a local function and an object expression) can access its *closure*, which includes the variables declared in the outer scope. The variables captured in the closure can be modified in the lambda:

```
var sum = 0
ints.filter { it > 0 }.forEach {
    sum += it
}
print(sum)
```

Function literals with receiver

Function types with receiver, such as `A.(B) -> C`, can be instantiated with a special form of function literals – function literals with receiver.

As mentioned above, Kotlin provides the ability to call an instance of a function type with receiver while providing the *receiver object*.

Inside the body of the function literal, the receiver object passed to a call becomes an *implicit* `this`, so that you can access the members of that receiver object without any additional qualifiers, or access the receiver object using a `this` expression.

This behavior is similar to that of extension functions, which also allow you to access the members of the receiver object inside the function body.

Here is an example of a function literal with receiver along with its type, where `plus` is called on the receiver object:

```
val sum: Int.(Int) -> Int = { other -> plus(other) }
```

The anonymous function syntax allows you to specify the receiver type of a function literal directly. This can be useful if you need to declare a variable of a function type with receiver, and then to use it later.

```
val sum = fun Int.(other: Int): Int = this + other
```

Lambda expressions can be used as function literals with receiver when the receiver type can be inferred from the context. One of the most important examples of their usage is type-safe builders:

```
class HTML {
    fun body() { ... }
}

fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()  // create the receiver object
    html.init()        // pass the receiver object to the lambda
    return html
}

html {       // lambda with receiver begins here
    body()   // calling a method on the receiver object
}
```

Type-safe builders

By using well-named functions as builders in combination with function literals with receiver, it is possible to create type-safe, statically-typed builders in Kotlin.

Type-safe builders allow creating Kotlin-based domain-specific languages (DSLs) suitable for building complex hierarchical data structures in a semi-declarative way. Sample use cases for the builders are:

- Generating markup with Kotlin code, such as HTML or XML
- Configuring routes for a web server: Ktor

Consider the following code:

```
package html

fun main() {
    val result = html {
        head {
            title { +"HTML encoding with Kotlin" }
        }
        body {
            h1 { +"HTML encoding with Kotlin" }
            p {
                +"this format can be used as an"
                +"alternative markup to HTML"
            }

            // An element with attributes and text content
            a(href = "http://kotlinlang.org") { +"Kotlin" }

            // Mixed content
            p {
                +"This is some"
                b { +"mixed" }
                +"text. For more see the"
                a(href = "http://kotlinlang.org") {
                    +"Kotlin"
                }
                +"project"
            }
            p {
                +"some text"
                ul {
                    for (i in 1..5)
                        li { +"${i}*2 = ${i*2}" }
                }
            }
        }
    }
    println(result)
```

```
}

interface Element {
    fun render(builder: StringBuilder, indent: String)
}

class TextElement(val text: String) : Element {
    override fun render(builder: StringBuilder, indent: String) {
        builder.append("$indent$text\n")
    }
}

@DslMarker
annotation class HtmlTagMarker

@HtmlTagMarker
abstract class Tag(val name: String) : Element {
    val children = arrayListOf<Element>()
    val attributes = hashMapOf<String, String>()

    protected fun <T : Element> initTag(tag: T, init: T.() -> Unit): T {
        tag.init()
        children.add(tag)
        return tag
    }

    override fun render(builder: StringBuilder, indent: String) {
        builder.append("$indent<$name${renderAttributes()}>\n")
        for (c in children) {
            c.render(builder, indent + "  ")
        }
        builder.append("$indent</$name>\n")
    }

    private fun renderAttributes(): String {
        val builder = StringBuilder()
        for ((attr, value) in attributes) {
            builder.append(" $attr=\"$value\"")
        }
        return builder.toString()
    }

    override fun toString(): String {
        val builder = StringBuilder()
        render(builder, "")
        return builder.toString()
    }
}

abstract class TagWithText(name: String) : Tag(name) {
    operator fun String.unaryPlus() {
        children.add(TextElement(this))
    }
}
class HTML() : TagWithText("html") {
    fun head(init: Head.() -> Unit) = initTag(Head(), init)
```

```
    fun body(init: Body.() -> Unit) = initTag(Body(), init)
}

class Head() : TagWithText("head") {
    fun title(init: Title.() -> Unit) = initTag(Title(), init)
}

class Title() : TagWithText("title")

abstract class BodyTag(name: String) : TagWithText(name) {
    fun b(init: B.() -> Unit) = initTag(B(), init)
    fun p(init: P.() -> Unit) = initTag(P(), init)
    fun h1(init: H1.() -> Unit) = initTag(H1(), init)
    fun ul(init: UL.() -> Unit) = initTag(UL(), init)
    fun a(href: String, init: A.() -> Unit) {
        val a = initTag(A(), init)
        a.href = href
    }
}

class Body() : BodyTag("body")
class UL() : BodyTag("ul") {
    fun li(init: LI.() -> Unit) = initTag(LI(), init)
}

class B() : BodyTag("b")
class LI() : BodyTag("li")
class P() : BodyTag("p")
class H1() : BodyTag("h1")

class A : BodyTag("a") {
    var href: String
        get() = attributes["href"]!!
        set(value) {
            attributes["href"] = value
        }
}

fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}
```

```
<html>
  <head>
    <title>
      HTML encoding with Kotlin
    </title>
  </head>
  <body>
    <h1>
      HTML encoding with Kotlin
    </h1>
```

```
      this format can be used as an
      alternative markup to HTML



      Kotlin



      This is some
      <b>
        mixed
      </b>
      text. For more see the

        Kotlin

      project




      some text



          1*2 = 2



          2*2 = 4



          3*2 = 6



          4*2 = 8



          5*2 = 10




  </body>
</html>
```

How it works

Assume that you need to implement a type-safe builder in Kotlin. First of all, define the model you want to build. In this case, you need to model HTML tags. It is easily done with a bunch of classes. For example, `HTML` is a class that describes the `<html>` tag defining children like `<head>` and `<body>`. (See its declaration below.)

Now, let's recall why you can say something like this in the code:

```
html {
 // ...
}
```

`html` is actually a function call that takes a lambda expression as an argument. This function is defined as follows:

```
fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}
```

This function takes one parameter named `init`, which is itself a function. The type of the function is `HTML.() -> Unit`, which is a *function type with receiver*. This means that you need to pass an instance of type `HTML` (a *receiver*) to the function, and you can call members of that instance inside the function.

The receiver can be accessed through the `this` keyword:

```
html {
    this.head { ... }
    this.body { ... }
}
```

(`head` and `body` are member functions of `HTML`.)

Now, `this` can be omitted, as usual, and you get something that looks very much like a builder already:

```
html {
    head { ... }
    body { ... }
}
```

So, what does this call do? Let's look at the body of `html` function as defined above. It creates a new instance of `HTML`, then it initializes it by calling the function that is passed as an argument (in this

example, this boils down to calling `head` and `body` on the `HTML` instance), and then it returns this instance. This is exactly what a builder should do.

The `head` and `body` functions in the `HTML` class are defined similarly to `html`. The only difference is that they add the built instances to the `children` collection of the enclosing `HTML` instance:

```
fun head(init: Head.() -> Unit): Head {
    val head = Head()
    head.init()
    children.add(head)
    return head
}

fun body(init: Body.() -> Unit): Body {
    val body = Body()
    body.init()
    children.add(body)
    return body
}
```

Actually, these two functions do just the same thing, so you can have a generic version, `initTag`:

```
protected fun <T : Element> initTag(tag: T, init: T.() -> Unit): T {
    tag.init()
    children.add(tag)
    return tag
}
```

So, now your functions are very simple:

```
fun head(init: Head.() -> Unit) = initTag(Head(), init)

fun body(init: Body.() -> Unit) = initTag(Body(), init)
```

And you can use them to build `<head>` and `<body>` tags.

One other thing to be discussed here is how you add text to tag bodies. In the example above, you say something like:

```
html {
    head {
        title {+"XML encoding with Kotlin"}
    }
    // ...
}
```

So basically, you just put a string inside a tag body, but there is this little `+` in front of it, so it is a function call that invokes a prefix `unaryPlus()` operation. That operation is actually defined by an

extension function `unaryPlus()` that is a member of the `TagWithText` abstract class (a parent of `Title`):

```
operator fun String.unaryPlus() {
    children.add(TextElement(this))
}
```

So, what the prefix `+` does here is wrapping a string into an instance of `TextElement` and adding it to the `children` collection, so that it becomes a proper part of the tag tree.

All this is defined in a package `com.example.html` that is imported at the top of the builder example above. In the last section, you can read through the full definition of this package.

Scope control: @DslMarker

When using DSLs, one might have come across the problem that too many functions can be called in the context. You can call methods of every available implicit receiver inside a lambda and therefore get an inconsistent result, like the tag `head` inside another `head`:

```
html {
    head {
        head {} // should be forbidden
    }
    // ...
}
```

In this example, only members of the nearest implicit receiver `this@head` must be available; `head()` is a member of the outer receiver `this@html`, so it must be illegal to call it.

To address this problem, there is a special mechanism to control receiver scope.

To make the compiler start controlling scopes, you only have to annotate the types of all receivers used in the DSL with the same marker annotation. For instance, for HTML Builders you declare an annotation `@HtmlTagMarker`:

```
@DslMarker
@Target(AnnotationTarget.CLASS)
annotation class HtmlTagMarker
```

An annotation class is called a DSL marker if it is annotated with the `@DslMarker` annotation.

The `@Target` annotation restricts where `@HtmlTagMarker` can be applied. DSL markers only affect scope control when applied to:

- Type declarations (`CLASS`): classes or interfaces used as DSL receivers.
- Type usages (`TYPE`): receiver types in function type signatures.

- Type aliases (`TYPEALIAS`): type aliases that expand to DSL receiver types.

Applying a DSL marker to other targets (such as functions or properties) has no effect on scope control.

> **Note:** For more details on how DSL marker works, see the corresponding KEEP document.

In our DSL, all the tag classes extend the same superclass `Tag`. It's enough to annotate only the superclass with `@HtmlTagMarker` and after that the Kotlin compiler will treat all the inherited classes as annotated:

```
@HtmlTagMarker
abstract class Tag(val name: String) { ... }
```

You don't have to annotate the `HTML` or `Head` classes with `@HtmlTagMarker` because their superclass is already annotated:

```
class HTML() : Tag("html") { ... }

class Head() : Tag("head") { ... }
```

After you've added this annotation, the Kotlin compiler knows which implicit receivers are part of the same DSL and allows to call members of the nearest receivers only:

```
html {
    head {
        head { } // error: a member of outer receiver
    }
    // ...
}
```

Note that it's still possible to call the members of the outer receiver, but to do that you have to specify this receiver explicitly:

```
html {
    head {
        this@html.head { } // possible
    }
    // ...
}
```

You can also apply the `@DslMarker` annotation directly to function types. This requires including `AnnotationTarget.TYPE` in the annotation targets:

```
@DslMarker
@Target(AnnotationTarget.CLASS, AnnotationTarget.TYPE)
annotation class HtmlTagMarker
```

As a result, the `@DslMarker` annotation can be applied to function types, most commonly to lambdas with receivers. For example:

```
fun html(init: @HtmlTagMarker HTML.() -> Unit): HTML { ... }

fun HTML.head(init: @HtmlTagMarker Head.() -> Unit): Head { ... }

fun Head.title(init: @HtmlTagMarker Title.() -> Unit): Title { ... }
```

When you call these functions, the `@DslMarker` annotation restricts access to outer receivers in the body of a lambda marked with it unless you specify them explicitly:

```
html {
    head {
        title {
            // Access to title, head or other functions of outer receivers is restricted her
e.
        }
    }
}
```

Only the nearest receiver's members and extensions are accessible within a lambda, preventing unintended interactions between nested scopes.

When both a member of an implicit receiver and a declaration from a context parameter are in a scope with the same name, the compiler reports a warning because the implicit receiver is shadowed by the context parameter. To resolve this, use a `this` qualifier to explicitly call the receiver, or use `contextOf<T>()` to call the context declaration:

```
interface HtmlTag {
    fun setAttribute(name: String, value: String)
}

// Declares a top-level function with the same name,
// which is available through a context parameter
context(tag: HtmlTag)
fun setAttribute(name: String, value: String) { tag.setAttribute(name, value) }

fun test(head: HtmlTag, extraInfo: HtmlTag) {
    with(head) {
        // Introduces a context value of the same type in an inner scope
        context(extraInfo) {
            // Reports a warning:
            // Uses an implicit receiver shadowed by a context parameter
            setAttribute("user", "1234")

            // Calls the receiver's member explicitly
            this.setAttribute("user", "1234")

            // Calls the context declaration explicitly
            contextOf<HtmlTag>().setAttribute("user", "1234")
```

```
        }
    }
}
```

Full definition of the com.example.html package

This is how the package `com.example.html` is defined (only the elements used in the example above). It builds an HTML tree. It makes heavy use of extension functions and lambdas with receiver.

```
package com.example.html

interface Element {
    fun render(builder: StringBuilder, indent: String)
}

class TextElement(val text: String) : Element {
    override fun render(builder: StringBuilder, indent: String) {
        builder.append("$indent$text\n")
    }
}

@DslMarker
@Target(AnnotationTarget.CLASS, AnnotationTarget.TYPE)
annotation class HtmlTagMarker

@HtmlTagMarker
abstract class Tag(val name: String) : Element {
    val children = arrayListOf<Element>()
    val attributes = hashMapOf<String, String>()

    protected fun <T : Element> initTag(tag: T, init: T.() -> Unit): T {
        tag.init()
        children.add(tag)
        return tag
    }

    override fun render(builder: StringBuilder, indent: String) {
        builder.append("$indent<$name${renderAttributes()}>\n")
        for (c in children) {
            c.render(builder, indent + "  ")
        }
        builder.append("$indent</$name>\n")
    }

    private fun renderAttributes(): String {
        val builder = StringBuilder()
        for ((attr, value) in attributes) {
            builder.append(" $attr=\"$value\"")
        }
        return builder.toString()
    }

    override fun toString(): String {
```

```
        val builder = StringBuilder()
        render(builder, "")
        return builder.toString()
    }
}

abstract class TagWithText(name: String) : Tag(name) {
    operator fun String.unaryPlus() {
        children.add(TextElement(this))
    }
}

class HTML : TagWithText("html") {
    fun head(init: Head.() -> Unit) = initTag(Head(), init)

    fun body(init: Body.() -> Unit) = initTag(Body(), init)
}

class Head : TagWithText("head") {
    fun title(init: Title.() -> Unit) = initTag(Title(), init)
}

class Title : TagWithText("title")

abstract class BodyTag(name: String) : TagWithText(name) {
    fun b(init: B.() -> Unit) = initTag(B(), init)
    fun p(init: P.() -> Unit) = initTag(P(), init)
    fun h1(init: H1.() -> Unit) = initTag(H1(), init)
    fun a(href: String, init: A.() -> Unit) {
        val a = initTag(A(), init)
        a.href = href
    }
}

class Body : BodyTag("body")
class B : BodyTag("b")
class P : BodyTag("p")
class H1 : BodyTag("h1")

class A : BodyTag("a") {
    var href: String
        get() = attributes["href"]!!
        set(value) {
            attributes["href"] = value
        }
}

fun html(init: HTML.() -> Unit): HTML {
    val html = HTML()
    html.init()
    return html
}
```

Using builders with builder type inference

Kotlin supports *builder type inference* (or builder inference), which can come in useful when you are working with generic builders. It helps the compiler infer the type arguments of a builder call based on the type information about other calls inside its lambda argument.

Consider this example of `buildMap()` usage:

```
fun addEntryToMap(baseMap: Map<String, Number>, additionalEntry: Pair<String, Int>?) {
    val myMap = buildMap {
        putAll(baseMap)
        if (additionalEntry != null) {
            put(additionalEntry.first, additionalEntry.second)
        }
    }
}
```

There is not enough type information here to infer type arguments in a regular way, but builder inference can analyze the calls inside the lambda argument. Based on the type information about `putAll()` and `put()` calls, the compiler can automatically infer type arguments of the `buildMap()` call into `String` and `Number`. Builder inference allows to omit type arguments while using generic builders.

Writing your own builders

Requirements for enabling builder inference

> **Note:** Before Kotlin 1.7.0, enabling builder inference for a builder function required `-Xenable-builder-inference` compiler option. In 1.7.0 the option is enabled by default.

To let builder inference work for your own builder, make sure its declaration has a builder lambda parameter of a function type with a receiver. There are also two requirements for the receiver type:

1. It should use the type arguments that builder inference is supposed to infer. For example:

   ```
   fun <V> buildList(builder: MutableList<V>.() -> Unit) { ... }
   ```

   > Note that passing the type parameter's type directly like `fun <T> myBuilder(builder: T.() -> Unit)` is not yet supported.

2. It should provide public members or extensions that contain the corresponding type parameters in their signature. For example:

```
class ItemHolder<T> {
    private val items = mutableListOf<T>()

    fun addItem(x: T) {
        items.add(x)
    }

    fun getLastItem(): T? = items.lastOrNull()
}

fun <T> ItemHolder<T>.addAllItems(xs: List<T>) {
    xs.forEach { addItem(it) }
}

fun <T> itemHolderBuilder(builder: ItemHolder<T>.() -> Unit): ItemHolder<T> =
    ItemHolder<T>().apply(builder)

fun test(s: String) {
    val itemHolder1 = itemHolderBuilder { // Type of itemHolder1 is ItemHolder<String>
        addItem(s)
    }
    val itemHolder2 = itemHolderBuilder { // Type of itemHolder2 is ItemHolder<String>
        addAllItems(listOf(s))
    }
    val itemHolder3 = itemHolderBuilder { // Type of itemHolder3 is ItemHolder<String?
>
        val lastItem: String? = getLastItem()
        // ...
    }
}
```

Supported features

Builder inference supports:

- Inferring several type arguments

  ```
  fun <K, V> myBuilder(builder: MutableMap<K, V>.() -> Unit): Map<K, V> { ... }
  ```

- Inferring type arguments of several builder lambdas within one call including interdependent ones

  ```
  fun <K, V> myBuilder(
      listBuilder: MutableList<V>.() -> Unit,
      mapBuilder: MutableMap<K, V>.() -> Unit
  ): Pair<List<V>, Map<K, V>> =
      mutableListOf<V>().apply(listBuilder) to mutableMapOf<K, V>().apply(mapBuilder)

  fun main() {
      val result = myBuilder(
          { add(1) },

      )
  ```

```
    // result has Pair<List<Int>, Map<String, Int>> type
}
```

- Inferring type arguments whose type parameters are lambda's parameter or return types

```
fun <K, V> myBuilder1(
    mapBuilder: MutableMap<K, V>.() -> K
): Map<K, V> = mutableMapOf<K, V>().apply { mapBuilder() }

fun <K, V> myBuilder2(
    mapBuilder: MutableMap<K, V>.(K) -> Unit
): Map<K, V> = mutableMapOf<K, V>().apply { mapBuilder(2 as K) }

fun main() {
    // result1 has the Map<Long, String> type inferred
    val result1 = myBuilder1 {
        put(1L, "value")
        2
    }
    val result2 = myBuilder2 {
        put(1, "value 1")
        // You can use `it` as "postponed type variable" type
        // See the details in the section below
        put(it, "value 2")
    }
}
```

How builder inference works

Postponed type variables

Builder inference works in terms of *postponed type variables*, which appear inside the builder lambda during builder inference analysis. A postponed type variable is a type argument's type, which is in the process of inferring. The compiler uses it to collect type information about the type argument.

Consider the example with `buildList()`:

```
val result = buildList {
    val x = get(0)
}
```

Here `x` has a type of postponed type variable: the `get()` call returns a value of type `E`, but `E` itself is not yet fixed. At this moment, a concrete type for `E` is unknown.

When a value of a postponed type variable gets associated with a concrete type, builder inference collects this information to infer the resulting type of the corresponding type argument at the end of the builder inference analysis. For example:

```
val result = buildList {
    val x = get(0)
    val y: String = x
} // result has the List<String> type inferred
```

After the postponed type variable gets assigned to a variable of the `String` type, builder inference gets the information that `x` is a subtype of `String`. This assignment is the last statement in the builder lambda, so the builder inference analysis ends with the result of inferring the type argument `E` into `String`.

Note that you can always call `equals()`, `hashCode()`, and `toString()` functions with a postponed type variable as a receiver.

Contributing to builder inference results

Builder inference can collect different varieties of type information that contribute to the analysis result. It considers:

- Calling methods on a lambda's receiver that use the type parameter's type

  ```
  val result = buildList {
      // Type argument is inferred into String based on the passed "value" argument
      add("value")
  } // result has the List<String> type inferred
  ```

- Specifying the expected type for calls that return the type parameter's type

  ```
  val result = buildList {
      // Type argument is inferred into Float based on the expected type
      val x: Float = get(0)
  } // result has the List<Float> type
  ```

  ```
  class Foo<T> {
      val items = mutableListOf<T>()
  }

  fun <K> myBuilder(builder: Foo<K>.() -> Unit): Foo<K> = Foo<K>().apply(builder)

  fun main() {
      val result = myBuilder {
          val x: List<CharSequence> = items
          // ...
      } // result has the Foo<CharSequence> type
  }
  ```

- Passing postponed type variables' types into methods that expect concrete types

  ```
  fun takeMyLong(x: Long) { ... }

  fun String.isMoreThan3() = length > 3
  ```

```
fun takeListOfStrings(x: List<String>) { ... }

fun main() {
    val result1 = buildList {
        val x = get(0)
        takeMyLong(x)
    } // result1 has the List<Long> type

    val result2 = buildList {
        val x = get(0)
        val isLong = x.isMoreThan3()
    // ...
    } // result2 has the List<String> type

    val result3 = buildList {
        takeListOfStrings(this)
    } // result3 has the List<String> type
}
```

- Taking a callable reference to the lambda receiver's member

```
fun main() {
    val result = buildList {
        val x: KFunction1<Int, Float> = ::get
    } // result has the List<Float> type
}
```

```
fun takeFunction(x: KFunction1<Int, Float>) { ... }

fun main() {
    val result = buildList {
        takeFunction(::get)
    } // result has the List<Float> type
}
```

At the end of the analysis, builder inference considers all collected type information and tries to merge it into the resulting type. See the example.

```
val result = buildList { // Inferring postponed type variable E
    // Considering E is Number or a subtype of Number
    val n: Number? = getOrNull(0)
    // Considering E is Int or a supertype of Int
    add(1)
    // E gets inferred into Int
} // result has the List<Int> type
```

The resulting type is the most specific type that corresponds to the type information collected during the analysis. If the given type information is contradictory and cannot be merged, the compiler reports an error.

Note that the Kotlin compiler uses builder inference only if regular type inference cannot infer a type argument. This means you can contribute type information outside a builder lambda, and

then builder inference analysis is not required. Consider the example:

```
fun someMap() = mutableMapOf<CharSequence, String>()

fun <E> MutableMap<E, String>.f(x: MutableMap<E, String>) { ... }

fun main() {
    val x: Map<in String, String> = buildMap {
        put("", "")
        f(someMap()) // Type mismatch (required String, found CharSequence)
    }
}
```

Here a type mismatch appears because the expected type of the map is specified outside the builder lambda. The compiler analyzes all the statements inside with the fixed receiver type `Map<in String, String>`.

Context parameters

> **Tip:** Context parameters replace an older experimental feature called context receivers. You can find their main differences in the design document for context parameters. To migrate from context receivers to context parameters, you can use assisted support in IntelliJ IDEA, as described in the related blog post.

Context parameters allow functions and properties to declare dependencies that are implicitly available in the surrounding context.

With context parameters, you don't need to manually pass around values, such as services or dependencies, that are shared and rarely change across sets of function calls.

To declare context parameters for properties and functions, use the `context` keyword followed by a list of parameters, with each parameter declared as `name: Type`. Here is an example with a dependency on the `UserService` interface:

```
// UserService defines the dependency required in context
interface UserService {
    fun log(message: String)
    fun findUserById(id: Int): String
}

// Declares a function with a context parameter
context(users: UserService)
fun outputMessage(message: String) {
    // Uses log from the context
    users.log("Log: $message")
}

// Declares a property with a context parameter
context(users: UserService)
val firstUser: String
    // Uses findUserById from the context
    get() = users.findUserById(1)

fun main() {
    val users = object : UserService {
        override fun log(message: String) {
            println(message)
        }

        override fun findUserById(id: Int): String {
            return "User $id"
        }
    }

    context(users) {
        outputMessage("Looking up the first user")
        println(firstUser)
```

```
        // User 1
    }
}
```

You can use `_` as a context parameter name when you don't need to refer to the parameter directly. An anonymous context parameter can still satisfy the context parameters required by called functions, but you can't access it by name. To access its value explicitly, use `contextOf<T>()`:

```
// Uses "_" as context parameter name
context(_: UserService)
fun logWelcome() {
    // The anonymous parameter satisfies the UserService context parameter
    // required by outputMessage()
    outputMessage("Welcome!")

    // Retrieves the UserService value explicitly
    contextOf<UserService>().log("Hi!")
}
```

Context parameters resolution

Kotlin resolves context parameters at the call site by searching for matching context values in the current scope. Kotlin matches them by their type. If multiple compatible values exist at the same scope level, the compiler reports an ambiguity:

```
// UserService defines the dependency required in context
interface UserService {
    fun log(message: String)
}

// Declares a function with a context parameter
context(users: UserService)
fun outputMessage(message: String) {
    users.log("Log: $message")
}

fun main() {
    // Implements UserService
    val serviceA = object : UserService {
        override fun log(message: String) = println("A: $message")
    }

    // Implements UserService
    val serviceB = object : UserService {
        override fun log(message: String) = println("B: $message")
    }

    // Both serviceA and serviceB match the expected UserService type at the call site
    context(serviceA, serviceB) {
        // This results in an ambiguity error
        outputMessage("This will not compile")
```

```
    }
}
```

Pass context arguments explicitly

When overloads differ only by context parameters, a call can become ambiguous if multiple matching context values are available.

To resolve the ambiguity, pass an explicit context argument at the call site:

```
class EmailSender
class SmsSender

context(emailSender: EmailSender)
fun sendNotification() {
    println("Sent email notification")
}

context(smsSender: SmsSender)
fun sendNotification() {
    println("Sent SMS notification")
}

context(defaultEmailSender: EmailSender, defaultSmsSender: SmsSender)
fun notifyUser() {
    // Selects the overload with the EmailSender context parameter
    sendNotification(emailSender = defaultEmailSender)

    // Selects the overload with the SmsSender context parameter
    sendNotification(smsSender = defaultSmsSender)
}
```

You can also use explicit context arguments to reduce nesting in some function calls:

- For a single call, use explicit context arguments to make the call easier to read.
- If multiple calls use the same context arguments, use the `context()` function.

This feature is Experimental. To opt in, add the following compiler option to your build file:

Gradle

```
kotlin {
    compilerOptions {
        freeCompilerArgs.add("-Xexplicit-context-arguments")
    }
}
```

Maven

```
<build>
    <plugins>
```

```
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <configuration>
                <args>
                    <arg>-Xexplicit-context-arguments</arg>
                </args>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Restrictions

Context parameters are in continuous improvement, and some of the current restrictions include:

- Constructors can't declare context parameters.
- Properties with context parameters can't have backing fields or initializers.
- Properties with context parameters can't use delegation.

Despite these restrictions, context parameters simplify managing dependencies through simplified dependency injection, improved DSL design, and scoped operations.

Inline functions

Using higher-order functions imposes certain runtime penalties: each function is an object, and it captures a closure. A closure is a scope of variables that can be accessed in the body of the function. Memory allocations (both for function objects and classes) and virtual calls introduce runtime overhead.

But it appears that in many cases this kind of overhead can be eliminated by inlining the lambda expressions. The functions shown below are good examples of this situation. The `lock()` function could be easily inlined at call-sites. Consider the following case:

```
lock(l) { foo() }
```

Instead of creating a function object for the parameter and generating a call, the compiler could emit the following code:

```
l.lock()
try {
    foo()
} finally {
    l.unlock()
}
```

To make the compiler do this, mark the `lock()` function with the `inline` modifier:

```
inline fun <T> lock(lock: Lock, body: () -> T): T { ... }
```

The `inline` modifier affects both the function itself and the lambdas passed to it: all of those will be inlined into the call site.

Inlining may cause the generated code to grow. However, if you do it in a reasonable way (avoiding inlining large functions), it will pay off in performance, especially at "megamorphic" call-sites inside loops.

noinline

If you don't want all of the lambdas passed to an inline function to be inlined, mark some of your function parameters with the `noinline` modifier:

```
inline fun foo(inlined: () -> Unit, noinline notInlined: () -> Unit) { ... }
```

Inlinable lambdas can only be called inside inline functions or passed as inlinable arguments. `noinline` lambdas, however, can be manipulated in any way you like, including being stored in

fields or passed around.

> **Note:** If an inline function has no inlinable function parameters and no reified type parameters, the compiler will issue a warning, since inlining such functions is very unlikely to be beneficial (you can use the `@Suppress("NOTHING_TO_INLINE")` annotation to suppress the warning if you are sure the inlining is needed).

Non-local jump expressions

Returns

In Kotlin, you can only use a normal, unqualified `return` to exit a named function or an anonymous function. To exit a lambda, use a label. A bare `return` is forbidden inside a lambda because a lambda cannot make the enclosing function `return`:

```
fun ordinaryFunction(block: () -> Unit) {
    println("hi!")
}
fun foo() {
    ordinaryFunction {
        return // ERROR: cannot make `foo` return here
    }
}
fun main() {
    foo()
}
```

But if the function the lambda is passed to is inlined, the return can be inlined, as well. So it is allowed:

```
inline fun inlined(block: () -> Unit) {
    println("hi!")
}
fun foo() {
    inlined {
        return // OK: the lambda is inlined
    }
}
fun main() {
    foo()
}
```

Such returns (located in a lambda, but exiting the enclosing function) are called *non-local* returns. This sort of construct usually occurs in loops, which inline functions often enclose:

```
fun hasZeros(ints: List<Int>): Boolean {
    ints.forEach {
        if (it == 0) return true // returns from hasZeros
    }
```

```
    return false
}
```

Note that some inline functions may call the lambdas passed to them as parameters not directly from the function body, but from another execution context, such as a local object or a nested function. In such cases, non-local control flow is also not allowed in the lambdas. To indicate that the lambda parameter of the inline function cannot use non-local returns, mark the lambda parameter with the `crossinline` modifier:

```
inline fun f(crossinline body: () -> Unit) {
    val f = object: Runnable {
        override fun run() = body()
    }
    // ...
}
```

Break and continue

Similar to non-local `return`, you can apply `break` and `continue` jump expressions in lambdas passed as arguments to an inline function that encloses a loop:

```
fun processList(elements: List<Int>): Boolean {
    for (element in elements) {
        val variable = element.nullableMethod() ?: run {
            log.warning("Element is null or invalid, continuing...")
            continue
        }
        if (variable == 0) return true
    }
    return false
}
```

Reified type parameters

Sometimes you need to access a type passed as a parameter:

```
fun <T> TreeNode.findParentOfType(clazz: Class<T>): T? {
    var p = parent
    while (p != null && !clazz.isInstance(p)) {
        p = p.parent
    }
    @Suppress("UNCHECKED_CAST")
    return p as T?
}
```

Here, you walk up a tree and use reflection to check whether a node has a certain type. It's all fine, but the call site is not very pretty:

```
treeNode.findParentOfType(MyTreeNode::class.java)
```

A better solution would be to simply pass a type to this function. You can call it as follows:

```
treeNode.findParentOfType<MyTreeNode>()
```

To enable this, inline functions support *reified type parameters*, so you can write something like this:

```
inline fun <reified T> TreeNode.findParentOfType(): T? {
    var p = parent
    while (p != null && p !is T) {
        p = p.parent
    }
    return p as T?
}
```

The code above qualifies the type parameter with the `reified` modifier to make it accessible inside the function, almost as if it were a normal class. Since the function is inlined, no reflection is needed and normal operators like `!is` and `as` are now available for you to use. Also, you can call the function as shown above: `myTree.findParentOfType<MyTreeNodeType>()`.

Though reflection may not be needed in many cases, you can still use it with a reified type parameter:

```
inline fun <reified T> membersOf() = T::class.members

fun main(s: Array<String>) {
    println(membersOf<StringBuilder>().joinToString("\n"))
}
```

Normal functions (not marked as inline) cannot have reified parameters. A type that does not have a run-time representation (for example, a non-reified type parameter or a fictitious type like `Nothing`) cannot be used as an argument for a reified type parameter.

Inline properties

The `inline` modifier can be used on accessors of properties that don't have backing fields. You can annotate individual property accessors:

```
val foo: Foo
    inline get() = Foo()

var bar: Bar
    get() = ...
    inline set(v) { ... }
```

You can also annotate an entire property, which marks both of its accessors as `inline`:

```
inline var bar: Bar
    get() = ...
    set(v) { ... }
```

At the call site, inline accessors are inlined as regular inline functions.

Restrictions for public API inline functions

When an inline function is `public` or `protected` but is not a part of a `private` or `internal` declaration, it is considered a module's public API. It can be called in other modules and is inlined at such call sites as well.

This imposes certain risks of binary incompatibility caused by changes in the module that declares an inline function in case the calling module is not re-compiled after the change.

To eliminate the risk of such incompatibility being introduced by a change in a *non*-public API of a module, public API inline functions are not allowed to use non-public-API declarations, i.e. `private` and `internal` declarations and their parts, in their bodies.

An `internal` declaration can be annotated with `@PublishedApi`, which allows its use in public API inline functions. When an `internal` inline function is marked as `@PublishedApi`, its body is checked too, as if it were public.

Operator overloading

Kotlin allows you to provide custom implementations for the predefined set of operators on types. These operators have predefined symbolic representation (like `+` or `*`) and precedence. To implement an operator, provide a member function or an extension function with a specific name for the corresponding type. This type becomes the left-hand side type for binary operations and the argument type for the unary ones.

To overload an operator, mark the corresponding function with the `operator` modifier:

```
interface IndexedContainer {
    operator fun get(index: Int)
}
```

When overriding your operator overloads, you can omit `operator`:

```
class OrdersList: IndexedContainer {
    override fun get(index: Int) { /*...*/ }
}
```

Unary operations

Unary prefix operators

Expression	Translated to
`+a`	`a.unaryPlus()`
`-a`	`a.unaryMinus()`
`!a`	`a.not()`

This table says that when the compiler processes, for example, an expression `+a`, it performs the following steps:

- Determines the type of `a`, let it be `T`.
- Looks up a function `unaryPlus()` with the `operator` modifier and no parameters for the receiver `T`, that means a member function or an extension function.
- If the function is absent or ambiguous, it is a compilation error.
- If the function is present and its return type is `R`, the expression `+a` has type `R`.

  Note: These operations, as well as all the others, are optimized for basic types and do not introduce overhead of function calls for them.

As an example, here's how you can overload the unary minus operator:

```
data class Point(val x: Int, val y: Int)

operator fun Point.unaryMinus() = Point(-x, -y)

val point = Point(10, 20)

fun main() {
   println(-point)  // prints "Point(x=-10, y=-20)"
}
```

Increments and decrements

Expression	Translated to
`a++`	`a.inc()` + see below
`a--`	`a.dec()` + see below

The `inc()` and `dec()` functions must return a value, which will be assigned to the variable on which the `++` or `--` operation was used. They shouldn't mutate the object on which the `inc` or `dec` was invoked.

The compiler performs the following steps for resolution of an operator in the *postfix* form, for example `a++`:

- Determines the type of `a`, let it be `T`.
- Looks up a function `inc()` with the `operator` modifier and no parameters, applicable to the receiver of type `T`.
- Checks that the return type of the function is a subtype of `T`.

The effect of computing the expression is:

- Store the initial value of `a` to a temporary storage `a0`.
- Assign the result of `a0.inc()` to `a`.
- Return `a0` as the result of the expression.

For `a--` the steps are completely analogous.

For the *prefix* forms `++a` and `--a` resolution works the same way, and the effect is:

- Assign the result of `a.inc()` to `a`.
- Return the new value of `a` as a result of the expression.

Binary operations

Arithmetic operators

Expression	Translated to
`a + b`	`a.plus(b)`
`a - b`	`a.minus(b)`
`a * b`	`a.times(b)`
`a / b`	`a.div(b)`
`a % b`	`a.rem(b)`
`a..b`	`a.rangeTo(b)`
`a..<b`	`a.rangeUntil(b)`

For the operations in this table, the compiler just resolves the expression in the *Translated to* column.

Below is an example `Counter` class that starts at a given value and can be incremented using the overloaded `+` operator:

```
data class Counter(val dayIndex: Int) {
    operator fun plus(increment: Int): Counter {
        return Counter(dayIndex + increment)
    }
}
```

in operator

Expression	Translated to
`a in b`	`b.contains(a)`
`a !in b`	`!b.contains(a)`

For `in` and `!in` the procedure is the same, but the order of arguments is reversed.

Indexed access operator

Expression	Translated to
`a[i]`	`a.get(i)`
`a[i, j]`	`a.get(i, j)`
`a[i_1, ..., i_n]`	`a.get(i_1, ..., i_n)`

Expression	Translated to
`a[i] = b`	`a.set(i, b)`
`a[i, j] = b`	`a.set(i, j, b)`
`a[i_1, ..., i_n] = b`	`a.set(i_1, ..., i_n, b)`

Square brackets are translated to calls to `get` and `set` with appropriate numbers of arguments.

invoke operator

Expression	Translated to
`a()`	`a.invoke()`
`a(i)`	`a.invoke(i)`
`a(i, j)`	`a.invoke(i, j)`
`a(i_1, ..., i_n)`	`a.invoke(i_1, ..., i_n)`

Parentheses are translated to calls to `invoke` with appropriate number of arguments.

Augmented assignments

Expression	Translated to
`a += b`	`a.plusAssign(b)`
`a -= b`	`a.minusAssign(b)`
`a *= b`	`a.timesAssign(b)`
`a /= b`	`a.divAssign(b)`
`a %= b`	`a.remAssign(b)`

For the assignment operations, for example `a += b`, the compiler performs the following steps:

- If the function from the right column is available:
  - If the corresponding binary function (that means `plus()` for `plusAssign()`) is available too, `a` is a mutable variable, and the return type of `plus` is a subtype of the type of `a`, report an error (ambiguity).
  - Make sure its return type is `Unit`, and report an error otherwise.
  - Generate code for `a.plusAssign(b)`.
- Otherwise, try to generate code for `a = a + b` (this includes a type check: the type of `a + b` must be a subtype of `a`).

  Note: Assignments are *NOT* expressions in Kotlin.

Equality and inequality operators

Expression	Translated to
`a == b`	`a?.equals(b) ?: (b === null)`
`a != b`	`!(a?.equals(b) ?: (b === null))`

These operators only work with the function `equals(other: Any?): Boolean`, which you can override to provide a custom equality check implementation. Any other function with the same name (like `equals(other: Foo)`) is ignored.

Kotlin calls `.equals()` when neither operand compares directly to `null` in the `==` expression and the comparison isn't between two floating-point types. Otherwise, Kotlin uses `===` for direct `null` comparisons and compares non-null floating-point values by numeric value.

> **Note:** `===` and `!==` (identity checks) aren't overloadable, so no conventions exist for them.

Comparison operators

Expression	Translated to
`a > b`	`a.compareTo(b) > 0`
`a < b`	`a.compareTo(b) < 0`
`a >= b`	`a.compareTo(b) >= 0`
`a <= b`	`a.compareTo(b) <= 0`

All comparisons are translated into calls to `compareTo`, that is required to return `Int`.

Property delegation operators

`provideDelegate`, `getValue` and `setValue` operator functions are described in Delegated properties.

Infix calls for named functions

You can simulate custom infix operations by using infix function calls.

Unused return value checker

> **Note:** This feature is planned to be stabilized and improved in future Kotlin releases. We would appreciate your feedback in our issue tracker YouTrack.
>
> For more information, see the related KEEP proposal.

The unused return value checker allows you to detect *ignored results*. These are values returned from expressions that produce something other than `Unit`, `Nothing`, or `Nothing?` and aren't:

- Stored in a variable or property.
- Returned or thrown.
- Passed as an argument to another function.
- Used as a receiver in a call or safe call.
- Checked in a condition such as `if`, `when`, or `while`.
- Used as the last statement of a lambda.

The checker doesn't report ignored results for increment operations like `++` and `--`, or for boolean shortcuts where the right-hand side exits the current function, for example, `condition || return`.

You can use the unused return value checker to catch bugs where a function call produces a meaningful result, but the result is silently dropped. This helps prevent unexpected behavior and makes such issues easier to track down.

Here's an example where a string is created but never used, so the checker reports it as an ignored result:

```
fun formatGreeting(name: String): String {
    if (name.isBlank()) return "Hello, anonymous user!"
    if (!name.contains(' ')) {
        // The checker reports a warning that this result is ignored:
        // "Unused return value of 'plus'."
        "Hello, " + name.replaceFirstChar(Char::titlecase) + "!"
    }
    val (first, last) = name.split(' ')
    return "Hello, $first! Or should I call you Dr. $last?"
}
```

Configure the unused return value checker

You can control how the compiler reports ignored results with the `-Xreturn-value-checker` compiler option.

It has the following modes:

- `disable` disables the unused return value checker (default).
- `check` enables the checker, and reports warnings for ignored results from marked functions.
- `full` enables the checker, treats all functions in your project as marked, and reports warnings for ignored results.

  Note: All marked functions are propagated as such, and ignored results are reported if the checker is enabled in projects that use your code as a dependency.

To use the unused return value checker in your project, add the compiler option to your build configuration file:

```
// build.gradle(.kts)
kotlin {
    compilerOptions {
        freeCompilerArgs.add("-Xreturn-value-checker=check")
    }
}
```

```
<plugin>
    <groupId>org.jetbrains.kotlin</groupId>
    ..
    <configuration>
        <args>
            <arg>-Xreturn-value-checker=check</arg>
        </args>
    </configuration>
</plugin>
```

Mark functions to check ignored results

When you set the `-Xreturn-value-checker` compiler option to `check`, the checker reports ignored results only from expressions that are marked, like most functions in the Kotlin standard library.

To mark your own code, use the `@MustUseReturnValues` annotation. You can apply it to a file, class, or function depending on the scope you want the checker to cover.

For example, you can mark an entire file:

```
// Marks all functions and classes in this file so the checker reports unused return values
@file:MustUseReturnValues

package my.project

fun someFunction(): String
```

Or a specific class:

```
// Marks all functions in this class so the checker reports unused return values
@MustUseReturnValues
class Greeter {
    fun greet(name: String): String = "Hello, $name"
}

fun someFunction(): Int = ...
```

> **Note:** You can apply the checker to your entire project by setting the `-Xreturn-value-checker` compiler option to `full`. With this option, you don't have to annotate your code with `@MustUseReturnValues`.

Suppress reports for ignored results

You can suppress reports on specific functions by annotating them with `@IgnorableReturnValue`. Annotate functions where ignoring the result is common and expected, such as `MutableList.add`:

```
@IgnorableReturnValue
fun <T> MutableList<T>.addAndIgnoreResult(element: T): Boolean {
    return add(element)
}
```

You can suppress a warning without annotating the function itself. To do this, assign the result to a special unnamed variable with an underscore syntax (`_`):

```
// Non-ignorable function
fun computeValue(): Int = 42

fun main() {

    // Reports a warning: result is ignored
    computeValue()

    // Suppresses the warning only at this call site with a special unused variable
    val _ = computeValue()
}
```

Ignored results in function overrides

When you override a function, the override inherits the reporting rules defined by the annotations on the base declaration. This also applies when the base declaration is part of the Kotlin standard library or of other library dependencies, so the checker reports ignored results for overrides of functions like `Any.hashCode()`.

Additionally, you can't override a function marked with `@IgnorableReturnValue` with another function that requires its return value to be used. However, you can mark an override with

`@IgnorableReturnValue` in a class or interface annotated with `@MustUseReturnValues` when its result can be safely ignored:

```
@MustUseReturnValues
interface Greeter {
    fun greet(name: String): String
}

object SilentGreeter : Greeter {
    @IgnorableReturnValue
    override fun greet(name: String): String = ""
}

fun check(g: Greeter) {
    // Reports a warning: unused return value
    g.greet("John")

    // No warning
    SilentGreeter.greet("John")
}
```

Check for unused results in higher-order functions

Some higher-order functions, such as the `let` scope function, return the result of a lambda. To check for unused lambda results of higher-order functions, add the Experimental `returnsResultOf()` contract to the function's contract.

> **Warning:** Kotlin contracts are Experimental. To opt in, add the `@OptIn(ExperimentalContracts::class)` annotation when declaring a function with a contract.

Here's an example:

```
import kotlin.contracts.ExperimentalContracts
import kotlin.contracts.contract

@OptIn(ExperimentalContracts::class)
inline fun <T, R> T.customLet(block: (T) -> R): R {
    contract {
        returnsResultOf(block)
    }
    return block(this)
}
```

You can then use a function with this contract, such as `.customLet()`, to check if the lambda result is used:

```
fun handleNullablePackageName(packageName: String?, builder: StringBuilder) {
    // The checker doesn't report a warning because the return value of append() can be igno
```

```
red
    packageName?.customLet { builder.append(it) }

    // The checker reports a warning because the returned string is unused
    packageName?.customLet { "kotlin.$it" }
}
```

> **Warning:** The `returnsResultOf()` contract requires a separate compiler option to opt in. Be aware that using it produces pre-release binaries that Kotlin compiler versions earlier than 2.4.0 can't read.

To opt in for your project, add the following compiler option to your build file:

Gradle

```
// build.gradle(.kts)
kotlin {
    compilerOptions {
        freeCompilerArgs.add("-Xallow-returns-result-of")
    }
}
```

Maven

```
<build>
    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <configuration>
                <args>
                    <arg>-Xallow-returns-result-of</arg>
                </args>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Interoperability with Java annotations

Some Java libraries use similar mechanisms with different annotations. The unused return value checker treats the following annotations as equivalent to using `@MustUseReturnValues`:

- `com.google.errorprone.annotations.CheckReturnValue`
- `edu.umd.cs.findbugs.annotations.CheckReturnValue`
- `org.jetbrains.annotations.CheckReturnValue`
- `org.springframework.lang.CheckReturnValue`
- `org.jooq.CheckReturnValue`

It also treats `com.google.errorprone.annotations.CanIgnoreReturnValue` as equivalent to using `@IgnorableReturnValue`.

Classes

> **Tip:** Before creating classes, consider using a data class if the purpose is to store data. Alternatively, think about extending an existing class with an extension, rather than creating a new one from scratch.

Like other object-oriented languages, Kotlin uses *classes* to encapsulate data (properties) and behavior (functions) for reusable, structured code.

Classes are blueprints or templates for objects, which you create via constructors. When you create an instance of a class, you are creating a concrete object based on that blueprint.

Kotlin offers concise syntax for declaring classes. To declare a class, use the `class` keyword followed by the class name:

```
class Person { /*...*/ }
```

The class declaration consists of:

- **Class header**, including but not limited to:
  - `class` keyword
  - Class name
  - Type parameters (if any)
  - Primary constructor (optional)
- **Class body** (optional), surrounded by curly braces `{}`, and including **class members** such as:
  - Secondary constructors
  - Initializer blocks
  - Functions
  - Properties
  - Nested and inner classes
  - Object declarations

You can keep both the class header and body to a bare minimum. If the class doesn't have a body, you can omit the curly braces `{}`:

```
// Class with primary constructor, but without a body
class Person(val name: String, var age: Int)
```

Here's an example that declares a class with a header and body, then creates an instance from it:

```
// Person class with a primary constructor
// that initializes the name property
class Person(val name: String) {
    // Class body with age property
```

```
    var age: Int = 0
}

fun main() {
    // Creates an instance of the Person class by calling the constructor
    val person = Person("Alice")

    // Accesses the instance's properties
    println(person.name)
    // Alice
    println(person.age)
    // 0
}
```

Creating instances

An instance is created when you use the class as a blueprint to build a real object to work with in your program.

To create an instance of a class, use the class name followed by parentheses `()`, similar to calling a function:

```
// Creates an instance of the Person class
val anonymousUser = Person()
```

In Kotlin, you can create instances:

- **Without arguments** (`Person()`): creates an instance using the default values, if they are declared in the class.
- **With arguments** (`Person(value)`): creates an instance by passing specific values.

You can assign the created instance to a mutable (`var`) or read-only (`val`) variable:

```
// Creates an instance using the default value
// and assigns it to a mutable variable
var anonymousUser = Person()

// Creates an instance by passing a specific value
// and assigns it to a read-only variable
val namedUser = Person("Joe")
```

It's possible to create instances wherever you need them, inside the `main()` function, within other functions, or inside another class. Additionally, you can create instances inside another function and call that function from `main()`.

The following code declares a `Person` class with a property for storing a name. It also demonstrates how to create an instance with both the default constructor's value and a specific value:

```
// Class header with a primary constructor
// that initializes name with a default value
class Person(val name: String = "Sebastian")

fun main() {
    // Creates an instance using the default constructor's value
    val anonymousUser = Person()

    // Creates an instance by passing a specific value
    val namedUser = Person("Joe")

    // Accesses the instances' name property
    println(anonymousUser.name)
    // Sebastian
    println(namedUser.name)
    // Joe
}
```

> **Note:** In Kotlin, unlike other object-oriented programming languages, there is no need for the `new` keyword when creating class instances.

For information about creating instances of nested, inner, and anonymous inner classes, see the Nested classes section.

Constructors and initializer blocks

When you create a class instance, you call one of its constructors. A class in Kotlin can have a *primary constructor* and one or more *secondary constructors*.

The primary constructor is the main way to initialize a class. You declare it in the class header. A secondary constructor provides additional initialization logic. You declare it in the class body.

Both primary and secondary constructors are optional, but a class must have at least one constructor.

Primary constructor

The primary constructor sets up the initial state of an instance when it's created.

To declare a primary constructor, place it in the class header after the class name:

```
class Person constructor(name: String) { /*...*/ }
```

If the primary constructor doesn't have any annotations or visibility modifiers, you can omit the `constructor` keyword:

```
class Person(name: String) { /*...*/ }
```

The primary constructor can declare parameters as properties. Use the `val` keyword before the argument name to declare a read-only property and the `var` keyword for a mutable property:

```
class Person(val name: String, var age: Int) { /*...*/ }
```

These constructor parameter properties are stored as part of the instance and are accessible from outside the class.

It's also possible to declare primary constructor parameters that are not properties. These parameters don't have `val` or `var` in front of them, so they are not stored in the instance and are available only within the class body:

```
// Primary constructor parameter that is also a property
class PersonWithProperty(val name: String) {
    fun greet() {
        println("Hello, $name")
    }
}

// Primary constructor parameter only (not stored as a property)
class PersonWithAssignment(name: String) {
    // Must be assigned to a property to be usable later
    val displayName: String = name

    fun greet() {
        println("Hello, $displayName")
    }
}
```

Properties declared in the primary constructor are accessible by member functions of the class:

```
// Class with a primary constructor declaring properties
class Person(val name: String, var age: Int) {
    // Member function accessing class properties
    fun introduce(): String {
        return "Hi, I'm $name and I'm $age years old."
    }
}
```

You can also assign default values to properties in the primary constructor:

```
class Person(val name: String = "John", var age: Int = 30) { /*...*/ }
```

If no value is passed to the constructor during instance creation, properties use their default value:

```
// Class with a primary constructor
// including default values for name and age
class Person(val name: String = "John", var age: Int = 30)

fun main() {
```

```
    // Creates an instance using default values
    val person = Person()
    println("Name: ${person.name}, Age: ${person.age}")
    // Name: John, Age: 30
}
```

You can use the primary constructor parameters to initialize additional class properties directly in the class body:

```
// Class with a primary constructor
// including default values for name and age
class Person(
    val name: String = "John",
    var age: Int = 30
) {
    // Initializes the description property
    // from the primary constructor parameters
    val description: String = "Name: $name, Age: $age"
}

fun main() {
    // Creates an instance of the Person class
    val person = Person()
    // Accesses the description property
    println(person.description)
    // Name: John, Age: 30
}
```

As with functions, you can use trailing commas in constructor declarations:

```
class Person(
    val name: String,
    val lastName: String,
    var age: Int,
) { /*...*/ }
```

Initializer blocks

The primary constructor initializes the class and sets its properties. In most cases, you can handle this with simple code.

If you need to perform more complex operations during instance creation, place that logic in *initializer blocks* inside the class body. These blocks run when the primary constructor executes.

Declare initializer blocks with the `init` keyword followed by curly braces `{}`. Write within the curly braces any code that you want to run during initialization:

```
// Class with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {
    init {
        // Initializer block runs when an instance is created
```

```
        println("Person created: $name, age $age.")
    }
}

fun main() {
    // Creates an instance of the Person class
    Person("John", 30)
    // Person created: John, age 30.
}
```

Add as many initializer blocks (`init {}`) as you need. They run in the order in which they appear in the class body, along with property initializers:

```
// Class with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {
    // First initializer block
    init {
        // Runs first when an instance is created
        println("Person created: $name, age $age.")
    }

    // Second initializer block
    init {
        // Runs after the first initializer block
        if (age < 18) {
            println("$name is a minor.")
        } else {
            println("$name is an adult.")
        }
    }
}

fun main() {
    // Creates an instance of the Person class
    Person("John", 30)
    // Person created: John, age 30.
    // John is an adult.
}
```

You can use primary constructor parameters in initializer blocks. For example, in the code above, the first and second initializers use the `name` and `age` parameters from the primary constructor.

A common use case for `init` blocks is data validation. For example, by calling the `require` function:

```
class Person(val age: Int) {
    init {
        require(age > 0) { "age must be positive" }
    }
}
```

Secondary constructors

In Kotlin, secondary constructors are additional constructors that a class can have beyond its primary constructor. Secondary constructors are useful when you need multiple ways to initialize a class or for Java interoperability.

To declare a secondary constructor, use the `constructor` keyword inside the class body with the constructor parameters within parentheses `()`. Add the constructor logic within curly braces `{}`:

```
// Class header with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {

    // Secondary constructor that takes age as a
    // String and converts it to an Int
    constructor(name: String, age: String) : this(name, age.toIntOrNull() ?: 0) {
        println("$name created with converted age: ${this.age}")
    }
}

fun main() {
    // Uses the secondary constructor with age as a String
    Person("Bob", "8")
    // Bob created with converted age: 8
}
```

> **Tip:** The expression `age.toIntOrNull() ?: 0` uses the Elvis operator. For more information, see Null safety.

In the code above, the secondary constructor delegates to the primary constructor via the `this` keyword, passing `name` and the `age` value converted to an integer.

In Kotlin, secondary constructors must delegate to the primary constructor. This delegation ensures that all primary constructor initialization logic is executed before any secondary constructor logic runs.

Constructor delegation can be:

- **Direct**, where the secondary constructor calls the primary constructor immediately.
- **Indirect**, where one secondary constructor calls another, which in turn delegates to the primary constructor.

Here's an example demonstrating how direct and indirect delegation works:

```
// Class header with a primary constructor that initializes name and age
class Person(
    val name: String,
    var age: Int
) {
    // Secondary constructor with direct delegation
    // to the primary constructor
```

```
    constructor(name: String) : this(name, 0) {
        println("Person created with default age: $age and name: $name.")
    }

    // Secondary constructor with indirect delegation:
    // this("Bob") -> constructor(name: String) -> primary constructor
    constructor() : this("Bob") {
        println("New person created with default age: $age and name: $name.")
    }
}

fun main() {
    // Creates an instance based on the direct delegation
    Person("Alice")
    // Person created with default age: 0 and name: Alice.

    // Creates an instance based on the indirect delegation
    Person()
    // Person created with default age: 0 and name: Bob.
    // New person created with default age: 0 and name: Bob.
}
```

In classes with initializer blocks (`init {}`), the code within these blocks becomes part of the primary constructor. Given that secondary constructors delegate to the primary constructor first, all initializer blocks and property initializers run before the body of the secondary constructor. Even if the class has no primary constructor, the delegation still happens implicitly:

```
// Class header with no primary constructor
class Person {
    // Initializer block runs when an instance is created
    init {
        // Runs before the secondary constructor
        println("1. First initializer block runs")
    }

    // Secondary constructor that takes an integer parameter
    constructor(i: Int) {
        // Runs after the initializer block
        println("2. Person $i is created")
    }
}

fun main() {
    // Creates an instance of the Person class
    Person(1)
    // 1. First initializer block runs
    // 2. Person 1 created
}
```

Classes without constructors

Classes that don't declare any constructors (primary or secondary) have an implicit primary constructor with no parameters:

```
// Class with no explicit constructors
class Person {
    // No primary or secondary constructors declared
}

fun main() {
    // Creates an instance of the Person class
    // using the implicit primary constructor
    val person = Person()
}
```

The visibility of this implicit primary constructor is public, meaning it can be accessed from anywhere. If you don't want your class to have a public constructor, declare an empty primary constructor with non-default visibility:

```
class Person private constructor() { /*...*/ }
```

> **Note:** On the JVM, if all primary constructor parameters have default values, the compiler implicitly provides a parameterless constructor that uses those default values.
>
> This makes it easier to use Kotlin with libraries such as Jackson or Spring Data JPA, which create class instances through parameterless constructors.
>
> In the following example, Kotlin implicitly provides a parameterless constructor `Person()` that uses the default value `""`:
>
> ```
> class Person(val personName: String = "")
> ```

Inheritance

Class inheritance in Kotlin allows you to create a new class (derived class) from an existing class (base class), inheriting its properties and functions while adding or modifying behavior.

For detailed information about inheritance hierarchies and how to use the `open` keyword, see the Inheritance section.

Abstract classes

In Kotlin, abstract classes are classes that can't be instantiated directly. They are designed to be inherited by other classes which define their actual behavior. This behavior is called an *implementation*.

An abstract class can declare abstract properties and functions, which must be implemented by subclasses.

Abstract classes can also have constructors. These constructors initialize class properties and enforce required parameters for subclasses. Declare an abstract class using the `abstract` keyword:

```
abstract class Person(val name: String, val age: Int)
```

An abstract class can have both abstract and non-abstract members (properties and functions). To declare a member as abstract, you must use the `abstract` keyword explicitly.

You don't need to annotate abstract classes or functions with the `open` keyword because they are implicitly inheritable by default. For more details about the `open` keyword, see Inheritance.

Abstract members don't have an implementation in the abstract class. You define the implementation in a subclass or inheriting class with an `override` function or property:

```
// Abstract class with a primary constructor that declares name and age
abstract class Person(
    val name: String,
    val age: Int
) {
    // Abstract member
    // Doesn't provide implementation,
    // and it must be implemented by subclasses
    abstract fun introduce()

    // Non-abstract member (has an implementation)
    fun greet() {
        println("Hello, my name is $name.")
    }
}

// Subclass that provides an implementation for the abstract member
class Student(
    name: String,
    age: Int,
    val school: String
) : Person(name, age) {
    override fun introduce() {
        println("I am $name, $age years old, and I study at $school.")
    }
}

fun main() {
```

```
    // Creates an instance of the Student class
    val student = Student("Alice", 20, "Engineering University")

    // Calls the non-abstract member
    student.greet()
    // Hello, my name is Alice.

    // Calls the overridden abstract member
    student.introduce()
    // I am Alice, 20 years old, and I study at Engineering University.
}
```

Companion objects

In Kotlin, each class can have a companion object. Companion objects are a type of object declaration that allows you to access its members using the class name without creating a class instance.

Suppose you need to write a function that can be called without creating an instance of a class, but it is still logically connected to the class (such as a factory function). In that case, you can declare it inside a companion object declaration within the class:

```
// Class with a primary constructor that declares the name property
class Person(
    val name: String
) {
    // Class body with a companion object
    companion object {
        fun createAnonymous() = Person("Anonymous")
    }
}

fun main() {
    // Calls the function without creating an instance of the class
    val anonymous = Person.createAnonymous()
    println(anonymous.name)
    // Anonymous
}
```

If you declare a companion object inside your class, you can access its members using only the class name as a qualifier.

For more information, see Companion objects.

Data classes

Data classes in Kotlin are primarily used to hold data. For each data class, the compiler automatically generates additional member functions that allow you to print an instance to readable output, compare instances, copy instances, and more. Data classes are marked with `data`:

```
data class User(val name: String, val age: Int)
```

The compiler automatically derives the following members from all properties declared in the primary constructor:

- `equals()`/`hashCode()` pair.
- `toString()` of the form `"User(name=John, age=42)"`.
- `componentN()` functions corresponding to the properties in their order of declaration.
- `copy()` function.

To ensure consistency and meaningful behavior of the generated code, data classes have to fulfill the following requirements:

- The primary constructor must have at least one parameter.
- All primary constructor parameters must be marked as `val` or `var`.
- Data classes can't be abstract, open, sealed, or inner.

Additionally, the generation of data class members follows these rules with regard to the members' inheritance:

- If there are explicit implementations of `equals()`, `hashCode()`, or `toString()` in the data class body or `final` implementations in a superclass, then these functions are not generated, and the existing implementations are used.
- If a supertype has `componentN()` functions that are `open` and return compatible types, the corresponding functions are generated for the data class and override those of the supertype. If the functions of the supertype cannot be overridden due to incompatible signatures or due to their being final, an error is reported.
- Providing explicit implementations for the `componentN()` and `copy()` functions is not allowed.

Data classes may extend other classes (see Sealed classes for examples).

> **Note:** On the JVM, if the generated class needs to have a parameterless constructor, default values for the properties have to be specified (see Constructors):
>
> ```
> data class User(val name: String = "", val age: Int = 0)
> ```

Properties declared in the class body

The compiler only uses the properties defined inside the primary constructor for the automatically generated functions. To exclude a property from the generated implementations, declare it inside the class body:

```
data class Person(val name: String) {
    var age: Int = 0
}
```

In the example below, only the `name` property is used by default inside the `toString()`, `equals()`, `hashCode()`, and `copy()` implementations, and there is only one component function, `component1()`. The `age` property is declared inside the class body and is excluded. Therefore, two `Person` objects with the same `name` but different `age` values are considered equal since `equals()` only evaluates properties from the primary constructor:

```
data class Person(val name: String) {
    var age: Int = 0
}
fun main() {
    val person1 = Person("John")
    val person2 = Person("John")
    person1.age = 10
    person2.age = 20

    println("person1 == person2: ${person1 == person2}")
    // person1 == person2: true

    println("person1 with age ${person1.age}: ${person1}")
    // person1 with age 10: Person(name=John)

    println("person2 with age ${person2.age}: ${person2}")
    // person2 with age 20: Person(name=John)
}
```

Copying

Use the `copy()` function to copy an object, allowing you to alter *some* of its properties while keeping the rest unchanged. The implementation of this function for the `User` class above would be as follows:

```
fun copy(name: String = this.name, age: Int = this.age) = User(name, age)
```

You can then write the following:

```
val jack = User(name = "Jack", age = 1)
val olderJack = jack.copy(age = 2)
```

The `copy()` function creates a *shallow* copy of the instance. In other words, it doesn't copy components recursively. As a result, references to other objects are shared.

For example, if a property holds a mutable list, changes made through the "original" value are also visible through the copy, and changes made through the copy are visible through the original:

```
data class Employee(val name: String, val roles: MutableList<String>)

fun main() {
    val original = Employee("Jamie", mutableListOf("developer"))
    val duplicate = original.copy()

    duplicate.roles.add("team lead")

    println(original)
    // Employee(name=Jamie, roles=[developer, team lead])
    println(duplicate)
    // Employee(name=Jamie, roles=[developer, team lead])
}
```

As you can see, modifying the `duplicate.roles` property also changes the `original.roles` property because both properties share the same list reference.

Data classes and destructuring declarations

Component functions generated for data classes make it possible to use them in destructuring declarations:

```
val jane = User("Jane", 35)
val (name, age) = jane
println("$name, $age years of age")
// Jane, 35 years of age
```

Standard data classes

The standard library provides the `Pair` and `Triple` classes. In most cases, though, named data classes are a better design choice because they make the code easier to read by providing meaningful names for the properties.

Extensions

Kotlin *extensions* let you extend a class or an interface with new functionality without using inheritance or design patterns like *Decorator*. They are useful when working with third-party libraries you can't modify directly. Once created, you call these extensions as if they were members of the original class or interface.

The most common forms of extensions are *extension functions* and *extension properties*.

Importantly, extensions don't modify the classes or interfaces they extend. When you define an extension, you don't add new members. You make new functions callable or new properties accessible using the same syntax.

Receivers

Extensions are always called on a receiver. The receiver has to have the same type as the class or interface being extended. To use an extension, prefix it with the receiver followed by a `.` and the function or property name.

For example, the `.appendLine()` extension function from the standard library extends the `StringBuilder` class. So in this case, the receiver is a `StringBuilder` instance, and the *receiver type* is `StringBuilder`:

```
fun main() {
    // builder is an instance of StringBuilder
    val builder = StringBuilder()
        // Calls .appendLine() extension function on builder
        .appendLine("Hello")
        .appendLine()
        .appendLine("World")
    println(builder.toString())
    // Hello
    //
    // World
}
```

Extension functions

Before creating your own extension functions, see if what you are looking for is already available in the Kotlin standard library. The standard library provides many useful extension functions for:

- Operating on collections: `.map()`, `.filter()`, `.reduce()`, `.fold()`, `.groupBy()`.
- Converting to strings: `.joinToString()`.
- Working with null values: `.filterNotNull()`.

To create your own extension function, prefix its name with a receiver type followed by a `.`. In this example, the `.truncate()` function extends the `String` class, so the receiver type is `String`:

```
fun String.truncate(maxLength: Int): String {
    return if (this.length <= maxLength) this else take(maxLength - 3) + "..."
}

fun main() {
    val shortUsername = "KotlinFan42"
    val longUsername = "JetBrainsLoverForever"

    println("Short username: ${shortUsername.truncate(15)}")
    // KotlinFan42
    println("Long username:  ${longUsername.truncate(15)}")
    // JetBrainsLov...
}
```

The `.truncate()` function truncates any string that it's called on by the number in the `maxLength` argument and adds an ellipsis `...`. If the string is shorter than `maxLength`, the function returns the original string.

In this example, the `.displayInfo()` function extends the `User` interface:

```
interface User {
    val name: String
    val email: String
}

fun User.displayInfo(): String = "User(name=$name, email=$email)"

// Inherits from and implements the properties of the User interface
class RegularUser(override val name: String, override val email: String) : User

fun main() {
    val user = RegularUser("Alice", "alice@example.com")
    println(user.displayInfo())
    // User(name=Alice, email=alice@example.com)
}
```

The `.displayInfo()` function returns a string containing the `name` and `email` of a `RegularUser` instance. Defining an extension on an interface like this is useful when you want to add functionality to all types that implement an interface only once.

In this example, the `.mostVoted()` function extends the `Map<String, Int>` class:

```
fun Map<String, Int>.mostVoted(): String? {
    return maxByOrNull { (key, value) -> value }?.key
}

fun main() {
    val poll = mapOf(
```

```
        "Cats" to 37,
        "Dogs" to 58,
        "Birds" to 22
    )

    println("Top choice: ${poll.mostVoted()}")
    // Dogs
}
```

The `.mostVoted()` function iterates through the key-value pairs of the map it's called on and uses the `maxByOrNull()` function to return the key of the pair containing the highest value. If the map is empty, the `maxByOrNull()` function returns `null`. The `mostVoted()` function uses a safe call `?.` to only access the `key` property when the `maxByOrNull()` function returns a non-null value.

Generic extension functions

To create generic extension functions, declare the generic type parameter before the function name to make it available in the receiver type expression. In this example, the `.endpoints()` function extends `List<T>` where `T` can be any type:

```
fun <T> List<T>.endpoints(): Pair<T, T> {
    return first() to last()
}

fun main() {
    val cities = listOf("Paris", "London", "Berlin", "Prague")
    val temperatures = listOf(21.0, 19.5, 22.3)

    val cityEndpoints = cities.endpoints()
    val tempEndpoints = temperatures.endpoints()

    println("First and last cities: $cityEndpoints")
    // (Paris, Prague)
    println("First and last temperatures: $tempEndpoints")
    // (21.0, 22.3)
}
```

The `.endpoints()` function returns a pair containing the first and last elements of the list that it's called on. Inside the function body, it calls the `first()` and `last()` functions and combines their returned values into a `Pair` using the `to` infix function.

For more information about generics, see generic functions.

Nullable receivers

You can define extension functions with a nullable receiver type, which allows you to call them on a variable even if its value is null. When the receiver is `null`, `this` is also `null`. Make sure to handle nullability correctly within your functions. For example, use `this == null` checks inside function bodies, safe calls `?.`, or the Elvis operator `?:`.

In this example, you can call the `.toString()` function without checking for `null` because the check already happens inside the extension function:

```
fun main() {
    // Extension function on nullable Any
    fun Any?.toString(): String {
        if (this == null) return "null"
        // After null check, `this` is smart-cast to non-nullable Any
        // So this call resolves to the regular toString() function
        return toString()
    }

    val number: Int? = 42
    val nothing: Any? = null

    println(number.toString())
    // 42
    println(nothing.toString())
    // null
}
```

Extension or member functions?

Since extension and member function calls have the same notation, how does the compiler know which one to use? Extension functions are dispatched *statically*, meaning the compiler determines which function to call based on the receiver type at compile time. For example:

```
fun main() {
    open class Shape
    class Rectangle: Shape()

    fun Shape.getName() = "Shape"
    fun Rectangle.getName() = "Rectangle"

    fun printClassName(shape: Shape) {
        println(shape.getName())
    }

    printClassName(Rectangle())
    // Shape
}
```

In this example, the compiler calls the `Shape.getName()` extension function because the parameter `shape` is declared as type `Shape`. Because extension functions are resolved statically, the compiler chooses the function based on the declared type, not the actual instance.

So even though the example passes a `Rectangle` instance, the `.getName()` function resolves to `Shape.getName()` since the variable is declared as type `Shape`.

If a class has a member function and there's an extension function with the same receiver type, the same name, and compatible arguments, the member function takes precedence. For example:

```
fun main() {
    class Example {
        fun printFunctionType() { println("Member function") }
    }

    fun Example.printFunctionType() { println("Extension function") }

    Example().printFunctionType()
    // Member function
}
```

However, extension functions can overload member functions that have the same name but a *different* signature:

```
fun main() {
    class Example {
        fun printFunctionType() { println("Member function") }
    }

    // Same name but different signature
    fun Example.printFunctionType(index: Int) { println("Extension function #$index") }

    Example().printFunctionType(1)
    // Extension function #1
}
```

In this example, since an `Int` is passed to the `.printFunctionType()` function, the compiler chooses the extension function because it matches the signature. The compiler ignores the member function, which takes no arguments.

Anonymous extension functions

You can define extension functions without giving them a name. This is useful when you want to avoid cluttering the global namespace or when you need to pass some extension behavior as a parameter.

For example, suppose you want to extend a data class with a one-time function to calculate shipping, without giving it a name:

```
fun main() {
    data class Order(val weight: Double)
    val calculateShipping = fun Order.(rate: Double): Double = this.weight * rate

    val order = Order(2.5)
    val cost = order.calculateShipping(3.0)
    println("Shipping cost: $cost")
```

```
    // Shipping cost: 7.5
}
```

To pass extension behavior as a parameter, use a lambda expression with a type annotation. For example, let's say you want to check if a number is within a range without defining a named function:

```
fun main() {
    val isInRange: Int.(min: Int, max: Int) -> Boolean = { min, max -> this in min..max }

    println(5.isInRange(1, 10))
    // true
    println(20.isInRange(1, 10))
    // false
}
```

In this example, the `isInRange` variable holds a function of type `Int.(min: Int, max: Int) -> Boolean`. The type is an extension function on the `Int` class that takes `min` and `max` parameters and returns a `Boolean`.

The lambda body `{ min, max -> this in min..max }` checks whether the `Int` value the function is called on falls within the range between `min` and `max` parameters. If the check is successful, the lambda returns `true`.

For more information, see Lambda expressions and anonymous functions.

Extension properties

Kotlin supports extension properties, which are useful for performing data transformations or creating UI display helpers without cluttering the class you're working with.

To create an extension property, write the name of the class that you want to extend, followed by a `.` and the name of your property.

For example, suppose you have a data class that represents a user with a first and last name, and you want to create a property that returns an email-style username when accessed. Your code might look like this:

```
data class User(val firstName: String, val lastName: String)

// An extension property to get a username-style email handle
val User.emailUsername: String
    get() = "${firstName.lowercase()}.${lastName.lowercase()}"

fun main() {
    val user = User("Mickey", "Mouse")
    // Calls extension property
    println("Generated email username: ${user.emailUsername}")
```

```
    // Generated email username: mickey.mouse
}
```

Since extensions don't actually add members to classes, there's no efficient way for an extension property to have a backing field. That's why initializers are not allowed for extension properties. You can define their behavior only by explicitly providing getters and setters. For example:

```
data class House(val streetName: String)

// Doesn't compile because there is no getter and setter
// var House.number = 1
// Error: Initializers are not allowed for extension properties

// Compiles successfully
val houseNumbers = mutableMapOf<House, Int>()
var House.number: Int
    get() = houseNumbers[this] ?: 1
    set(value) {
        println("Setting house number for ${this.streetName} to $value")
        houseNumbers[this] = value
    }

fun main() {
    val house = House("Maple Street")

    // Shows the default
    println("Default number: ${house.number} ${house.streetName}")
    // Default number: 1 Maple Street

    house.number = 99
    // Setting house number for Maple Street to 99

    // Shows the updated number
    println("Updated number: ${house.number} ${house.streetName}")
    // Updated number: 99 Maple Street
}
```

In this example, the getter uses the Elvis operator to return the house number if it exists in the `houseNumbers` map or `1`. To learn more about how to write getters and setters, see Custom getters and setters.

Companion object extensions

If a class defines a companion object, you can also define extension functions and properties for the companion object. Just like regular members of the companion object, you can call them using only the class name as the qualifier. The compiler names the companion object `Companion` by default:

```
class Logger {
    companion object { }
```

```
}

fun Logger.Companion.logStartupMessage() {
    println("Application started.")
}

fun main() {
    Logger.logStartupMessage()
    // Application started.
}
```

Declaring extensions as members

You can declare extensions for one class inside another. Extensions like this have multiple *implicit receivers*. An implicit receiver is an object whose members you can access without qualifying them with `this`:

- The class where you declare the extension is the *dispatch receiver*.
- The extension function's receiver type is the *extension receiver*.

Consider this example where the `Connection` class has an extension function for the `Host` class called `printConnectionString()`:

```
class Host(val hostname: String) {
    fun printHostname() { print(hostname) }
}

class Connection(val host: Host, val port: Int) {
    fun printPort() { print(port) }

    // Host is the extension receiver
    fun Host.printConnectionString() {
        // Calls Host.printHostname()
        printHostname()
        print(":")
        // Calls Connection.printPort()
        // Connection is the dispatch receiver
        printPort()
    }

    fun connect() {
        /*...*/
        // Calls the extension function
        host.printConnectionString()
    }
}

fun main() {
    Connection(Host("kotl.in"), 443).connect()
    // kotl.in:443

    // Triggers an error because the extension function isn't available outside Connection
```

```
    // Host("kotl.in").printConnectionString()
    // Unresolved reference 'printConnectionString'.
}
```

This example declares the `printConnectionString()` function inside the `Connection` class, so the `Connection` class is the dispatch receiver. The extension function's receiver type is the `Host` class, so the `Host` class is the extension receiver.

If the dispatch receiver and the extension receiver have members with the same name, the extension receiver's member takes precedence. To access the dispatch receiver explicitly, use the qualified `this` syntax:

```
class Connection {
    fun Host.getConnectionString() {
        // Calls Host.toString()
        toString()
        // Calls Connection.toString()
        this@Connection.toString()
    }
}
```

Overriding member extensions

You can declare member extensions as `open` and override them in subclasses, which is useful when you want to customize the extension's behavior for each subclass. The compiler handles each receiver type differently:

Receiver type	Resolution time	Dispatch type
Dispatch receiver	Runtime	Virtual
Extension receiver	Compile time	Static

Consider this example, where the `User` class is `open` and the `Admin` class inherits from it. The `NotificationSender` class defines `sendNotification()` extension functions for both `User` and `Admin` classes, and the `SpecialNotificationSender` class overrides them:

```
open class User

class Admin : User()

open class NotificationSender {
    open fun User.sendNotification() {
        println("Sending user notification from normal sender")
    }

    open fun Admin.sendNotification() {
        println("Sending admin notification from normal sender")
    }
```

```
    fun notify(user: User) {
        user.sendNotification()
    }
}

class SpecialNotificationSender : NotificationSender() {
    override fun User.sendNotification() {
        println("Sending user notification from special sender")
    }

    override fun Admin.sendNotification() {
        println("Sending admin notification from special sender")
    }
}

fun main() {
    // Dispatch receiver is NotificationSender
    // Extension receiver is User
    // Resolves to User.sendNotification() in NotificationSender
    NotificationSender().notify(User())
    // Sending user notification from normal sender

    // Dispatch receiver is SpecialNotificationSender
    // Extension receiver is User
    // Resolves to User.sendNotification() in SpecialNotificationSender
    SpecialNotificationSender().notify(User())
    // Sending user notification from special sender

    // Dispatch receiver is SpecialNotificationSender
    // Extension receiver is User NOT Admin
    // The notify() function declares user as type User
    // Statically resolves to User.sendNotification() in SpecialNotificationSender
    SpecialNotificationSender().notify(Admin())
    // Sending user notification from special sender
}
```

The dispatch receiver is resolved at runtime using virtual dispatch, which makes the behavior in the `main()` function easier to follow. What may surprise you is that when you call the `notify()` function on an `Admin` instance, the compiler chooses the extension based on the declared type: `user: User`, because it resolves the extension receiver statically.

Extensions and visibility modifiers

Extensions use the same visibility modifiers as regular functions declared in the same scope, including extensions declared as members of other classes.

For example, an extension declared at the top level of a file can access other `private` top-level declarations in the same file:

```
// File: StringUtils.kt
```

```
private fun removeWhitespace(input: String): String {
    return input.replace("\\s".toRegex(), "")
}

fun String.cleaned(): String {
    return removeWhitespace(this)
}

fun main() {
    val rawEmail = "  user @example. com  "
    val cleaned = rawEmail.cleaned()
    println("Raw:     '$rawEmail'")
    // Raw:     '  user @example. com  '
    println("Cleaned: '$cleaned'")
    // Cleaned: 'user@example.com'
    println("Looks like an email: ${cleaned.contains("@") && cleaned.contains(".")}")
    // Looks like an email: true
}
```

And if an extension is declared outside its receiver type, it can't access the receiver's `private` or `protected` members:

```
class User(private val password: String) {
    fun isLoggedIn(): Boolean = true
    fun passwordLength(): Int = password.length
}

// Extension declared outside the class
fun User.isSecure(): Boolean {
    // Can't access password because it's private:
    // return password.length >= 8

    // Instead, we rely on public members:
    return passwordLength() >= 8 && isLoggedIn()
}

fun main() {
    val user = User("supersecret")
    println("Is user secure: ${user.isSecure()}")
    // Is user secure: true
}
```

If an extension is marked as `internal`, it's only accessible within its module:

```
// Networking module
// JsonParser.kt
internal fun String.parseJson(): Map<String, Any> {
    return mapOf("fakeKey" to "fakeValue")
}
```

Scope of extensions

In most cases, you define extensions on the top level, directly under packages:

```
package org.example.declarations

fun List<String>.getLongestString() { /*...*/}
```

To use an extension outside its declaring package, import it at the call site:

```
package org.example.usage

import org.example.declarations.getLongestString

fun main() {
    val list = listOf("red", "green", "blue")
    list.getLongestString()
}
```

For more information, see Imports.

Interfaces

Interfaces in Kotlin can contain declarations of abstract methods, as well as method implementations. What makes them different from abstract classes is that interfaces cannot store state. They can have properties, but these need to be abstract or provide accessor implementations.

An interface is defined using the keyword `interface`:

```
interface MyInterface {
    fun bar()
    fun foo() {
      // optional body
    }
}
```

Implementing interfaces

A class or object can implement one or more interfaces:

```
class Child : MyInterface {
    override fun bar() {
        // body
    }
}
```

Properties in interfaces

You can declare properties in interfaces. A property declared in an interface can either be abstract or provide implementations for accessors. Properties declared in interfaces can't have backing fields, and therefore accessors declared in interfaces can't reference them:

```
interface MyInterface {
    val prop: Int // abstract

    val propertyWithImplementation: String
        get() = "foo"

    fun foo() {
        print(prop)
    }
}

class Child : MyInterface {
```

```
    override val prop: Int = 29
}
```

Interfaces Inheritance

An interface can derive from other interfaces, meaning it can both provide implementations for their members and declare new functions and properties. Quite naturally, classes implementing such an interface are only required to define the missing implementations:

```
interface Named {
    val name: String
}

interface Person : Named {
    val firstName: String
    val lastName: String

    override val name: String get() = "$firstName $lastName"
}

data class Employee(
    // implementing 'name' is not required
    override val firstName: String,
    override val lastName: String,
    val position: Position
) : Person
```

Resolving overriding conflicts

When you declare many types in your supertype list, you may inherit more than one implementation of the same method:

```
interface A {
    fun foo() { print("A") }
    fun bar()
}

interface B {
    fun foo() { print("B") }
    fun bar() { print("bar") }
}

class C : A {
    override fun bar() { print("bar") }
}

class D : A, B {
    override fun foo() {
        super<A>.foo()
        super<B>.foo()
```

```
    }

    override fun bar() {
        super<B>.bar()
    }
}
```

Interfaces *A* and *B* both declare functions *foo()* and *bar()*. Both of them implement *foo()*, but only *B* implements *bar()* (*bar()* is not marked as abstract in *A*, because this is the default for interfaces if the function has no body). Now, if you derive a concrete class *C* from *A*, you have to override *bar()* and provide an implementation.

However, if you derive *D* from *A* and *B*, you need to implement all the methods that you have inherited from multiple interfaces, and you need to specify how exactly *D* should implement them. This rule applies both to methods for which you've inherited a single implementation (*bar()*) and to those for which you've inherited multiple implementations (*foo()*).

JVM default method generation for interface functions

On the JVM, functions declared in interfaces are compiled to default methods. You can control this behavior using the `-jvm-default` compiler option with the following values:

- `enable` (default): generates default implementations in interfaces and includes bridge functions in subclasses and `DefaultImpls` classes. Use this mode to maintain binary compatibility with older Kotlin versions.
- `no-compatibility`: generates only default implementations in interfaces. This mode skips compatibility bridges and `DefaultImpls` classes, making it suitable for new Kotlin code.
- `disable`: skips default methods and generates only compatibility bridges and `DefaultImpls` classes.

To configure the `-jvm-default` compiler option, set the `jvmDefault` property in your Gradle Kotlin DSL:

```
kotlin {
    compilerOptions {
        jvmDefault = JvmDefaultMode.NO_COMPATIBILITY
    }
}
```

Delegation

The Delegation pattern has proven to be a good alternative to implementation inheritance, and Kotlin supports it natively requiring zero boilerplate code.

A class `Derived` can implement an interface `Base` by delegating all of its public members to a specified object:

```
interface Base {
    fun print()
}

class BaseImpl(val x: Int) : Base {
    override fun print() { print(x) }
}

class Derived(b: Base) : Base by b

fun main() {
    val base = BaseImpl(10)
    val derived = Derived(base)

    derived.print()
    // 10
}
```

The `by`-clause in the supertype list for `Derived` indicates that `b` will be stored internally in objects of `Derived` and the compiler will generate all the methods of `Base` that forward to `b`.

Overriding a member of an interface implemented by delegation

Overrides work as you expect: the compiler will use your `override` implementations instead of those in the delegate object. If you want to add `override fun printMessage() { print("abc") }` to `Derived`, the program would print *abc* instead of *10* when `printMessage` is called:

```
interface Base {
    fun printMessage()
    fun printMessageLine()
}

class BaseImpl(val x: Int) : Base {
    override fun printMessage() { print(x) }
    override fun printMessageLine() { println(x) }
}

class Derived(b: Base) : Base by b {
    override fun printMessage() { println("abc") }
}
```

```
fun main() {
    val base = BaseImpl(10)
    val derived = Derived(base)

    derived.printMessage()
    // abc
    derived.printMessageLine()
    // 10
}
```

Note, however, that members overridden in this way do not get called from the members of the delegate object, which can only access its own implementations of the interface members:

```
interface Base {
    val message: String
    fun print()
}

class BaseImpl(x: Int) : Base {
    override val message = "BaseImpl: x = $x"
    override fun print() { println(message) }
}

class Derived(b: Base) : Base by b {
    // This property is not accessible
    // from b's implementation of `print()`
    override val message = "Message of Derived"
}

fun main() {
    val base = BaseImpl(10)
    val derived = Derived(base)

    derived.print()
    // BaseImpl: x = 10
    println(derived.message)
    // Message of Derived
}
```

Learn more about delegated properties.

Inheritance

> **Tip:** Before creating an inheritance hierarchy with classes, consider using abstract classes or interfaces. You can inherit from abstract classes and interfaces by default. They are designed so that other classes can inherit their members and implement them.

All classes in Kotlin have a common superclass, `Any`, which is the default superclass for a class with no supertypes declared:

```
class Example // Implicitly inherits from Any
```

`Any` has three methods: `equals()`, `hashCode()`, and `toString()`. Thus, these methods are defined for all Kotlin classes.

By default, Kotlin classes are final – they can't be inherited. To make a class inheritable, mark it with the `open` keyword:

```
open class Base // Class is open for inheritance
```

For more information, see Open keyword.

To declare an explicit supertype, place the type after a colon in the class header:

```
open class Base(p: Int)

class Derived(p: Int) : Base(p)
```

If the derived class has a primary constructor, the base class can (and must) be initialized in that primary constructor according to its parameters.

If the derived class has no primary constructor, then each secondary constructor has to initialize the base type using the `super` keyword or it has to delegate to another constructor which does. Note that in this case, different secondary constructors can call different constructors of the base type:

```
class MyView : View {
    constructor(ctx: Context) : super(ctx)

    constructor(ctx: Context, attrs: AttributeSet) : super(ctx, attrs)
}
```

Open keyword

In Kotlin, the `open` keyword indicates that a class or a member (function or property) can be overridden in subclasses. By default, Kotlin classes and their members are *final*, meaning they cannot be inherited from (for classes) or overridden (for members) unless you explicitly mark them as `open`:

```
// Base class with the open keyword to allow inheritance
open class Person(
    val name: String
) {
    // Open function that can be overridden in a subclass
    open fun introduce() {
        println("Hello, my name is $name.")
    }
}

// Subclass inheriting from Person and overriding the introduce() function
class Student(
    name: String,
    val school: String
) : Person(name) {
    override fun introduce() {
        println("Hi, I'm $name, and I study at $school.")
    }
}
```

If you override a member of a base class, the overriding member is also open by default. If you want to change this and forbid the subclasses of your class from overriding your implementation, you can explicitly mark the overriding member as `final`:

```
// Base class with the open keyword to allow inheritance
open class Person(val name: String) {
    // Open function that can be overridden in a subclass
    open fun introduce() {
        println("Hello, my name is $name.")
    }
}

//  A subclass that inherits from Person and overrides the introduce() function
class Student(name: String, val school: String) : Person(name) {
    // The final keyword prevents further overrides in subclasses
    final override fun introduce() {
        println("Hi, I'm $name, and I study at $school.")
    }
}
```

Overriding methods

Kotlin requires explicit modifiers for overridable members and overrides:

```
open class Shape {
    open fun draw() { /*...*/ }
    fun fill() { /*...*/ }
}

class Circle() : Shape() {
    override fun draw() { /*...*/ }
}
```

The `override` modifier is required for `Circle.draw()`. If it's missing, the compiler will complain. If there is no `open` modifier on a function, like `Shape.fill()`, declaring a method with the same signature in a subclass is not allowed, either with `override` or without it. The `open` modifier has no effect when added to members of a final class – a class without an `open` modifier.

A member marked `override` is itself open, so it may be overridden in subclasses. If you want to prohibit re-overriding, use `final`:

```
open class Rectangle() : Shape() {
    final override fun draw() { /*...*/ }
}
```

Overriding properties

The overriding mechanism works on properties in the same way that it does on methods. Properties declared on a superclass that are then redeclared on a derived class must be prefaced with `override`, and they must have a compatible type. Each declared property can be overridden by a property with an initializer or by a property with a `get` method:

```
open class Shape {
    open val vertexCount: Int = 0
}

class Rectangle : Shape() {
    override val vertexCount = 4
}
```

You can also override a `val` property with a `var` property, but not vice versa. This is allowed because a `val` property essentially declares a `get` method, and overriding it as a `var` additionally declares a `set` method in the derived class.

Note that you can use the `override` keyword as part of the property declaration in a primary constructor:

```
interface Shape {
    val vertexCount: Int
}
```

```
class Rectangle(override val vertexCount: Int = 4) : Shape // Always has 4 vertices

class Polygon : Shape {
    override var vertexCount: Int = 0  // Can be set to any number later
}
```

Derived class initialization order

During the construction of a new instance of a derived class, the base class initialization is done as the first step (preceded only by evaluation of the arguments for the base class constructor), which means that it happens before the initialization logic of the derived class is run.

```
open class Base(val name: String) {

    init { println("Initializing a base class") }

    open val size: Int =
        name.length.also { println("Initializing size in the base class: $it") }
}

class Derived(
    name: String,
    val lastName: String,
) : Base(name.replaceFirstChar { it.uppercase() }.also { println("Argument for the base clas
s: $it") }) {

    init { println("Initializing a derived class") }

    override val size: Int =
        (super.size + lastName.length).also { println("Initializing size in the derived clas
s: $it") }
}
fun main() {
    println("Constructing the derived class(\"hello\", \"world\")")
    Derived("hello", "world")
}
```

This means that when the base class constructor is executed, the properties declared or overridden in the derived class have not yet been initialized. Using any of those properties in the base class initialization logic (either directly or indirectly through another overridden `open` member implementation) may lead to incorrect behavior or a runtime failure. When designing a base class, you should therefore avoid using `open` members in the constructors, property initializers, or `init` blocks.

Calling the superclass implementation

Code in a derived class can call its superclass functions and property accessor implementations using the `super` keyword:

```
open class Rectangle {
    open fun draw() { println("Drawing a rectangle") }
    val borderColor: String get() = "black"
}

class FilledRectangle : Rectangle() {
    override fun draw() {
        super.draw()
        println("Filling the rectangle")
    }

    val fillColor: String get() = super.borderColor
}
```

Inside an inner class, accessing the superclass of the outer class is done using the `super` keyword qualified with the outer class name: `super@Outer`:

```
open class Rectangle {
    open fun draw() { println("Drawing a rectangle") }
    val borderColor: String get() = "black"
}
class FilledRectangle: Rectangle() {
    override fun draw() {
        val filler = Filler()
        filler.drawAndFill()
    }

    inner class Filler {
        fun fill() { println("Filling") }
        fun drawAndFill() {
            super@FilledRectangle.draw() // Calls Rectangle's implementation of draw()
            fill()
            println("Drawn a filled rectangle with color ${super@FilledRectangle.borderColo
r}") // Uses Rectangle's implementation of borderColor's get()
        }
    }
}
fun main() {
    val fr = FilledRectangle()
        fr.draw()
}
```

Overriding rules

In Kotlin, implementation inheritance is regulated by the following rule: if a class inherits multiple implementations of the same member from its immediate superclasses, it must override this member and provide its own implementation (perhaps, using one of the inherited ones).

To denote the supertype from which the inherited implementation is taken, use `super` qualified by the supertype name in angle brackets, such as `super<Base>`:

```
open class Rectangle {
    open fun draw() { /* ... */ }
}

interface Polygon {
    fun draw() { /* ... */ } // interface members are 'open' by default
}

class Square() : Rectangle(), Polygon {
    // The compiler requires draw() to be overridden:
    override fun draw() {
        super<Rectangle>.draw() // call to Rectangle.draw()
        super<Polygon>.draw() // call to Polygon.draw()
    }
}
```

It's fine to inherit from both `Rectangle` and `Polygon`, but both of them have their implementations of `draw()`, so you need to override `draw()` in `Square` and provide a separate implementation for it to eliminate the ambiguity.

Object declarations and expressions

In Kotlin, objects allow you to define a class and create an instance of it in a single step. This is useful when you need either a reusable singleton instance or a one-time object. To handle these scenarios, Kotlin provides two key approaches: *object declarations* for creating singletons and *object expressions* for creating anonymous, one-time objects.

> **Tip:** A singleton ensures that a class has only one instance and provides a global point of access to it.

Object declarations and object expressions are best used for scenarios when:

- **Using singletons for shared resources:** You need to ensure that only one instance of a class exists throughout the application. For example, managing a database connection pool.
- **Creating factory methods:** You need a convenient way to create instances efficiently. Companion objects allow you to define class-level functions and properties tied to a class, simplifying the creation and management of these instances.
- **Modifying existing class behavior temporarily:** You want to modify the behavior of an existing class without the need to create a new subclass. For example, adding temporary functionality to an object for a specific operation.
- **Type-safe design is required:** You require one-time implementations of interfaces or abstract classes using object expressions. This can be useful for scenarios like a button click handler.

Object declarations

You can create single instances of objects in Kotlin using object declarations, which always have a name following the `object` keyword. This allows you to define a class and create an instance of it in a single step, which is useful for implementing singletons:

```
// Declares a Singleton object to manage data providers
object DataProviderManager {
    private val providers = mutableListOf<DataProvider>()

    // Registers a new data provider
    fun registerDataProvider(provider: DataProvider) {
        providers.add(provider)
    }

    // Retrieves all registered data providers
    val allDataProviders: Collection<DataProvider>
        get() = providers
}
// Example data provider interface
interface DataProvider {
```

```
    fun provideData(): String
}

// Example data provider implementation
class ExampleDataProvider : DataProvider {
    override fun provideData(): String {
        return "Example data"
    }
}

fun main() {
    // Creates an instance of ExampleDataProvider
    val exampleProvider = ExampleDataProvider()

    // To refer to the object, use its name directly
    DataProviderManager.registerDataProvider(exampleProvider)

    // Retrieves and prints all data providers
    println(DataProviderManager.allDataProviders.map { it.provideData() })
    // [Example data]
}
```

Tip: The initialization of an object declaration is thread-safe and done on first access.

To refer to the `object`, use its name directly:

```
DataProviderManager.registerDataProvider(exampleProvider)
```

Object declarations can also have supertypes, similar to how anonymous objects can inherit from existing classes or implement interfaces:

```
object DefaultListener : MouseAdapter() {
    override fun mouseClicked(e: MouseEvent) { ... }

    override fun mouseEntered(e: MouseEvent) { ... }
}
```

Like variable declarations, object declarations are not expressions, so they cannot be used on the right-hand side of an assignment statement:

```
// Syntax error: An object expression cannot bind a name.
val myObject = object MySingleton {
    val name = "Singleton"
}
```

Object declarations cannot be local, which means they cannot be nested directly inside a function. However, they can be nested within other object declarations or non-inner classes.

Data objects

When printing a plain object declaration in Kotlin, the string representation contains both its name and the hash of the `object`:

```
object MyObject

fun main() {
    println(MyObject)
    // MyObject@hashcode
}
```

However, by marking an object declaration with the `data` modifier, you can instruct the compiler to return the actual name of the object when calling `toString()`, the same way it works for data classes:

```
data object MyDataObject {
    val number: Int = 3
}

fun main() {
    println(MyDataObject)
    // MyDataObject
}
```

Additionally, the compiler generates several functions for your `data object`:

- `toString()` returns the name of the data object
- `equals()`/`hashCode()` enables equality checks and hash-based collections

    You can't provide a custom `equals` or `hashCode` implementation for a `data object`.

The `equals()` function for a `data object` ensures that all objects that have the type of your `data object` are considered equal. In most cases, you will only have a single instance of your `data object` at runtime, since a `data object` declares a singleton. However, in the edge case where another object of the same type is generated at runtime (for example, by using platform reflection with `java.lang.reflect` or a JVM serialization library that uses this API under the hood), this ensures that the objects are treated as being equal.

> **Warning:** Make sure that you only compare `data objects` structurally (using the `==` operator) and never by reference (using the `===` operator). This helps you to avoid pitfalls when more than one instance of a data object exists at runtime.

```
import java.lang.reflect.Constructor

data object MySingleton

fun main() {
```

```
    val evilTwin = createInstanceViaReflection()

    println(MySingleton)
    // MySingleton

    println(evilTwin)
    // MySingleton

    // Even when a library forcefully creates a second instance of MySingleton,
    // its equals() function returns true:
    println(MySingleton == evilTwin)
    // true

    // Don't compare data objects using ===
    println(MySingleton === evilTwin)
    // false
}

fun createInstanceViaReflection(): MySingleton {
    // Kotlin reflection does not permit the instantiation of data objects.
    // This creates a new MySingleton instance "by force" (using Java platform reflection)
    // Don't do this yourself!
    return (MySingleton.javaClass.declaredConstructors[0].apply { isAccessible = true } as C
onstructor<MySingleton>).newInstance()
}
```

The generated `hashCode()` function has a behavior that is consistent with the `equals()` function, so that all runtime instances of a `data object` have the same hash code.

Differences between data objects and data classes

While `data object` and `data class` declarations are often used together and have some similarities, there are some functions that are not generated for a `data object`:

- No `copy()` function. Because a `data object` declaration is intended to be used as singletons, no `copy()` function is generated. Singletons restrict the instantiation of a class to a single instance, which would be violated by allowing copies of the instance to be created.
- No `componentN()` function. Unlike a `data class`, a `data object` does not have any data properties. Since attempting to destructure such an object without data properties wouldn't make sense, no `componentN()` functions are generated.

Use data objects with sealed hierarchies

Data object declarations are particularly useful for sealed hierarchies like sealed classes or sealed interfaces. They allow you to maintain symmetry with any data classes you may have defined alongside the object.

In this example, declaring `EndOfFile` as a `data object` instead of a plain `object` means that it will get the `toString()` function without the need to override it manually:

```
sealed interface ReadResult
data class Number(val number: Int) : ReadResult
data class Text(val text: String) : ReadResult
data object EndOfFile : ReadResult

fun main() {
    println(Number(7))
    // Number(number=7)
    println(EndOfFile)
    // EndOfFile
}
```

Companion objects

Companion objects allow you to define class-level functions and properties. This makes it easy to create factory methods, hold constants, and access shared utilities.

An object declaration inside a class can be marked with the `companion` keyword:

```
class MyClass {
    companion object Factory {
        fun create(): MyClass = MyClass()
    }
}
```

Members of the `companion object` can be called simply by using the class name as the qualifier:

```
class User(val name: String) {
    // Defines a companion object that acts as a factory for creating User instances
    companion object Factory {
        fun create(name: String): User = User(name)
    }
}

fun main(){
    // Calls the companion object's factory method using the class name as the qualifier.
    // Creates a new User instance
    val userInstance = User.create("John Doe")
    println(userInstance.name)
    // John Doe
}
```

The name of the `companion object` can be omitted, in which case the name `Companion` is used:

```
class User(val name: String) {
    // Defines a companion object without a name
    companion object { }
}

// Accesses the companion object
val companionUser = User.Companion
```

Class members can access `private` members of their corresponding `companion object`:

```
class User(val name: String) {
    companion object {
        private val defaultGreeting = "Hello"
    }

    fun sayHi() {
        println(defaultGreeting)
    }
}
User("Nick").sayHi()
// Hello
```

When a class name is used by itself, it acts as a reference to the companion object of the class, regardless of whether the companion object is named or not:

```
class User1 {
    // Defines a named companion object
    companion object Named {
        fun show(): String = "User1's Named Companion Object"
    }
}

// References the companion object of User1 using the class name
val reference1 = User1

class User2 {
    // Defines an unnamed companion object
    companion object {
        fun show(): String = "User2's Companion Object"
    }
}

// References the companion object of User2 using the class name
val reference2 = User2
fun main() {
    // Calls the show() function from the companion object of User1
    println(reference1.show())
    // User1's Named Companion Object

    // Calls the show() function from the companion object of User2
    println(reference2.show())
    // User2's Companion Object
}
```

Although members of companion objects in Kotlin look like static members from other languages, they are actually instance members of the companion object, meaning they belong to the object itself. This allows companion objects to implement interfaces:

```
interface Factory<T> {
    fun create(name: String): T
```

```
}

class User(val name: String) {
    // Defines a companion object that implements the Factory interface
    companion object : Factory<User> {
        override fun create(name: String): User = User(name)
    }
}

fun main() {
    // Uses the companion object as a Factory
    val userFactory: Factory<User> = User
    val newUser = userFactory.create("Example User")
    println(newUser.name)
    // Example User
}
```

However, on the JVM, you can have members of companion objects generated as real static methods and fields if you use the `@JvmStatic` annotation. See the Java interoperability section for more detail.

Object expressions

Object expressions declare a class and create an instance of that class, but without naming either of them. These classes are useful for one-time use. They can either be created from scratch, inherit from existing classes, or implement interfaces. Instances of these classes are also called *anonymous objects* because they are defined by an expression, not a name.

Create anonymous objects from scratch

Object expressions start with the `object` keyword.

If the object doesn't extend any classes or implement interfaces, you can define an object's members directly inside curly braces after the `object` keyword:

```
fun main() {
    val helloWorld = object {
        val hello = "Hello"
        val world = "World"
        // Object expressions extend the Any class, which already has a toString() function,
        // so it must be overridden
        override fun toString() = "$hello $world"
    }

    print(helloWorld)
    // Hello World
}
```

Inherit anonymous objects from supertypes

To create an anonymous object that inherits from some type (or types), specify this type after `object` and a colon `:`. Then implement or override the members of this class as if you were inheriting from it:

```
window.addMouseListener(object : MouseAdapter() {
    override fun mouseClicked(e: MouseEvent) { /*...*/ }

    override fun mouseEntered(e: MouseEvent) { /*...*/ }
})
```

If a supertype has a constructor, pass the appropriate constructor parameters to it. Multiple supertypes can be specified, separated by commas, after the colon:

```
// Creates an open class BankAccount with a balance property
open class BankAccount(initialBalance: Int) {
    open val balance: Int = initialBalance
}

// Defines an interface Transaction with an execute() function
interface Transaction {
    fun execute()
}

// A function to perform a special transaction on a BankAccount
fun specialTransaction(account: BankAccount) {
    // Creates an anonymous object that inherits from the BankAccount class and implements t
he Transaction interface
    // The balance of the provided account is passed to the BankAccount superclass construct
or
    val temporaryAccount = object : BankAccount(account.balance), Transaction {

        override val balance = account.balance + 500  // Temporary bonus

        // Implements the execute() function from the Transaction interface
        override fun execute() {
            println("Executing special transaction. New balance is $balance.")
        }
    }
    // Executes the transaction
    temporaryAccount.execute()
}
fun main() {
    // Creates a BankAccount with an initial balance of 1000
    val myAccount = BankAccount(1000)
    // Performs a special transaction on the created account
    specialTransaction(myAccount)
    // Executing special transaction. New balance is 1500.
}
```

Use anonymous objects as return and value types

When you return an anonymous object from a local or `private` function or property, all the members of that anonymous object are accessible through that function or property:

```
class UserPreferences {
    private fun getPreferences() = object {
        val theme: String = "Dark"
        val fontSize: Int = 14
    }

    fun printPreferences() {
        val preferences = getPreferences()
        println("Theme: ${preferences.theme}, Font Size: ${preferences.fontSize}")
    }
}
fun main() {
    val userPreferences = UserPreferences()
    userPreferences.printPreferences()
    // Theme: Dark, Font Size: 14
}
```

This allows you to return an anonymous object with specific properties, offering a simple way to encapsulate data or behavior without creating a separate class.

If a function or property that returns an anonymous object has `public`, `protected`, or `internal` visibility, its actual type is:

- `Any` if the anonymous object doesn't have a declared supertype.
- The declared supertype of the anonymous object, if there is exactly one such type.
- The explicitly declared type if there is more than one declared supertype.

In all these cases, members added in the anonymous object are not accessible. Overridden members are accessible if they are declared in the actual type of the function or property. For example:

```
interface Notification {
    // Declares notifyUser() in the Notification interface
    fun notifyUser()
}

interface DetailedNotification

class NotificationManager {
    // The return type is Any. The message property is not accessible.
    // When the return type is Any, only members of the Any class are accessible.
    fun getNotification() = object {
        val message: String = "General notification"
    }

    // The return type is Notification because the anonymous object implements only one inte
```

```
rface
    // The notifyUser() function is accessible because it is part of the Notification interf
ace
    // The message property is not accessible because it is not declared in the Notification
interface
    fun getEmailNotification() = object : Notification {
        override fun notifyUser() {
            println("Sending email notification")
        }
        val message: String = "You've got mail!"
    }

    // The return type is DetailedNotification. The notifyUser() function and the message pr
operty are not accessible
    // Only members declared in the DetailedNotification interface are accessible
    fun getDetailedNotification(): DetailedNotification = object : Notification, DetailedNot
ification {
        override fun notifyUser() {
            println("Sending detailed notification")
        }
        val message: String = "Detailed message content"
    }
}
fun main() {
    // This produces no output
    val notificationManager = NotificationManager()

    // The message property is not accessible here because the return type is Any
    // This produces no output
    val notification = notificationManager.getNotification()

    // The notifyUser() function is accessible
    // The message property is not accessible here because the return type is Notification
    val emailNotification = notificationManager.getEmailNotification()
    emailNotification.notifyUser()
    // Sending email notification

    // The notifyUser() function and message property are not accessible here because the re
turn type is DetailedNotification
    // This produces no output
    val detailedNotification = notificationManager.getDetailedNotification()
}
```

Access variables from anonymous objects

Code within the body of object expressions can access variables from the enclosing scope:

```
import java.awt.event.MouseAdapter
import java.awt.event.MouseEvent

fun countClicks(window: JComponent) {
    var clickCount = 0
    var enterCount = 0

    // MouseAdapter provides default implementations for mouse event functions
```

```
    // Simulates MouseAdapter handling mouse events
    window.addMouseListener(object : MouseAdapter() {
        override fun mouseClicked(e: MouseEvent) {
            clickCount++
        }

        override fun mouseEntered(e: MouseEvent) {
            enterCount++
        }
    })
    // The clickCount and enterCount variables are accessible within the object expression
}
```

Behavior difference between object declarations and expressions

There are differences in the initialization behavior between object declarations and object expressions:

- Object expressions are executed (and initialized) *immediately*, where they are used.
- Object declarations are initialized *lazily*, when accessed for the first time.
- A companion object is initialized when the corresponding class is loaded (resolved) that matches the semantics of a Java static initializer.

This expressions

The `this` expression refers to the current receiver (the object with which the function works). How you use `this` depends on the context:

- In a member of a class, `this` refers to the current object of that class.

  For example, in the following code, `this.name` means the `name` property of the current `Language` object:

  ```
  class Language(val name: String) {
      fun printName() {
          println(this.name)
      }
  }

  fun main() {
      val language = Language("Kotlin")
      language.printName()
      // Kotlin
  }
  ```

- In an extension function or a function literal with receiver, `this` refers to the receiver.

  In the following example, `lastCharacter()` is called on the string `"Kotlin"`. Therefore, `"Kotlin"` is the receiver. Inside the extension function, `this` refers to `"Kotlin"`. This means that `this.length` is the length of `"Kotlin"`:

  ```
  fun String.lastCharacter(): Char {
      println(this.length)
      // 6
      return this[this.length - 1]
  }

  fun main() {
      println("Kotlin".lastCharacter())
      // n
  }
  ```

  Tip: In simple cases, you don't need to write `this` explicitly. Kotlin resolves it from the current scope. Learn more about .

Qualified `this`

Your code can have several receivers available at the same time when receiver scopes are nested. Kotlin can use any available receiver to access its members implicitly, but receivers from inner

scopes have higher priority. To explicitly refer to a particular receiver, use a qualified `this`. It is especially useful when multiple receivers have members with the same name, and you need to access a member of an outer receiver.

To use a qualified `this`, add a label qualifier:

```
this@label
```

The label tells the compiler which receiver to access. You can use the name of an enclosing class or extension function. For example, `this@foo` refers to the receiver of an enclosing extension function named `foo`.

Access an outer class from an inner class

In an inner class, unqualified `this` refers to the inner class instance. To access the outer class object, use qualified `this`:

```
class User(val name: String) {
    inner class Age(val value: Int) {
        fun printInfo() {

            // Refers to the value property of the current Age object
            println(this.value)
            // 22

            // Refers to the name property of the outer User object
            println(this@User.name)
            // Jane Doe
        }
    }
}

fun main() {
    val user = User("Jane Doe")
    val age = user.Age(22)
    age.printInfo()
}
```

> **Note:** Only inner classes hold a reference to an outer class instance. Regular nested classes don't have access to `this` from the outer class.

Access a class from an extension function

If you declare an extension function inside a class, two receivers are available:

- The extension receiver: the value on which you call the extension function.
- The dispatch receiver: the current object of the class where you declare the extension function.

Use qualified `this` to specify which receiver you need:

```
class User(val name: String) {
    val prefix = "Name"

    fun String.formatName(): String {
        return "${this@User.prefix}: ${this.uppercase()}"
    }

    fun printName() {
        println(name.formatName())
    }
}

fun main() {
    val user = User("Jane Doe")
    user.printName()
    // Name: JANE DOE
}
```

Here:

- `this` refers to the extension receiver (`String`), so `this.uppercase()` converts the string to uppercase.
- `this@User` refers to the current `User` object.
- `this@User.prefix` accesses the `prefix` property of the current `User` object.

Access from a lambda

Unlike a regular lambda, a lambda with receiver introduces a receiver into its scope. As a result, `this` inside the lambda refers to the lambda's receiver, not to one from an enclosing scope. If a lambda with receiver is nested inside another receiver scope, add a label to the lambda and use a qualified `this` to refer explicitly to the lambda's receiver or to a receiver from an enclosing scope:

```
class User(val name: String) {
    fun printWithPrefix() {
        val printString: String.() -> Unit = stringLabel@ {
            println("${this@stringLabel}: ${this@User.name}")
        }

        printString("User")
    }
}

fun main() {
    val user = User("Jane Doe")
    user.printWithPrefix()
    // User: Jane Doe
}
```

Here:

- `stringLabel@` labels the lambda.
- `this@stringLabel` refers to the string receiver of the lambda.
- `this@User` refers to the current `User` object.

The label doesn't call anything or change how the lambda works. It only helps you refer to the lambda's receiver.

Access an anonymous object or its outer class

An anonymous object has its own receiver scope. Inside the object body, unqualified `this` refers to the anonymous object itself. However, since anonymous objects don't have class names, you can't use them as `this` qualifiers. Therefore, you can refer to the anonymous object only with unqualified `this`:

```
interface UserPrinter {
    fun print()
}

fun main() {
    val printer = object : UserPrinter {
        val prefix = "User"

        override fun print() {
            // `this` refers to the anonymous object
            // `this.prefix` accesses its `prefix` property
            println(this.prefix)
        }
    }
    printer.print()
    // User
}
```

If you declare an anonymous object inside another class, you can use qualified `this` to access the enclosing object:

```
interface UserPrinter {
    fun print()
}

class User(val name: String) {
    fun createPrinter(): UserPrinter {
        return object : UserPrinter {
            override fun print() {
                // `this@User` refers to the enclosing `User` object
                // `this@User.name` accesses its `name` property
                println(this@User.name)
            }
        }
    }
}
```

```
fun main() {
    val printer = User("Jane Doe").createPrinter()
    printer.print()
    // Jane Doe
}
```

Implicit `this`

When you call a member function on `this`, you can omit the `this.` qualifier. However, if another callable with the same name is available in a closer lexical scope, Kotlin resolves the unqualified call to that callable instead of the member function. To explicitly call the member function, use the `this.` qualifier:

```
fun main() {
    class A {
        fun printLine() {
            println("Member function")
        }

        fun invokePrintLine() {
            fun printLine() {
                println("Local function")
            }

            printLine()
            // Local function

            this.printLine()
            // Member function
        }
    }

    A().invokePrintLine()
}
```

Sealed classes and interfaces

Sealed classes and interfaces provide controlled inheritance of your class hierarchies. All direct subclasses of a sealed class are known at compile time. No other subclasses may appear outside the module and package within which the sealed class is defined. The same logic applies to sealed interfaces and their implementations: once a module with a sealed interface is compiled, no new implementations can be created.

> **Note:** Direct subclasses are classes that immediately inherit from their superclass.
>
> Indirect subclasses are classes that inherit from more than one level down from their superclass.

When you combine sealed classes and interfaces with the `when` expression, you can cover the behavior of all possible subclasses and ensure that no new subclasses are created to affect your code adversely.

Sealed classes are best used for scenarios when:

- **Limited class inheritance is desired:** You have a predefined, finite set of subclasses that extend a class, all of which are known at compile time.
- **Type-safe design is required:** Safety and pattern matching are crucial in your project. Particularly for state management or handling complex conditional logic. For an example, check out Use sealed classes with when expressions.
- **Working with closed APIs:** You want robust and maintainable public APIs for libraries that ensure that third-party clients use the APIs as intended.

For more detailed practical applications, see Use case scenarios.

> **Tip:** Java 15 introduced a similar concept, where sealed classes use the `sealed` keyword along with the `permits` clause to define restricted hierarchies.

Declare a sealed class or interface

To declare a sealed class or interface, use the `sealed` modifier:

```
// Create a sealed interface
sealed interface Error

// Create a sealed class that implements sealed interface Error
sealed class IOError(): Error

// Define subclasses that extend sealed class 'IOError'
class FileReadError(val file: File): IOError()
class DatabaseError(val source: DataSource): IOError()
```

```
// Create a singleton object implementing the 'Error' sealed interface
object RuntimeError : Error
```

This example could represent a library's API that contains error classes to let library users handle errors that it can throw. If the hierarchy of such error classes includes interfaces or abstract classes visible in the public API, then nothing prevents other developers from implementing or extending them in the client code. Since the library doesn't know about errors declared outside of it, it can't treat them consistently with its own classes. However, with a **sealed** hierarchy of error classes, library authors can be sure that they know all the possible error types and that other error types can't appear later.

The hierarchy of the example looks like this:

!Hierarchy illustration of sealed classes and interfaces

Constructors

A sealed class itself is always an abstract class, and as a result, can't be instantiated directly. However, it may contain or inherit constructors. These constructors aren't for creating instances of the sealed class itself but for its subclasses. Consider the following example with a sealed class called `Error` and its several subclasses, which we instantiate:

```
sealed class Error(val message: String) {
    class NetworkError : Error("Network failure")
    class DatabaseError : Error("Database cannot be reached")
    class UnknownError : Error("An unknown error has occurred")
}

fun main() {
    val errors = listOf(Error.NetworkError(), Error.DatabaseError(), Error.UnknownError())
    errors.forEach { println(it.message) }
}
// Network failure
// Database cannot be reached
// An unknown error has occurred
```

You can use `enum` classes within your sealed classes to use enum constants to represent states and provide additional detail. Each enum constant exists only as a **single** instance, while subclasses of a sealed class may have **multiple** instances. In the example, the `sealed class Error` along with its several subclasses, employs an `enum` to denote error severity. Each subclass constructor initializes the `severity` and can alter its state:

```
enum class ErrorSeverity { MINOR, MAJOR, CRITICAL }

sealed class Error(val severity: ErrorSeverity) {
    class FileReadError(val file: File): Error(ErrorSeverity.MAJOR)
    class DatabaseError(val source: DataSource): Error(ErrorSeverity.CRITICAL)
```

```
    object RuntimeError : Error(ErrorSeverity.CRITICAL)
    // Additional error types can be added here
}
```

Constructors of sealed classes can have one of two visibilities: `protected` (by default) or `private`:

```
sealed class IOError {
    // A sealed class constructor has protected visibility by default. It's visible inside t
his class and its subclasses
    constructor() { /*...*/ }

    // Private constructor, visible inside this class only.
    // Using a private constructor in a sealed class allows for even stricter control over i
nstantiation, enabling specific initialization procedures within the class.
    private constructor(description: String): this() { /*...*/ }

    // This will raise an error because public and internal constructors are not allowed in
sealed classes
    // public constructor(code: Int): this() {}
}
```

Inheritance

Direct subclasses of sealed classes and interfaces must be declared in the same package. They may be top-level or nested inside any number of other named classes, named interfaces, or named objects. Subclasses can have any visibility as long as they are compatible with normal inheritance rules in Kotlin, including those for overriding properties.

Subclasses of sealed classes must have a properly qualified name. They can't be local or anonymous objects.

> **Note:** `enum` classes can't extend a sealed class, or any other class. However, they can implement sealed interfaces:
>
> ```
> sealed interface Error
>
> // enum class extending the sealed interface Error
> enum class ErrorType : Error {
>     FILE_ERROR, DATABASE_ERROR
> }
> ```

These restrictions don't apply to indirect subclasses. If a direct subclass of a sealed class is not marked as sealed, it can be extended in any way that its modifiers allow:

```
// Sealed interface 'Error' has implementations only in the same package and module
sealed interface Error

// Sealed class 'IOError' extends 'Error' and is extendable only within the same package
sealed class IOError(): Error
```

```
// Open class 'CustomError' extends 'Error' and can be extended anywhere it's visible
open class CustomError(): Error
```

Inheritance in multiplatform projects

There is one more inheritance restriction in multiplatform projects: direct subclasses of sealed classes must reside in the same source set. It applies to sealed classes without the expected and actual modifiers.

If a sealed class is declared as `expect` in a common source set and have `actual` implementations in platform source sets, both `expect` and `actual` versions can have subclasses in their source sets. Moreover, if you use a hierarchical structure, you can create subclasses in any source set between the `expect` and `actual` declarations.

Learn more about the hierarchical structure of multiplatform projects.

Use sealed classes with when expression

The key benefit of using sealed classes comes into play when you use them in a `when` expression. The `when` expression, used with a sealed class, allows the Kotlin compiler to check exhaustively that all possible cases are covered. In such cases, you don't need to add an `else` clause:

```
// Sealed class and its subclasses
sealed class Error {
    class FileReadError(val file: String): Error()
    class DatabaseError(val source: String): Error()
    object RuntimeError : Error()
}
// Function to log errors
fun log(e: Error) = when(e) {
    is Error.FileReadError -> println("Error while reading file ${e.file}")
    is Error.DatabaseError -> println("Error while reading from database ${e.source}")
    Error.RuntimeError -> println("Runtime error")
    // No `else` clause is required because all the cases are covered
}
// List all errors
fun main() {
    val errors = listOf(
        Error.FileReadError("example.txt"),
        Error.DatabaseError("usersDatabase"),
        Error.RuntimeError
    )

    errors.forEach { log(it) }
}
```

> **Tip:** To reduce repetition in `when` expressions, try out context-sensitive resolution (currently in preview). This feature allows you to omit the type name when matching

sealed class members if the expected type is known.

For more information, see Preview of context-sensitive resolution or the related KEEP proposal.

When using sealed classes with `when` expressions, you can also add guard conditions to include additional checks in a single branch. For more information, see Guard conditions in when expressions.

> **Note:** In multiplatform projects, if you have a sealed class with a `when` expression as an expected declaration in your common code, you still need an `else` branch. This is because subclasses of `actual` platform implementations may extend sealed classes that aren't known in the common code.

Inspect sealed subclasses with reflection

You can use Reflection functionality to inspect the direct subclasses of a sealed class or interface at runtime. It helps a framework code discover the variants of a sealed hierarchy. For example, you can register a serializer or verify that every variant has a handler.

If you need to inspect a known direct subclass, use sealedSubclasses. The method returns a `KClass` that can represent a regular class or an object declaration. For an object declaration, `objectInstance` returns its singleton instance:

```
sealed interface Status
data class Send(val message: String) : Status
data object Start : Status
data object Stop : Status

fun main() {
    // Get the direct subclasses known to the sealed hierarchy
    val subclasses = Status::class.sealedSubclasses
    println(subclasses.mapNotNull { it.simpleName })
    // [Send, Start, Stop]

    val singletonStatus = subclasses.mapNotNull { subclass ->
        // Return null for Send because it is a regular class
        subclass.objectInstance
    }
    println(singletonStatus)
    // [Start, Stop]
}
```

In this example, `objectInstance` filters out `Send`, because every `Send` value must be constructed separately. Since `Start` and `Stop` are object declarations, the compiler returns their existing singleton instances.

Tip: If one of the direct subclasses is also sealed, traverse its `sealedSubclasses` recursively to collect the complete hierarchy.

Use case scenarios

Let's explore some practical scenarios where sealed classes and interfaces can be particularly useful.

State management in UI applications

You can use sealed classes to represent different UI states in an application. This approach allows for structured and safe handling of UI changes. This example demonstrates how to manage various UI states:

```
sealed class UIState {
    data object Loading : UIState()
    data class Success(val data: String) : UIState()
    data class Error(val exception: Exception) : UIState()
}

fun updateUI(state: UIState) {
    when (state) {
        is UIState.Loading -> showLoadingIndicator()
        is UIState.Success -> showData(state.data)
        is UIState.Error -> showError(state.exception)
    }
}
```

Payment method handling

In practical business applications, handling various payment methods efficiently is a common requirement. You can use sealed classes with `when` expressions to implement such business logic. By representing different payment methods as subclasses of a sealed class, it establishes a clear and manageable structure for processing transactions:

```
sealed class Payment {
    data class CreditCard(val number: String, val expiryDate: String) : Payment()
    data class PayPal(val email: String) : Payment()
    data object Cash : Payment()
}

fun processPayment(payment: Payment) {
    when (payment) {
        is Payment.CreditCard -> processCreditCardPayment(payment.number, payment.expiryDat
e)
        is Payment.PayPal -> processPayPalPayment(payment.email)
        is Payment.Cash -> processCashPayment()
```

```
    }
}
```

`Payment` is a sealed class that represents different payment methods in an e-commerce system: `CreditCard`, `PayPal`, and `Cash`. Each subclass can have its specific properties, like `number` and `expiryDate` for `CreditCard`, and `email` for `PayPal`.

The `processPayment()` function demonstrates how to handle different payment methods. This approach ensures that all possible payment types are considered, and the system remains flexible for new payment methods to be added in the future.

API request-response handling

You can use sealed classes and sealed interfaces to implement a user authentication system that handles API requests and responses. The user authentication system has login and logout functionalities. The `ApiRequest` sealed interface defines specific request types: `LoginRequest` for login, and `LogoutRequest` for logout operations. The sealed class, `ApiResponse`, encapsulates different response scenarios: `UserSuccess` with user data, `UserNotFound` for absent users, and `Error` for any failures. The `handleRequest` function processes these requests in a type-safe manner using a `when` expression, while `getUserById` simulates user retrieval:

```
// Import necessary modules
import io.ktor.server.application.*
import io.ktor.server.resources.*

import kotlinx.serialization.*

// Define the sealed interface for API requests using Ktor resources
@Resource("api")
sealed interface ApiRequest

@Serializable
@Resource("login")
data class LoginRequest(val username: String, val password: String) : ApiRequest

@Serializable
@Resource("logout")
object LogoutRequest : ApiRequest

// Define the ApiResponse sealed class with detailed response types
sealed class ApiResponse {
    data class UserSuccess(val user: UserData) : ApiResponse()
    data object UserNotFound : ApiResponse()
    data class Error(val message: String) : ApiResponse()
}

// User data class to be used in the success response
data class UserData(val userId: String, val name: String, val email: String)

// Function to validate user credentials (for demonstration purposes)
fun isValidUser(username: String, password: String): Boolean {
```

```
    // Some validation logic (this is just a placeholder)
    return username == "validUser" && password == "validPass"
}

// Function to handle API requests with detailed responses
fun handleRequest(request: ApiRequest): ApiResponse {
    return when (request) {
        is LoginRequest -> {
            if (isValidUser(request.username, request.password)) {
                ApiResponse.UserSuccess(UserData("userId", "userName", "userEmail"))
            } else {
                ApiResponse.Error("Invalid username or password")
            }
        }
        is LogoutRequest -> {
            // Assuming logout operation always succeeds for this example
            ApiResponse.UserSuccess(UserData("userId", "userName", "userEmail")) // For demo
nstration
        }
    }
}

// Function to simulate a getUserById call
fun getUserById(userId: String): ApiResponse {
    return if (userId == "validUserId") {
        ApiResponse.UserSuccess(UserData("validUserId", "John Doe", "john@example.com"))
    } else {
        ApiResponse.UserNotFound
    }
    // Error handling would also result in an Error response.
}

// Main function to demonstrate the usage
fun main() {
    val loginResponse = handleRequest(LoginRequest("user", "pass"))
    println(loginResponse)

    val logoutResponse = handleRequest(LogoutRequest)
    println(logoutResponse)

    val userResponse = getUserById("validUserId")
    println(userResponse)

    val userNotFoundResponse = getUserById("invalidId")
    println(userNotFoundResponse)
}
```

What's next

Learn more about inheritance in Kotlin.

Enum classes

Enum classes represent a fixed set of possible values. Use an enum class when a value can only be one of several predefined options, such as available states or modes.

Each value in an enum class is called an *enum constant*. Enum constants behave like singleton objects of the enum class type, so they can have properties, functions, and custom behavior.

Enum classes are the best fit when all possible values are known in advance and have the same structure. If you need to hold different data or have a different structure for each case, use sealed classes or interfaces.

Declare enum classes

To create an enum class, use the `enum` keyword and follow the usual class syntax with a body enclosed in curly braces. Inside the class body, list the enum constants separated by commas:

```
enum class Direction {
    NORTH, SOUTH, WEST, EAST
}
```

In this example, `Direction` is the enum class, and `NORTH`, `SOUTH`, `WEST`, and `EAST` are enum constants.

By convention, enum constants are usually written in uppercase because they represent constant values.

You can access an enum constant by using the enum class name followed by the constant name:

```
enum class Direction {
    NORTH, SOUTH, WEST, EAST
}

fun main() {
    // `Direction.NORTH` is an enum constant of type `Direction`.
    val direction: Direction = Direction.NORTH

    println(direction)
    // NORTH
}
```

Every enum class in Kotlin inherits from the `Enum<T>` base class, where `T` is the enum class itself. For example, the `Direction` enum class inherits from `Enum<Direction>`. This is why enum constants have built-in properties, such as `name` and `ordinal`.

Working with enum constants

Since enum constants are values, you can assign them to variables, print them, pass them to functions, compare them, and use them in `when` expressions.

Declare enum constants

To declare enum constants, first define properties in the enum class constructor and then pass values to each enum constant in parentheses. Unlike some other languages, Kotlin doesn't use assignment syntax, for example `RED = "#FF0000"`.

Enum constants can have associated values of any type. Strings and numbers are common examples, but you can also use other types, such as `Boolean`, another enum class, or a custom class.

Consider the `Color` enum class that stores a hexadecimal color code for each color:

```
enum class Color(val hex: String) {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF")
}
```

The value passed to each enum constant must match the constructor parameter type. Here, `hex` is a string property of the `Color` enum class. Each enum constant passes its own string value for this property.

You can also associate numeric values with enum constants. For example, declare the `Int` type in the constructor and provide an `Int` value for each enum constant:

```
enum class Priority(val level: Int) {
    LOW(0),
    MEDIUM(1),
    HIGH(2)
}
```

Access enum constants and their properties

You can access an enum constant through the enum class name. To access a property associated with an enum constant, use the dot notation, such as `color.hex` or `Color.GREEN.hex`:

```
enum class Color(val hex: String) {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF")
}

fun main() {
```

```
    val color: Color = Color.RED

    println(color)
    // RED
    println(color.hex)
    // #FF0000
    println(Color.GREEN.hex)
    // #00FF00
}
```

Here, `Color.RED` is an enum constant of the `Color` type. The color variable stores this enum constant.

Besides any properties you define, every enum constant also has the built-in `name` and `ordinal` properties for getting its name and position (starting from `0`) in the enum class declaration:

```
enum class RGB { RED, GREEN, BLUE }

fun main() {
    println(RGB.RED.name)
    // RED
    println(RGB.RED.ordinal)
    // 0
}
```

Pass enum constants to functions

Because enum constants are values, you can pass them to functions. This way, a function accepts only the fixed set of options defined in the enum class, ensuring type-safe code:

```
enum class Color(val hex: String) {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF")
}
fun printColor(color: Color) {
    println("Color: $color")
    println("Hex code: ${color.hex}")
}

fun main() {
    printColor(Color.BLUE)
    // Color: BLUE
    // Hex code: #0000FF
}
```

Here, the `printColor()` function accepts a value of type `Color`, so you can pass any `Color` enum constant to it.

Although enum constants behave like singleton objects, the compiler treats them as values of the enum class type. You can use the enum class name itself as a type, but you can't use enum

constants as enum types:

```
enum class Color {
    RED, GREEN, BLUE
}

fun printColor(color: Color) {
    println(color)
}

fun printRed(color: Color.RED) {
    println(color)
    // Error: enum entry cannot be used as a type
}
```

Use enum constants in `when` expressions

Enum classes work best with `when` expressions when you want to handle each constant separately:

```
enum class Color(val hex: String) {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF")
}
fun describeColor(color: Color): String {
    return when (color) {
        Color.RED -> "Red is a warm color"
        Color.GREEN -> "Green is a natural color"
        Color.BLUE -> "Blue is a cool color"
    }
}

fun main() {
    println(describeColor(Color.RED))
    // Red is a warm color
}
```

When you use all enum constants in a when expression, you don't need an `else` branch.

> **Tip:** To reduce repetition when working with enum entries, try context-sensitive resolution (currently in preview). This feature allows you to omit the enum class name when the expected type is known, such as in `when` expressions or when assigning to a typed variable.
>
> For more information, see Preview of context-sensitive resolution or the related KEEP proposal.

Find enum constants

Sometimes you need to get an enum constant from a string, an index, or one of its associated values. Kotlin provides built-in APIs to look up constants by name, position, or custom values.

For example, consider an enum class where each color has an associated RGB value. To find an enum constant by its name, use the `valueOf()` function:

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF)
}

fun main() {
    val color = Color.valueOf("RED")

    println(color)
    // RED
}
```

The name passed to `valueOf()` must match the enum constant name exactly. If there is no enum constant with the specified name, `valueOf()` throws an `IllegalArgumentException`.

To find an enum constant by its position in the enum declaration, use the `getOrNull()` function on the enum's `entries` property:

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF)
}
fun main() {
    val color = Color.entries.getOrNull(0)

    println(color)
    // RED
}
```

Enum positions start from `0`. In this example, `RED` has the position `0`, `GREEN` is at `1`, and `BLUE` is at `2`.

This is useful when you have an integer, for example from a file or user input, that represents an enum constant position. Unlike some other languages, Kotlin doesn't let you cast an `Int` directly to an enum constant. Instead, use the integer as an index and look up the constant with `entries.getOrNull(index)`.

If the integer represents a value that should remain stable even if you reorder the enum constants, define an explicit numeric property, for example, `rgb` or `code` and search for the constant with the

matching value.

Since `entries` is a specialized `List`, you can use standard collection APIs with it. For example, to find an enum constant by an associated value, search through entries using `first()`:

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF)
}
fun main() {
    val color = Color.entries.first { it.rgb == 0xFF0000 }

    println(color)
    // RED
}
```

The `first()` function throws a `NoSuchElementException` if no matching constant is found. To get `null` instead, use `firstOrNull()`.

To get the number of enum constants, use the `size` property. For example:

```
enum class RGB { RED, GREEN, BLUE }

fun main() {
    println(RGB.entries)
    // [RED, GREEN, BLUE]
    println(RGB.entries.size)
    // 3
    println("The first color is: ${RGB.valueOf("RED")}")
    // "The first color is: RED"
}
```

If you often need to look up enum constants by name, position, or associated value, add helper functions in a companion object:

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF);

    companion object {
        fun fromName(name: String): Color? =
            entries.find { it.name == name }

        fun fromPosition(position: Int): Color? =
            entries.getOrNull(position)

        fun fromRgb(rgb: Int): Color? =
            entries.find { it.rgb == rgb }
    }
}
```

```
fun main() {
    println(Color.fromName("RED"))
    // RED
    println(Color.fromPosition(1))
    // GREEN
    println(Color.fromRgb(0x0000FF))
    // BLUE
    println(Color.fromRgb(0xABCDEF))
    // null
}
```

Companion object helper functions are useful when you want safe lookups that return `null` instead of throwing an exception.

The lookup APIs used above, such as `entries` and `valueOf()`, are examples of *synthetic* members. In this context, synthetic means that Kotlin provides these members automatically, even though you don't declare them yourself. This is why every enum class can list its constants with the `entries` property and get a constant by name with the `valueOf()` function without the need to write extra code.

You can access the constants in an enum class using generic helper functions such as `enumEntries<T>()` and `enumValueOf<T>()`. These functions use reified type parameters. Such parameters keep the actual enum type available inside a generic inline function, so the helper functions can work with the enum type `T` directly:

Function	Description
`enumEntries<T>()`	(Recommended) Returns all enum entries of the enum type `T`. Every call returns the same list.
`enumValues<T>()`	Returns an array with all enum entries of the enum type `T`. Every call `enumValues<T>()` creates a new array.
`enumValueOf<T>()`	Returns a single enum entry by its name, throwing an `IllegalArgumentException` if no enum entry matches.

For example:

```
import kotlin.enums.enumEntries

enum class RGB { RED, GREEN, BLUE }

inline fun <reified T : Enum<T>> printAllValues() {
    println(enumEntries<T>().joinToString { it.name })
}

inline fun <reified T : Enum<T>> findByName(name: String): T = enumValueOf<T>(name)

fun main() {
    printAllValues<RGB>()
```

```
    // RED, GREEN, BLUE
    println(findByName<RGB>("GREEN"))
    // GREEN
}
```

For more information about inline functions and reified type parameters, see Inline functions.

Compare and sort enum constants

Use the `==` structural equality operator to compare enum constants:

```
enum class Color(val hex: String) {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF")
}
fun main() {
    val color = Color.RED

    println(color == Color.RED)
    // true
    println(color == Color.BLUE)
    // false
}
```

Since each enum constant behaves like a singleton object, comparing enum constants checks whether both values refer to the same constant.

All enum classes implement the `Comparable` interface by default, so you can compare and sort enum constants. Constants are ordered by their position in the enum declaration (their `ordinal` value), which means the constant declared first is considered the smallest:

```
enum class Priority {
    LOW, MEDIUM, HIGH
}

fun main() {
    println(Priority.LOW < Priority.HIGH)
    // true
    println(Priority.HIGH > Priority.MEDIUM)
    // true
}
```

Sorting follows the same declaration order. For example, `entries.sorted()` returns the constants in the order in which they are declared, regardless of their names:

```
enum class Priority {
    HIGH, LOW, MEDIUM
}

fun main() {
```

```
    println(Priority.entries.sorted())
    // [HIGH, LOW, MEDIUM]
}
```

The `Enum<T>` class provides the `compareTo()`, `equals()`, and `hashCode()` functions, and you can't override them to customize their behavior like in normal classes. Comparison always follows the declaration order.

If you need a different order, don't rely on the declaration order. Instead, define an explicit property and sort by it. For example, sort colors by their brightness:

```
enum class Color(val brightness: Int) {
    RED(1),
    GREEN(3),
    BLUE(2)
}

fun main() {
    println(Color.entries.sortedBy { it.brightness })
    // [RED, BLUE, GREEN]
}
```

For more information, see Ordering.

Add functions to enum classes

Just like properties, enum classes can have functions. You can add functions that are shared by all enum constants, combine them with properties, or define operator functions.

Add functions shared by all constants

To add behavior that every enum constant shares, define a function in the enum class body. If the enum class defines any members, separate the constant definitions from the member definitions with a semicolon:

```
enum class Direction {
    NORTH, SOUTH, WEST, EAST;

    fun isVertical(): Boolean = this == NORTH || this == SOUTH
}

fun main() {
    println(Direction.NORTH.isVertical())
    // true
    println(Direction.EAST.isVertical())
    // false
}
```

Every enum constant can call the shared function. Inside the function, `this` refers to the enum constant it's called on.

You can combine constructor properties with functions to associate data with each constant and add behavior that uses that data:

```
enum class Color(val hex: String) {
    RED("#FF0000"),
    GREEN("#00FF00"),
    BLUE("#0000FF");

    fun describe(): String = "$name has hex code $hex"
}

fun main() {
    println(Color.RED.describe())
    // RED has hex code #FF0000
}
```

Here, each constant stores its own `hex` value, and the shared `describe()` function uses both the built-in `name` property and the `hex` property.

Add operator functions

Enum classes can also define operator functions, so you can use enum constants with operators. For example, define the `not()` operator function to return the opposite direction with the `!` operator:

```
enum class Direction {
    NORTH, SOUTH, WEST, EAST;

    operator fun not(): Direction = when (this) {
        NORTH -> SOUTH
        SOUTH -> NORTH
        WEST -> EAST
        EAST -> WEST
    }
}

fun main() {
    println(!Direction.NORTH)
    // SOUTH
}
```

Use anonymous classes

Enum constants can declare their own anonymous classes with their corresponding functions, as well as with overriding base functions. With anonymous classes, you write the class body directly after the enum constant name and Kotlin infers the enum class as the supertype.

This is useful when you declare an abstract function in the enum class and require each constant to provide its own implementation. Each constant overrides the abstract function inside its own anonymous class:

```
enum class ProtocolState {
    WAITING {
        override fun signal() = TALKING
    },

    TALKING {
        override fun signal() = WAITING
    };

    abstract fun signal(): ProtocolState
}

fun main() {
    var state = ProtocolState.WAITING

    println(state)
    // WAITING
    state = state.signal()
    println(state)
    // TALKING
}
```

Here, each constant implements the abstract `signal()` function differently, so calling `signal()` returns a different next state depending on the constant.

Although enum constants behave like singleton objects, the type of an enum constant is the enum class itself, not its own anonymous class. That's why you can't access members declared inside the body of an anonymous class:

```
enum class ProtocolState {
    WAITING {
        val waitingMessage = "Waiting for a signal"
        override fun signal() = TALKING
    },

    TALKING {
        override fun signal() = WAITING
    };

    abstract fun signal(): ProtocolState
}

fun main() {
    println(ProtocolState.WAITING.waitingMessage)
    // Error: unresolved reference 'waitingMessage'
}
```

To expose data or behavior for every constant, declare it in the enum class body, using an abstract member when each constant needs its own implementation.

Implement interfaces in enum classes

An enum class can implement an interface, but it cannot inherit from a class. You can provide a common implementation of the interface members for all enum constants or let each constant provide its own implementation in an anonymous class.

To implement an interface, add it to the enum class declaration:

```
import java.util.function.BinaryOperator
import java.util.function.IntBinaryOperator
enum class IntArithmetics : BinaryOperator<Int>, IntBinaryOperator {
    PLUS {
        override fun apply(t: Int, u: Int): Int = t + u
    },
    TIMES {
        override fun apply(t: Int, u: Int): Int = t * u
    };

    override fun applyAsInt(t: Int, u: Int) = apply(t, u)
}
fun main() {
    val a = 13
    val b = 31
    for (f in IntArithmetics.entries) {
        println("$f($a, $b) = ${f.apply(a, b)}")
    }
}
```

In this example, the `IntArithmetics` enum class implements two interfaces in the enum class declaration: `BinaryOperator<Int>` and `IntBinaryOperator`. Each constant can override interface members inside its own anonymous class body, as `PLUS` and `TIMES` do for `apply()`, while `applyAsInt()` provides a shared implementation for all constants.

Inline value classes

Sometimes it is useful to wrap a value in a class to create a more domain-specific type. However, it introduces runtime overhead due to additional heap allocations. Moreover, if the wrapped type is primitive, the performance hit is significant, because primitive types are usually heavily optimized by the runtime, while their wrappers don't get any special treatment.

To solve such issues, Kotlin introduces a special kind of class called an *inline class*. Inline classes are a subset of value-based classes. They don't have an identity and can only hold values.

To declare an inline class, use the `value` modifier before the name of the class:

```
value class Password(private val s: String)
```

To declare an inline class for the JVM backend, use the `value` modifier along with the `@JvmInline` annotation before the class declaration:

```
// For JVM backends
@JvmInline
value class Password(private val s: String)
```

An inline class must have a single property initialized in the primary constructor. At runtime, instances of the inline class will be represented using this single property (see details about runtime representation below):

```
// No actual instantiation of class 'Password' happens
// At runtime 'securePassword' contains just 'String'
val securePassword = Password("Don't try this in production")
```

This is the main feature of inline classes, which inspired the name *inline*: data of the class is *inlined* into its usages (similar to how the content of inline functions is inlined to call sites).

Members

Inline classes support some functionality of regular classes. In particular, they are allowed to declare properties and functions, have an `init` block and secondary constructors:

```
@JvmInline
value class Person(private val fullName: String) {
    init {
        require(fullName.isNotEmpty()) {
            "Full name shouldn't be empty"
        }
    }
```

```
    constructor(firstName: String, lastName: String) : this("$firstName $lastName") {
        require(lastName.isNotBlank()) {
            "Last name shouldn't be empty"
        }
    }

    val length: Int
        get() = fullName.length

    fun greet() {
        println("Hello, $fullName")
    }
}

fun main() {
    val name1 = Person("Kotlin", "Mascot")
    val name2 = Person("Kodee")
    name1.greet() // the `greet()` function is called as a static method
    println(name2.length) // property getter is called as a static method
}
```

Inline class properties cannot have backing fields. They can only have simple computable properties (no `lateinit`/delegated properties).

Inheritance

Inline classes are allowed to inherit from interfaces:

```
interface Printable {
    fun prettyPrint(): String
}

@JvmInline
value class Name(val s: String) : Printable {
    override fun prettyPrint(): String = "Let's $s!"
}

fun main() {
    val name = Name("Kotlin")
    println(name.prettyPrint()) // Still called as a static method
}
```

It is forbidden for inline classes to participate in a class hierarchy. This means that inline classes cannot extend other classes and are always `final`.

Representation

In generated code, the Kotlin compiler keeps a *wrapper* for each inline class. Inline class instances can be represented at runtime either as wrappers or as the underlying type. This is similar to how

`Int` can be represented either as a primitive `int` or as the wrapper `Integer`.

The Kotlin compiler will prefer using underlying types instead of wrappers to produce the most performant and optimized code. However, sometimes it is necessary to keep wrappers around. As a rule of thumb, inline classes are boxed whenever they are used as another type.

```
interface I

@JvmInline
value class Foo(val i: Int) : I

fun asInline(f: Foo) {}
fun <T> asGeneric(x: T) {}
fun asInterface(i: I) {}
fun asNullable(i: Foo?) {}

fun <T> id(x: T): T = x

fun main() {
    val f = Foo(42)

    asInline(f)    // unboxed: used as Foo itself
    asGeneric(f)   // boxed: used as generic type T
    asInterface(f) // boxed: used as type I
    asNullable(f)  // boxed: used as Foo?, which is different from Foo

    // below, 'f' first is boxed (while being passed to 'id') and then unboxed (when returne
d from 'id')
    // In the end, 'c' contains unboxed representation (just '42'), as 'f'
    val c = id(f)
}
```

Because inline classes may be represented both as the underlying value and as a wrapper, referential equality is pointless for them and is therefore prohibited.

Inline classes can also have a generic type parameter as the underlying type. In this case, the compiler maps it to `Any?` or, generally, to the upper bound of the type parameter.

```
@JvmInline
value class UserId<T>(val value: T)

fun compute(s: UserId<String>) {} // compiler generates fun compute-<hashcode>(s: Any?)
```

Mangling

Since inline classes are compiled to their underlying type, it may lead to various obscure errors, for example, unexpected platform signature clashes:

```
@JvmInline
value class UInt(val x: Int)
```

```
// Represented as 'public final void compute(int x)' on the JVM
fun compute(x: Int) { }

// Also represented as 'public final void compute(int x)' on the JVM!
fun compute(x: UInt) { }
```

To mitigate such issues, functions using inline classes are *mangled* by adding some stable hashcode to the function name. Therefore, `fun compute(x: UInt)` will be represented as `public final void compute-<hashcode>(int x)`, which solves the clash problem.

Calling from Java code

You can call functions that accept inline classes from Java code. To do so, you should manually disable mangling: add the `@JvmName` annotation before the function declaration:

```
@JvmInline
value class UInt(val x: Int)

fun compute(x: Int) { }

@JvmName("computeUInt")
fun compute(x: UInt) { }
```

By default, Kotlin compiles inline classes using **unboxed representations**, which makes them difficult to access from Java. To learn how to compile inline classes into **boxed representations** that are accessible from Java, see the guide to Calling Kotlin from Java.

Inline classes vs type aliases

At first sight, inline classes seem very similar to type aliases. Indeed, both seem to introduce a new type and both will be represented as the underlying type at runtime.

However, the crucial difference is that type aliases are *assignment-compatible* with their underlying type (and with other type aliases with the same underlying type), while inline classes are not.

In other words, inline classes introduce a truly *new* type, contrary to type aliases which only introduce an alternative name (alias) for an existing type:

```
typealias NameTypeAlias = String

@JvmInline
value class NameInlineClass(val s: String)

fun acceptString(s: String) {}
fun acceptNameTypeAlias(n: NameTypeAlias) {}
fun acceptNameInlineClass(p: NameInlineClass) {}
```

```
fun main() {
    val nameAlias: NameTypeAlias = ""
    val nameInlineClass: NameInlineClass = NameInlineClass("")
    val string: String = ""

    acceptString(nameAlias) // OK: pass alias instead of underlying type
    acceptString(nameInlineClass) // Not OK: can't pass inline class instead of underlying t
ype

    // And vice versa:
    acceptNameTypeAlias(string) // OK: pass underlying type instead of alias
    acceptNameInlineClass(string) // Not OK: can't pass underlying type instead of inline cl
ass
}
```

Inline classes and delegation

Implementation by delegation to inlined value of inlined class is allowed with interfaces:

```
interface MyInterface {
    fun bar()
    fun foo() = "foo"
}

@JvmInline
value class MyInterfaceWrapper(val myInterface: MyInterface) : MyInterface by myInterface

fun main() {
    val my = MyInterfaceWrapper(object : MyInterface {
        override fun bar() {
            // body
        }
    })
    println(my.foo()) // prints "foo"
}
```

Nested and inner classes

Classes can be nested in other classes:

```
class Outer {
    private val bar: Int = 1
    class Nested {
        fun foo() = 2
    }
}

val demo = Outer.Nested().foo() // == 2
```

You can also use interfaces with nesting. All combinations of classes and interfaces are possible: You can nest interfaces in classes, classes in interfaces, and interfaces in interfaces.

```
interface OuterInterface {
    class InnerClass
    interface InnerInterface
}

class OuterClass {
    class InnerClass
    interface InnerInterface
}
```

Inner classes

A nested class marked as `inner` can access the members of its outer class. Inner classes carry a reference to an object of an outer class:

```
class Outer {
    private val bar: Int = 1
    inner class Inner {
        fun foo() = bar
    }
}

val demo = Outer().Inner().foo() // == 1
```

See Qualified `this` expressions to learn about disambiguation of `this` in inner classes.

Anonymous inner classes

Anonymous inner class instances are created using an object expression:

```
window.addMouseListener(object : MouseAdapter() {

    override fun mouseClicked(e: MouseEvent) { ... }

    override fun mouseEntered(e: MouseEvent) { ... }
})
```

Note: On the JVM, if the object is an instance of a functional Java interface (that means a Java interface with a single abstract method), you can create it using a lambda expression prefixed with the type of the interface:

```
val listener = ActionListener { println("clicked") }
```

Functional (SAM) interfaces

An interface with only one abstract member function is called a *functional interface*, or a *Single Abstract Method (SAM) interface*. The functional interface can have several non-abstract member functions but only one abstract member function.

To declare a functional interface in Kotlin, use the `fun` modifier.

```
fun interface KRunnable {
    fun invoke()
}
```

SAM conversions

For functional interfaces, you can use SAM conversions that help make your code more concise and readable by using lambda expressions.

Instead of creating a class that implements a functional interface manually, you can use a lambda expression. With a SAM conversion, Kotlin can convert any lambda expression whose signature matches the signature of the interface's single method into the code, which dynamically instantiates the interface implementation.

For example, consider the following Kotlin functional interface:

```
fun interface IntPredicate {
    fun accept(i: Int): Boolean
}
```

If you don't use a SAM conversion, you will need to write code like this:

```
// Creating an instance of a class
val isEven = object : IntPredicate {
    override fun accept(i: Int): Boolean {
        return i % 2 == 0
    }
}
```

By leveraging Kotlin's SAM conversion, you can write the following equivalent code instead:

```
// Creating an instance using lambda
val isEven = IntPredicate { it % 2 == 0 }
```

A short lambda expression replaces all the unnecessary code.

```
fun interface IntPredicate {
    fun accept(i: Int): Boolean
}

val isEven = IntPredicate { it % 2 == 0 }

fun main() {
    println("Is 7 even? - ${isEven.accept(7)}")
}
```

You can also use SAM conversions for Java interfaces.

Pass callable references to SAM parameters

If the required behavior already exists as a named function, you can pass a callable reference. Kotlin adapts a compatible callable reference to the expected functional interface:

```
fun interface Action {
    fun run()
}

fun execute(action: Action) {
    action.run()
}

fun start() {
    println("Run from a callable reference")
}

fun main() {
    // Kotlin converts the lambda to an Action instance
    execute { start() }
    // Run from a callable reference

    // Kotlin adapts the reference to an Action instance
    execute(::start)
    // Run from a callable reference
}
```

The lambda in execute `{ start() }` is a function literal, so Kotlin uses SAM conversion to create an `Action` instance. In `execute(::start)`, `::start` is a callable reference to the existing `start()` function. Because `execute()` expects an `Action`, the compiler creates a compatible `Action` instance that uses `start()` as the implementation of `Action.run()`. This process is called *SAM adaptation*.

Migration from an interface with constructor function to a functional interface

Starting from 1.6.20, Kotlin supports callable references to functional interface constructors, which adds a source-compatible way to migrate from an interface with a constructor function to a

functional interface. Consider the following code:

```
interface Printer {
    fun print()
}

fun Printer(block: () -> Unit): Printer = object : Printer {
    override fun print() = block()
}
```

With callable references to functional interface constructors enabled, this code can be replaced with just a functional interface declaration:

```
fun interface Printer {
    fun print()
}
```

Its constructor will be created implicitly, and any code using the `::Printer` function reference will compile. For example:

```
documentsStorage.addPrinter(::Printer)
```

Preserve the binary compatibility by marking the legacy function `Printer` with the `@Deprecated` annotation with `DeprecationLevel.HIDDEN`:

```
@Deprecated(message = "Your message about the deprecation", level = DeprecationLevel.HIDDEN)
fun Printer(...) {...}
```

Functional interfaces vs. type aliases

You can also simply rewrite the above using a type alias for a functional type:

```
typealias IntPredicate = (i: Int) -> Boolean

val isEven: IntPredicate = { it % 2 == 0 }

fun main() {
    println("Is 7 even? - ${isEven(7)}")
}
```

However, functional interfaces and type aliases serve different purposes. Type aliases are just names for existing types – they don't create a new type, while functional interfaces do. You can provide extensions that are specific to a particular functional interface to be inapplicable for plain functions or their type aliases.

Type aliases can have only one member, while functional interfaces can have multiple non-abstract member functions and one abstract member function. Functional interfaces can also

implement and extend other interfaces.

Functional interfaces are more flexible and provide more capabilities than type aliases, but they can be more costly both syntactically and at runtime because they can require conversions to a specific interface. When you choose which one to use in your code, consider your needs:

- If your API needs to accept a function (any function) with some specific parameter and return types – use a simple functional type or define a type alias to give a shorter name to the corresponding functional type.
- If your API accepts a more complex entity than a function – for example, it has non-trivial contracts and/or operations on it that can't be expressed in a functional type's signature – declare a separate functional interface for it.

Properties

In Kotlin, properties let you store and manage data without writing functions to access or change the data. You can use properties in classes, interfaces, objects, companion objects, and even outside these structures as top-level properties.

Every property has a name, a type, and an automatically generated `get()` function called a getter. You can use the getter to read the property's value. If the property is mutable, it also has a `set()` function called a setter, which allows you to change the property's value.

> **Tip:** Getters and setters are called *accessors*.

Declaring properties

Properties can be mutable (`var`) or read-only (`val`). You can declare them as a top-level property in a `.kt` file. Think of a top-level property as a global variable that belongs to a package:

```
// File: Constants.kt
package my.app

val pi = 3.14159
var counter = 0
```

You can also declare properties inside a class, interface, or object:

```
// Class with properties
class Address {
    var name: String = "Holmes, Sherlock"
    var street: String = "Baker"
    var city: String = "London"
}

// Interface with a property
interface ContactInfo {
    val email: String
}

// Object with properties
object Company {
    var name: String = "Detective Inc."
    val country: String = "UK"
}

// Class implementing the interface
class PersonContact : ContactInfo {
    override val email: String = "sherlock@example.com"
}
```

To use a property, refer to it by its name:

```
class Address {
    var name: String = "Holmes, Sherlock"
    var street: String = "Baker"
    var city: String = "London"
}

interface ContactInfo {
    val email: String
}

object Company {
    var name: String = "Detective Inc."
    val country: String = "UK"
}

class PersonContact : ContactInfo {
    override val email: String = "sherlock@example.com"
}
fun copyAddress(address: Address): Address {
    val result = Address()
    // Accesses properties in the result instance
    result.name = address.name
    result.street = address.street
    result.city = address.city
    return result
}

fun main() {
    val sherlockAddress = Address()
    val copy = copyAddress(sherlockAddress)
    // Accesses properties in the copy instance
    println("Copied address: ${copy.name}, ${copy.street}, ${copy.city}")
    // Copied address: Holmes, Sherlock, Baker, London

    // Accesses properties in the Company object
    println("Company: ${Company.name} in ${Company.country}")
    // Company: Detective Inc. in UK

    val contact = PersonContact()
    // Access properties in the contact instance
    println("Email: ${contact.email}")
    // Email: sherlock@email.com
}
```

In Kotlin, we recommend initializing properties when you declare them to keep your code safe and easy to read. However, you can initialize them later in special cases.

Declaring the property type is optional if the compiler can infer it from the initializer or the getter's return type:

```
var initialized = 1 // The inferred type is Int
```

```
var allByDefault    // ERROR: Property must be initialized.
```

Custom getters and setters

By default, Kotlin automatically generates getters and setters. You can define your own custom accessors when you need extra logic, such as validation, formatting, or calculations based on other properties.

A custom getter runs every time the property is accessed:

```
class Rectangle(val width: Int, val height: Int) {
    val area: Int
        get() = this.width * this.height
}
fun main() {
    val rectangle = Rectangle(3, 4)
    println("Width=${rectangle.width}, height=${rectangle.height}, area=${rectangle.area}")
}
```

You can omit the type if the compiler can infer it from the getter:

```
val area get() = this.width * this.height
```

A custom setter runs every time you assign a value to the property, except during initialization. By convention, the name of the setter parameter is `value`, but you can choose a different name:

```
class Point(var x: Int, var y: Int) {
    var coordinates: String
        get() = "$x,$y"
        set(value) {
            val parts = value.split(",")
            x = parts[0].toInt()
            y = parts[1].toInt()
        }
}

fun main() {
    val location = Point(1, 2)
    println(location.coordinates)
    // 1,2

    location.coordinates = "10,20"
    println("${location.x}, ${location.y}")
    // 10, 20
}
```

Changing visibility or adding annotations

In Kotlin, you can change accessor visibility or add annotations without replacing the default implementation. You don't have to make these changes within a body `{}`.

To change the visibility of an accessor, use the modifier before the `get` or `set` keyword:

```
class BankAccount(initialBalance: Int) {
    var balance: Int = initialBalance
        // Only the class can modify the balance
        private set

    fun deposit(amount: Int) {
        if (amount > 0) balance += amount
    }

    fun withdraw(amount: Int) {
        if (amount > 0 && amount <= balance) balance -= amount
    }
}

fun main() {
    val account = BankAccount(100)
    println("Initial balance: ${account.balance}")
    // 100

    account.deposit(50)
    println("After deposit: ${account.balance}")
    // 150

    account.withdraw(70)
    println("After withdrawal: ${account.balance}")
    // 80

    // account.balance = 1000
    // Error: cannot assign because setter is private
}
```

To annotate an accessor, use the annotation before the `get` or `set` keyword:

```
// Defines an annotation that can be applied to a getter
@Target(AnnotationTarget.PROPERTY_GETTER)
annotation class Inject

class Service {
    var dependency: String = "Default Service"
        // Annotates the getter
        @Inject get
}

fun main() {
    val service = Service()
    println(service.dependency)
    // Default service
```

```
    println(service::dependency.getter.annotations)
    // [@Inject()]
    println(service::dependency.setter.annotations)
    // []
}
```

This example uses reflection to show which annotations are present on the getter and setter.

Backing fields

The compiler automatically generates backing fields for properties when a value needs to be stored in memory.

For example, the compiler creates a backing field when you use the default `get()` and `set()` functions because they read and write the stored value:

```
var count = 0
```

You can access backing fields by using the `field` keyword in a custom `get()` or `set()` function. For example, you can add extra logic to a getter or setter, or trigger an additional action when a property changes.

In this example, the `score` property uses the backing field inside the `set()` function so that updating the value also triggers a log event:

```
class Scoreboard {
    var score: Int = 0
        set(value) {
            field = value
            // Adds logging when updating the value
            println("Score updated to $field")
        }
}

fun main() {
    val board = Scoreboard()
    board.score = 10
    // Score updated to 10
    board.score = 20
    // Score updated to 20
}
```

Backing fields aren't created by default for all properties because they might not need them. For example, the `isEmpty` property doesn't have a backing field because the value is calculated from the `size` property each time you access it:

```
val isEmpty: Boolean
    get() = this.size == 0
```

Explicit backing fields

Sometimes you might need more flexibility. For example, if you have an API where you want to be able to modify the property internally but not externally. In such cases, you can use an *explicit backing field*.

In the following example, the `ShoppingCart` class has an `items` property that represents everything in the shopping cart. The class exposes the `items` property as a read-only list of strings, but internally it stores the data in a mutable list with an explicit backing field:

```
class ShoppingCart {
    // Public read-only view with explicit backing field
    val items: List<String>
        field = mutableListOf()

    fun addItem(item: String) {
        items.add(item)
    }

    fun removeItem(item: String) {
        items.remove(item)
    }
}

fun main() {
    val cart = ShoppingCart()
    cart.addItem("Apple")
    cart.addItem("Banana")

    println(cart.items)
    // [Apple, Banana]

    cart.removeItem("Apple")
    println(cart.items)
    // [Banana]
}
```

In this example, the compiler infers the type of the backing field from the `mutableListOf()` call: `MutableList<String>`. You can also declare the type of the backing field explicitly:

```
val items: List<String>
    // Explicit backing field with explicit type
    field: MutableList<String> = mutableListOf()
```

In the example of the `ShoppingCart` class, the compiler smart casts the `items` property to the `MutableList<String>` type, so the class can add and remove items from the cart through the `add()` and `remove()` functions. Outside the class, the compiler uses the public property type `List<String>`, so API users can only read what's in the `items` list.

Limitations

To use explicit backing fields, their properties and the backing fields themselves must follow certain rules. Properties can have explicit backing fields only if they:

- Don't have a custom getter.
- Are read-only (`val`).
- Aren't `open`.
- Aren't a delegated property.
- Aren't compile-time constants.

In addition, the backing field type must be a subtype of the property's type and have `private` visibility.

You can work around these restrictions by using backing properties instead.

Backing properties

If explicit backing fields don't fit your use case, you can try using a coding pattern called a *backing property*.

For example, if your property needs a custom getter:

```
class UserDirectory {
    private val _users = mutableListOf(
        "sarah",
        "mike",
        "emma"
    )

    val users: List<String>
        get() = _users.sorted()

    fun addUser(username: String) {
        _users.add(username)
    }
}

fun main() {
    val directory = UserDirectory()

    directory.addUser("alex")
    println(directory.users)
    // [alex, emma, mike, sarah]
}
```

> **Tip:** Use a leading underscore when naming backing properties to follow Kotlin coding conventions.

In this example, the `UserDirectory` class has a read-only `users` property that lists every user in the directory. The `_users` variable is the private backing property containing the real list. The getter for the public `users` property sorts the entries before returning them.

Compile-time constants

If the value of a read-only property is known at compile time, mark it as a *compile-time constant* using the `const` modifier. Compile-time constants are inlined at compile time, so each reference is replaced with its actual value. They are accessed more efficiently because no getter is called:

```
// File: AppConfig.kt
package com.example

// Compile-time constant
const val MAX_LOGIN_ATTEMPTS = 3
```

Compile-time constants must meet the following requirements:

- They must be either a top-level property, or a member of an `object` declaration or a companion object.
- They must be initialized with a value of type `String` or a primitive type.
- They can't have a custom getter.

Compile-time constants still have a backing field, so you can interact with them using reflection.

You can also use these properties in annotations:

```
const val SUBSYSTEM_DEPRECATED: String = "This subsystem is deprecated"

@Deprecated(SUBSYSTEM_DEPRECATED) fun processLegacyOrders() { ... }
```

Late-initialized properties and variables

Normally, you must initialize properties in the constructor. However, this isn't always convenient. For example, you might initialize properties through dependency injection or inside the setup method of a unit test.

To handle these situations, mark the property with the `lateinit` modifier:

```
public class OrderServiceTest {
    lateinit var orderService: OrderService

    @SetUp fun setup() {
        orderService = OrderService()
    }
```

```
    @Test fun processesOrderSuccessfully() {
        // Calls orderService directly without checking for null
        // or initialization
        orderService.processOrder()
    }
}
```

You can use the `lateinit` modifier on `var` properties declared as:

- Top-level properties.
- Local variables.
- Properties inside the body of a class.

For class properties:

- You can't declare them in the primary constructor.
- They must not have a custom getter or setter.

In all cases, the property or variable must be non-nullable and must not be a primitive type.

If you access a `lateinit` property before initializing it, Kotlin throws a specific exception that identifies the uninitialized property being accessed:

```
class ReportGenerator {
    lateinit var report: String

    fun printReport() {
        // Throws an exception as it's accessed before
        // initialization
        println(report)
    }
}

fun main() {
    val generator = ReportGenerator()
    generator.printReport()
    // Exception in thread "main" kotlin.UninitializedPropertyAccessException: lateinit prop
erty report has not been initialized
}
```

To check whether a `lateinit var` has already been initialized, use the `isInitialized` property on the reference to that property:

```
class WeatherStation {
    lateinit var latestReading: String

    fun printReading() {
        // Checks whether the property is initialized
        if (this::latestReading.isInitialized) {
            println("Latest reading: $latestReading")
        } else {
            println("No reading available")
```

```
        }
    }
}

fun main() {
    val station = WeatherStation()

    station.printReading()
    // No reading available
    station.latestReading = "22°C, sunny"
    station.printReading()
    // Latest reading: 22°C, sunny
}
```

You can only use `isInitialized` on a property if you can already access that property in your code. The property must be declared in the same class, in an outer class, or as a top-level property in the same file.

Overriding properties

See Overriding properties.

Delegated properties

To reuse logic and reduce code duplication, you can delegate the responsibility of getting and setting a property to a separate object.

Delegating accessor behavior keeps the property's accessor logic centralized, making it easier to reuse. This approach is useful when implementing behaviors like:

- Computing a value lazily.
- Reading from a map by a given key.
- Accessing a database.
- Notifying a listener when a property is accessed.

You can implement these common behaviors in libraries yourself or use existing delegates provided by external libraries. For more information, see delegated properties.

Delegated properties

With some common kinds of properties, even though you can implement them manually every time you need them, it is more helpful to implement them once, add them to a library, and reuse them later. For example:

- *Lazy* properties: the value is computed only on first access.
- *Observable* properties: listeners are notified about changes to this property.
- Storing properties in a *map* instead of a separate field for each property.

To cover these (and other) cases, Kotlin supports *delegated properties*:

```
class Example {
    var p: String by Delegate()
}
```

The syntax is: `val/var <property name>: <Type> by <expression>`. The expression after `by` is a *delegate*, because the `get()` (and `set()`) that correspond to the property will be delegated to its `getValue()` and `setValue()` methods. Property delegates don't have to implement an interface, but they have to provide a `getValue()` function (and `setValue()` for `var`s).

For example:

```
import kotlin.reflect.KProperty

class Delegate {
    operator fun getValue(thisRef: Any?, property: KProperty<*>): String {
        return "$thisRef, thank you for delegating '${property.name}' to me!"
    }

    operator fun setValue(thisRef: Any?, property: KProperty<*>, value: String) {
        println("$value has been assigned to '${property.name}' in $thisRef.")
    }
}
```

When you read from `p`, which delegates to an instance of `Delegate`, the `getValue()` function from `Delegate` is called. Its first parameter is the object you read `p` from, and the second parameter holds a description of `p` itself (for example, you can take its name).

```
val e = Example()
println(e.p)
```

This prints:

```
Example@33a17727, thank you for delegating 'p' to me!
```

Similarly, when you assign to `p`, the `setValue()` function is called. The first two parameters are the same, and the third holds the value being assigned:

```
e.p = "NEW"
```

This prints:

```
NEW has been assigned to 'p' in Example@33a17727.
```

The specification of the requirements for the delegated object can be found below.

You can declare a delegated property inside a function or code block; it doesn't have to be a member of a class. Below you can find an example.

Standard delegates

The Kotlin standard library provides factory methods for several useful kinds of delegates.

Lazy properties

`lazy()` is a function that takes a lambda and returns an instance of `Lazy<T>`, which can serve as a delegate for implementing a lazy property. The first call to `get()` executes the lambda passed to `lazy()` and remembers the result. Subsequent calls to `get()` simply return the remembered result.

```
val lazyValue: String by lazy {
    println("computed!")
    "Hello"
}

fun main() {
    println(lazyValue)
    println(lazyValue)
}
```

By default, the evaluation of lazy properties is *synchronized*: the value is computed only in one thread, but all threads will see the same value. If the synchronization of the initialization delegate is not required to allow multiple threads to execute it simultaneously, pass `LazyThreadSafetyMode.PUBLICATION` as a parameter to `lazy()`.

If you're sure that the initialization will always happen in the same thread as the one where you use the property, you can use `LazyThreadSafetyMode.NONE`. It doesn't incur any thread-safety guarantees and related overhead.

Observable properties

`Delegates.observable()` takes two arguments: the initial value and a handler for modifications.

The handler is called every time you assign to the property (*after* the assignment has been performed). It has three parameters: the property being assigned to, the old value, and the new value:

```
import kotlin.properties.Delegates

class User {
    var name: String by Delegates.observable("<no name>") {
        prop, old, new ->
        println("$old -> $new")
    }
}

fun main() {
    val user = User()
    user.name = "first"
    user.name = "second"
}
```

If you want to intercept assignments and *veto* them, use `vetoable()` instead of `observable()`. The handler passed to `vetoable` will be called *before* the assignment of a new property value.

Delegating to another property

A property can delegate its getter and setter to another property. Such delegation is available for both top-level and class properties (member and extension). The delegate property can be:

- A top-level property
- A member or an extension property of the same class
- A member or an extension property of another class

To delegate a property to another property, use the `::` qualifier in the delegate name, for example, `this::delegate` or `MyClass::delegate`.

```
var topLevelInt: Int = 0
class ClassWithDelegate(val anotherClassInt: Int)

class MyClass(var memberInt: Int, val anotherClassInstance: ClassWithDelegate) {
    var delegatedToMember: Int by this::memberInt
    var delegatedToTopLevel: Int by ::topLevelInt

    val delegatedToAnotherClass: Int by anotherClassInstance::anotherClassInt
}
var MyClass.extDelegated: Int by ::topLevelInt
```

This may be useful, for example, when you want to rename a property in a backward-compatible way: introduce a new property, annotate the old one with the `@Deprecated` annotation, and delegate its implementation.

```
class MyClass {
    var newName: Int = 0
    @Deprecated("Use 'newName' instead", ReplaceWith("newName"))
    var oldName: Int by this::newName
}
fun main() {
    val myClass = MyClass()
    // Notification: 'oldName: Int' is deprecated.
    // Use 'newName' instead
    myClass.oldName = 42
    println(myClass.newName) // 42
}
```

Storing properties in a map

One common use case is storing the values of properties in a map. This comes up often in applications for things like parsing JSON or performing other dynamic tasks. In this case, you can use the map instance itself as the delegate for a delegated property.

```
class User(val map: Map<String, Any?>) {
    val name: String by map
    val age: Int     by map
}
```

In this example, the constructor takes a map:

```
val user = User(mapOf(
    "name" to "John Doe",
    "age"  to 25
))
```

Delegated properties take values from this map through string keys, which are associated with the names of properties:

```
class User(val map: Map<String, Any?>) {
    val name: String by map
    val age: Int     by map
}

fun main() {
    val user = User(mapOf(
        "name" to "John Doe",
        "age"  to 25
    ))
    println(user.name) // Prints "John Doe"
    println(user.age)  // Prints 25
}
```

This also works for `var`'s properties if you use a `MutableMap` instead of a read-only `Map`:

```
class MutableUser(val map: MutableMap<String, Any?>) {
    var name: String by map
    var age: Int     by map
}
```

Local delegated properties

You can declare local variables as delegated properties. For example, you can make a local variable lazy:

```
fun example(computeFoo: () -> Foo) {
    val memoizedFoo by lazy(computeFoo)

    if (someCondition && memoizedFoo.isValid()) {
        memoizedFoo.doSomething()
    }
}
```

The `memoizedFoo` variable will be computed on first access only. If `someCondition` fails, the variable won't be computed at all.

Property delegate requirements

For a *read-only* property (`val`), a delegate should provide an operator function `getValue()` with the following parameters:

- `thisRef` must be the same type as, or a supertype of, the *property owner* (for extension properties, it should be the type being extended).
- `property` must be of type `KProperty<*>` or its supertype.

`getValue()` must return the same type as the property (or its subtype).

```
class Resource

class Owner {
    val valResource: Resource by ResourceDelegate()
}

class ResourceDelegate {
    operator fun getValue(thisRef: Owner, property: KProperty<*>): Resource {
        return Resource()
    }
}
```

For a *mutable* property (`var`), a delegate has to additionally provide an operator function `setValue()` with the following parameters:

- `thisRef` must be the same type as, or a supertype of, the *property owner* (for extension properties, it should be the type being extended).
- `property` must be of type `KProperty<*>` or its supertype.
- `value` must be of the same type as the property (or its supertype).

```
class Resource

class Owner {
    var varResource: Resource by ResourceDelegate()
}

class ResourceDelegate(private var resource: Resource = Resource()) {
    operator fun getValue(thisRef: Owner, property: KProperty<*>): Resource {
        return resource
    }
    operator fun setValue(thisRef: Owner, property: KProperty<*>, value: Any?) {
        if (value is Resource) {
            resource = value
        }
    }
}
```

`getValue()` and/or `setValue()` functions can be provided either as member functions of the delegate class or as extension functions. The latter is handy when you need to delegate a property to an object that doesn't originally provide these functions. Both of the functions need to be marked with the `operator` keyword.

You can create delegates as anonymous objects without creating new classes, by using the interfaces `ReadOnlyProperty` and `ReadWriteProperty` from the Kotlin standard library. They provide the required methods: `getValue()` is declared in `ReadOnlyProperty`; `ReadWriteProperty` extends it and adds `setValue()`. This means you can pass a `ReadWriteProperty` whenever a `ReadOnlyProperty` is expected.

```
fun resourceDelegate(resource: Resource = Resource()): ReadWriteProperty<Any?, Resource> =
    object : ReadWriteProperty<Any?, Resource> {
        var curValue = resource
        override fun getValue(thisRef: Any?, property: KProperty<*>): Resource = curValue
        override fun setValue(thisRef: Any?, property: KProperty<*>, value: Resource) {
            curValue = value
        }
    }

val readOnlyResource: Resource by resourceDelegate()  // ReadWriteProperty as val
var readWriteResource: Resource by resourceDelegate()
```

Translation rules for delegated properties

Under the hood, the Kotlin compiler generates auxiliary properties for some kinds of delegated properties and then delegates to them.

> **Note:** For optimization purposes, the compiler *does not* generate auxiliary properties in several cases. Learn about the optimization on the example of delegating to another property.

For example, for the property `prop` it generates the hidden property `prop$delegate`, and the code of the accessors simply delegates to this additional property:

```
class C {
    var prop: Type by MyDelegate()
}

// this code is generated by the compiler instead:
class C {
    private val prop$delegate = MyDelegate()
    var prop: Type
        get() = prop$delegate.getValue(this, this::prop)
        set(value: Type) = prop$delegate.setValue(this, this::prop, value)
}
```

The Kotlin compiler provides all the necessary information about `prop` in the arguments: the first argument `this` refers to an instance of the outer class `C`, and `this::prop` is a reflection object of the `KProperty` type describing `prop` itself.

Optimized cases for delegated properties

The `$delegate` field will be omitted if a delegate is:

- A referenced property:

  ```
  class C<Type> {
      private var impl: Type = ...
      var prop: Type by ::impl
  }
  ```

- A named object:

  ```
  object NamedObject {
      operator fun getValue(thisRef: Any?, property: KProperty<*>): String = ...
  }

  val s: String by NamedObject
  ```

- A final `val` property with a backing field and a default getter in the same module:

```
val impl: ReadOnlyProperty<Any?, String> = ...

class A {
    val s: String by impl
}
```

- A constant expression, enum entry, `this`, `null`. The example of `this`:

```
class A {
    operator fun getValue(thisRef: Any?, property: KProperty<*>) ...

    val s by this
}
```

Translation rules when delegating to another property

When delegating to another property, the Kotlin compiler generates immediate access to the referenced property. This means that the compiler doesn't generate the field `prop$delegate`. This optimization helps save memory.

Take the following code, for example:

```
class C<Type> {
    private var impl: Type = ...
    var prop: Type by ::impl
}
```

Property accessors of the `prop` variable invoke the `impl` variable directly, skipping the delegated property's `getValue` and `setValue` operators, and thus the `KProperty` reference object is not needed.

For the code above, the compiler generates the following code:

```
class C<Type> {
    private var impl: Type = ...

    var prop: Type
        get() = impl
        set(value) {
            impl = value
        }

    fun getProp$delegate(): Type = impl // This method is needed only for reflection
}
```

Providing a delegate

By defining the `provideDelegate` operator, you can extend the logic for creating the object to which the property implementation is delegated. If the object used on the right-hand side of `by` defines

`provideDelegate` as a member or extension function, that function will be called to create the property delegate instance.

One of the possible use cases of `provideDelegate` is to check the consistency of the property upon its initialization.

For example, to check the property name before binding, you can write something like this:

```
class ResourceDelegate<T> : ReadOnlyProperty<MyUI, T> {
    override fun getValue(thisRef: MyUI, property: KProperty<*>): T { ... }
}

class ResourceLoader<T>(id: ResourceID<T>) {
    operator fun provideDelegate(
            thisRef: MyUI,
            prop: KProperty<*>
    ): ReadOnlyProperty<MyUI, T> {
        checkProperty(thisRef, prop.name)
        // create delegate
        return ResourceDelegate()
    }

    private fun checkProperty(thisRef: MyUI, name: String) { ... }
}

class MyUI {
    fun <T> bindResource(id: ResourceID<T>): ResourceLoader<T> { ... }

    val image by bindResource(ResourceID.image_id)
    val text by bindResource(ResourceID.text_id)
}
```

The parameters of `provideDelegate` are the same as those of `getValue`:

- `thisRef` must be the same type as, or a supertype of, the *property owner* (for extension properties, it should be the type being extended);
- `property` must be of type `KProperty<*>` or its supertype.

The `provideDelegate` method is called for each property during the creation of the `MyUI` instance, and it performs the necessary validation right away.

Without this ability to intercept the binding between the property and its delegate, to achieve the same functionality you'd have to pass the property name explicitly, which isn't very convenient:

```
// Checking the property name without "provideDelegate" functionality
class MyUI {
    val image by bindResource(ResourceID.image_id, "image")
    val text by bindResource(ResourceID.text_id, "text")
}

fun <T> MyUI.bindResource(
        id: ResourceID<T>,
```

```
        propertyName: String
): ReadOnlyProperty<MyUI, T> {
    checkProperty(this, propertyName)
    // create delegate
}
```

In the generated code, the `provideDelegate` method is called to initialize the auxiliary `prop$delegate` property. Compare the generated code for the property declaration `val prop: Type by MyDelegate()` with the generated code above (when the `provideDelegate` method is not present):

```
class C {
    var prop: Type by MyDelegate()
}

// this code is generated by the compiler
// when the 'provideDelegate' function is available:
class C {
    // calling "provideDelegate" to create the additional "delegate" property
    private val prop$delegate = MyDelegate().provideDelegate(this, this::prop)
    var prop: Type
        get() = prop$delegate.getValue(this, this::prop)
        set(value: Type) = prop$delegate.setValue(this, this::prop, value)
}
```

Note that the `provideDelegate` method affects only the creation of the auxiliary property and doesn't affect the code generated for the getter or the setter.

With the `PropertyDelegateProvider` interface from the standard library, you can create delegate providers without creating new classes.

```
val provider = PropertyDelegateProvider { thisRef: Any?, property ->
    ReadOnlyProperty<Any?, Int> {_, property -> 42 }
}
val delegate: Int by provider
```

Null safety

Null safety is a Kotlin feature designed to significantly reduce the risk of null references, also known as The Billion-Dollar Mistake.

One of the most common pitfalls in many programming languages, including Java, is that accessing a member of a null reference results in a null reference exception. In Java, this would be the equivalent of a `NullPointerException`, or an *NPE* for short.

Kotlin explicitly supports nullability as part of its type system, meaning you can explicitly declare which variables or properties are allowed to be `null`. Also, when you declare non-null variables, the compiler enforces that these variables cannot hold a `null` value, preventing an NPE.

Kotlin's null safety ensures safer code by catching potential null-related issues at compile time rather than runtime. This feature improves code robustness, readability, and maintainability by explicitly expressing `null` values, making the code easier to understand and manage.

The only possible causes of an NPE in Kotlin are:

- An explicit call to `throw NullPointerException()`.
- Usage of the not-null assertion operator `!!`.
- Data inconsistency during initialization, such as when:
  - An uninitialized `this` available in a constructor is used somewhere else (a "leaking `this`").
  - A superclass constructor calling an open member whose implementation in the derived class uses an uninitialized state.
- Java interoperation:
  - Attempts to access a member of a `null` reference of a platform type.
  - Nullability issues with generic types. For example, a piece of Java code adding `null` into a Kotlin `MutableList<String>`, which would require `MutableList<String?>` to handle it properly.
  - Other issues caused by external Java code.

Tip: Besides NPE, another exception related to null safety is `UninitializedPropertyAccessException`. Kotlin throws this exception when you try to access a property that has not been initialized, ensuring that non-nullable properties are not used until they are ready. This typically happens with `lateinit` properties.

Nullable types and non-nullable types

In Kotlin, the type system distinguishes between types that can hold `null` (nullable types) and those that cannot (non-nullable types). For example, a regular variable of type `String` cannot hold

`null`:

```
fun main() {
    // Assigns a non-null string to a variable
    var a: String = "abc"
    // Attempts to re-assign null to the non-nullable variable
    a = null
    print(a)
    // Null can not be a value of a non-null type String
}
```

You can safely call a method or access a property on `a`. It's guaranteed not to cause an NPE because `a` is a non-nullable variable. The compiler ensures that `a` always holds a valid `String` value, so there's no risk of accessing its properties or methods when it's `null`:

```
fun main() {
    // Assigns a non-null string to a variable
    val a: String = "abc"
    // Returns the length of a non-nullable variable
    val l = a.length
    print(l)
    // 3
}
```

To allow `null` values, declare a variable with a `?` sign right after the variable type. For example, you can declare a nullable string by writing `String?`. This expression makes `String` a type that can accept `null`:

```
fun main() {
    // Assigns a nullable string to a variable
    var b: String? = "abc"
    // Successfully re-assigns null to the nullable variable
    b = null
    print(b)
    // null
}
```

If you try accessing `length` directly on `b`, the compiler reports an error. This is because `b` is declared as a nullable variable and can hold `null` values. Attempting to access properties on nullables directly leads to an NPE:

```
fun main() {
    // Assigns a nullable string to a variable
    var b: String? = "abc"
    // Re-assigns null to the nullable variable
    b = null
    // Tries to directly return the length of a nullable variable
    val l = b.length
    print(l)
    // Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of
```

```
type String?
}
```

In the example above, the compiler requires you to use safe calls to check for nullability before accessing properties or performing operations. There are several ways to handle nullables:

- Check for `null` with the `if` conditional
- Safe call operator `?.`
- Elvis operator `?:`
- Not-null assertion operator `!!`
- Nullable receiver
- `let` function
- Safe casts `as?`
- Collections of a nullable type

Read the next sections for details and examples of `null` handling tools and techniques.

Check for null with the if conditional

When working with nullable types, you need to handle nullability safely to avoid an NPE. One way to handle this is checking for nullability explicitly with the `if` conditional expression.

For example, check whether `b` is `null` and then access `b.length`:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability first and then accesses length
    val l = if (b != null) b.length else -1
    print(l)
    // -1
}
```

In the example above, the compiler performs a smart cast to change the type from nullable `String?` to non-nullable `String`. It also tracks the information about the check you performed and allows the call to `length` inside the `if` conditional.

More complex conditions are supported as well:

```
fun main() {
    // Assigns a nullable string to a variable
    val b: String? = "Kotlin"

    // Checks for nullability first and then accesses length
    if (b != null && b.length > 0) {
        print("String of length ${b.length}")
        // String of length 6
    } else {
```

```
        // Provides alternative if the condition is not met
        print("Empty string")
    }
}
```

Note that the example above only works when the compiler can guarantee that `b` doesn't change between the check and its usage, same as the smart cast prerequisites.

Safe call operator

The safe call operator `?.` allows you to handle nullability safely in a shorter form. Instead of throwing an NPE, if the object is `null`, the `?.` operator simply returns `null`:

```
fun main() {
    // Assigns a nullable string to a variable
    val a: String? = "Kotlin"
    // Assigns null to a nullable variable
    val b: String? = null

    // Checks for nullability and returns length or null
    println(a?.length)
    // 6
    println(b?.length)
    // null
}
```

The `b?.length` expression checks for nullability and returns `b.length` if `b` is non-null, or `null` otherwise. The type of this expression is `Int?`.

You can use the `?.` operator with both `var` and `val` variables in Kotlin:

- A nullable `var` can hold a `null` (for example, `var nullableValue: String? = null`) or a non-null value (for example, `var nullableValue: String? = "Kotlin"`). If it's a non-null value, you can change it to `null` at any point.
- A nullable `val` can hold a `null` (for example, `val nullableValue: String? = null`) or a non-null value (for example, `val nullableValue: String? = "Kotlin"`). If it's a non-null value, you cannot change it to `null` subsequently.

Safe calls are useful in chains. For example, Bob is an employee who may be assigned to a department (or not). That department may, in turn, have another employee as a department head. To obtain the name of Bob's department head (if there is one), you write the following:

```
bob?.department?.head?.name
```

This chain returns `null` if any of its properties are `null`.

You can also place a safe call on the left side of an assignment:

```
person?.department?.head = managersPool.getManager()
```

In the example above, if one of the receivers in the safe call chain is `null`, the assignment is skipped, and the expression on the right is not evaluated at all. For example, if either `person` or `person.department` is `null`, the function is not called. Here's the equivalent of the same safe call but with the `if` conditional:

```
if (person != null && person.department != null) {
    person.department.head = managersPool.getManager()
}
```

Elvis operator

When working with nullable types, you can check for `null` and provide an alternative value. For example, if `b` is not `null`, access `b.length`. Otherwise, return an alternative value:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability. If not null, returns length. If null, returns 0
    val l: Int = if (b != null) b.length else 0
    println(l)
    // 0
}
```

Instead of writing the complete `if` expression, you can handle this in a more concise way with the Elvis operator `?:`:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability. If not null, returns length. If null, returns a non-null valu
e
    val l = b?.length ?: 0
    println(l)
    // 0
}
```

If the expression to the left of `?:` is not `null`, the Elvis operator returns it. Otherwise, the Elvis operator returns the expression to the right. The expression on the right-hand side is evaluated only if the left-hand side is `null`.

Since `throw` and `return` are expressions in Kotlin, you can also use them on the right-hand side of the Elvis operator. This can be handy, for example, when checking function arguments:

```
fun foo(node: Node): String? {
    // Checks for getParent(). If not null, it's assigned to parent. If null, returns null
```

```
    val parent = node.getParent() ?: return null
    // Checks for getName(). If not null, it's assigned to name. If null, throws exception
    val name = node.getName() ?: throw IllegalArgumentException("name expected")
    // ...
}
```

Not-null assertion operator

The not-null assertion operator `!!` converts any value to a non-nullable type.

When you apply the `!!` operator to a variable whose value is not `null`, it's safely handled as a non-nullable type, and the code executes normally. However, if the value is `null`, the `!!` operator forces it to be treated as non-nullable, which results in an NPE.

When `b` is not `null` and the `!!` operator makes it return its non-null value (which is a `String` in this example), it accesses `length` correctly:

```
fun main() {
    // Assigns a nullable string to a variable
    val b: String? = "Kotlin"
    // Treats b as non-null and accesses its length
    val l = b!!.length
    println(l)
    // 6
}
```

When `b` is `null` and the `!!` operator makes it return its non-null value, and an NPE occurs:

```
fun main() {
    // Assigns null to a nullable variable
    val b: String? = null
    // Treats b as non-null and tries to access its length
    val l = b!!.length
    println(l)
    // Exception in thread "main" java.lang.NullPointerException
}
```

The `!!` operator is particularly useful when you are confident that a value is not `null` and there's no chance of getting an NPE, but the compiler cannot guarantee this due to certain rules. In such cases, you can use the `!!` operator to explicitly tell the compiler that the value is not `null`.

Nullable receiver

You can use extension functions with a nullable receiver type, allowing these functions to be called on variables that might be `null`.

By defining an extension function on a nullable receiver type, you can handle `null` values within the function itself instead of checking for `null` at every place where you call the function.

For example, the `.toString()` extension function can be called on a nullable receiver. When invoked on a `null` value, it safely returns the string `"null"` without throwing an exception:

```
fun main() {
    // Assigns null to a nullable Person object stored in the person variable
    val person: Person? = null

    // Applies .toString to the nullable person variable and prints a string
    println(person.toString())
    // null
}

// Defines a simple Person class
data class Person(val name: String)
```

In the example above, even though `person` is `null`, the `.toString()` function safely returns the string `"null"`. This can be helpful for debugging and logging.

If you expect the `.toString()` function to return a nullable string (either a string representation or `null`), use the safe-call operator `?.`. The `?.` operator calls `.toString()` only if the object is not `null`, otherwise it returns `null`:

```
fun main() {
    // Assigns a nullable Person object to a variable
    val person1: Person? = null
    val person2: Person? = Person("Alice")

    // Prints "null" if person is null; otherwise prints the result of person.toString()
    println(person1?.toString())
    // null
    println(person2?.toString())
    // Person(name=Alice)
}

// Defines a Person class
data class Person(val name: String)
```

The `?.` operator allows you to safely handle potential `null` values while still accessing properties or functions of objects that might be `null`.

Let function

To handle `null` values and perform operations only on non-null types, you can use the safe call operator `?.` together with the `let` function.

This combination is useful for evaluating an expression, check the result for `null`, and execute code only if it's not `null`, avoiding manual null checks:

```
fun main() {
    // Declares a list of nullable strings
    val listWithNulls: List<String?> = listOf("Kotlin", null)

    // Iterates over each item in the list
    for (item in listWithNulls) {
        // Checks if the item is null and only prints non-null values
        item?.let { println(it) }
        //Kotlin
    }
}
```

Safe casts

The regular Kotlin operator for type casts is the `as` operator. However, regular casts can result in an exception if the object is not of the target type.

You can use the `as?` operator for safe casts. It tries to cast a value to the specified type and returns `null` if the value is not of that type:

```
fun main() {
    // Declares a variable of type Any, which can hold any type of value
    val a: Any = "Hello, Kotlin!"

    // Safe casts to Int using the 'as?' operator
    val aInt: Int? = a as? Int
    // Safe casts to String using the 'as?' operator
    val aString: String? = a as? String

    println(aInt)
    // null
    println(aString)
    // "Hello, Kotlin!"
}
```

The code above prints `null` because `a` is not an `Int`, so the cast fails safely. It also prints `"Hello, Kotlin!"` because it matches the `String?` type, so the safe cast succeeds.

Collections of a nullable type

If you have a collection of nullable elements and want to keep only the non-null ones, use the `filterNotNull()` function:

```
fun main() {
    // Declares a list containing some null and non-null integer values
```

```
    val nullableList: List<Int?> = listOf(1, 2, null, 4)

    // Filters out null values, resulting in a list of non-null integers
    val intList: List<Int> = nullableList.filterNotNull()

    println(intList)
    // [1, 2, 4]
}
```

What's next?

- Learn how to handle nullability in Java and Kotlin.
- Learn about generic types that are definitely non-nullable.

Equality

In Kotlin, there are two types of equality:

- *Structural* equality (`==`) - a check for the `equals()` function
- *Referential* equality (`===`) - a check for two references pointing to the same object

Structural equality

Structural equality verifies if two objects have the same content or structure. Structural equality is checked by the `==` operation and its negated counterpart `!=`. By convention, an expression like `a == b` is translated to:

```
a?.equals(b) ?: (b === null)
```

If `a` is not `null`, it calls the `equals(Any?)` function. Otherwise (`a` is `null`), it checks that `b` is referentially equal to `null`:

```
fun main() {
    var a = "hello"
    var b = "hello"
    var c = null
    var d = null
    var e = d

    println(a == b)
    // true
    println(a == c)
    // false
    println(c == e)
    // true
}
```

Note that there's no point in optimizing your code when comparing to `null` explicitly: `a == null` will be automatically translated to `a === null`.

In Kotlin, the `equals()` function is inherited by all classes from the `Any` class. By default, the `equals()` function implements referential equality. However, classes in Kotlin can override the `equals()` function to provide a custom equality logic and, in this way, implement structural equality.

Value classes and data classes are two specific Kotlin types that automatically override the `equals()` function. That's why they implement structural equality by default.

However, in the case of data classes, if the `equals()` function is marked as `final` in the parent class, its behavior remains unchanged.

Distinctly, non-data classes (those not declared with the `data` modifier) do not override the `equals()` function by default. Instead, non-data classes implement referential equality behavior inherited from the `Any` class. To implement structural equality, non-data classes require a custom equality logic to override the `equals()` function.

To provide a custom equals check implementation, override the `equals(other: Any?): Boolean` function:

```
class Point(val x: Int, val y: Int) {
    override fun equals(other: Any?): Boolean {
        if (this === other) return true
        if (other !is Point) return false

        // Compares properties for structural equality
        return this.x == other.x && this.y == other.y
    }
}
```

> **Note:** When overriding the equals() function, you should also override the hashCode() function to keep consistency between equality and hashing and ensure the proper behavior of these functions.

Functions with the same name and other signatures (like `equals(other: Foo)`) don't affect equality checks with the operators `==` and `!=`.

Structural equality has nothing to do with comparison defined by the `Comparable<...>` interface, so only a custom `equals(Any?)` implementation may affect the behavior of the operator.

Referential equality

Referential equality verifies the memory addresses of two objects to determine if they are the same instance.

Referential equality is checked by the `===` operation and its negated counterpart `!==`. `a === b` evaluates to true if and only if `a` and `b` point to the same object:

```
fun main() {
    var a = "Hello"
    var b = a
    var c = "world"
    var d = "world"

    println(a === b)
    // true
    println(a === c)
```

```
    // false
    println(c === d)
    // true

}
```

For values represented by primitive types at runtime (for example, `Int`), the `===` equality check is equivalent to the `==` check.

> **Tip:** The referential equality is implemented differently in Kotlin/JS. For more information about equality, see the Kotlin/JS documentation.

Floating-point numbers equality

When the operands of an equality check are statically known to be `Float` or `Double` (nullable or not), the check follows the IEEE 754 Standard for Floating-Point Arithmetic.

The behavior is different for operands that are not statically typed as floating-point numbers. In these cases, structural equality is implemented. As a result, checks with operands not statically typed as floating-point numbers differ from the IEEE standard. In this scenario:

- `NaN` is equal to itself
- `NaN` is greater than any other element (including `POSITIVE_INFINITY`)
- `-0.0` is not equal to `0.0`

For more information, see Floating-point numbers comparison.

Array equality

To compare whether two arrays have the same elements in the same order, use `contentEquals()`.

For more information, see Compare arrays.

Generics: in, out, where

Classes in Kotlin can have type parameters, just like in Java:

```
class Box<T>(t: T) {
    var value = t
}
```

To create an instance of such a class, simply provide the type arguments:

```
val box: Box<Int> = Box<Int>(1)
```

If the compiler can infer the type arguments, for example, from the constructor arguments, you don’t need to specify them explicitly:

```
val box = Box(1) // 1 has type Int, so the compiler figures out that it is Box<Int>
```

Variance

One of the trickiest aspects of Java's type system is the wildcard types (see Java Generics FAQ). Kotlin doesn't have these. Instead, Kotlin has declaration-site variance and type projections.

Variance and wildcards in Java

Let's think about why Java needs these mysterious wildcards. First, generic types in Java are *invariant*, meaning that `List<String>` is *not* a subtype of `List<Object>`. If `List` were not *invariant*, it would have been no better than Java's arrays, as the following code would have compiled but caused an exception at runtime:

```
// Java
List<String> strs = new ArrayList<String>();

// Java reports a type mismatch here at compile-time.
List<Object> objs = strs;

// What if it didn't?
// We would be able to put an Integer into a list of Strings.
objs.add(1);

// And then at runtime, Java would throw
// a ClassCastException: Integer cannot be cast to String
String s = strs.get(0);
```

Java prohibits such things to guarantee runtime safety. But this has implications. For example, consider the `addAll()` method from the `Collection` interface. What's the signature of this method?

Intuitively, you'd write it this way:

```
// Java
interface Collection<E> ... {
    void addAll(Collection<E> items);
}
```

But then, you would not be able to do the following (which is perfectly safe):

```
// Java

// The following would not compile with the naive declaration of addAll:
// Collection<String> is not a subtype of Collection<Object>
void copyAll(Collection<Object> to, Collection<String> from) {
    to.addAll(from);
}
```

That's why the actual signature of `addAll()` is the following:

```
// Java
interface Collection<E> ... {
    void addAll(Collection<? extends E> items);
}
```

The *wildcard type argument* `? extends E` indicates that this method accepts a collection of objects of `E` *or a subtype of* `E`, not just `E` itself. This means that you can safely *read* `E`'s from items (elements of this collection are instances of a subclass of E), but *cannot write* to it as you don't know what objects comply with that unknown subtype of `E`. In return for this limitation, you get the desired behavior: `Collection<String>` *is* a subtype of `Collection<? extends Object>`. In other words, the wildcard with an *extends*-bound (*upper* bound) makes the type *covariant*.

The key to understanding why this works is rather simple: if you can only *take* items from a collection, then using a collection of `String`s and reading `Object`s from it is fine. Conversely, if you can only *put* items into the collection, it's okay to take a collection of `Object`s and put `String`s into it: in Java there is `List<? super String>`, which accepts `String`s or any of its supertypes.

The latter is called *contravariance*, and you can only call methods that take `String` as an argument on `List<? super String>` (for example, you can call `add(String)` or `set(int, String)`). If you call something that returns `T` in `List<T>`, you don't get a `String`, but rather an `Object`.

Joshua Bloch, in his book Effective Java, 3rd Edition, explains the problem well (Item 31: "Use bounded wildcards to increase API flexibility"). He gives the name *Producers* to objects you only *read from* and *Consumers* to those you only *write to*. He recommends:

> "For maximum flexibility, use wildcard types on input parameters that represent producers or consumers."

He then proposes the following mnemonic: *PECS* stands for *Producer-Extends, Consumer-Super.*

> **Note:** If you use a producer-object, say, `List<? extends Foo>`, you are not allowed to call `add()` or `set()` on this object, but this does not mean that it is *immutable*: for example, nothing prevents you from calling `clear()` to remove all the items from the list, since `clear()` does not take any parameters at all.
>
> The only thing guaranteed by wildcards (or other types of variance) is *type safety*. Immutability is a completely different story.

Declaration-site variance

Let's suppose that there is a generic interface `Source<T>` that does not have any methods that take `T` as a parameter, only methods that return `T`:

```
// Java
interface Source<T> {
    T nextT();
}
```

Then, it would be perfectly safe to store a reference to an instance of `Source<String>` in a variable of type `Source<Object>` - there are no consumer-methods to call. But Java does not know this, and still prohibits it:

```
// Java
void demo(Source<String> strs) {
    Source<Object> objects = strs; // !!! Not allowed in Java
    // ...
}
```

To fix this, you should declare objects of type `Source<? extends Object>`. Doing so is meaningless, because you can call all the same methods on such a variable as before, so there's no value added by the more complex type. But the compiler does not know that.

In Kotlin, there is a way to explain this sort of thing to the compiler. This is called *declaration-site variance*: you can annotate the *type parameter* `T` of `Source` to make sure that it is only *returned* (produced) from members of `Source<T>`, and never consumed. To do this, use the `out` modifier:

```
interface Source<out T> {
    fun nextT(): T
}

fun demo(strs: Source<String>) {
    val objects: Source<Any> = strs // This is OK, since T is an out-parameter
    // ...
}
```

The general rule is this: when a type parameter `T` of a class `C` is declared `out`, it may occur only in the *out*-position in the members of `C`, but in return `C<Base>` can safely be a supertype of `C<Derived>`.

In other words, you can say that the class `C` is *covariant* in the parameter `T`, or that `T` is a *covariant* type parameter. You can think of `C` as being a *producer* of `T`'s, and NOT a *consumer* of `T`'s.

The `out` modifier is called a *variance annotation*, and since it is provided at the type parameter declaration site, it provides *declaration-site variance*. This is in contrast with Java's *use-site variance* where wildcards in the type usages make the types covariant.

In addition to `out`, Kotlin provides a complementary variance annotation: `in`. It makes a type parameter *contravariant*, meaning it can only be consumed and never produced. A good example of a contravariant type is `Comparable`:

```
interface Comparable<in T> {
    operator fun compareTo(other: T): Int
}

fun demo(x: Comparable<Number>) {
    x.compareTo(1.0) // 1.0 has type Double, which is a subtype of Number
    // Thus, you can assign x to a variable of type Comparable<Double>
    val y: Comparable<Double> = x // OK!
}
```

The words *in* and *out* seem to be self-explanatory (as they've already been used successfully in C# for quite some time), and so the mnemonic mentioned above is not really needed. It can in fact be rephrased at a higher level of abstraction:

The Existential Transformation: Consumer in, Producer out! :-)

Type projections

Use-site variance: type projections

It is very easy to declare a type parameter `T` as `out` and avoid trouble with subtyping on the use site, but some classes *can't* actually be restricted to only return `T`'s! A good example of this is `Array`:

```
class Array<T>(val size: Int) {
    operator fun get(index: Int): T { ... }
    operator fun set(index: Int, value: T) { ... }
}
```

This class can be neither co- nor contravariant in `T`. And this imposes certain inflexibilities. Consider the following function:

```
fun copy(from: Array<Any>, to: Array<Any>) {
    assert(from.size == to.size)
    for (i in from.indices)
        to[i] = from[i]
}
```

This function is supposed to copy items from one array to another. Let's try to apply it in practice:

```
val ints: Array<Int> = arrayOf(1, 2, 3)
val any = Array<Any>(3) { "" }
copy(ints, any)
//   ^ type is Array<Int> but Array<Any> was expected
```

Here you run into the same familiar problem: `Array<T>` is *invariant* in `T`, and so neither `Array<Int>` nor `Array<Any>` is a subtype of the other. Why not? Again, this is because `copy` could have an unexpected behavior, for example, it may attempt to write a `String` to `from`, and if you actually pass an array of `Int` there, a `ClassCastException` will be thrown later.

To prohibit the `copy` function from *writing* to `from`, you can do the following:

```
fun copy(from: Array<out Any>, to: Array<Any>) { ... }
```

This is *type projection*, which means that `from` is not a simple array, but is rather a restricted (*projected*) one. You can only call methods that return the type parameter `T`, which in this case means that you can only call `get()`. This is our approach to *use-site variance*, and it corresponds to Java's `Array<? extends Object>` while being slightly simpler.

You can project a type with `in` as well:

```
fun fill(dest: Array<in String>, value: String) { ... }
```

`Array<in String>` corresponds to Java's `Array<? super String>`. This means that you can pass an array of `String`, `CharSequence`, or `Object` to the `fill()` function.

Star-projections

Sometimes you want to say that you know nothing about the type argument, but you still want to use it in a safe way. The safe way here is to define such a projection of the generic type, that every concrete instantiation of that generic type will be a subtype of that projection.

Kotlin provides so-called *star-projection* syntax for this:

- For `Foo<out T : TUpper>`, where `T` is a covariant type parameter with the upper bound `TUpper`, `Foo<*>` is equivalent to `Foo<out TUpper>`. This means that when the `T` is unknown, you can safely *read* values of `TUpper` from `Foo<*>`.

- For `Foo<in T>`, where `T` is a contravariant type parameter, `Foo<*>` is equivalent to `Foo<in Nothing>`. This means there is nothing you can *write* to `Foo<*>` in a safe way when `T` is unknown.
- For `Foo<T : TUpper>`, where `T` is an invariant type parameter with the upper bound `TUpper`, `Foo<*>` is equivalent to `Foo<out TUpper>` for reading values and to `Foo<in Nothing>` for writing values.

If a generic type has several type parameters, each of them can be projected independently. For example, if the type is declared as `interface Function<in T, out U>` you could use the following star-projections:

- `Function<*, String>` means `Function<in Nothing, String>`.
- `Function<Int, *>` means `Function<Int, out Any?>`.
- `Function<*, *>` means `Function<in Nothing, out Any?>`.

Note: Star-projections are very much like Java's raw types, but safe.

Captured types

When you use a type projection, such as `out T` or `in T`, the compiler internally represents the unknown concrete type as a *captured type*. A captured type is an unknown type with known upper and lower bounds.

Captured types are non-denotable, so you can't write them directly in Kotlin code. Instead, you can most frequently see captured types in compiler diagnostics, such as `CapturedType(out X)`. For example, the following type mismatch diagnostic contains a captured type:

```
val array: Array<out CharSequence> = arrayOf("str")
val item: Int = array.get(0)
// Initializer type mismatch: expected 'Int', actual 'CapturedType(out CharSequence)'
```

The compiler uses the captured type's upper bound for read operations and its lower bound to determine which values are type-safe for write operations:

```
// The projected type is Array<out CharSequence>
val array: Array<out CharSequence> = arrayOf("Kotlin")

// The get() read operation uses the captured type's upper bound, CharSequence
val item = array.get(0)

// The set() write operation uses the captured type's lower bound, Nothing,
// which results in an error
array.set(0, "New value")
// Receiver type 'Array<out CharSequence>' contains out projection
// which prohibits the use of 'fun set(index: Int, value: T): Unit'
```

In this example:

- The variable `array` has the projected type `Array<out CharSequence>`. The compiler represents the projected type argument `out CharSequence` as a captured type with `CharSequence` as its upper bound and `Nothing` as its lower bound.
- For the `get()` operation, the compiler approximates the captured type to its upper bound, `CharSequence`, and infers `CharSequence` as the type of `item`.
- For the `set()` operation, the captured type has `Nothing` as its lower bound. Since `Nothing` has no instances, writing a value to the projected type isn't type-safe and results in an error.

Generic functions

Classes aren't the only declarations that can have type parameters. Functions can, too. Type parameters are placed *before* the name of the function:

```
fun <T> singletonList(item: T): List<T> {
    // ...
}

fun <T> T.basicToString(): String { // extension function
    // ...
}
```

To call a generic function, specify the type arguments at the call site *after* the name of the function:

```
val l = singletonList<Int>(1)
```

Type arguments can be omitted if they can be inferred from the context, so the following example works as well:

```
val l = singletonList(1)
```

Generic constraints

The set of all possible types that can be substituted for a given type parameter may be restricted by *generic constraints*.

Upper bounds

The most common type of constraint is an *upper bound*, which corresponds to Java's `extends` keyword:

```
fun <T : Comparable<T>> sort(list: List<T>) {  ... }
```

The type specified after a colon is the *upper bound*, indicating that only a subtype of `Comparable<T>` can be substituted for `T`. For example:

```
sort(listOf(1, 2, 3)) // OK. Int is a subtype of Comparable<Int>
sort(listOf(HashMap<Int, String>())) // Error: HashMap<Int, String> is not a subtype of Comparable<HashMap<Int, String>>
```

The default upper bound (if there was none specified) is `Any?`. Only one upper bound can be specified inside the angle brackets. If the same type parameter needs more than one upper bound, you need a separate *where*-clause:

```
fun <T> copyWhenGreater(list: List<T>, threshold: T): List<String>
    where T : CharSequence,
          T : Comparable<T> {
    return list.filter { it > threshold }.map { it.toString() }
}
```

The passed type must satisfy all conditions of the `where` clause simultaneously. In the above example, the `T` type must implement *both* `CharSequence` and `Comparable`.

Definitely non-nullable types

To make interoperability with generic Java classes and interfaces easier, Kotlin supports declaring a generic type parameter as **definitely non-nullable**.

To declare a generic type `T` as definitely non-nullable, declare the type with `& Any`. For example: `T & Any`.

A definitely non-nullable type must have a nullable upper bound.

The most common use case for declaring definitely non-nullable types is when you want to override a Java method that contains `@NotNull` as an argument. For example, consider the `load()` method:

```
import org.jetbrains.annotations.*;

public interface Game<T> {
    public T save(T x) {}
    @NotNull
    public T load(@NotNull T x) {}
}
```

To override the `load()` method in Kotlin successfully, you need `T1` to be declared as definitely non-nullable:

```
interface ArcadeGame<T1> : Game<T1> {
    override fun save(x: T1): T1
```

```
    // T1 is definitely non-nullable
    override fun load(x: T1 & Any): T1 & Any
}
```

When working only with Kotlin, it's unlikely that you will need to declare definitely non-nullable types explicitly because Kotlin's type inference takes care of this for you.

Type erasure

The type safety checks that Kotlin performs for generic declaration usages are done at compile time. At runtime, the instances of generic types do not hold any information about their actual type arguments. The type information is said to be *erased*. For example, the instances of `Foo<Bar>` and `Foo<Baz?>` are erased to just `Foo<*>`.

Generics type checks and casts

Due to the type erasure, there is no general way to check whether an instance of a generic type was created with certain type arguments at runtime, and the compiler prohibits such `is`-checks such as `ints is List<Int>` or `list is T` (type parameter). However, you can check an instance against a star-projected type:

```
if (something is List<*>) {
    something.forEach { println(it) } // The items are typed as `Any?`
}
```

Similarly, when you already have the type arguments of an instance checked statically (at compile time), you can make an `is`-check or a cast that involves the non-generic part of the type. Note that angle brackets are omitted in this case:

```
fun handleStrings(list: MutableList<String>) {
    if (list is ArrayList) {
        // `list` is smart-cast to `ArrayList<String>`
    }
}
```

The same syntax but with the type arguments omitted can be used for casts that do not take type arguments into account: `list as ArrayList`.

The type arguments of generic function calls are also only checked at compile time. Inside the function bodies, the type parameters cannot be used for type checks, and type casts to type parameters (`foo as T`) are unchecked. The only exclusion is inline functions with reified type parameters, which have their actual type arguments inlined at each call site. This enables type checks and casts for the type parameters. However, the restrictions described above still apply for instances of generic types used inside checks or casts. For example, in the type check `arg is T`, if `arg` is an instance of a generic type itself, its type arguments are still erased.

```
inline fun <reified A, reified B> Pair<*, *>.asPairOf(): Pair<A, B>? {
    if (first !is A || second !is B) return null
    return first as A to second as B
}

val somePair: Pair<Any?, Any?> = "items" to listOf(1, 2, 3)

val stringToSomething = somePair.asPairOf<String, Any>()
val stringToInt = somePair.asPairOf<String, Int>()
val stringToList = somePair.asPairOf<String, List<*>>()
val stringToStringList = somePair.asPairOf<String, List<String>>() // Compiles but breaks ty
pe safety!
// Expand the sample for more details
fun main() {
    println("stringToSomething = " + stringToSomething)
    println("stringToInt = " + stringToInt)
    println("stringToList = " + stringToList)
    println("stringToStringList = " + stringToStringList)
    //println(stringToStringList?.second?.forEach() {it.length}) // This will throw ClassCas
tException as list items are not String
}
```

Unchecked casts

Type casts to generic types with concrete type arguments such as `foo as List<String>` cannot be checked at runtime.
These unchecked casts can be used when type safety is implied by the high-level program logic but cannot be inferred directly by the compiler. See the example below.

```
fun readDictionary(file: File): Map<String, *> = file.inputStream().use {
    TODO("Read a mapping of strings to arbitrary elements.")
}

// We saved a map with `Int`s into this file
val intsFile = File("ints.dictionary")

// Warning: Unchecked cast: `Map<String, *>` to `Map<String, Int>`
val intsDictionary: Map<String, Int> = readDictionary(intsFile) as Map<String, Int>
```

A warning appears for the cast in the last line. The compiler can't fully check it at runtime and provides no guarantee that the values in the map are `Int`.

To avoid unchecked casts, you can redesign the program structure. In the example above, you could use the `DictionaryReader<T>` and `DictionaryWriter<T>` interfaces with type-safe implementations for different types. You can introduce reasonable abstractions to move unchecked casts from the call site to the implementation details. Proper use of generic variance can also help.

For generic functions, using reified type parameters makes casts like `arg as T` checked, unless `arg`'s type has *its own* type arguments that are erased.

An unchecked cast warning can be suppressed by annotating the statement or the declaration where it occurs with `@Suppress("UNCHECKED_CAST")`:

```
inline fun <reified T> List<*>.asListOfType(): List<T>? =
    if (all { it is T })
        @Suppress("UNCHECKED_CAST")
        this as List<T> else
        null
```

> **Note: On the JVM**: array types (`Array<Foo>`) retain information about the erased type of their elements, and type casts to an array type are partially checked: the nullability and actual type arguments of the element type are still erased. For example, the cast `foo as Array<List<String>?>` will succeed if `foo` is an array holding any `List<*>`, whether it is nullable or not.

Underscore operator for type arguments

The underscore operator `_` can be used for type arguments. Use it to automatically infer a type of the argument when other types are explicitly specified:

```
abstract class SomeClass<T> {
    abstract fun execute() : T
}

class SomeImplementation : SomeClass<String>() {
    override fun execute(): String = "Test"
}

class OtherImplementation : SomeClass<Int>() {
    override fun execute(): Int = 42
}

object Runner {
    inline fun <reified S: SomeClass<T>, T> run() : T {
        return S::class.java.getDeclaredConstructor().newInstance().execute()
    }
}

fun main() {
    // T is inferred as String because SomeImplementation derives from SomeClass<String>
    val s = Runner.run<SomeImplementation, _>()
    assert(s == "Test")

    // T is inferred as Int because OtherImplementation derives from SomeClass<Int>
    val n = Runner.run<OtherImplementation, _>()
    assert(n == 42)
}
```

Asynchronous control flow and suspending functions

An application can become unresponsive when it waits for an operation to finish before handling other work.

On the JVM, code runs on threads. If the thread responsible for handling events has to wait for an operation to finish, it can't do other work in the meantime. An operation that causes the thread to wait is known as a *blocking operation*.

To remain responsive, an application needs to let independent work continue while an operation is in progress. *Asynchronous control flow* determines which work can continue and which work must wait for the operation to finish.

> **Note:** The following sections use examples from a small reminder application to demonstrate different ways to manage asynchronous control flow and how suspending functions make it easier to express.
>
> The example implementations are intended for teaching purposes and aren't production-ready. To get started with writing production-ready asynchronous code in Kotlin, see Coroutine basics.

Concurrent computations

The simplest form of asynchronous control flow is running two operations concurrently. Two operations run concurrently when one can start before the other has finished.

When a JVM application starts, the JVM invokes its `main()` function on the *main thread*. You can start another operation on a separate thread with the `Thread` class while the main thread continues running other code.

In `src/main/kotlin/Example1.kt` of the reminder application, each reminder starts on a new thread while the main thread continues processing user input:

```
// Starts a new thread that waits for the specified duration before printing the reminder
fun scheduleReminder(remindIn: Duration, reminderText: List<String>) {
    Thread {
        // Waits for the specified duration on a separate thread
        Thread.sleep(remindIn.inWholeMilliseconds)

        println(/* reminder text */)
    }.apply {
        // Keeps the JVM running until the reminder thread finishes
        isDaemon = false
```

```
        // Starts the thread
        start()
    }
}

fun main() {
    while (true) {
        // The main thread continues accepting commands
        println("Please enter your input:")
        val input = readlnOrNull() ?: return

        when (/* parsed command */) {
            "remind" -> {
                // The reminder starts on a new thread
                scheduleReminder(remindInParsed, command)
            }
        }
    }
}
```

In this example, you can continue interacting with the application while a reminder is pending. The `start()` method starts each reminder thread, and setting the `isDaemon` property to `false` keeps the JVM running until all reminder threads finish. When a reminder is triggered, the reminder thread prints its text to the same console that the main thread uses for other output.

The application also has a `fun_animation` command that prints a sequence of frames on the main thread:

```
"fun_animation" -> {
    for (frame in funAnimationFrames) {
        // The main thread prints each animation frame
        println(frame)

        // The main thread waits 200 milliseconds before printing the next frame
        Thread.sleep(200)
    }
}
```

Because reminders run concurrently with the animation, a reminder can be triggered while the animation is being printed. As a result, the reminder text can appear in the middle of the animation:

```
==============================


       _---_
*** REMINDER (SCHEDULED 2m AGO): Turn off the stove! ***
     / o o o \
    <==========>
     /       \
    /         \
```

```
   /           \

==============================
```

This interleaving shows a problem with concurrent access to shared resources: although the operations can run independently, they can interfere with each other.

In real applications, similar interference can occur when concurrent operations update shared state. For example, one operation might recalculate a UI layout, while another might change the displayed content. Without coordination, the result can depend on which operation finishes first, leaving the UI in an inconsistent state. This type of unpredictable interference is known as a *race condition*.

Here's the complete implementation for this version of the reminder application:

```
import kotlin.time.Clock
import kotlin.time.Duration

fun scheduleReminder(remindIn: Duration, reminderText: List<String>) {
    Thread {
        Thread.sleep(remindIn.inWholeMilliseconds)
        println(buildString {
            append("*** REMINDER (SCHEDULED $remindIn AGO)")
            if (reminderText.isNotEmpty()) {
                append(": ")
                for (word in reminderText) {
                    append(word)
                    append(' ')
                }
            } else {
                append(' ')
            }
            append("***")
        })
    }.apply {
        isDaemon = false
        start()
    }
}

fun main() {
    while (true) {
        println("Please enter your input:")
        val input = readlnOrNull() ?: return
        val command = input
            .trim().split("\\s".toRegex())
            .toMutableList()
        val operation = command.removeFirstOrNull() ?: continue
        when (val operationInLowerCase = operation.lowercase()) {
            "remind" -> {
                val remindIn = command.removeFirstOrNull()
                val remindInParsed = remindIn?.let(Duration::parseOrNull)
                if (remindInParsed == null) {
```

```
                    println(buildString {
                        append("The 'remind' command requires a 'Duration' argument")
                        if (remindIn != null) {
                            append(", got '$remindIn'")
                        }
                    })
                    println()
                    printRemindSyntax()
                } else {
                    scheduleReminder(remindInParsed, command)
                    println("Scheduled a reminder in $remindInParsed")
                    println("    (fires at ${Clock.System.now() + remindInParsed})")
                }
            }
            "help" -> printHelp()
            "fun_animation" -> {
                for (frame in funAnimationFrames) {
                    println(frame)
                    Thread.sleep(200) // 200 milliseconds
                }
            }
            "quit" -> break
            else -> {
                println("Unknown command '$operationInLowerCase'")
                println()
                printHelp()
            }
        }
    }
    println("All done! The program will exit once all pending reminders fire.")
}

private fun printHelp() {
    println("Usage:")
    printRemindSyntax()
    printFunAnimationSyntax()
    printHelpSyntax()
    printQuitSyntax()
    println()
}

private fun printRemindSyntax() {
    println("remind duration [message] - remind about something")
    println("\texample: remind 10m Turn off stove")
}

private fun printFunAnimationSyntax() {
    println("fun_animation - show a fun animation to pass the time")
}

private fun printHelpSyntax() {
    println("help - show a list of available commands")
}

private fun printQuitSyntax() {
    println("quit - exit this program")
```

```
}

private val funAnimationFrames = (
    "#       _---_#     / o o o \\#    <==========>####       o#    " +
    "  /|\\#      / \\#~@##       _---_#     / o o o \\#    <=======" +
    "==>###       o#      /|\\#      / \\#~@###       _---_#     / " +
    "o o o \\#    <==========>##       o#      /|\\#      / \\#~@## " +
    "      _---_#     / o o o \\#    <==========>#     /       \\# " +
    "   /         \\#   /    o      \\#       /|\\#       / \\#~@## " +
    "      _---_#     / o o o \\#    <==========>#     /       \\# " +
    "   /   o     \\#   /   /|\\     \\#       / \\##~@##       _---" +
    "_#     / o o o \\#    <==========>#     /  o    \\#    /  /|\\" +
    "    \\#   /   / \\     \\###~@##       _---_#     / o o o \\#  " +
    "  <==========>#     /       \\#    /         \\#   /          " +
    " \\###~@##       _---_#     / - - - \\#    <==========>######~" +
    "@#       _---_#     / - - - \\#    <==========>#       * * *#" +
    "#####~@       * * *#         *#########~"
).replace("~", "==============================")
    .replace("#", "\n")
    .split("@")
```

Actor-based shared resource coordination

When concurrent operations use the same resource, they need to coordinate how they use it.

One way to coordinate access to a shared resource is to use an *actor-based approach*, where a single component controls access to the resource and processes requests one at a time. If the component is already processing a request, new requests wait in a queue.

> **Tip:** Many UI frameworks use a similar approach with a *UI thread*. The UI thread processes requests to update UI elements one at a time.

In `src/main/kotlin/Example2.kt`, the reminder application uses a `SimpleActor` class to coordinate access to the console. An actor runs requests on a dedicated thread in the order it receives them:

```
// Coordinates access to a shared resource by processing requests one at a time
class SimpleActor {
    private val requests = mutableListOf<() -> Unit>()

    init {
        Thread {
            while (true) {
                // Gets the next request from the queue when available
                val request = /* ... */

                // Executes one request at a time
                request()
            }
        }.apply {
            // Allows the JVM to exit while the actor thread is still running
            isDaemon = true
```

```
            // Starts the actor thread
            start()
        }
    }

    fun sendRequest(request: () -> Unit) {
        // Adds the request to the queue
        /* ... */
    }
}
```

Instead of calling the `println()` function directly, operations send requests to the same actor. For example, a reminder sends its output to the actor after its specified duration elapses:

```
fun scheduleReminder(
    printlnActor: SimpleActor,
    remindIn: Duration,
    reminderText: List<String>
) {
    Thread {
        Thread.sleep(remindIn.inWholeMilliseconds)

        val stringToPrint = /* Creates the reminder text */

        // Sends the reminder output to the actor
        printlnActor.sendRequest {
            println(stringToPrint)
        }
    }.apply {
        isDaemon = false
        start()
    }
}
```

The `fun_animation` command also sends its output to the actor, with all animation frames grouped into a single request:

```
// Sends the animation output to the actor as a single request
"fun_animation" -> printlnActor.sendRequest {
    for (frame in funAnimationFrames) {
        println(frame)
        Thread.sleep(200)
    }
}
```

In this example, the `sendRequest()` function submits a request to the actor and returns without waiting for the request to finish.

The actor processes requests one at a time. If a reminder is triggered while the actor is processing another request, its print request waits in the queue until the current request finishes. This prevents interleaving issues, such as reminder text appearing in the middle of the animation.

However, the actor only coordinates the requests it processes. It doesn't determine when the main thread continues, so the main thread can keep accepting user input while the actor is still processing an earlier request. This matters when work on the main thread depends on that request finishing.

Here's the complete implementation for this version of the reminder application:

```
import kotlin.time.Clock
import kotlin.time.Duration

@Suppress("PLATFORM_CLASS_MAPPED_TO_KOTLIN")
class SimpleActor {
    private val requests = mutableListOf<() -> Unit>()

    init {
        Thread {
            while (true) {
                val request = synchronized(requests) {
                    val request = requests.removeFirstOrNull()
                    if (request == null) {
                        (requests as Object).wait()
                        continue
                    }
                    request
                }
                request()
            }
        }.apply {
            isDaemon = true
            start()
        }
    }

    /**
     * Sends a request that will execute on the thread of this actor.
     *
     * Requests to any given actor execute one-by-one,
     * never in parallel.
     */
    fun sendRequest(
        request: () -> Unit,
    ) {
        synchronized(requests) {
            requests.add(request)
            (requests as Object).notify()
        }
    }
}

fun scheduleReminder(
    printlnActor: SimpleActor,
    remindIn: Duration,
    reminderText: List<String>
) {
```

```
    Thread {
        Thread.sleep(remindIn.inWholeMilliseconds)
        val stringToPrint = buildString {
            append("*** REMINDER (SCHEDULED $remindIn AGO)")
            if (reminderText.isNotEmpty()) {
                append(": ")
                for (word in reminderText) {
                    append(word)
                    append(' ')
                }
            } else {
                append(' ')
            }
            append("***")
        }
        printlnActor.sendRequest {
            println(stringToPrint)
        }
    }.apply {
        isDaemon = false
        start()
    }
}

fun main() {
    val printlnActor = SimpleActor()
    while (true) {
        printlnActor.sendRequest {
            println("Please enter your input:")
        }
        val input = readlnOrNull() ?: return
        val command = input
            .trim().split("\\s".toRegex())
            .toMutableList()
        val operation = command.removeFirstOrNull() ?: continue
        when (val operationInLowerCase = operation.lowercase()) {
            "remind" -> {
                val remindIn = command.removeFirstOrNull()
                val remindInParsed = remindIn?.let(Duration::parseOrNull)
                if (remindInParsed == null) {
                    printlnActor.sendRequest {
                        println(buildString {
                            append("The 'remind' command requires a 'Duration' argument")
                            if (remindIn != null) {
                                append(", got '$remindIn'")
                            }
                        })
                        println()
                        printRemindSyntax()
                    }
                } else {
                    scheduleReminder(printlnActor, remindInParsed, command)
                    printlnActor.sendRequest {
                        println("Scheduled a reminder in $remindInParsed")
                        println("    (fires at ${Clock.System.now() + remindInParsed})")
                    }
```

```
                }
            }
            "help" -> printlnActor.sendRequest {
                printHelp()
            }
            "fun_animation" -> printlnActor.sendRequest {
                for (frame in funAnimationFrames) {
                    println(frame)
                    Thread.sleep(200) // 200 milliseconds
                }
            }
            "quit" -> break
            else -> printlnActor.sendRequest {
                println("Unknown command '$operationInLowerCase'")
                println()
                printHelp()
            }
        }
    }
    printlnActor.sendRequest {
        println("All done! The program will exit once all pending reminders fire.")
    }
}

private fun printHelp() {
    println("Usage:")
    printRemindSyntax()
    printFunAnimationSyntax()
    printHelpSyntax()
    printQuitSyntax()
    println()
}

private fun printRemindSyntax() {
    println("remind duration [message] - remind about something")
    println("\texample: remind 10m Turn off stove")
}

private fun printFunAnimationSyntax() {
    println("fun_animation - show a fun animation to pass the time")
}

private fun printHelpSyntax() {
    println("help - show a list of available commands")
}

private fun printQuitSyntax() {
    println("quit - exit this program")
}

private val funAnimationFrames = (
    "#       _---_#     / o o o \\#    <==========>####       o#    " +
    "  /|\\#      / \\#~@##       _---_#     / o o o \\#    <=======" +
    "==>###       o#      /|\\#      / \\#~@###       _---_#     / " +
    "o o o \\#    <==========>##       o#      /|\\#      / \\#~@## " +
    "      _---_#     / o o o \\#    <==========>#     /       \\# " +
```

```
    "   /         \\#   /    o      \\#       /|\\#       / \\#~@## " +
    "      _---_#     / o o o \\#    <==========>#     /       \\# " +
    "   /   o     \\#   /   /|\\     \\#       / \\##~@##       _---" +
    "_#     / o o o \\#    <==========>#     /  o    \\#    /  /|\\" +
    "    \\#   /   / \\     \\###~@##       _---_#     / o o o \\#  " +
    "  <==========>#     /       \\#    /         \\#   /          " +
    " \\###~@##       _---_#     / - - - \\#    <==========>######~" +
    "@#       _---_#     / - - - \\#    <==========>#       * * *#" +
    "#####~@       * * *#         *#########~"
).replace("~", "==============================")
    .replace("#", "\n")
    .split("@")
```

Callbacks

When some work has to start after an asynchronous operation finishes, an application needs to control when that subsequent work runs. You can define the subsequent work in a *callback*. A callback is a function that you pass to another function to invoke later when a specified condition is met, such as after an operation finishes.

In `src/main/kotlin/Example3.kt`, the reminder application uses callbacks to resume processing user input only after the animation finishes. To achieve this, the example adds a second `SimpleActor` for processing user input:

```
fun main() {
    val printlnActor = SimpleActor()
    val userInputActor = SimpleActor()

    // Passes a callback that starts processing user input
    userInputActor.sendRequest {
        processUserInput(
            printlnActor,
            userInputActor,
            // Runs after input processing stops
            doLast = {
                printlnActor.sendRequest {
                    println("All done! The program will exit once all pending reminders fir
e.")
                }
            }
        )
    }

    // Keeps the main thread running until the application can terminate
    while (!shouldTerminate.load()) {
        Thread.sleep(100)
    }
}
```

The `userInputActor` calls the `processUserInput()` function on its dedicated thread. Unlike the `while` loop in the actor-based shared resource coordination implementation, each call to the

`processUserInput()` function reads and processes only one command.

After processing a `remind` command, a `help` command, or an unknown command, the function sends another request to `userInputActor`:

```
fun processUserInput(
    printlnActor: SimpleActor,
    userInputActor: SimpleActor,
    doLast: () -> Unit
) {
    // Processes the current command
    /* ... */

    // Schedules processing of the next command
    userInputActor.sendRequest {
        processUserInput(printlnActor, userInputActor, doLast)
    }
}
```

The `userInputActor` queues the callback instead of invoking it immediately. The current call to the `processUserInput()` function returns before the actor processes the next request. This starts another input-processing operation, similar to starting the next iteration of a loop.

For the `fun_animation` command, the application delays the next input request until the animation finishes. The request sent to `printlnActor` prints all animation frames before it requests processing of the next command:

```
"fun_animation" -> {
    // Sends a request to print the animation
    printlnActor.sendRequest {
        for (frame in funAnimationFrames) {
            println(frame)
            Thread.sleep(200)
        }

        // Passes a callback that processes the next command
        // after the animation finishes
        userInputActor.sendRequest {
            processUserInput(printlnActor, userInputActor, doLast)
        }
    }

    // Prevents processUserInput() from scheduling another input request
    // before the animation finishes
    return
}
```

For the `quit` command, the function calls the `doLast` callback instead of scheduling another input request:

```
"quit" -> {
    // Invokes the callback that runs after input processing stops
    doLast()
    return
}
```

With callbacks, each execution path must explicitly schedule the work that follows it. The application has to schedule another input-processing operation, start an asynchronous operation that schedules input processing when it finishes, or invoke the final callback.

As control flows become more complex, omitting one of these actions, performing it more than once, or performing it on the wrong execution path can change the application's behavior. This makes the code harder to maintain and more error-prone.

Here's the complete implementation for this version of the reminder application:

```
@file:OptIn(kotlin.concurrent.atomics.ExperimentalAtomicApi::class)
import kotlin.concurrent.atomics.AtomicBoolean
import kotlin.time.Clock
import kotlin.time.Duration

@Suppress("PLATFORM_CLASS_MAPPED_TO_KOTLIN")
class SimpleActor {
    private val requests = mutableListOf<() -> Unit>()

    init {
        Thread {
            while (true) {
                val request = synchronized(requests) {
                    val request = requests.removeFirstOrNull()
                    if (request == null) {
                        (requests as Object).wait()
                        continue
                    }
                    request
                }
                request()
            }
        }.apply {
            isDaemon = true
            start()
        }
    }

    /**
     * Sends a request that will execute on the thread of this actor.
     *
     * Requests to any given actor execute one-by-one,
     * never in parallel.
     */
    fun sendRequest(
        request: () -> Unit,
    ) {
```

```
        synchronized(requests) {
            requests.add(request)
            (requests as Object).notify()
        }
    }
}

fun scheduleReminder(
    printlnActor: SimpleActor,
    remindIn: Duration,
    reminderText: List<String>
) {
    Thread {
        Thread.sleep(remindIn.inWholeMilliseconds)
        val stringToPrint = buildString {
            append("*** REMINDER (SCHEDULED $remindIn AGO)")
            if (reminderText.isNotEmpty()) {
                append(": ")
                for (word in reminderText) {
                    append(word)
                    append(' ')
                }
            } else {
                append(' ')
            }
            append("***")
        }
        printlnActor.sendRequest {
            println(stringToPrint)
        }
    }.apply {
        isDaemon = false
        start()
    }
}

val shouldTerminate = AtomicBoolean(false)

fun processUserInput(
    printlnActor: SimpleActor,
    userInputActor: SimpleActor,
    doLast: () -> Unit
) {
    printlnActor.sendRequest {
        println("Please enter your input:")
    }
    val input = readlnOrNull() ?: return
    val command = input
        .trim().split("\\s".toRegex())
        .toMutableList()
    val operation = command.removeFirstOrNull()
    if (operation == null) {
        /* We emulate the `continue` from the earlier versions */
        userInputActor.sendRequest {
            processUserInput(printlnActor, userInputActor, doLast)
        }
```

```
        return
    }
    when (val operationInLowerCase = operation.lowercase()) {
        "remind" -> {
            val remindIn = command.removeFirstOrNull()
            val remindInParsed = remindIn?.let(Duration::parseOrNull)
            if (remindInParsed == null) {
                printlnActor.sendRequest {
                    println(buildString {
                        append("The 'remind' command requires a 'Duration' argument")
                        if (remindIn != null) {
                            append(", got '$remindIn'")
                        }
                    })
                    println()
                    printRemindSyntax()
                }
            } else {
                scheduleReminder(printlnActor, remindInParsed, command)
                printlnActor.sendRequest {
                    println("Scheduled a reminder in $remindInParsed")
                    println("    (fires at ${Clock.System.now() + remindInParsed})")
                }
            }
        }
        "help" -> printlnActor.sendRequest {
            printHelp()
        }
        "fun_animation" -> {
            printlnActor.sendRequest {
                for (frame in funAnimationFrames) {
                    println(frame)
                    Thread.sleep(200) // 200 milliseconds
                }
                userInputActor.sendRequest {
                    processUserInput(printlnActor, userInputActor, doLast)
                }
            }
            /* We don't need to reschedule reading user input, because the
            println actor will do that on its own once it finishes. */
            return
        }
        "quit" -> {
            /* We emulate the `break` from the earlier versions by exiting the
            current iteration without rescheduling a new one, instead scheduling
            the code that comes *after* the loop. */
            doLast()
            return
        }
        else -> printlnActor.sendRequest {
            println("Unknown command '$operationInLowerCase'")
            println()
            printHelp()
        }
    }
    // Scheduling the next iteration of the loop
```

```
    userInputActor.sendRequest {
        processUserInput(printlnActor, userInputActor, doLast)
    }
}

fun main() {
    val printlnActor = SimpleActor()
    val userInputActor = SimpleActor()
    userInputActor.sendRequest {
        processUserInput(printlnActor, userInputActor, doLast = {
            printlnActor.sendRequest {
                println("All done! The program will exit once all pending reminders fire.")
                // Allow the main thread to exit
                shouldTerminate.store(true)
            }
        })
    }
    // Keep the program alive: if we simply exit the main thread,
    // everything will terminate.
    while (!shouldTerminate.load()) {
        Thread.sleep(100)
    }
}

private fun printHelp() {
    println("Usage:")
    printRemindSyntax()
    printFunAnimationSyntax()
    printHelpSyntax()
    printQuitSyntax()
    println()
}

private fun printRemindSyntax() {
    println("remind duration [message] - remind about something")
    println("\texample: remind 10m Turn off stove")
}

private fun printFunAnimationSyntax() {
    println("fun_animation - show a fun animation to pass the time")
}

private fun printHelpSyntax() {
    println("help - show a list of available commands")
}

private fun printQuitSyntax() {
    println("quit - exit this program")
}

private val funAnimationFrames = (
    "#       _---_#     / o o o \\#    <==========>####       o#    " +
    "  /|\\#      / \\#~@##       _---_#     / o o o \\#    <=======" +
    "==>###       o#      /|\\#      / \\#~@###       _---_#     / " +
    "o o o \\#    <==========>##       o#      /|\\#      / \\#~@## " +
    "      _---_#     / o o o \\#    <==========>#     /       \\# " +
```

```
    "   /         \\#   /    o      \\#       /|\\#       / \\#~@## " +
    "      _---_#     / o o o \\#    <==========>#     /       \\# " +
    "   /   o     \\#   /   /|\\     \\#       / \\##~@##       _---" +
    "_#     / o o o \\#    <==========>#     /  o    \\#    /  /|\\" +
    "    \\#   /   / \\     \\###~@##       _---_#     / o o o \\#  " +
    "  <==========>#     /       \\#    /         \\#   /          " +
    " \\###~@##       _---_#     / - - - \\#    <==========>######~" +
    "@#       _---_#     / - - - \\#    <==========>#       * * *#" +
    "#####~@       * * *#         *#########~"
).replace("~", "==============================")
    .replace("#", "\n")
    .split("@")
```

Control-flow abstractions

To make callback-based control flow easier to follow, you can move repeated scheduling logic into helper functions that represent familiar control-flow structures, such as loops.

In `src/main/kotlin/Example4.kt`, the reminder application uses a `runInfiniteLoop()` function to process user input asynchronously. Each loop iteration returns a `LoopIterationResult` enum value that determines whether the loop continues, exits, or resumes later after an asynchronous operation finishes:

```
enum class LoopIterationResult {
    BREAK,
    CONTINUE,
    WILL_BE_RESUMED_ASYNCHRONOUSLY
}
```

The `runInfiniteLoop()` function handles each result and schedules the next iteration when necessary:

```
// Runs the input-processing loop asynchronously
fun runInfiniteLoop(
    schedule: (RequestType) -> Unit,
    iteration: (scheduleNextIteration: () -> Unit) -> LoopIterationResult,
    actionOnLoopExit: () -> Unit,
) {
    fun helper() {
        when (iteration { schedule(::helper) }) {
            LoopIterationResult.BREAK -> {
                // Runs the action that follows the loop
                actionOnLoopExit()
            }

            LoopIterationResult.CONTINUE -> {
                // Schedules the next loop iteration
                schedule(::helper)
            }

            LoopIterationResult.WILL_BE_RESUMED_ASYNCHRONOUSLY -> {
```

```
                // Another operation schedules the next iteration
            }
        }
    }

    // Schedules the first loop iteration
    schedule(::helper)
}
```

The reminder application passes its input-processing logic to the `runInfiniteLoop()` function. For execution paths that can continue immediately, the iteration returns `CONTINUE`, similarly to calling `continue` in a loop:

```
val operation = command.removeFirstOrNull()
    ?: return@iteration LoopIterationResult.CONTINUE
```

For operations that resume later, the iteration returns `WILL_BE_RESUMED_ASYNCHRONOUSLY`. For example, the `fun_animation` command schedules the next iteration after the animation finishes:

```
"fun_animation" -> {
    printlnActor.sendRequest {
        for (frame in funAnimationFrames) {
            println(frame)
            Thread.sleep(200)
        }

        // Schedules the next iteration after the animation finishes
        userInputActor.sendRequest {
            scheduleNextIteration()
        }
    }

    return@iteration LoopIterationResult.WILL_BE_RESUMED_ASYNCHRONOUSLY
}
```

Finally, to stop processing user input, the iteration returns `BREAK`:

```
"quit" -> {
    return@iteration LoopIterationResult.BREAK
}
```

As you can see, the `runInfiniteLoop()` function centralizes the logic for continuing and exiting the loop instead of requiring each execution path to schedule the next operation directly.

However, correctly implementing control-flow abstractions using functions such as `runInfiniteLoop()` can be difficult. For example, this implementation doesn't handle exceptions thrown from the loop iteration, the schedule callback, or the `actionOnLoopExit` callback.

Here's the complete implementation for this version of the reminder application:

```
@file:OptIn(kotlin.concurrent.atomics.ExperimentalAtomicApi::class)
import kotlin.concurrent.atomics.AtomicBoolean
import kotlin.time.Clock
import kotlin.time.Duration

typealias RequestType = () -> Unit

@Suppress("PLATFORM_CLASS_MAPPED_TO_KOTLIN")
class SimpleActor {
    private val requests = mutableListOf<RequestType>()

    init {
        Thread {
            while (true) {
                val request = synchronized(requests) {
                    val request = requests.removeFirstOrNull()
                    if (request == null) {
                        (requests as Object).wait()
                        continue
                    }
                    request
                }
                request()
            }
        }.apply {
            isDaemon = true
            start()
        }
    }

    /**
     * Sends a request that will execute on the thread of this actor.
     *
     * Requests to any given actor execute one-by-one,
     * never in parallel.
     */
    fun sendRequest(
        request: RequestType,
    ) {
        synchronized(requests) {
            requests.add(request)
            (requests as Object).notify()
        }
    }
}

fun scheduleReminder(
    printlnActor: SimpleActor,
    remindIn: Duration,
    reminderText: List<String>
) {
    Thread {
        Thread.sleep(remindIn.inWholeMilliseconds)
        val stringToPrint = buildString {
            append("*** REMINDER (SCHEDULED $remindIn AGO)")
            if (reminderText.isNotEmpty()) {
```

```
                append(": ")
                for (word in reminderText) {
                    append(word)
                    append(' ')
                }
            } else {
                append(' ')
            }
            append("***")
        }
        println1Actor.sendRequest {
            println(stringToPrint)
        }
    }.apply {
        isDaemon = false
        start()
    }
}

// NEW APIS ////////////////////////////////////////////////////////////////

enum class LoopIterationResult {
    BREAK,
    CONTINUE,
    WILL_BE_RESUMED_ASYNCHRONOUSLY
}

/**
 * Runs an infinite loop asynchronously.
 *
 * [schedule] is the command that will be invoked to schedule
 * the next loop iteration.
 *
 * [iteration] is the loop body.
 * It accepts a value of type `() -> Unit` invoking which will cause
 * the next iteration to be scheduled.
 * Use this in conjunction with
 * [LoopIterationResult.WILL_BE_RESUMED_ASYNCHRONOUSLY].
 *
 * [actionOnLoopExit] will be run once the loop exits.
 *
 * This function returns immediately after scheduling the first iteration.
 */
fun runInfiniteLoop(
    schedule: (RequestType) -> Unit,
    iteration: (scheduleNextIteration: () -> Unit) -> LoopIterationResult,
    actionOnLoopExit: () -> Unit,
) {
    fun helper() {
        when (iteration { schedule(::helper) }) {
            LoopIterationResult.BREAK -> {
                /* We exited the loop. No need to reschedule anything. */
                actionOnLoopExit()
            }
            LoopIterationResult.CONTINUE -> {
                /* Schedule the next loop iteration */
```

```
                schedule(::helper)
            }
            LoopIterationResult.WILL_BE_RESUMED_ASYNCHRONOUSLY -> {
                /* No need to do anything, someone else will resume the loop. */
            }
        }
    }
    schedule(::helper)
}

// NEW API USAGE ////////////////////////////////////////////////////////////

val shouldTerminate = AtomicBoolean(false)

fun main() {
    val printlnActor = SimpleActor()
    val userInputActor = SimpleActor()

    runInfiniteLoop(
        schedule = userInputActor::sendRequest,
        iteration = iteration@{ scheduleNextIteration ->
            printlnActor.sendRequest {
                println("Please enter your input:")
            }
            val input = readlnOrNull()
                ?: return@iteration LoopIterationResult.BREAK
            val command = input
                .trim().split("\\s".toRegex())
                .toMutableList()
            val operation = command.removeFirstOrNull()
                ?: return@iteration LoopIterationResult.CONTINUE
            when (val operationInLowerCase = operation.lowercase()) {
                "remind" -> {
                    val remindIn = command.removeFirstOrNull()
                    val remindInParsed = remindIn?.let(Duration::parseOrNull)
                    if (remindInParsed == null) {
                        printlnActor.sendRequest {
                            println(buildString {
                                append("The 'remind' command requires a 'Duration' argumen
t")
                                if (remindIn != null) {
                                    append(", got '$remindIn'")
                                }
                            })
                            println()
                            printRemindSyntax()
                        }
                    } else {
                        scheduleReminder(printlnActor, remindInParsed, command)
                        printlnActor.sendRequest {
                            println("Scheduled a reminder in $remindInParsed")
                            println("    (fires at ${Clock.System.now() + remindInParsed})")
                        }
                    }
                }
                "help" -> printlnActor.sendRequest {
```

```
                    printHelp()
                }
                "fun_animation" -> {
                    printlnActor.sendRequest {
                        for (frame in funAnimationFrames) {
                            println(frame)
                            Thread.sleep(200) // 200 milliseconds
                        }
                        userInputActor.sendRequest {
                            scheduleNextIteration()
                        }
                    }
                    return@iteration LoopIterationResult.WILL_BE_RESUMED_ASYNCHRONOUSLY
                }
                "quit" -> {
                    return@iteration LoopIterationResult.BREAK
                }
                else -> printlnActor.sendRequest {
                    println("Unknown command '$operationInLowerCase'")
                    println()
                    printHelp()
                }
            }
            LoopIterationResult.CONTINUE
        },
        actionOnLoopExit = {
            printlnActor.sendRequest {
                println("All done! The program will exit once all pending reminders fire.")
                // Allow the main thread to exit
                shouldTerminate.store(true)
            }
        }
    )
    // Keep the program alive: if we simply exit the main thread,
    // everything will terminate.
    while (!shouldTerminate.load()) {
        Thread.sleep(100)
    }
}

private fun printHelp() {
    println("Usage:")
    printRemindSyntax()
    printFunAnimationSyntax()
    printHelpSyntax()
    printQuitSyntax()
    println()
}

private fun printRemindSyntax() {
    println("remind duration [message] - remind about something")
    println("\texample: remind 10m Turn off stove")
}

private fun printFunAnimationSyntax() {
    println("fun_animation - show a fun animation to pass the time")
```

```
}

private fun printHelpSyntax() {
    println("help - show a list of available commands")
}

private fun printQuitSyntax() {
    println("quit - exit this program")
}

private val funAnimationFrames = (
    "#       _---_#     / o o o \\#    <=========>####       o#    " +
    "  /|\\#      / \\#~@##       _---_#     / o o o \\#    <=======" +
    "==>###       o#      /|\\#      / \\#~@###       _---_#     / " +
    "o o o \\#    <=========>##       o#      /|\\#      / \\#~@## " +
    "       _---_#     / o o o \\#    <=========>#     /       \\# " +
    "   /         \\#   /    o      \\#       /|\\#       / \\#~@## " +
    "       _---_#     / o o o \\#    <=========>#     /       \\# " +
    "   /   o     \\#   /   /|\\     \\#       / \\##~@##       _---" +
    "_#     / o o o \\#    <=========>#     /  o    \\#    /  /|\\" +
    "    \\#   /   / \\     \\###~@##       _---_#     / o o o \\#  " +
    "  <=========>#     /       \\#    /         \\#   /          " +
    " \\###~@##       _---_#     / - - - \\#    <=========>######~" +
    "@#       _---_#     / - - - \\#    <=========>#       * * *#" +
    "#####~@       * * *#         *#########~"
).replace("~", "==============================")
    .replace("#", "\n")
    .split("@")
```

Suspending functions

Callback-based control flow can become difficult to maintain as the number of dependent operations grows. Kotlin provides *suspending functions*, which let you express asynchronous control flow in a clear, sequential style.

With callbacks, you define the work that follows an asynchronous operation separately and schedule it as a callback. A suspending function can instead pause at a *suspension point* and continue later from that point. This preserves familiar control-flow structures without requiring you to express each subsequent step as a separately scheduled callback.

To declare a suspending function, use the `suspend` keyword.

In `src/main/kotlin/Example5.kt`, the reminder application uses suspending functions to process user input in the `suspendMain()` function:

```
// Processes user input in a suspending function
suspend fun suspendMain(
    printlnActor: SimpleActor,
    userInputActor: SimpleActor
) {
    while (true) {
```

```
        printlnActor.sendRequest {
            println("Please enter your input:")
        }

        val input = readlnOrNull() ?: break
        val command = input
            .trim().split("\\s".toRegex())
            .toMutableList()
        val operation = command.removeFirstOrNull() ?: continue

        when (val operationInLowerCase = operation.lowercase()) {
            // Handles other commands
            /* ... */

            "quit" -> break
        }
    }

    printlnActor.sendRequest {
        println("All done! The program will exit once all pending reminders fire.")
        shouldTerminate.store(true)
    }
}
```

The `suspendMain()` function can use a `while` loop, `continue`, and `break` directly instead of manually coordinating each execution path with callbacks. Suspending functions can also use familiar constructs such as `return`, `try`, `catch`, and `finally`, and call other suspending functions.

> **Note:** Recreating this behavior with control-flow abstractions such as the `runInfiniteLoop()` helper function would require increasingly complex scheduling logic.

For the `fun_animation` command, the `suspendCoroutine()` function creates a suspension point, suspends while the animation runs, and resumes after it finishes:

```
"fun_animation" -> {
    // Suspends execution until another operation resumes it
    suspendCoroutine { continuation ->
        printlnActor.sendRequest {
            for (frame in funAnimationFrames) {
                println(frame)
                Thread.sleep(200)
            }

            userInputActor.sendRequest {
                // Resumes execution after the animation finishes
                continuation.resumeWith(Result.success(Unit))
            }
        }
    }
}
```

Here, the `suspendCoroutine()` function suspends execution and provides an implementation of the `Continuation` interface that represents the execution that follows the suspension point. This is similar to how the `runInfiniteLoop()` function in control-flow abstractions uses the `helper()` function to represent the next loop iteration to schedule.

Compared to using callbacks, suspending functions don't require you to define the subsequent work separately. The compiler creates a continuation that represents the execution following a suspension point.

After the animation finishes, the `Continuation.resumeWith()` function resumes that execution with a result. The `suspendMain()` function then continues after the suspension point and starts the next loop iteration.

> **Warning:** The `suspendCoroutine()` function is used here only to demonstrate how suspension works. Don't use it in production code that needs to support cancellation.

To run the suspending `suspendMain()` function from the `main()` function, the application starts a *coroutine*, a suspendable computation that can pause and resume execution. The `main()` function wraps the `suspendMain()` call in a suspending lambda and starts the coroutine with the `startCoroutineUninterceptedOrReturn()` function:

```
userInputActor.sendRequest {
    suspend {
        suspendMain(printlnActor, userInputActor)
    }.startCoroutineUninterceptedOrReturn(
        Continuation<Unit>(EmptyCoroutineContext) {
        }
    )
}
```

With suspending functions, the application can express dependencies between asynchronous operations without manually scheduling each execution path. This keeps the order of operations visible in the code and reduces the amount of control-flow logic you need to maintain.

Here's the complete implementation for this version of the reminder application:

```
@file:OptIn(kotlin.concurrent.atomics.ExperimentalAtomicApi::class)
import kotlin.concurrent.atomics.AtomicBoolean
import kotlin.time.Clock
import kotlin.time.Duration
import kotlin.coroutines.*
import kotlin.coroutines.intrinsics.startCoroutineUninterceptedOrReturn

typealias RequestType = () -> Unit

@Suppress("PLATFORM_CLASS_MAPPED_TO_KOTLIN")
class SimpleActor {
    private val requests = mutableListOf<RequestType>()
```

```
    init {
        Thread {
            while (true) {
                val request = synchronized(requests) {
                    val request = requests.removeFirstOrNull()
                    if (request == null) {
                        (requests as Object).wait()
                        continue
                    }
                    request
                }
                request()
            }
        }.apply {
            isDaemon = true
            start()
        }
    }

    /**
     * Sends a request that will execute on the thread of this actor.
     *
     * Requests to any given actor execute one-by-one,
     * never in parallel.
     */
    fun sendRequest(
        request: RequestType,
    ) {
        synchronized(requests) {
            requests.add(request)
            (requests as Object).notify()
        }
    }
}

fun scheduleReminder(
    printlnActor: SimpleActor,
    remindIn: Duration,
    reminderText: List<String>
) {
    Thread {
        Thread.sleep(remindIn.inWholeMilliseconds)
        val stringToPrint = buildString {
            append("*** REMINDER (SCHEDULED $remindIn AGO)")
            if (reminderText.isNotEmpty()) {
                append(": ")
                for (word in reminderText) {
                    append(word)
                    append(' ')
                }
            } else {
                append(' ')
            }
            append("***")
        }
        printlnActor.sendRequest {
```

```
            println(stringToPrint)
        }
    }.apply {
        isDaemon = false
        start()
    }
}

val shouldTerminate = AtomicBoolean(false)

suspend fun suspendMain(printlnActor: SimpleActor, userInputActor: SimpleActor) {
    while (true) {
        printlnActor.sendRequest {
            println("Please enter your input:")
        }
        val input = readlnOrNull() ?: break
        val command = input
            .trim().split("\\s".toRegex())
            .toMutableList()
        val operation = command.removeFirstOrNull() ?: continue
        when (val operationInLowerCase = operation.lowercase()) {
            "remind" -> {
                val remindIn = command.removeFirstOrNull()
                val remindInParsed = remindIn?.let(Duration::parseOrNull)
                if (remindInParsed == null) {
                    printlnActor.sendRequest {
                        println(buildString {
                            append("The 'remind' command requires a 'Duration' argument")
                            if (remindIn != null) {
                                append(", got '$remindIn'")
                            }
                        })
                        println()
                        printRemindSyntax()
                    }
                } else {
                    scheduleReminder(printlnActor, remindInParsed, command)
                    printlnActor.sendRequest {
                        println("Scheduled a reminder in $remindInParsed")
                        println("    (fires at ${Clock.System.now() + remindInParsed})")
                    }
                }
            }
            "help" -> printlnActor.sendRequest {
                printHelp()
            }
            "fun_animation" -> {
                suspendCoroutine { cont ->
                    printlnActor.sendRequest {
                        for (frame in funAnimationFrames) {
                            println(frame)
                            Thread.sleep(200) // 200 milliseconds
                        }
                        userInputActor.sendRequest {
                            cont.resumeWith(Result.success(Unit))
                        }
```

```
                    }
                }
            }
            "quit" -> {
                break
            }
            else -> printlnActor.sendRequest {
                println("Unknown command '$operationInLowerCase'")
                println()
                printHelp()
            }
        }
    }
    printlnActor.sendRequest {
        println("All done! The program will exit once all pending reminders fire.")
        // Allow the main thread to exit
        shouldTerminate.store(true)
    }
}

fun main() {
    val printlnActor = SimpleActor()
    val userInputActor = SimpleActor()

    userInputActor.sendRequest {
        suspend {
            suspendMain(printlnActor, userInputActor)
        }.startCoroutineUninterceptedOrReturn(
            Continuation<Unit>(EmptyCoroutineContext) {
            }
        )
    }

    // Keep the program alive: if we simply exit the main thread,
    // everything will terminate.
    while (!shouldTerminate.load()) {
        Thread.sleep(100)
    }
}

private fun printHelp() {
    println("Usage:")
    printRemindSyntax()
    printFunAnimationSyntax()
    printHelpSyntax()
    printQuitSyntax()
    println()
}

private fun printRemindSyntax() {
    println("remind duration [message] - remind about something")
    println("\texample: remind 10m Turn off stove")
}

private fun printFunAnimationSyntax() {
    println("fun_animation - show a fun animation to pass the time")
```

```
}

private fun printHelpSyntax() {
    println("help - show a list of available commands")
}

private fun printQuitSyntax() {
    println("quit - exit this program")
}

private val funAnimationFrames = (
    "#       _---_#     / o o o \\#    <=========>####       o#    " +
    "  /|\\#      / \\#~@##       _---_#     / o o o \\#    <=======" +
    "==>###       o#      /|\\#      / \\#~@###       _---_#     / " +
    "o o o \\#    <=========>##       o#      /|\\#      / \\#~@## " +
    "       _---_#     / o o o \\#    <=========>#     /       \\# " +
    "   /         \\#   /    o      \\#       /|\\#       / \\#~@## " +
    "       _---_#     / o o o \\#    <=========>#     /       \\# " +
    "   /   o     \\#   /   /|\\     \\#       / \\##~@##       _---" +
    "_#     / o o o \\#    <=========>#     /  o    \\#    /  /|\\" +
    "    \\#   /   / \\     \\###~@##       _---_#     / o o o \\#  " +
    "  <=========>#     /       \\#    /         \\#   /          " +
    " \\###~@##       _---_#     / - - - \\#    <=========>######~" +
    "@#       _---_#     / - - - \\#    <=========>#       * * *#" +
    "#####~@       * * *#         *#########~"
).replace("~", "==============================")
    .replace("#", "\n")
    .split("@")
```

What's next

Learn more about coroutines and how they support asynchronous and concurrent programming in Coroutines.

Coroutines

Applications often need to perform multiple tasks at the same time, such as responding to user input, loading data, or updating the screen. To support this, they rely on concurrency, which allows operations to run independently without blocking each other.

The most common way to run tasks concurrently is by using threads, which are independent paths of execution managed by the operating system. However, threads are relatively heavy, and creating many of them can lead to performance issues.

To support efficient concurrency, Kotlin uses asynchronous programming built around *coroutines*, which let you write asynchronous code in a natural, sequential style using suspending functions. Coroutines are lightweight alternatives to threads. They can suspend without blocking system resources and are resource-friendly, making them better suited for fine-grained concurrency.

Most coroutine features are provided by the `kotlinx.coroutines` library, which includes tools for launching coroutines, handling concurrency, working with asynchronous streams, and more.

If you're new to coroutines in Kotlin, start with the Coroutine basics guide before diving into more complex topics. This guide introduces the key concepts of suspending functions, coroutine builders, and structured concurrency through simple examples:

Get started with coroutines

> **Tip:** Check out the KotlinConf app for a sample project to see how coroutines are used in practice.

Coroutine concepts

The `kotlinx.coroutines` library provides the core building blocks for running tasks concurrently, structuring coroutine execution, and managing shared state.

Suspending functions and coroutine builders

Coroutines in Kotlin are built on suspending functions, which allow code to pause and resume without blocking a thread. The `suspend` keyword marks functions that can perform long-running operations asynchronously.

To launch new coroutines, use coroutine builders like `.launch()` and `.async()`. These builders are extension functions on `CoroutineScope`, which defines the coroutine's lifecycle and provides the coroutine context.

You can learn more about these builders in Coroutine basics and Composing suspend functions.

Coroutine context and behavior

Launching a coroutine from a `CoroutineScope` creates a context that governs its execution. Builder functions like `.launch()` and `.async()` automatically create a set of elements that define how the coroutine behaves:

- The `Job` interface tracks the coroutine's lifecycle and enables structured concurrency.
- `CoroutineDispatcher` controls where the coroutine runs, such as on a background thread or the main thread in UI applications.
- `CoroutineExceptionHandler` handles uncaught exceptions.

These, along with other possible elements, make up the *coroutine context*, which is inherited by default from the coroutine's parent. This context forms a hierarchy that enables structured concurrency, where related coroutines can be canceled together or handle exceptions as a group.

Asynchronous flow and shared mutable state

Kotlin provides several ways for coroutines to communicate. Use one of the following options based on how you want to share values between coroutines:

- `Flow` produces values only when a coroutine actively collects them.
- `Channel` allows multiple coroutines to send and receive values, with each value delivered to exactly one coroutine.
- `SharedFlow` continuously shares every value with all active collecting coroutines.

When multiple coroutines need to access or update the same data, they *share mutable state*. Without coordination, this can lead to race conditions, where operations interfere with each other in unpredictable ways. To safely manage shared mutable state, use `StateFlow` to wrap the shared data. Then, you can update it from one coroutine and collect its latest value from others.

For more information, see Flows, Channels, and the Coroutines and channels tutorial.

What's next

- Learn the fundamentals of coroutines, suspending functions, and builders in the Coroutine basics guide.
- Explore how to combine suspending functions and build coroutine pipelines in Composing suspending functions.
- Learn how to debug coroutines using built-in tools in IntelliJ IDEA.
- For flow-specific debugging, see the Debug Kotlin Flow using IntelliJ IDEA tutorial.
- Read the Guide to UI programming with coroutines to learn about coroutine-based UI development.
- Review best practices for using coroutines in Android.
- Check out the `kotlinx.coroutines` API reference.

Destructuring declarations

Sometimes it is convenient to *destructure* an object into a number of variables, for example:

```
val (name, age) = person
```

This syntax is called a *destructuring declaration*. A destructuring declaration creates multiple variables at once. You have declared two new variables: `name` and `age`, and can use them independently:

```
println(name)
println(age)
```

A destructuring declaration is compiled down to the following code:

```
val name = person.component1()
val age = person.component2()
```

The `component1()` and `component2()` functions are another example of the *principle of conventions* widely used in Kotlin (see operators like `+` and `*`, `for`-loops as an example). Anything can be on the right-hand side of a destructuring declaration, as long as the required number of component functions can be called on it. And, of course, there can be `component3()` and `component4()` and so on.

> **Note:** The `componentN()` functions need to be marked with the `operator` keyword to allow using them in a destructuring declaration.

Destructuring declarations also work in `for`-loops:

```
for ((a, b) in collection) { ... }
```

Variables `a` and `b` get the values returned by `component1()` and `component2()` called on elements of the collection.

Example: returning two values from a function

Assume that you need to return two things from a function - for example, a result object and a status of some sort. A compact way of doing this in Kotlin is to declare a data class and return its instance:

```
data class Result(val result: Int, val status: Status)
fun function(...): Result {
    // computations
```

```
    return Result(result, status)
}

// Now, to use this function:
val (result, status) = function(...)
```

Since data classes automatically declare `componentN()` functions, destructuring declarations work here.

> **Note:** You could also use the standard class `Pair` and have `function()` return `Pair<Int, Status>`, but it's often better to have your data named properly.

Example: destructuring declarations and maps

Probably the nicest way to traverse a map is this:

```
for ((key, value) in map) {
   // do something with the key and the value
}
```

To make this work, you should

- Present the map as a sequence of values by providing an `iterator()` function.
- Present each of the elements as a pair by providing functions `component1()` and `component2()`.

And indeed, the standard library provides such extensions:

```
operator fun <K, V> Map<K, V>.iterator(): Iterator<Map.Entry<K, V>> = entrySet().iterator()
operator fun <K, V> Map.Entry<K, V>.component1() = getKey()
operator fun <K, V> Map.Entry<K, V>.component2() = getValue()
```

So you can freely use destructuring declarations in `for`-loops with maps (as well as collections of data class instances or similar).

Underscore for unused variables

If you don't need a variable in the destructuring declaration, you can place an underscore instead of its name:

```
val (_, status) = getResult()
```

The `componentN()` operator functions are not called for the components that are skipped in this way.

Destructuring in lambdas

You can use the destructuring declarations syntax for lambda parameters. If a lambda has a parameter of the `Pair` type (or `Map.Entry`, or any other type that has the appropriate `componentN` functions), you can introduce several new parameters instead of one by putting them in parentheses:

```
map.mapValues { entry -> "${entry.value}!" }
map.mapValues { (key, value) -> "$value!" }
```

Note the difference between declaring two parameters and declaring a destructuring pair instead of a parameter:

```
{ a -> ... } // one parameter
{ a, b -> ... } // two parameters
{ (a, b) -> ... } // a destructured pair
{ (a, b), c -> ... } // a destructured pair and another parameter
```

If a component of the destructured parameter is unused, you can replace it with the underscore to avoid inventing its name:

```
map.mapValues { (_, value) -> "$value!" }
```

You can specify the type for the whole destructured parameter or for a specific component separately:

```
map.mapValues { (_, value): Map.Entry<Int, String> -> "$value!" }

map.mapValues { (_, value: String) -> "$value!" }
```

Name-based destructuring

Kotlin supports *name-based destructuring declarations*, where variables match properties by name instead of the position defined by `componentN()` functions in *position-based* destructuring.

> **Tip:** For more information about name-based destructuring, see the feature's KEEP.

In position-based destructuring, variables correspond to the order of `componentN()` functions, for example:

```
data class User(val username: String, val email: String)

fun main() {
    val user = User("alice", "alice@example.com")

    val (email, username) = user
```

```
    println(email)
    // alice

    println(username)
    // alice@example.com
}
```

In this example, because destructuring relies on the order of `componentN()` functions, `email` receives the value of `username`, and `username` receives the value of `email`.

With name-based destructuring, property names determine which values are extracted rather than the position of `componentN()` functions:

```
fun main() {
    val user = User("alice", "alice@example.com")

    // Uses name-based destructuring with explicit form
    (val mail = email, val name = username) = user

    println(name)
    // alice

    println(mail)
    // alice@example.com
}
```

Name-based destructuring is Experimental. When you enable this feature, it also introduces a new syntax for position-based destructuring using square brackets. Use this syntax for types where the order of elements matters, such as lists and other ordered collections, as well as unnamed tuples like `Pair` or `Triple`:

```
val point = Pair(10, 20)

// Uses position-based destructuring
val [x, y] = point
```

You can control how the compiler interprets destructuring declarations with the `-Xname-based-destructuring` compiler option.

It has the following modes:

- `only-syntax` enables the explicit form of name-based destructuring without changing the behavior of existing destructuring declarations.
- `name-mismatch` reports warnings when position-based destructuring in data classes uses variable names that don't match the property names.
- `complete` enables short-form name-based destructuring with parentheses and continues supporting position-based destructuring with square bracket syntax.

> **Tip:** Before enabling `complete` mode, review and resolve the warnings reported in `name-mismatch` mode. These warnings show which destructuring declarations the compiler interprets differently in `complete` mode and include suggestions for rewriting those declarations accordingly.

If you use `complete` mode, the short-form destructuring syntax with parentheses matches variables to property names instead of relying on position:

```
val (email, username) = user
```

To enable name-based destructuring in your project, add the compiler option to your build configuration file:

Gradle

```
kotlin {
    compilerOptions {
        freeCompilerArgs.add("-Xname-based-destructuring=only-syntax")
    }
}
```

Maven

```
<build>
    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <configuration>
                <args>
                    <arg>-Xname-based-destructuring=only-syntax</arg>
                </args>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Android

Kotlin for Android

Android mobile development has been Kotlin-first since Google I/O in 2019.

Over 50% of professional Android developers use Kotlin as their primary language, while only 30% use Java as their main language. 70% of developers whose primary language is Kotlin say that Kotlin makes them more productive.

Using Kotlin for Android development, you can benefit from:

- **Less code combined with greater readability**. Spend less time writing your code and working to understand the code of others.
- **Fewer common errors**. Apps built with Kotlin are 20% less likely to crash based on Google's internal data.
- **Kotlin support in Jetpack libraries**. Jetpack Compose is Android's recommended modern toolkit for building native UI in Kotlin. KTX extensions add Kotlin language features, like coroutines, extension functions, lambdas, and named parameters to existing Android libraries.
- **Support for multiplatform development**. Kotlin Multiplatform allows development for not only Android but also iOS, backend, and web applications. Some Jetpack libraries are already multiplatform. Compose Multiplatform, JetBrains' declarative UI framework based on Kotlin and Jetpack Compose, makes it possible to share UIs across platforms – iOS, Android, desktop, and web.
- **Mature language and environment**. Since its creation in 2011, Kotlin has developed continuously, not only as a language but as a whole ecosystem with robust tooling. Now it's seamlessly integrated into Android Studio and is actively used by many companies for developing Android applications.
- **Interoperability with Java**. You can use Kotlin along with the Java programming language in your applications without needing to migrate all your code to Kotlin.
- **Easy learning**. Kotlin is very easy to learn, especially for Java developers.
- **Big community**. Kotlin has great support and many contributions from the community, which is growing all over the world. Over 95% of the top thousand Android apps use Kotlin.

Many startups and Fortune 500 companies have already developed Android applications using Kotlin, see the list on the Google website for Android developers.

To start using Kotlin for:

- Android development, read Google's documentation for developing Android apps with Kotlin.
- Developing cross-platform mobile applications, see Create an app with shared logic and native UI.